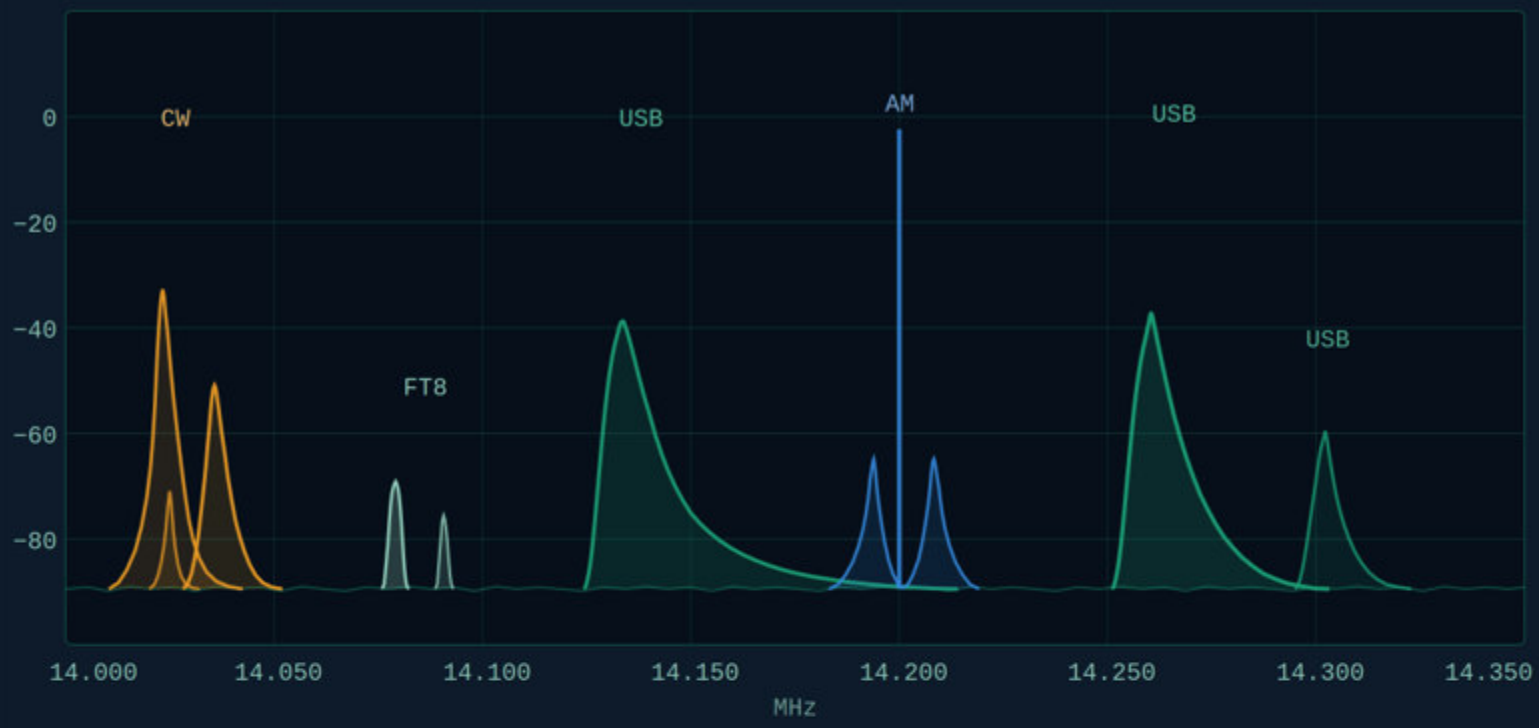


WDSP

DIGITAL SIGNAL PROCESSING LIBRARY

Reference Manual

Warren C. Pratt, NR0V
warren@pratt.one

Revision 2.1.1
2026

Copyright © 2016 – 2026 Warren C. Pratt, NR0V

Open Source — GPL v2

1 Table of Contents

2 WDSP Organization 7

3 Operation of the Channel 8

3.1 Instantiation and Initialization 8

3.2 Setting the Channel State 8

3.2.1 Important Additional Information 10

3.3 Setting Other Channel Parameters 11

3.4 Exchanging Data 13

4 WDSP Wisdom Calculations 15

5 The RXA Receiver Unit 16

5.1 Channel Parameters for RXA 16

5.2 Signal Levels for RXA 17

5.3 Block Diagram & Controls 18

5.3.1 Frequency Shifter 20

5.3.2 Resamplers 21

5.3.3 Signal Generator 22

5.3.4 Input, S, and AGC Meters 23

5.3.5 Notched Bandpass Filter 24

5.3.6 Post-filter Display Sender 30

5.3.7 AM Squelch 31

5.3.8 AM/SAM Demodulator 32

5.3.9 Voice Squelch 33

5.3.10 FM Broadcast Stereo Demodulator 34

5.3.11 FM Demodulator 35

5.3.12 FM Squelch 37

5.3.13 Spectral Noise Blanker 38

5.3.14 Equalizer 39

5.3.15 Equalizer – Using Higher-Degree Curves 40

5.3.16 AGC 42

5.3.17 Automatic Notch Filter 48
5.3.18 LMS Noise Reduction 50
5.3.19 Spectral Noise Reduction 52
5.3.20 Neural Noise Reduction 57
5.3.21 Bandpass Filter 67
5.3.22 Scope/Phase Display Sender 68
5.3.23 AM Carrier Block 69
5.3.24 CW Peaking Filter 70
5.3.25 Dolly Filter 74
5.3.26 Patch Panel - Audio Output Configuration 75
5.3.27 RXA Collectives & General Controls 76
6 The TXA Transmitter Unit 77
6.1 Channel Parameters for TXA 77
6.2 Signal Characteristics & Levels for TXA 78
6.3 Block Diagram & Controls 79
6.3.1 Resamplers 80
6.3.2 Signal Generators 81
6.3.3 PatchPanel 85
6.3.4 Meters 86
6.3.5 Noise Gate 87
6.3.6 Equalizer 88
6.3.7 Equalizer – Using Higher-Degree Curves 89
6.3.8 FM Pre-emphasis 91
6.3.9 Leveler 93
6.3.10 Phase Rotator 94
6.3.11 Continuous Frequency Compressor (CFC) 96
6.3.12 Bandpass Filters 99
6.3.13 Speech Processor 102
6.3.14 CESSB Overshoot Control 103

6.3.15 ALC 104
6.3.16 AM Modulator 105
6.3.17 FM Modulator 106
6.3.18 Up-Slew 107
6.3.19 Siphon 108
6.3.20 PureSignal Adaptive Predistortion 110
6.3.21 CFIR Filter 117
6.3.22 TXA Collectives & General Controls 118
7 Panadapter & Other Frequency-Domain Displays 119
7.1 Creating and Destroying a Display 120
7.2 Setting Display Parameters 121
7.3 Supplying Input Data 125
7.4 Retrieving Pixel Values 126
7.5 Clearing the Display Pipeline 126
7.6 Retrieving Maximum Bin Value (dB) Within a Frequency Range 126
8 Blocks Used Outside Channels 128
8.1 Preemptive Wideband Noise Blanker 128
8.2 Interpolating Wideband Noise Blanker 133
8.3 Resampler 138
8.3.1 Complex Double-Precision Resampler 138
8.3.2 Non-Complex (Audio) Single-Precision Resampler 139
8.4 Variable Ratio Resampler 141
8.5 Sample-Rate Matcher 144
8.6 Diversity Mixer 149
8.7 EER 152
8.8 Combined VOX & Downward Expander 154
8.9 Bandpass Filter Characterization Utility 159
8.10 Filter Impulse Response Cache 160
9 WDSP Version Information 161

10 Appendix A: Higher-Degree Curve Interpolation (NURBS / B-Spline) 162
10.1 Control Points 163
10.2 Degree 163
10.3 Weights 163
10.4 Knot Placement (umethod) 163
10.5 Graphical Output 164
10.6 User Interface Guidance 164
11 Appendix B: Phase Response in Audio Filters 165
11.1 Introduction: What Phase Response Actually Is 165
11.1.1 Group Delay: The Practical Measure 165
11.1.2 The Two Approaches 165
11.1.3 What the Research Says About Audibility 166
11.2 Bandpass Filters 166
11.2.1 SSB Voice 166
11.2.2 AM 167
11.2.3 CW 168
11.2.4 Digital Modes 168
11.3 The TX Audio Console: Equalizer, Compressor, and Post-EQ 169
11.3.1 The Transmit Equalizer 169
11.3.2 The Compressor and Post-EQ 169
11.3.3 Latency Considerations 170
11.4 Summary and Recommendations 170
11.5 References 172
12 Appendix C: The Transmit Audio Chain 173
12.1 Microphone / Audio Input 175
12.2 Phase Rotator 175
12.3 Noise Gate, VOX, and Anti-VOX 178
12.4 Transmit Equalizer 182
12.5 Leveler 185

12.6 Continuous Frequency Compressor and Post-EQ 187
12.7 Speech Processor 192
12.8 CESSB Overshoot Control 194
12.9 ALC 198
12.10 Adaptive Predistortion and In-band Distortion 200
13 Appendix D: Intermodulation Distortion (IMD) and PureSignal 204
13.1 What IMD Is, and Why It Matters 204
13.2 Predistortion and PureSignal 204
13.3 Measuring IMD: Two-Tone, Noise, and Speech 205
13.4 Memory Effects 206
13.5 PureSignal 3 versus PureSignal 2 209
13.6 Interpreting Your Own Measurements 213
Revision History 215

2 WDSP Organization

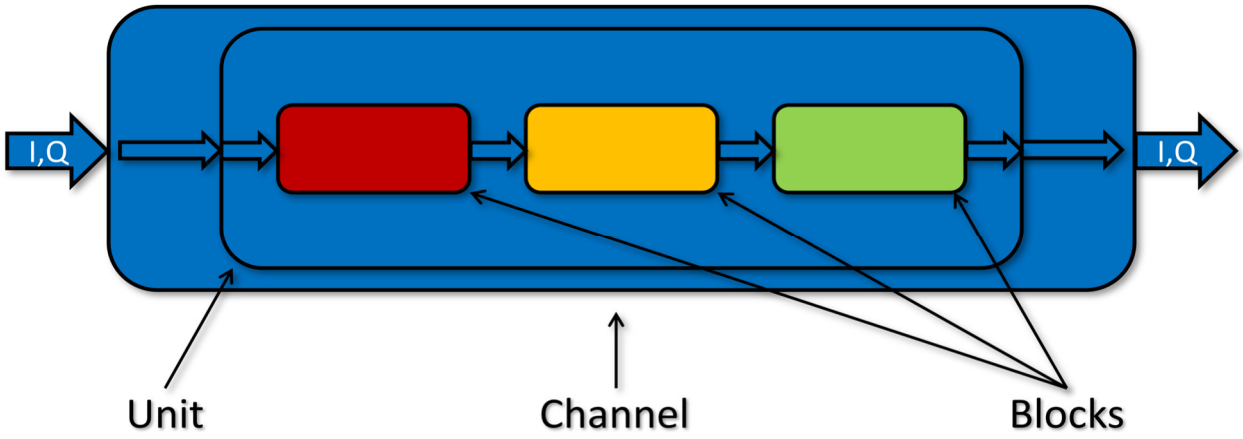


Figure 1: WDSP architecture overview — relationship between Channels, Units, and Blocks.

Within WDSP, a “channel” is the home for a single “unit”. The channel provides input/output buffering to permit the unit to run asynchronously from the outside world and it provides to the unit a thread for its computations. When a channel is “opened”, it is told which type of unit, from a selection of types, it is to house. Channels are independent of each other. Each has its own fundamental parameters, e.g., buffer sizes and sample rates. Channels can operate in parallel, in series, or have no particular relationship at all to each other.

Units are generally major functional items such as a receiver, a transmitter, an interstellar exhorborator, or whatever other major signal processing one requires. However, should one need to wrap a single simple function within the channel structure, a unit could certainly be defined to do so.

The unit houses a number of blocks and buffers as needed to implement its function. For example, in the case of a transmitter, some of the blocks would be a compressor, a band-pass filter, and ALC.

The channel remains exactly the same no matter what unit is housed within. To function within a channel, there is, of course, a specific structure expected of the unit. Once written, the unit can be replicated in as many channels as desired. Likewise, there is a specific structure expected of the blocks that are housed within the units. This block structure supports multiple instances of any block within a unit; for example, you could write a block that implements a particular type of filter and then use three different instances of that filter with the same unit. Likewise, once written, a block can be instantiated within multiple types of units.

3 Operation of the Channel

3.1 Instantiation and Initialization

A channel is instantiated as the result of a call to OpenChannel(...).

```
void OpenChannel (int channel, int in_size, int dsp_size, int
input_samplerate, int dsp_rate, int output_samplerate, int type, int state,
double tdelayup, double tslewup, double tdelaydown, double tslewdown, int
bfo)
```

channel: The unique identifier you wish to assign to this channel. Currently a number ranging 0 to 31. This range can be extended as needed.

in_size: The size (as a count of complex samples) that is to be provided as input to the channel each time an exchange function is called. This value is required to be a power of two.

dsp_size: The size (as a count of complex samples) that is used internally for DSP processing. This sets the size of buffers of data that are passed through the unit and therefore through its blocks. This parameter is required to be a power of two.

input_samplerate: The sample rate (samples / second) of the input data to the channel.

dsp_rate: The sample rate used internally within the blocks of the unit for DSP processing. This rate may affect certain performance characteristics such as filter shape factors. It must be related to the channel input and output sample-rates by powers of two. It may either be a sub-multiple, equal, or a multiple; however, the ratio must be a power of two.

output_samplerate: The sample rate of the output data from the channel (may or may not be the same as the input sample rate). Note, however, that the output sample rate is required to be related to the input sample rate by a power of two. It may either be a sub-multiple, equal, or a multiple; however, the ratio must be a power of two.

type: A number that has been assigned to the particular type of unit that is to be housed within this particular channel. At this time, two channel types are predefined and available. For the RXA Receiver the type is 0 and for the TXA transmitter the type is 1.

state: If set to 0 in OpenChannel(…), the channel will be created but will not operate until the state is later changed to 1; if set to 1 in OpenChannel(…), after creation, the channel will start, up-slew, and continue to operate as soon as it is provided with input data via an exchange function call.

tdelayup: Time (in seconds) to delay before beginning an up-slew.

tslewup: Length (in seconds) of an up-slew.

tdelaydown: Time (in seconds) to delay before beginning a down-slew.

tslewdown: Length (in seconds) of a down-slew.

bfo: If non-zero, the call to an exchange function will NOT return until output data is available. This is useful in situations where input data is not guaranteed to arrive at regular intervals.

3.2 Setting the Channel State

The state variable of the channel is changed using

```
int SetChannelState (int channel, int state, int dmode)
```

return value: the PRIOR value of state, i.e., the value at the time the call was made.

channel: identifier/number of the channel being accessed

state: 0 to turn the channel OFF; 1 to turn the channel ON

dmode: 0 - the function returns quickly, prior to completing shutdown; 1 - the function does not return until shutdown is complete, including a shutdown begun by an earlier call made with dmode == 0 (see 3.2.1).

Setting channel state is used anytime it is desired to turn ON or OFF the operation of a channel. (This is the ONLY call to turn ON/OFF a channel.) One common use is for transmit-receive transitions.

Example 1: Receive to Transmit

For the receiver: SetChannelState (0, 0, 1);

[Deactivate receiver hardware and fully activate transmit hardware here]

Then, for the transmitter: SetChannelState (5, 1, 0);

The receiver will (1) wait tdelaydown before taking action, (2) down-slew its audio output during time tslewdown, (3) flush the channel and zero the output buffer, and (4) stop operating entirely so as to not use CPU cycles. With dmode set to 1, the function SetChannelState(...) will NOT return until these actions have been completed. Therefore, with the receiver and transmitter calls made in sequence, we know that the transmitter cannot begin it's up-slew until the receiver is completely OFF. (More on dmode == 0 later.) **Note that the receiver hardware should remain FULLY ACTIVE until the down-slew is complete; without samples flowing, it is impossible for the down-slew and a subsequent channel flush to occur.**

The transmitter will then (1) wait to see non-zero samples coming in, (2) wait tdelayup before taking action, (3) up-slew its MIC audio input during time tslewup, and (4) remain ON until its state is again changed. **Note that the transmitter hardware should be FULLY ACTIVE before the up-slew begins; otherwise, an abrupt turn-on in transmitter power (a discontinuity) would result.**

Example 2: Transmit to Receive

For the transmitter: SetChannelState (5, 0, 1);

[Deactivate transmitter hardware and fully activate receiver hardware here]

Then, for the receiver: SetChannelState (0, 1, 0);

The transmitter will (1) wait tdelaydown before taking action, (2) down-slew its I-Q output during time tslewdown, (3) flush the channel and zero the output buffer, and (4) stop operating entirely so as to not use CPU cycles. With dmode set to 1, the function SetChannelState(...) will NOT return until these actions have been completed. Therefore, with the transmitter and receiver calls made in sequence, we know that the receiver cannot begin it's up-slew until the transmitter is completely OFF. (More on dmode == 0 later.) **Note that the transmitter hardware should remain FULLY ACTIVE until the down-slew is complete; without samples flowing, it is impossible for the down-slew and a subsequent channel flush to occur.**

The receiver will then (1) wait to see non-zero samples coming in, (2) wait tdelayup before taking action, (3) up-slew its I-Q input during time tslewup, and (4) remain ON until its state is again changed. **Note that the receiver hardware should be FULLY ACTIVE before the up-slew begins; otherwise, an abrupt turn-on in received signal (a discontinuity) would result.**

Example 3: Receive to Transmit, when we want to turn off multiple receivers

The primary difference here is that if we were to turn-off all receivers with dmode set to 1, each would down-slew and turn-off in sequence. This would mean more delay than necessary in making the

transition from receive to transmit. The receivers should slew down and turn-off in parallel. To do this, first begin the turn-off of every receiver with dmode set to 0. Then call SetChannelState(...) again for each receiver, still with state 0 but now with dmode set to 1, to wait for that receiver's turn-off to complete.

For the first receiver: SetChannelState (0, 0, 0);
For the second receiver: SetChannelState (1, 0, 0);
[Any console work associated with turning off the receivers can be done here]
Wait for the first receiver: SetChannelState (0, 0, 1);
Wait for the second receiver: SetChannelState (1, 0, 1);
[Deactivate receiver hardware and fully activate transmit hardware here]
For the transmitter: SetChannelState (5, 1, 0);

The first two calls begin the turn-off of both receivers and return immediately, so both receivers delay, down-slew, and flush at the same time. The third call does not return until the first receiver is completely OFF, and the fourth call does not return until the second receiver is completely OFF. If a receiver has already finished by the time its waiting call is made, that call returns immediately. Therefore, when the last waiting call returns, all receivers are completely OFF, and the total time is approximately that of the slowest receiver rather than the sum of all of them. This holds regardless of differences between the receivers in delay and slew times, sample rates, buffer sizes, or processing load. The same pattern can be used for any number of receivers: begin the turn-off of all of them with dmode == 0, then wait for each of them with dmode == 1.

> **Earlier revisions of this manual turned off the first receiver with dmode = 0 and the second with dmode = 1, assuming both would finish at about the same time. That is not guaranteed. Differences in timing and processing load between receivers can leave the first receiver still turning off when the second call returns. Consoles using that sequence should change to the one above, which requires WDSP version 2.1.1 or later. With earlier versions, the waiting calls return immediately.**

3.2.1 Important Additional Information

- When you call SetChannelState(...) with state == 1, dmode has no effect. The call begins the turn-on sequence and returns immediately. (Exception: If you call to begin the turn-on sequence while a turn-off sequence is in process, the function waits until the turnoff completes, initiates the turn-on, then returns.)
- If you call SetChannelState(...) with state == 1 when its state is already 1, nothing new happens; the function just returns. If an up-slew was in process, it just continues.
- If you call SetChannelState(...) with state == 0 when its state is already 0, nothing new happens; the function just returns. If a down-slew was in process, it just continues. Also, note here that if dmode is set to 1 to "shadow" a prior call to turn OFF a receiver, there will be no delay in the return of the call and the "shadowing" will not happen. You can check for this condition since the call returns the prior state.

- SetChannelState(...) has a "Time-Out." If no input data is being provided to the channel, a turn-off can never complete, because without data the channel cannot complete the down-slew and subsequent flush. In the absence of a Time-Out, the function would then never return in any case where it waits for a turn-off to complete: when it is called to set state to 0 with dmode == 1; when it is called with state == 0 and dmode == 1 to wait for a turn-off already in process; and when it is called to set state to 1 while a previous turn-off has not completed. The Time-Out causes the function to return after an extended time, currently 100 waits of a nominal 1 mS each (roughly 100–200 mS, depending upon the operating system's timer resolution). When the Time-Out occurs, the incomplete turn-off is abandoned without flushing the channel's buffers, so a brief remnant of earlier data may be output when the channel is next turned ON. This Time-Out feature is useful if the console is, for some reason, trying to manipulate the channel state in the absence of dataflow, for example during console start-up before data is flowing. However, each such call is delayed by the full Time-Out, so, where possible, avoid turning a channel OFF and back ON before data is flowing.

3.3 Setting Other Channel Parameters

Primary channel parameters are those that are initially specified in the OpenChannel(...) call that creates the channel. These parameters can be reset to different values at later times using appropriate calls. Resetting the channel state has been discussed above. Resetting of other parameters is discussed here.

```
void SetDSPBuffsize (int channel, int dsp_size)
```

channel: identifier/number of the channel being accessed
dsp_size: size (in complex samples) of the buffers of samples passed through the blocks of the channel

Sets the buffer size used for DSP internal processing. dsp_size is required to be a power of two. This restriction helps to achieve highest efficiency in the frequency FFT operations within most units.

This function can be called at any time, **on the control thread**, including when data is flowing on the data input thread. It sets the channel state to 0, makes the necessary changes, and then sets the channel state back to its previous value. If data is flowing, it will necessarily pause channel processing to make the changes; however, it will not block the data input thread during this time. Data output will properly slew to zero, flush the channel, stay at zero during the changes, and properly slew back up to full strength.

Data flow should NOT be terminated during the time this function is executing. Doing so could result in a "broken" down-slew and the lack of a channel flush. This should be easily manageable when the flow of data is controlled on the same control thread that calls this function.

```
void SetDSPSamplerate (int channel, int dsp_rate)
```

channel: identifier/number of the channel being accessed
dsp_rate: sample rate of samples provided to the blocks for processing

Sets the sample rate used for internal DSP processing. Note that, among other things, dsp_rate will affect the shape factor of filters.

This function can be called at any time, **on the control thread**, including when data is flowing on the data input thread. It sets the channel state to 0, makes the necessary changes, and then sets the channel state back to its previous value. If data is flowing, it will necessarily pause channel processing to make the changes; however, it will not block the data input thread during this time. Data output will properly slew to zero, flush the channel, stay at zero during the changes, and properly slew back up to full strength.

Data flow should NOT be terminated during the time this function is executing. Doing so could result in a "broken" down-slew and the lack of a channel flush. This should be easily manageable when the flow of data is controlled on the same control thread that calls this function.

```
void SetInputSamplerate (int channel, int in_rate)
```
channel: identifier/number of the channel being accessed
in_rate: sample rate of data provided from the outside to the channel for processing

The channel's state must be set to 0 (use dmode == 1 if calling SetChannelState(...)to be sure the operation is complete) before this function is called. It may be useful to make other changes that have a similar requirement, for example changing the channel's "Input Buffsize," while the channel state is still 0 and then returning it to 1 if appropriate.

```
void SetInputBuffsize (int channel, int in_size)
```
channel: identifier/number of the channel being accessed
in_size: size (in complex samples) of the buffers of data provided from the outside to the channel for processing

The channel's state must be set to 0 (use dmode == 1 if calling SetChannelState(...) to be sure the operation is complete) before this function is called. It may be useful to make other changes that have a similar requirement, for example changing the channel's "Input Samplerate," while the channel state is still 0 and then returning it to 1 if appropriate.

```
void SetOutputSamplerate (int channel, int out_rate)
```
channel: identifier/number of the channel being accessed
out_rate: sample rate of the samples the channel is to provide at its output

The channel's state must be set to 0 (use dmode == 1 if calling SetChannelState(...) to be sure the operation is complete) before this function is called.

```
void SetAllRates (int channel, int in_rate, int dsp_rate, int out_rate)
```
channel: identifier/number of the channel being accessed
in_rate: sample rate of data provided from the outside to the channel for processing
dsp_rate: sample rate of samples provided to the blocks for processing
out_rate: sample rate of the samples the channel is to provide at its output

This alternative call can be used to set all three channel rates at the same time. The channel's state must be set to 0 (use dmode == 1 if calling SetChannelState(...) to be sure the operation is complete) before this function is called.

```
void SetChannelTDelayUp (int channel, double time)
```
channel: identifier/number of the channel being accessed
time: time (seconds) to delay before beginning an up-slew

```
void SetChannelTSlewUp (int channel, double time)
```
channel: identifier/number of the channel being accessed
time: length of the up-slew (seconds)

```
void SetChannelTDelayDown (int channel, double time)
```
channel: identifier/number of the channel being accessed
time: time (seconds) to delay before beginning a down-slew

```
void SetChannelTSlewDown (int channel, double time)
```
channel: identifier/number of the channel being accessed
time: length of the down-slew (seconds)

```
void SetType (int channel, int type)
```
channel: identifier/number of the channel being accessed
type: 0 for RXA receiver; 1 for TXA transmitter; or per your own defined type

Changing the channel type requires completely destroying the old unit and building a new one. While the channel parameters as specified in the OpenChannel(...) call will be preserved, all parameters that had been stored in the blocks of the unit will be destroyed and the new unit will be created with default values.

> **Best Practice:** Several of the above function definitions state that "**Data flow should NOT be terminated during the time this function is executing.**" The reasons are simple. Certain operations such as a down-slew or up-slew of dataflow amplitude can not possibly be accomplished without data flowing. A best practice is to provide **continuous data flow**. This is generally simple to do and provides a natural and simple interface with channel operation. During transmit, receive data continues to flow, uninterrupted by the transition. Likewise, during receive, MIC or other source data continues to flow to the transmitter channel, uninterrupted.

3.4 Exchanging Data

The channel uses a time-slice-exchange model meaning that, immediately after a channel consumes a set of samples representing the signal for a certain window of time, a set of samples representing that same amount of time will be written out in exchange. Note that the number of in-samples may be different than the number of out-samples if a sample-rate change occurred somewhere in the channel processing. For example, in the case of a receiver channel, the incoming I/Q sample rate might be 384K and the output audio sample rate might be 48K. Therefore, 1024 input samples would be exchanged for 128 output samples.

All exchanges are for a power-of-two quantity of complex samples. The input samples will be consumed just BEFORE the output samples are written out. Therefore, the same buffer can be used for input and output samples if desired. There is a choice of exchange calls, depending upon the organization of the data. For example, if your data is organized in buffers of datatype double with interleaved I and Q, you can call:

//double, interleaved I/Q

```
void fexchange0 (int channel, double* in, double* out, int* error)
```
channel: identifier/number of the channel being accessed
in: pointer to input buffer of complex samples, {double I, double Q} samples

out: pointer to output buffer where complex samples are to be placed, {double I, double Q} samples
error: pointer to int location where error report from the exchange is to be placed

The error indicator will indicate if the unit provided data or whether zeros were returned instead. For example, suppose a unit isn't getting enough CPU cycles to keep up with the rate of data flow, in that case an error will be indicated and zeros will be returned. *error will be a non-zero value in the event of an *error and will otherwise be returned as 0.

As another example, suppose your data is stored in separate I and Q buffers and the datatype is float. Then you'd use:

```
//separate I/Q buffers
void fexchange2 (int channel, INREAL* Iin, INREAL* Qin, OUTREAL* Iout,
OUTREAL* Qout, int* error)
```

channel: identifier/number of the channel being accessed
Iin: pointer to input buffer of I-only samples
Qin: pointer to input buffer of Q-only samples
Iout: pointer to output buffer for I samples
Qout: pointer to output buffer for Q samples
error: pointer to int location where error report from the exchange is to be placed

INREAL and OUTREAL are typically defined as float and that definition is made in the file "comm.h".

When changing "fundamental channel parameters", calls to a 'fexchange' function may continue; however, no new data will be accepted until the reconfiguration is complete. The thread calling 'fexchange' will not be blocked during the execution of the channel parameter reconfiguration.

> Note that the fexchange0(...) function (interleaved, double, I and Q) is the preferred method of providing samples. Interleaved values is the standard for type 'complex' and the FFTW library uses this format. Using fexchange2(...) with its separate I and Q buffers also requires more CPU cycles since a conversion to and from interleaved format must be made internally to WDSP.

4 WDSP Wisdom Calculations

Several of the Blocks provided make use of the FFTW library for FFT operations. A function is provided to calculate the FFTW wisdom file.

```
int WDSPwisdom (char* directory)
```

return value: 0 if a pre-existing wisdom file is to be used; 1 if the wisdom file was just rebuilt; −1 if directory is NULL or the resulting file path would be longer than 1023 characters, in which case no wisdom is loaded or built.

directory: pointer to a string containing the path to the directory where the wisdom file is to be stored

This function should be called each time the application is opened. It will load the wisdom file if it is already present or create it if it is not.

The wisdom file includes all Forward/Backward, Complex/Real wisdom required for both RXA / TXA processing and for display processing.

A console program will report progress as the wisdom file is initially generated. In addition, a string containing information on the most recent progress in creating the wisdom file can be periodically read. This is accomplished with the call:

```
char* wisdom_get_status ()
```

A pointer to a status string is returned.

5 The RXA Receiver Unit

5.1 Channel Parameters for RXA

The channel code serves as the interface between the unit it houses and the outside world. Considering this, some of the channel parameters relate only to the outside world. However, others impact the operation or perceived operation of the unit, they specify the environment for the unit. It is this latter category of parameters that we discuss again here.

dsp_size: The unit only processes data when dsp_size samples have been accumulated. This parameter therefore impacts the latency incurred in DSP processing. Generally, using a lower dsp_size will reduce latency. However, these lower values mean a larger number of buffers are processed per unit time and each buffer requires some amount of processing overhead. Therefore, lower dsp_size implies some modest increase in the CPU load. For RXA, running at a dsp_rate of 48000, dsp_size ranging from a minimum of 64 to a maximum of 1024 offers a reasonable range of trade-offs -- 64 for minimum latency and 1024 to minimize CPU load.

Note, however, that there is an important relationship among dsp_size, dsp_rate, in_size, and input_samplerate. For example, consider the case where dsp_size = 64, dsp_rate = 48000, input_samplerate = 192000 and in_size = 256. As discussed below, a resampler will be automatically invoked to resample the input from 192000 down to 48000. For each four samples at the 192000 input rate, only one sample at the 48000 rate will be produced. Therefore, 256 input samples (in_size) will be required to populate a buffer of 64 such that DSP processing can proceed. This implies that making in_size less than 256 will not speed-up DSP processing. In this example, making it 128 would only mean that input must be supplied twice before a single DSP buffer could be processed.

dsp_rate: dsp_rate not only impacts latency, as described just above, it also impacts the results of filtering operations. Doubling dsp_rate doubles the width of filter transition bands. This can be precisely compensated by doubling the number of filter coefficients. (See later sections on band-pass filters.) Of course, increasing the rate and increasing the complexity of the filter, both require more CPU cycles. RXA has been designed to run optimally at a dsp_rate of 48000 for modes other than FM modes and at a rate of 192000 for FM modes.

tdelayup, tslewup, tdelaydown, tslewdown: Typical settings for these parameters are given here. Note that for non-CW modes they are more generous to give a very smooth sound while for CW they are more restricted for fast turnaround.

CW: tdelayup = 0.005; tslewup = 0.010; tdelaydown = 0.000; tslewdown = 0.005.

Other modes: tdelayup = 0.010; tslewup = 0.025; tdelaydown = 0.000; tslewdown = 0.010.

5.2 Signal Levels for RXA

RXA assumes that an input signal magnitude of 1.0 corresponds to 0dBm. Magnitude = sqrt (I^2 + Q^2). **It is recommended that the input be scaled to conform to this.**

Since all computations in WDSP are done in double-precision floating point, and assuming levels stay well within that range, most RXA blocks are level-independent. However, that is not exclusively true. There are a few places where absolute level matters:

1) Metering outputs. The meter outputs provide a 0 dBm output for a signal magnitude of 1.0. If your input scaling differs from this, you can compensate externally by adding or subtracting some fixed dBm value from the metering outputs.
2) Spectrum display (data following the band-pass filter). This output will need to be scaled accordingly if your input scaling is different than that prescribed above.
3) Squelch Thresholds. These will need to be scaled accordingly if your input scaling is different than that prescribed above.
4) AGC. Various AGC features such as "Slope" and "Hang" require specification of a maximum input level. By default, this is set to 1.0. If your maximum input level is different, you can change this value by calling:

```
void SetRXAAGCMaxInputLevel (int channel, double level)
```

channel: number of the receiver channel being accessed
level: maximum input level.[default = 1.0]

5.3 Block Diagram & Controls

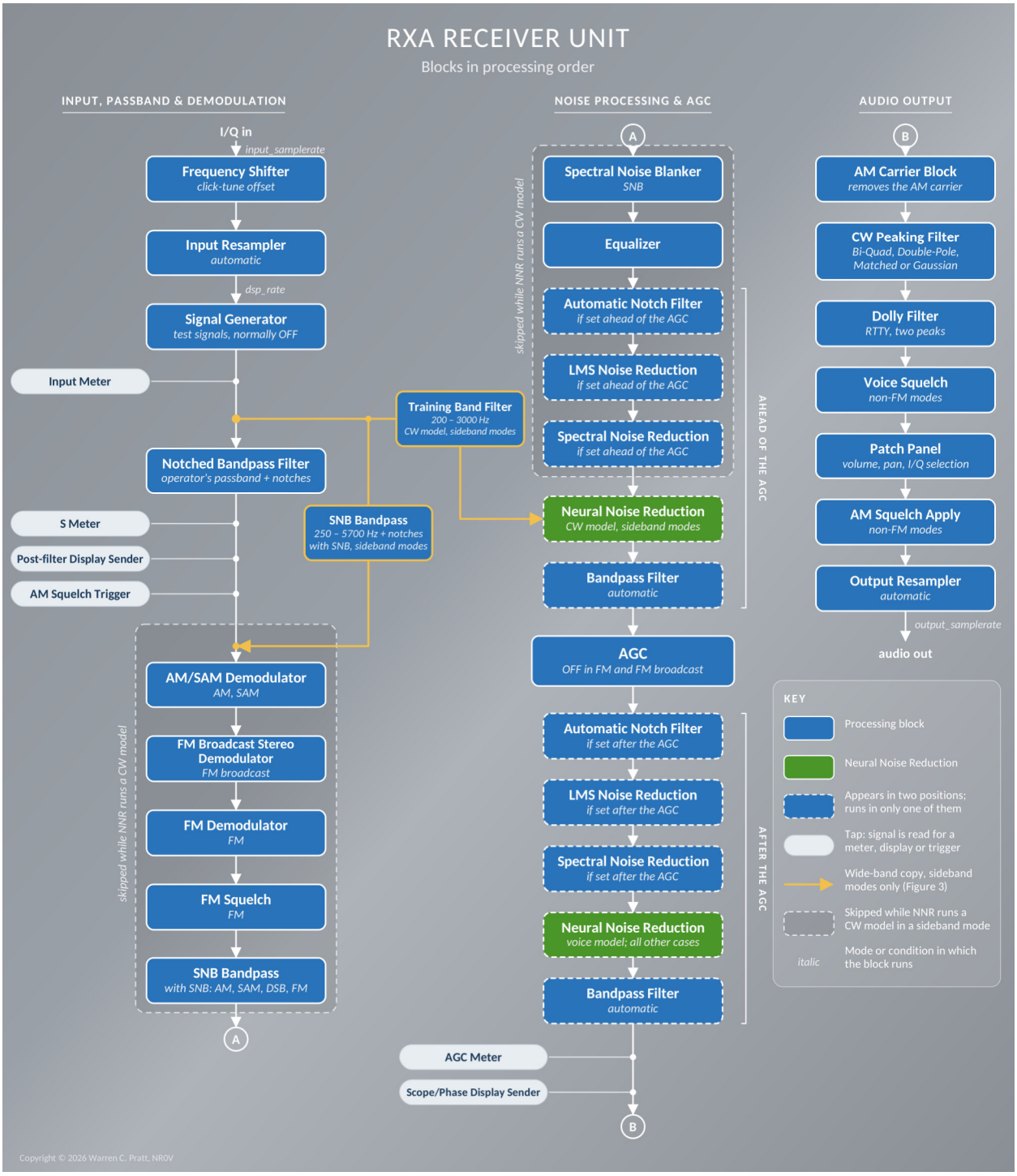


Figure 2: Block diagram of the RXA Receiver Unit — every block, in processing order.

Figure 2 shows every block in the RXA unit in the order in which it processes each buffer. The italic line in each block notes when it runs. Taps read the signal for a meter, a display or the AM squelch without changing it. Five blocks appear twice, ahead of and after the AGC, and each runs at only one of those places.

In the sideband modes, the Spectral Noise Blanker and the CW models of Neural Noise Reduction are fed from a wider copy of the signal taken ahead of the Notched Bandpass Filter. The Notched Bandpass Filter then serves only the S-meter, the display and the AM squelch trigger, and the second Bandpass Filter sets what is heard. Figure 3 shows these paths.

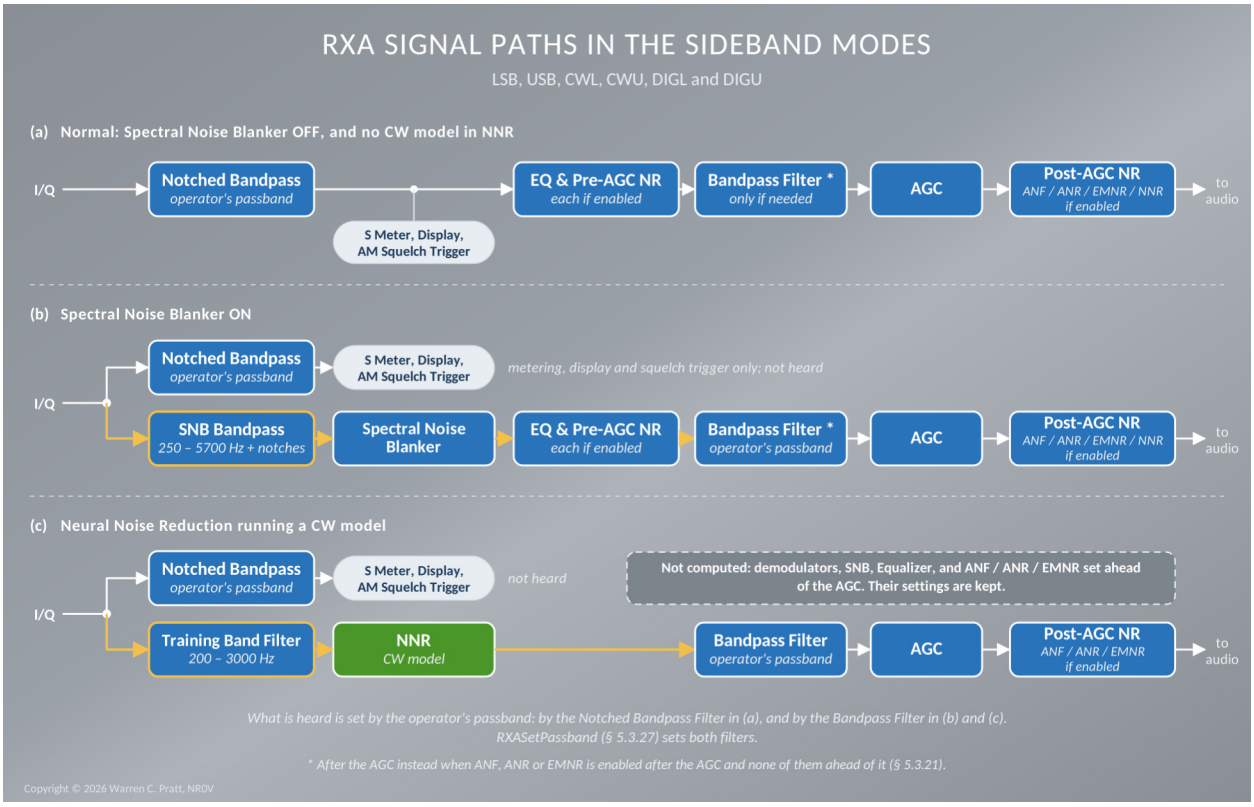


Figure 3: RXA signal paths in the sideband modes — normal, with the Spectral Noise Blanker ON, and with Neural Noise Reduction running a CW model.

The purpose/function of each block and the calls to control each block will now be explained.

5.3.1 Frequency Shifter

The frequency shifter generates a complex oscillator and multiplies the incoming sample stream by this oscillator. In analog radio terms, it's functions would be that of a "local oscillator" and "mixer." This process effectively rotates the incoming frequency spectrum by the frequency of the complex oscillator. There are several uses of this functionality, for example: (1) Click-Tune functionality where the panadapter display spectrum is static but the user clicks on a particular signal to receive, and (2) applying a final frequency offset at the output of the DFC (Direct Fourier Conversion) process.

> Note that this is done BEFORE the resampler. Therefore, if resampling is to be done to a lower rate for internal DSP processing, a portion of the total input spectrum can be selected for further processing rather than only being able to rotate the portion of the spectrum available at the lower rate.

```
void SetRXAShiftRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: turns the shifter OFF/ON, 0 for OFF, 1 for ON [default = 1]

```
void SetRXAShiftFreq (int channel, double fshift)
```

channel: number of the receiver channel being accessed
fshift: frequency of the complex oscillator in Hertz [default = 0.0]

5.3.2 Resamplers

In the discussion above of channel parameters, three distinct rates are mentioned:

input_samplerate: sample rate of data entering the channel
dsp_rate: internal rate at which the data is processed within the unit
output_samplerate: sample rate of data exiting the channel

Two resamplers are provided within the RXA unit to convert among these rates. The first of these, the "Input Resampler," converts from the input_samplerate to the dsp_rate. The second of these, the "Output Resampler," converts from the dsp_rate to the output_samplerate.

No external calls are provided or needed to control these resamplers. The three rates are set by the channel parameters and the resamplers are automatically activated when rates differ and therefore resampling is required.

As of later revisions of WDSP, the RXA input resampler has been replaced with a new half-band resampler offering improved alias rejection, increased output bandwidth, and higher input sample rates. This resampler has been configured to accept input rates up to and including 6144K. However, note that increased CPU cycles (especially floating point operations) are required to decimate PROPERLY from such high rates, as this resampler does. The new resampler is used for all RXA input decimation.

5.3.3 Signal Generator

The signal generator is available to support testing and experimentation on RXA. When it is turned ON, it replaces the incoming samples with the signal samples from the generator output. It is not used for normal receiver operation and is turned OFF by default. Therefore, its control calls can be completely ignored if the console does not desire to expose this functionality.

The signal generator can produce quite a few different types of signal outputs. However, only a subset of those have thus far been deemed useful for RXA and only that subset is currently supported with a full set of controls for RXA. The total set of possible outputs comprises: Tone, Two-Tone, Noise, Sweep, Sawtooth, Triangle, Pulse (can also pulse a tone on and off), and Silence. Sawtooth, Triangle, and Pulse produce an I-only (audio) output with Q = 0.0.

The set of outputs currently supported for RXA comprises: Tone, Noise, Sweep, and Silence.

```
void SetRXAPreGenRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 to turn the generator OFF, 1 to turn the generator ON [default = 0]

```
void SetRXAPreGenMode (int channel, int mode)
```
channel: number of the receiver channel being accessed
mode: Tone = 0, Noise = 2, Sweep = 3, Silence = 99 [default = 2]

```
void SetRXAPreGenToneMag (int channel, double mag)
```
channel: number of the receiver channel being accessed
mag: peak magnitude value of the single-tone (mode = 0). Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetRXAPreGenToneFreq (int channel, double freq)
```
channel: number of the receiver channel being accessed
freq: frequency (Hertz) of the single-tone (mode = 0). Can be either positive or negative. [default = +1000.0]

```
void SetRXAPreGenNoiseMag (int channel, double mag)
```
channel: number of the receiver channel being accessed
mag: magnitude value of the noise (mode = 2). Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetRXAPreGenSweepMag (int channel, double mag)
```
channel: number of the receiver channel being accessed
mag: peak magnitude of swept single tone (mode = 3). Typical range is 0.0 to 1.0.[default = 1.0]

```
void SetRXAPreGenSweepFreq (int channel, double freq1, double freq2)
```
channel: number of the receiver channel being accessed
freq1, freq2: frequency limits of the sweep range (Hertz). These may be either positive or negative. Sweep progresses from freq1 to freq2 and then repeats.[defaults: freq1 = -20000.0, freq2 = +20000.0]

```
void SetRXAPreGenSweepRate (int channel, double rate)
```
channel: number of the receiver channel being accessed
rate: rate at which the sweep progresses (Hertz / Second).[default = +4000.0]

5.3.4 Input, S, and AGC Meters

As shown in the block diagram, three sets of RXA metering data are available to the console: Input-level, S-level (after the bandpass and notch filtering), and AGC-level (after the AGC). For Input, Input_peak and Input_average are available. For S, S_peak and S_average are available. For AGC, AGC_peak, AGC_average, and AGC_gain are available.

The console can call for individual values any time it wants to update the respective meter user interfaces.

```
double GetRXAMeter (int channel, int mt)
```

return value: requested meter value (dBm, except dB for AGC_gain)
channel: number of the receiver channel being accessed
mt: type of meter data being requested. mt values are:

- S_peak = 0
- S_average = 1
- Input_peak = 2
- Input_average = 3
- AGC_gain = 4
- AGC_peak = 5
- AGC_average = 6

5.3.5 Notched Bandpass Filter

The Notched Bandpass Filter provides both the primary bandpass filtering AND the fixed notch filtering (pre-specified notch frequencies as opposed to automatic notch filtering). Therefore, both the lower and upper bandpass limits and all the notch filtering information must be specified.

The "sharpness" of these filters, when running RXA at a dsp_rate of 48000, is shown in the following two figures. The first of these figures shows the response curves for different impulse response lengths, all using the 4-term Blackman-Harris window. The second figure is the same except using the 7-term Blackman-Harris window. The 4-term yields slightly sharper cutoff; however, less cutoff depth. The 4-term would be preferred in most situations since its cutoff depth approaches the dynamic range of the receiver hardware; the 7-term is overkill on cutoff depth for most situations.

> Note that the transition bands (regions between the pass-band and the cutoff bands) are the most important part to observe in these figures. These transition bands will maintain the same width independent of the width of the filter pass-band.

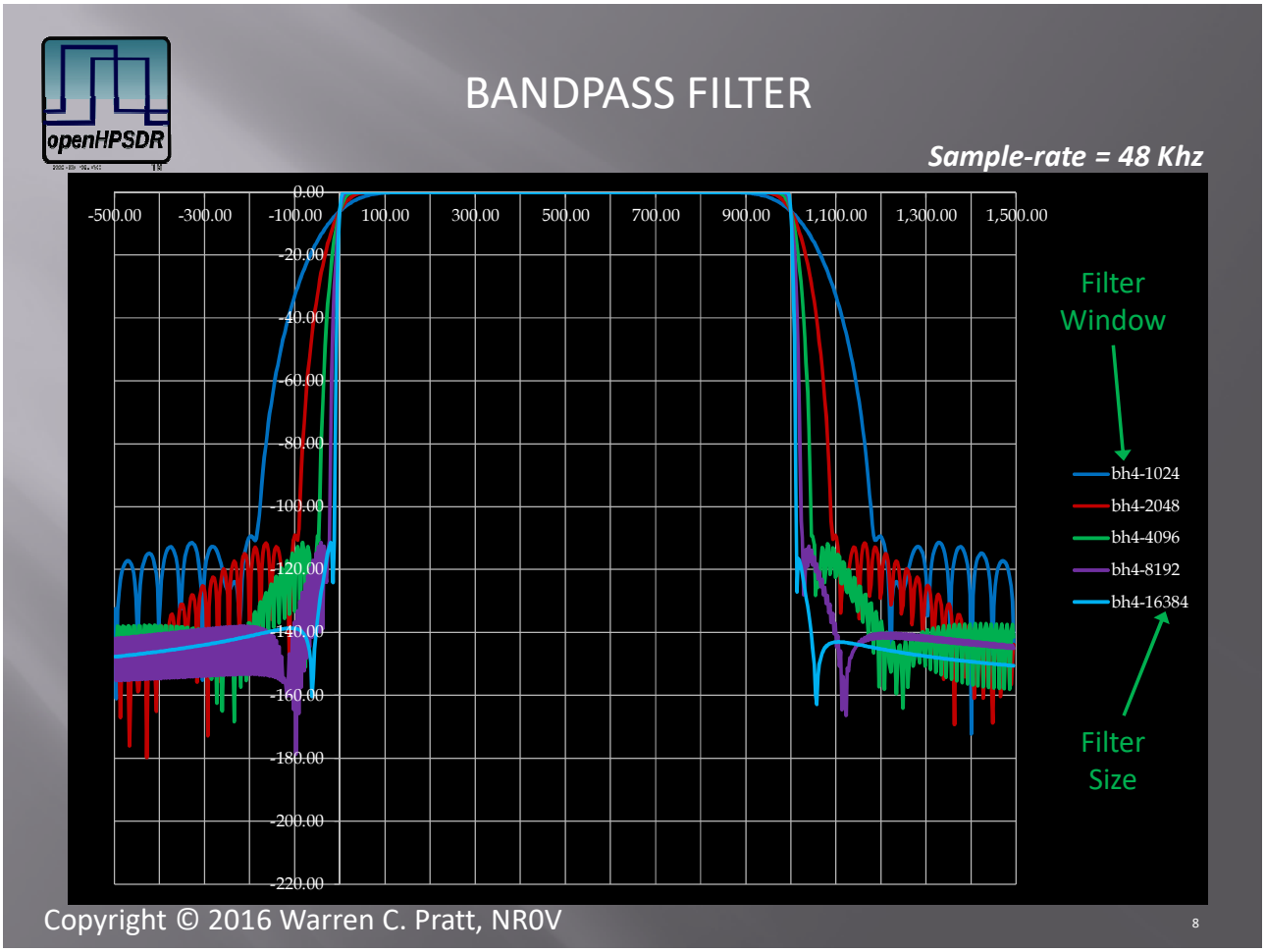


Figure 4: RXA Bandpass Filter frequency response, 4-term Blackman-Harris window, 48 kHz sample rate.

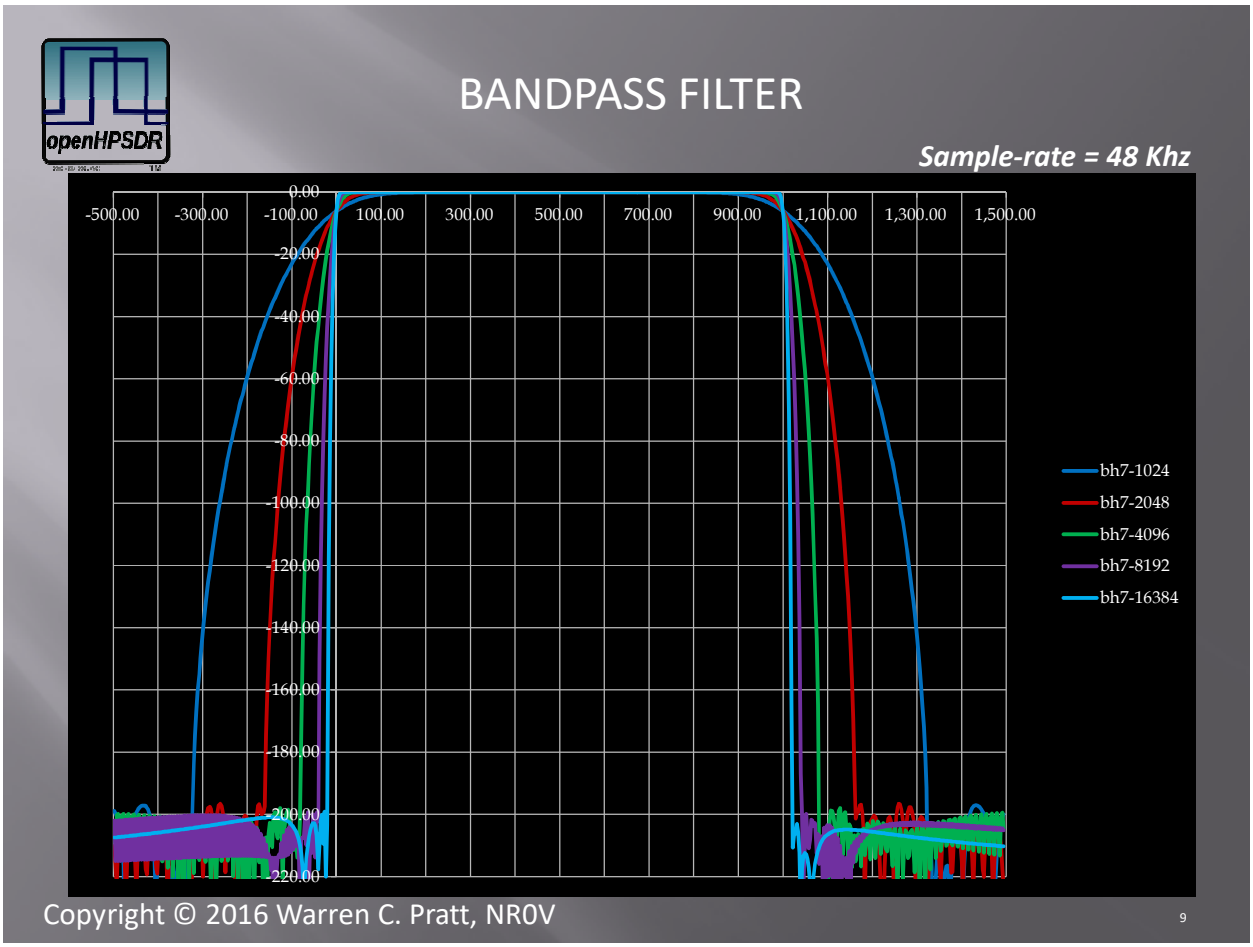


Figure 5: RXA Bandpass Filter frequency response, 7-term Blackman-Harris window, 48 kHz sample rate.

The following figure displays an example of a Notched Bandpass Filter with three notches of three different widths. This example uses a filter size of 2048 at a dsp_rate of 48000. The notch at -1 Khz relative to the receiver tune frequency is 200 Hz in width, the notch at -2 Khz is 400 Hz in width, and the notch at -4 Khz is 800 Hz in width. Note that the width specifies the -6 dB points.

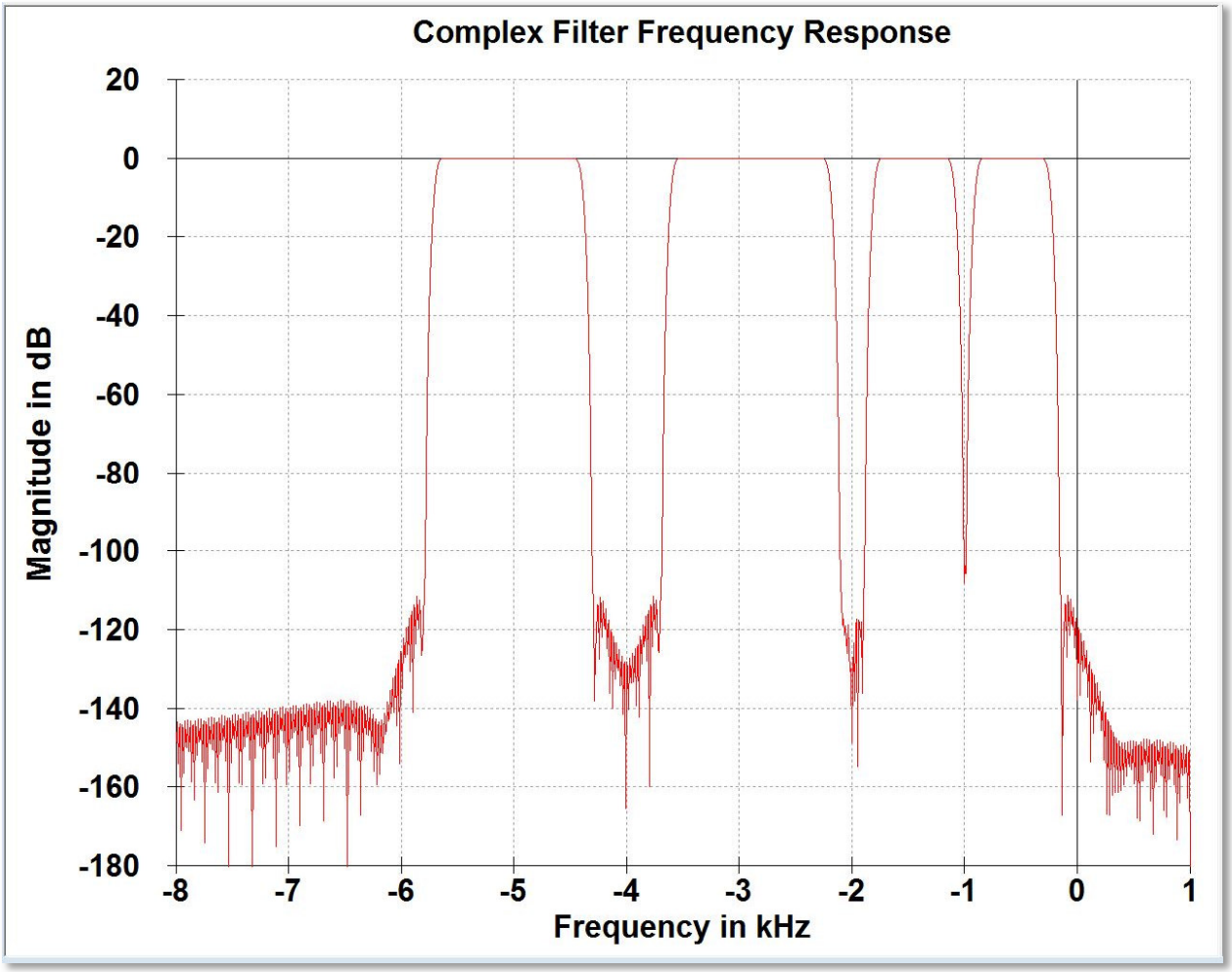


Figure 6: Example Notched Bandpass Filter with three notches at −1, −2, and −4 kHz relative to the tune frequency.

Bandpass Filter Settings

```
void RXANBPSetRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 to turn the filter OFF; 1 to turn the filter ON. [default = 1]

```
void RXANBPSetFreqs (int channel, double f_low, double f_high)  — DEPRECATED
```
channel: number of the receiver channel being accessed
f_low: lower of the two frequencies (Hertz). May be negative or positive. This is a -6 dB point of the filter.
[default = -4150.0]
f_high: higher of the two frequencies (Hertz). May be negative or positive. This is a -6 dB point of the filter.
[default = -150.0]

The preferred way to set the lower and upper bandpass limits is to use a "Collective" call which not only sets this filter properly but also sets the bandpass limits for other blocks that need that same information. See the section of this document on "RXA Collectives" for this preferred call.

This is especially important when Neural Noise Reduction runs with a CW model. The operator's passband must then also be set in the second Bandpass Filter, which decides what is heard; setting it here alone narrows the S-meter but not the audio (§ 5.3.20).

```
void RXANBPSetWindow (int channel, int wintype)
```
channel: number of the receiver channel being accessed
wintype: 0 for 4-term Blackman-Harris window; 1 for 7-term Blackman-Harris window.[default = 0]

```
void RXANBPSetNC (int channel, int nc)  — DEPRECATED
```
channel: number of the receiver channel being accessed
nc: length of the desired filter impulse response. This must be a power of two AND must be greater than or equal to the DSP Buffer Size in use.[default = 2048]

```
void RXANBPSetMP (int channel, int mp)  — DEPRECATED
```
channel: number of the receiver channel being accessed
mp: 0 for Linear Phase; 1 for Low Latency (Minimum Phase)[default = 0]

Notch Filter Settings

```
void RXANBPSetNotchesRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 to turn OFF all specified notches; 1 to turn ON all specified notches that are 'active'[default = 0]

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, the notches have no effect on what is heard (§ 5.3.20).

The width of each notch is specified and implies the frequencies at the edges of the notch where the response is 6dB down. The transition bands of notches are not perfectly vertical; the transition bands have some non-zero width. Therefore, if essentially full-cutoff (>100dB) is to be attained at the center of a notch, the notch must be specified with some minimum width. The value of this minimum width can be found using the call:

```
void RXANBPGetMinNotchWidth (int channel, double* minwidth)
```
channel: number of the receiver channel being accessed
minwidth: pointer to the location into which the minimum notch width is to be stored. The value stored is in Hertz.

The width of all notches will be automatically increased, when necessary, to that required to achieve >100dB cutoff at the notches' centers if the "autoincr" flag is set.

```
void RXANBPSetAutoIncrease (int channel, int autoincr)
```
channel: number of the receiver channel being accessed
autoincr: 0 sets Auto-Increase to OFF; 1 sets Auto-Increase to ON[default = 1]

Notches are specified at absolute RF frequencies. For example, if there is always an interfering carrier on 145.275 Mhz, a notch of a desired width may be specified at that frequency. The Notched Bandpass Filter then compares each notch frequency with the current RF frequency to which the radio is tuned to determine which notches are within the passband and where within the passband they are placed. To perform this comparison, the current radio Tune Frequency and any Shift Frequency offset to that must be specified. The following two calls are used for that purpose.

```
void RXANBPSetTuneFrequency (int channel, double tunefreq)
```
channel: number of the receiver channel being accessed
tunefreq: current RF tune frequency of the radio (Hertz)[default = 0.0]

```
void RXANBPSetShiftFrequency (int channel, double shift)
```
channel: number of the receiver channel being accessed
shift: current RF shift frequency (Hertz)[default = 0.0]

The Notched Bandpass Filter maintains a database of the desired notches. Several calls are provided to manage this database. Note that RXA currently supports a maximum of 1024 notches.

The following call can be made to determine the number of notches currently stored in the database.

```
void RXANBPGetNumNotches (int channel, int* nnotches)
```
channel: number of the receiver channel being accessed
nnotches: pointer to location where the number of notches currently in the database is to be stored

A new notch is added to the database using:

```
int RXANBPAddNotch (int channel, int notch, double fcenter, double fwidth,
int active)
```
return value: 0 if the operation was successful; -1 if the specified value of 'notch' was >= the total (new, including this addition) number of notches, i.e., it was out-of-range, OR, if adding another notch would exceed the maximum number of notches that are supported.
channel: number of the receiver channel being accessed
notch: identifier for the notch to be added. Notch identifiers begin at 0 and are positive. Notches in the database are required to be consecutive, i.e., with no gaps in used notch identifiers. Therefore, the value of 'notch' must be less than the total (new, including this addition) number of notches. If 'notch' is equal to new_number_of_notches minus one, this notch will be added at the end of the list. If 'notch' is less than new_number_of_notches minus one, this notch will be added with the 'notch' identifier specified and the notches with higher identifiers will be pushed-up in the list and their identifiers incremented accordingly.
fcenter: center frequency of the notch (Hertz)
fwidth: width of the notch (Hertz)
active: 0 implies that even though the notch will be stored in the database, it will not be used in filtering operations. 1 implies that it will be stored and will be used in filtering operations.

The current parameters of a specified notch can be obtained using:

```
int RXANBPGetNotch (int channel, int notch, double* fcenter, double* fwidth,
int* active)
```
return value: 0 if the operation was successful; -1 if the value of 'notch' was >= the current number of notches in the database.
channel: number of the receiver channel being accessed
notch: identifier for the notch to be read
fcenter: pointer to location to store center value of this notch (Hertz)
fwidth: pointer to location to store width of this notch (Hertz)
active: pointer to location to store the active flag for this notch

A specified notch can be deleted from the database using the following call. Note that any notches with higher identifiers will be pushed down in the list and their identifiers decremented accordingly.

```
int RXANBPDeleteNotch (int channel, int notch)
```
return value: 0 if the operation was successful; -1 if the value of 'notch' was >= the current number of notches in the database.

channel: number of the receiver channel being accessed
notch: identifier for the notch to be deleted

One or more parameters of a specified notch can be changed using the following call. Note that ALL the parameters must be supplied even though perhaps only some of them are being changed in value.

```
int RXANBPEditNotch (int channel, int notch, double fcenter, double fwidth,
int active)
```

return value: 0 if the operation was successful; -1 if the value of 'notch' was >= the current number of notches in the database.
channel: number of the receiver channel being accessed
notch: identifier for the notch to be edited
fcenter: center frequency of the notch (Hertz)
fwidth: width of the notch (Hertz)
active: 0 implies that even though the notch will be stored in the database, it will not be used in filtering operations. 1 implies that it will be stored and will be used in filtering operations.

5.3.6 Post-filter Display Sender

The Post-filter Display Sender is used to transfer samples of the signal, taken after the Notched Bandpass Filter, to a WDSP Display Unit. This can provide a frequency-domain display showing the signal(s) within the passband and displaying the operation of the Notched Bandpass Filter and any other active preceding blocks in RXA.

> Note that Display Units are instantiated and configured separately from RXA and TXA. See the appropriate section of this document for additional information on setup, configuration, and use of Display Units.

A single Display Unit may be used to process multiple data streams at different times, e.g., a full-bandwidth panadapter stream at some times as well as this post-filter data stream at others. Therefore, a 'flag' to turn the sender OFF/ON is needed.

Parameters are also needed to specify the target Display Unit, the target sub-span (in the event the display unit is set up for stitched displays), and which LO-position (in the event that spur-elimination mode is being used).

All this information is supplied in a single call:

```
void SetRXASpectrum (int channel, int flag, int disp, int ss, int LO)
```

channel: number of the receiver channel being accessed
flag: 0 to turn the sender OFF, 1 to turn the sender ON[default = 0]
disp: identifier of the display unit. The overall configuration *might* be such that there is one display unit per DSP channel and the number is the same as the channel number. However, this is not necessarily the case.
ss: the sub-span which is to receive the data, normally this will be set to 0.
LO: the LO position to receive the data, normally this will be set to 0.

5.3.7 AM Squelch

The AM Squelch is provided for modes OTHER THAN FM. Special features of the WDSP implementation include raised-cosine OFF-ON-OFF transitions and a variable tail-length which depends upon the strength of the signal. Weaker signals get a longer tail-length to attempt to avoid the squelch turning the audio OFF/ON as the strength slightly varies.

Note that a newer alternative squelch algorithm is described in § 5.3.9. It is considerably more advanced in voice detection than this volume-based approach. Take your choice!

```
void SetRXAAMSQRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 for AM Squelch OFF; 1 for AM Squelch ON

```
void SetRXAAMSQThreshold (int channel, double threshold)
```
channel: number of the receiver channel being accessed
threshold: squelch threshold, i.e., if the passband signal strength is below this level the audio will be squelched (turned-off). 'threshold' is a negative dBm value.[default = -40.0]

```
void SetRXAAMSQMaxTail (int channel, double tail)
```
channel: number of the receiver channel being accessed
tail: maximum tail length to be applied (seconds)[default = 1.50]

5.3.8 AM/SAM Demodulator

This demodulator provides either AM demodulation or Synchronous-AM demodulation, dependent upon settings. Note that the **SetRXAMode(...)** collective makes the choice of AM or SAM, dependent upon the overall RXA Mode setting. (See the "RXA Collectives" section of this document.)

```
void SetRXAAMDRun (int channel, int run)  – DEPRECATED
```
channel: number of the receiver channel being accessed
run: 0 to turn OFF the demodulator; 1 to turn ON the demodulator

For SAM ONLY, a choice of whether to receive both sidebands or only the lower or the upper sideband can be made. This is often useful in avoiding interference on one side or the other. Note that this is NOT a bandpass filtering operation; therefore, the received station does not have to be exactly centered on frequency for the sideband selection to function. This is particularly useful in AM round-tables where not all stations are on exactly the same frequency and interference exists on one side.

```
void SetRXAAMDSBMode (int channel, int sbmode)
```
channel: number of the receiver channel being accessed
sbmode: 0 = receive both sidebands; 1 = receive LSB only; 2 = receive USB only[default = 0]

For both AM and SAM demodulation, there is an option to "level" the carrier in situations of carrier fading. This is accomplished by stripping out the varying carrier and replacing it with a stable carrier. This feature is deactivated/activated with the following call.

```
void SetRXAAMDFadeLevel (int channel, int levelfade)
```
channel: number of the receiver channel being accessed
levelfade: 0 = OFF; 1 = ON[default = 1]

5.3.9 Voice Squelch

This Voice Squelch is offered as an alternative to the simple volume-based AM Squelch described in § 5.3.7. It operates by attempting to distinguish a voice signal from noise and various types of interference. This is for use for non-FM modes. Consider the two alternative systems and take your choice!

Four controls are provided.

```
void SetRXASSQLRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 for Voice Squelch OFF; 1 for Voice Squelch ON

```
void SetRXASSQLThreshold (int channel, double threshold)
```
channel: number of the receiver channel being accessed
threshold: squelch threshold. The value should be between 0.0 and 1.0. A typical value would be 0.16.

```
void SetRXASSQLTauMute (int channel, double tau_mute)
```
channel: number of the receiver channel being accessed
tau_mute: time constant (seconds) to be used in muting the squelch. A reasonable (but wide) range would be 0.1 to 2.0. Typical values are near the low-end of the range.

```
void SetRXASSQLTauUnMute (int channel, double tau_unmute)
```
channel: number of the receiver channel being accessed
tau_unmute: time constant (seconds) to be used in un-muting the squelch. A reasonable (but wide) range would be 0.1 to 1.0. Typical values are near the low-end of the range.

5.3.10 FM Broadcast Stereo Demodulator

While not an Amateur Radio feature, this is provided for those users who use this library with hardware and console software that cover the FM Broadcast band. This demodulator does not process digital content included in some FM broadcasts. Enabling it is very simple within an existing console. Requirements:

- Select the RXA Mode '12'.
- 'dsp_rate' MUST be set to specifically 192K, the same rate as for narrowband FM demodulation.
- 'input_samplerate' MUST be 192K or greater. Note that an FM Broadcast signal consumes approximately 180K of bandwidth. Hence, 192K is the minimum that can possibly support this.
- There are only two added calls for this demodulator. Both are optional and described below.

Some additional notes about this mode:

- Bandpass filters within WDSP are intentionally disabled for this mode – they would only restrict the edges of the bandwidth that this demodulation needs to operate correctly.
- Enabling this also enables its OWN INTERNAL SQUELCH. It is not expected that any additional squelch capability will be used when this mode is active.
- Comment on 'input_samplerate': 192K is ideal, assuming the source has good signal quality, meaning good anti-aliasing. If input signal quality is questionable, it may be better to use a higher 'input_samplerate' and let the WDSP decimator automatically bring that down to 192K.

```
void SetRXAWBFMdmph (int channel, int dmph_run, int dmph_continent)
```

channel: number of the receiver channel being accessed
dmph_run: 1 for use deemphasis; 0 for de-activate deemphasis. Deemphasis is normally used in all FM broadcast. [default = 1].
dmph_continent: North America and Europe use slightly different standards for deemphasis. 0 for North America; 1 for Europe. [default = 0].

```
int GetRXAWBFMStereoIndicator (int channel)
```

Return value: 0 if broadcast is NOT detected as Stereo; 1 if Stereo detected.

5.3.11 FM Demodulator

The FM Demodulator supports the standard FM audio bandwidth of 300 Hz to 3000 Hz and applies the standard de-emphasis of 6 dB / octave.

The FM demodulator is turned OFF/ON by using the **SetRXAMode(...)** collective. Hence, no call is provided here to accomplish that function.

The expected maximum deviation of the received signal is set as follows:

```
void SetRXAFMDeviation (int channel, double deviation)
```
channel: number of the receiver channel being accessed
deviation: deviation in Hertz

FM signals may have CTCSS tones used to trigger repeaters. Those may be passed through by the repeaters and it is sometimes desirable to notch those out of the audio output. A special notch filter is provided to perform this task. Two controls are provided for this filter:

```
void SetRXACTCSSRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = turn OFF the CTCSS notch filter; 1 = turn ON the CTCSS notch filter

```
void SetRXACTCSSFreq (int channel, double freq)
```
channel: number of the receiver channel being accessed
freq: frequency of the tone and notch (Hertz)

```
void SetRXAFMNCde (int channel, int nc)  – DEPRECATED
```
channel: number of the receiver channel being accessed

```
void SetRXAFMMPde (int channel, int mp)  – DEPRECATED
```
channel: number of the receiver channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency
channel: number of the receiver channel being accessed

```
void SetRXAFMMPaud (int channel, int mp)  – DEPRECATED
```
channel: number of the receiver channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency

The FM demodulator also includes a “volume limiter” that can be used to level the noise and signal volumes during the squelch tail or when squelch is OFF to listen for very weak signals. There are two controls for this functionality.

```
void SetRXAFMLimRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = turn OFF the volume limiter; 1 = turn ON the volume limiter

```
void SetRXAFMLimGain (int channel, double gaindB)
```
channel: number of the receiver channel being accessed
gaindB: the gain in dB that the limiter should apply. A reasonable range is 0dB to 30dB.

The normal frequency range for narrow-band FM communication is 300Hz to 3000Hz. This can be adjusted, if desired, to offer higher fidelity. However, note that going lower means that the receive bandwidth overlaps the range in which CTCSS tones are transmitted. Going higher in frequency

increases the necessary receive bandwidth which, per Carson's Rule, is 2 * (peak_deviation + highest audio frequency).

```
void SetRXAFMAFFilter (int channel, double low, double high)
```

channel: number of the receiver channel being accessed
low: low-cut frequency (Hz)
high: high-cut frequency (Hz)

5.3.12 FM Squelch

FM Squelch is turned OFF/ON using the following call:

```
void SetRXAFMSQRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 = turn OFF the FM Squelch; 1 = turn ON the FM Squelch[default = 0]

```
void SetRXAFMSQThreshold (int channel, double threshold)
```

channel: number of the receiver channel being accessed
threshold: squelch threshold. This is a positive value which normally ranges from 1.0 to 0.01. It can be generated with a slider from which the output value range is 0.0 to 100.0 and then applying the equation: threshold = pow (10.0, -2.0 * slider_value / 100.0);

```
void SetRXAFMSQNC (int channel, int nc)  – DEPRECATED
```

channel: number of the receiver channel being accessed

```
void SetRXAFMSQMP (int channel, int mp)  – DEPRECATED
```

channel: number of the receiver channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency

5.3.13 Spectral Noise Blanker

The Spectral Noise Blanker uses Linear Predictive Coding (LPC) to detect impulses or other severe abnormalities in signals and to replace the corrupted samples with an estimate of the original signal.

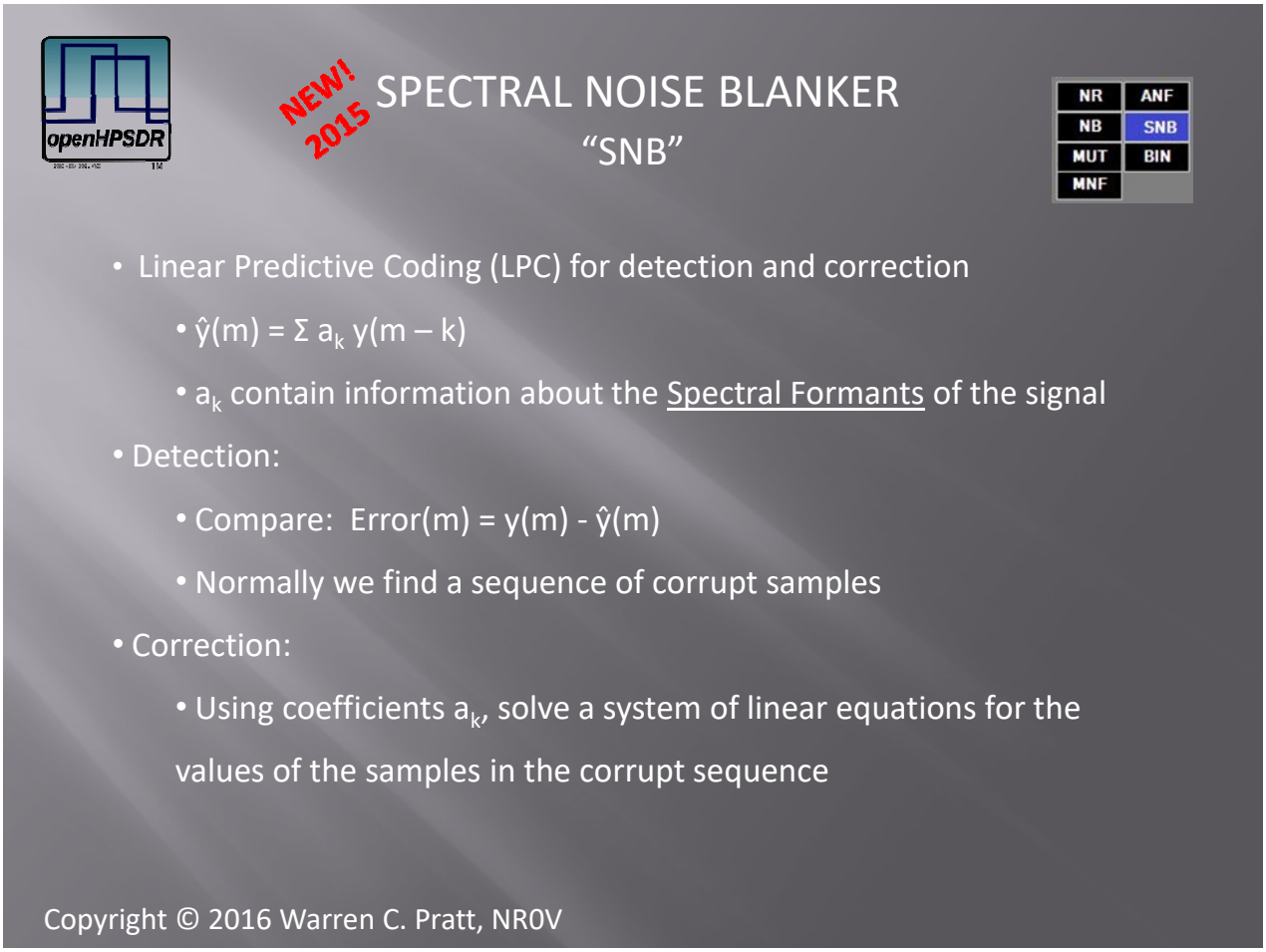


Figure 7: Spectral Noise Blanker (SNB) — Linear Predictive Coding detection and waveform correction.

The Spectral Noise Blanker is turned OFF/ON by calls to:

```
void SetRXASNBARun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 for OFF; 1 for ON[default = 0]

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, the Spectral Noise Blanker has no effect and is not computed (§ 5.3.20).

```
void SetRXASNBAOutputBandwidth (int channel, double flow, double fhigh)  –
DEPRECATED
```

channel: number of the receiver channel being accessed
flow: lower bandpass limit (Hertz)
fhigh: upper bandpass limit (Hertz)

5.3.14 Equalizer

The equalizer uses a continuous frequency response algorithm meaning that the gain is specified at any number of frequencies and the response is interpolated between the specified points. This is in contrast to a frequency "band" approach.

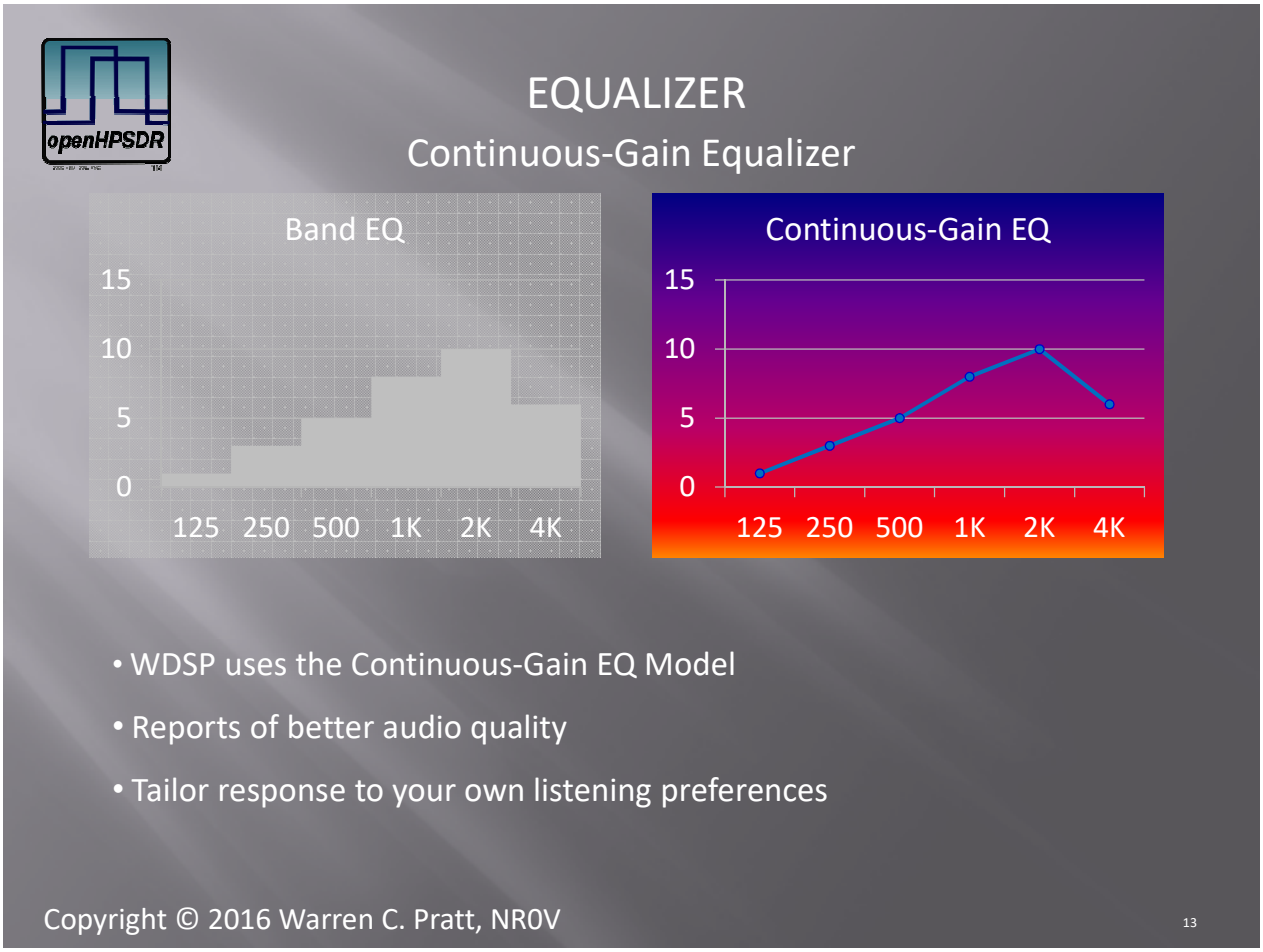


Figure 8: RXA Continuous-Gain Equalizer panel comparing Band EQ and Continuous-Gain EQ response models.

The equalizer is turned OFF/ON using calls to:

```
void SetRXAEQRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 for OFF; 1 for ON [default = 0]

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, the Equalizer has no effect and is not computed (§ 5.3.20).

```
void SetRXAEQNC (int channel, int nc)  – DEPRECATED
```

channel: number of the receiver channel being accessed
nc: length of the EQ filter impulse response. 'nc' must be a power-of-two and must be greater than or equal to the DSP Buffer Size in use. [default = AUTOMATIC]

```
void SetRXAEQMP (int channel, int mp)  – DEPRECATED
```
channel: number of the receiver channel being accessed
mp: 0 for Linear-Phase; 1 for Low-Latency (Minimum Phase)[default = 1 (Minimum Phase) - AUTOMATIC]

The window-type used in the generation of the EQ filter can be selected. The Kaiser window (wintype = 2) provides slightly better roll-off than the Blackman-Harris options. This setting should normally be left at the default value.

```
void SetRXAEQWintype (int channel, int wintype)
```
channel: number of the receiver channel being accessed
wintype: 0 = 4-term Blackman-Harris; 1 = 7-term Blackman-Harris; 2 = Kaiser [default = 2 (Kaiser)]

The cutoff-mode used in the generation of the EQ filter can be selected. This setting should normally just be left at the default value.

```
void SetRXAEQCtfmode (int channel, int mode)
```
channel: number of the receiver channel being accessed
mode: 0 = 4th-power roll-off on each side of the specified passband; 1 = no roll-off outside the specified passband [default = 0]

The following call is the primary and preferred method of setting the equalizer response.

```
void SetRXAEQProfile (int channel, int nfreqs, double* F, double* G)
```
channel: number of the receiver channel being accessed
nfreqs: number of frequency/gain pairs to be specified
F: pointer to an array of frequencies (Hertz). NOTE: The 0th element of this vector is NOT USED. So, for example, if nfreqs is 5, F[0] is not used and the five frequencies will be stored in F[1] ... F[5]. The F[] array has (nfreqs + 1) values.
Note that acceptable values are between 0.0 Hz and dsp_rate/2.0 Hz. It is acceptable to specify the same frequency multiple times; however, if different gains are specified for the same frequency, it is indeterminate as to which of the gains will be applied at that frequency. Frequency/Gain pairs do not necessarily have to be specified in order of increasing frequency. If they are not, they will be sorted into the appropriate order prior to application.
G: pointer to a corresponding array of gain values (dB). The 0th element is used to specify a "preamp gain" value to be applied to all frequencies. The G[] array has (nfreqs + 1) values.

```
void SetRXAGrphEQ (int channel, int* rxeq)  – DEPRECATED
```
channel: number of the receiver channel being accessed

```
void SetRXAGrphEQ10 (int channel, int* rxeq)  – DEPRECATED
```
channel: number of the receiver channel being accessed

5.3.15 Equalizer – Using Higher-Degree Curves

The RX Equalizer supports higher-degree B-spline / NURBS curve interpolation between control points. For a complete description — including degrees, weights, knot placement, and graphical output — see Appendix A. The following calls provide access to this functionality.

> Note that the control points are provided by the calls above, setting the 'F' and 'G' arrays.

```
void SetRXAEQCurve (int channel, int degree, int r, int umethod)
```
channel: number of the receiver channel being accessed
degree: degree of the curve segments [default = 0]

0 – Use previous method, linear, no graphical output.

1 – Linear, high-degree method with graphical output.

2, 3, … , 16 – Degree of curve segments.

r: 0 – do NOT use Weights; 1 – use Weights [default = 0]
umethod: method used to position knots. [default = 0, recommended]

0 – calculated positions based upon control point activity.

1 – equally-spaced in 'u' parameter space.

```
void SetRXAEQWeights (int channel, int ncp, double* weights)
```

channel: number of the receiver channel being accessed
ncp: number of control points (note: one weight per control point)
weights: vector of weights [defaults = 1.0]

> NOTE: There must be a weight, for each control point, satisfying these conditions when weights are used:

- weights[i] > 0.0
- weights[i] < 100.0

```
void GetRXAEQDraw (int channel, double* X, double* Y)
```

channel: number of the receiver channel being accessed
X, Y: pointers to vectors into which X (frequency) and Y (gain) values will be copied.

5.3.16 AGC

WDSP provides a full-featured AGC including:

- Exponential attack / decay
- Hang for quieting when listening to relatively strong stations
- Dual-decay for suppression of certain impulses not removable using wideband noise blankers
- Controls for on-panadapter adjustment of the AGC Threshold and Hang Level
- Zero Overshoot for avoidance of un-natural sound and AGC "pumping"
- Slope to allow stronger signals to yield higher audio output than weaker signals

The following figure illustrates the exponential attack and decay. The attack is typically set to be much faster than the normal decay. Note that the blue upper plot is of the AGC input signal and the lower red plot is of the AGC output signal.

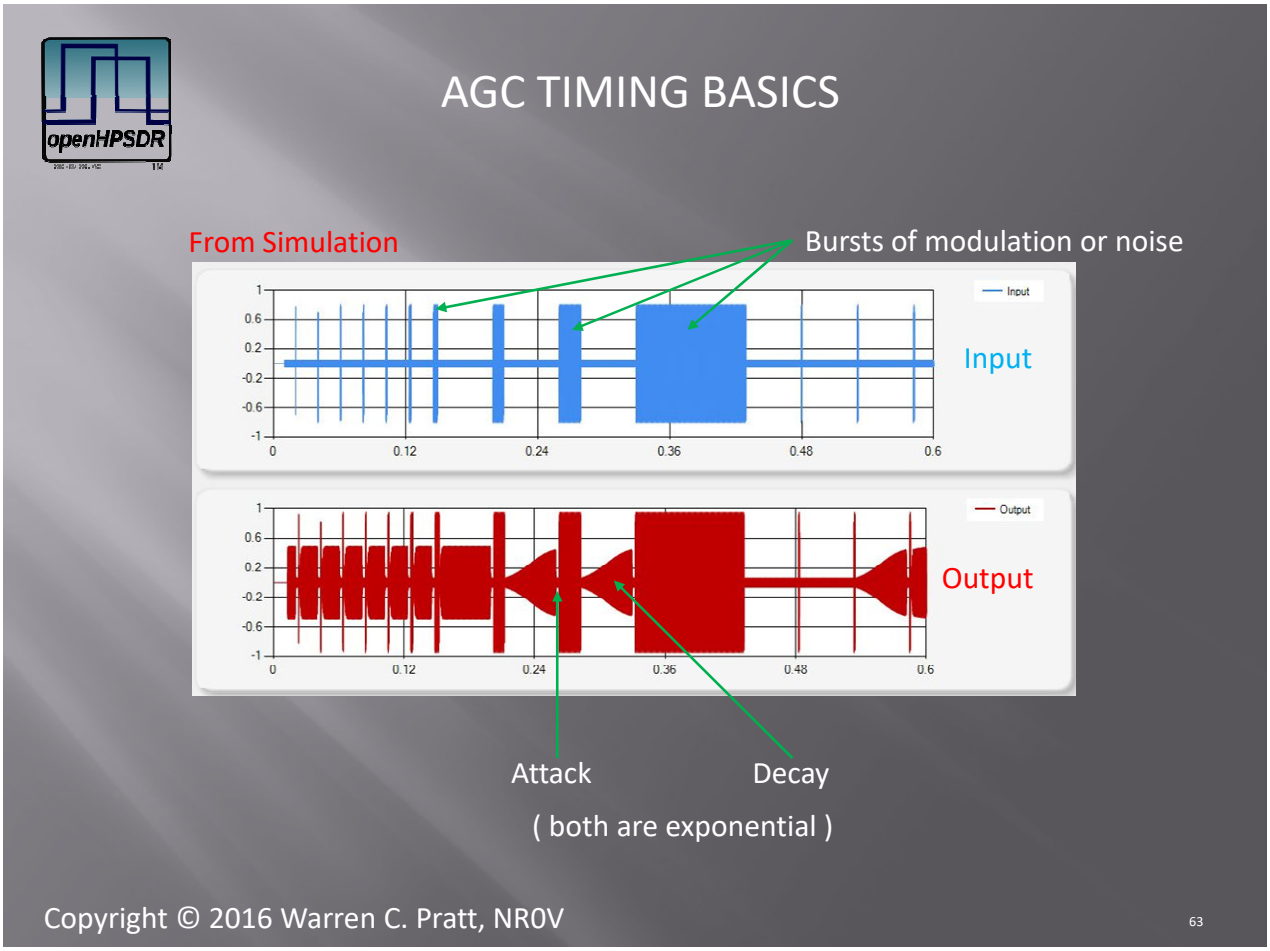


Figure 9: AGC exponential attack and decay waveforms illustrating attack and decay time-constant behavior.

The following figure illustrates additional features of Zero Overshoot, Dual-Decay, and Hang. The Dual-Decay is used in the suppression of certain impulses that cannot be removed with wideband noise blankers. If the AGC judges a short burst to be an impulse, a fast-decay is invoked to avoid the AGC from blocking the reception of weak signals for an extended time following the impulse. Otherwise, the normal decay is used. Hang maintains the gain at a constant fixed level for a specified amount of time after a strong burst of reception.

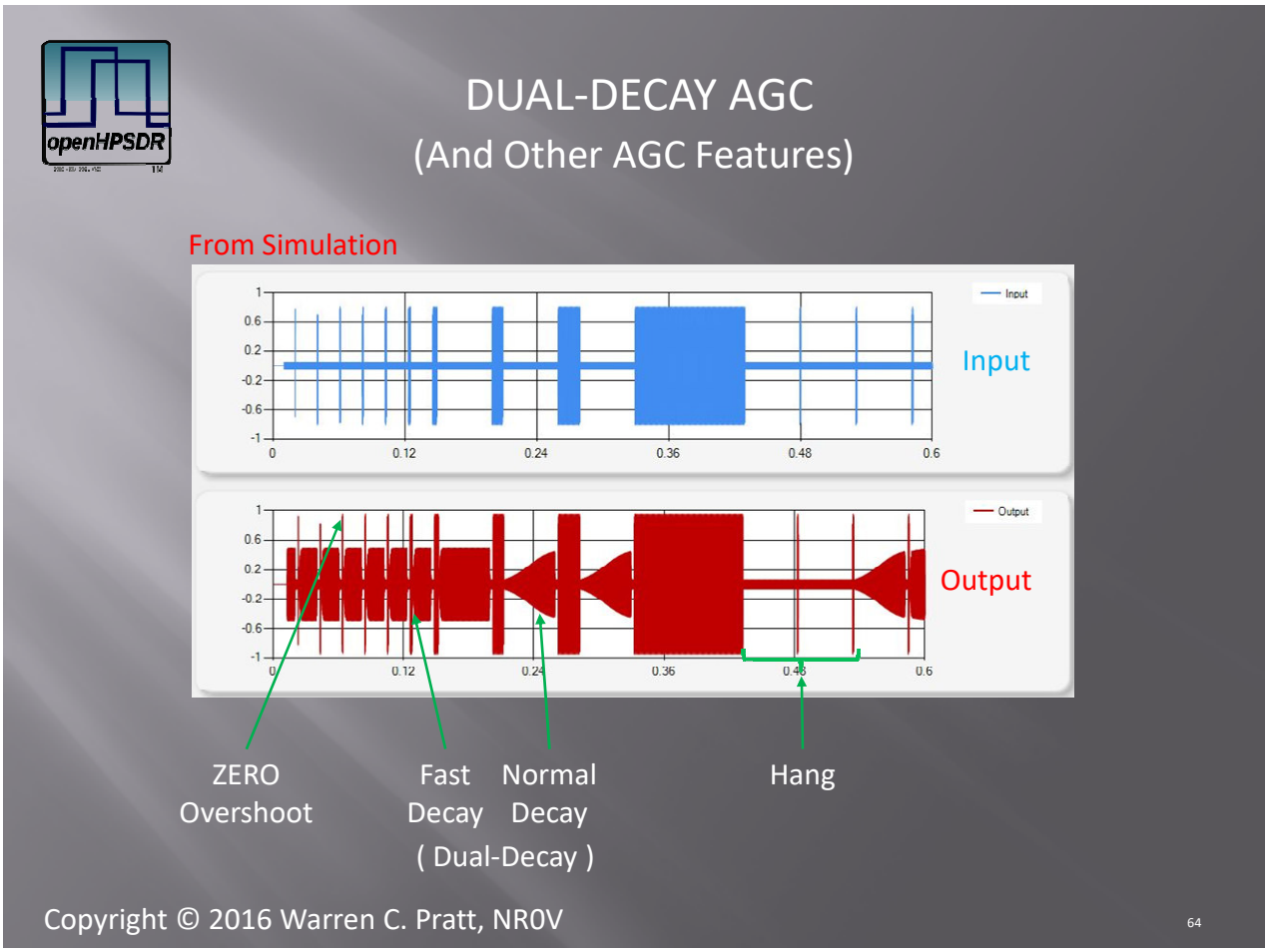


Figure 10: AGC Zero Overshoot, Dual-Decay, and Hang — independent fast and slow decay paths with hold period.

The following figure demonstrates a situation in which the Dual-Decay feature provides a major improvement in reception. The IQ.wav file for this example is available from Leif Asbrink, SM5BSZ, and is posted on his website.

Shown are snapshots of the frequency spectrum taken at three different times. The two strong carriers render wideband noise blankers useless. The key clicks from the two pulsed carriers are the impulses we want to suppress. Strong fan noise on the strong signal implies that we need a relatively long decay as the normal decay for the AGC such that the fan noise is suppressed between words and phrases of the strong station. However, a long AGC time-constant would cause the AGC to block the weak station for an extended time after each impulse. Fortunately, the fast decay is invoked after the impulses and the weak station is not blocked.

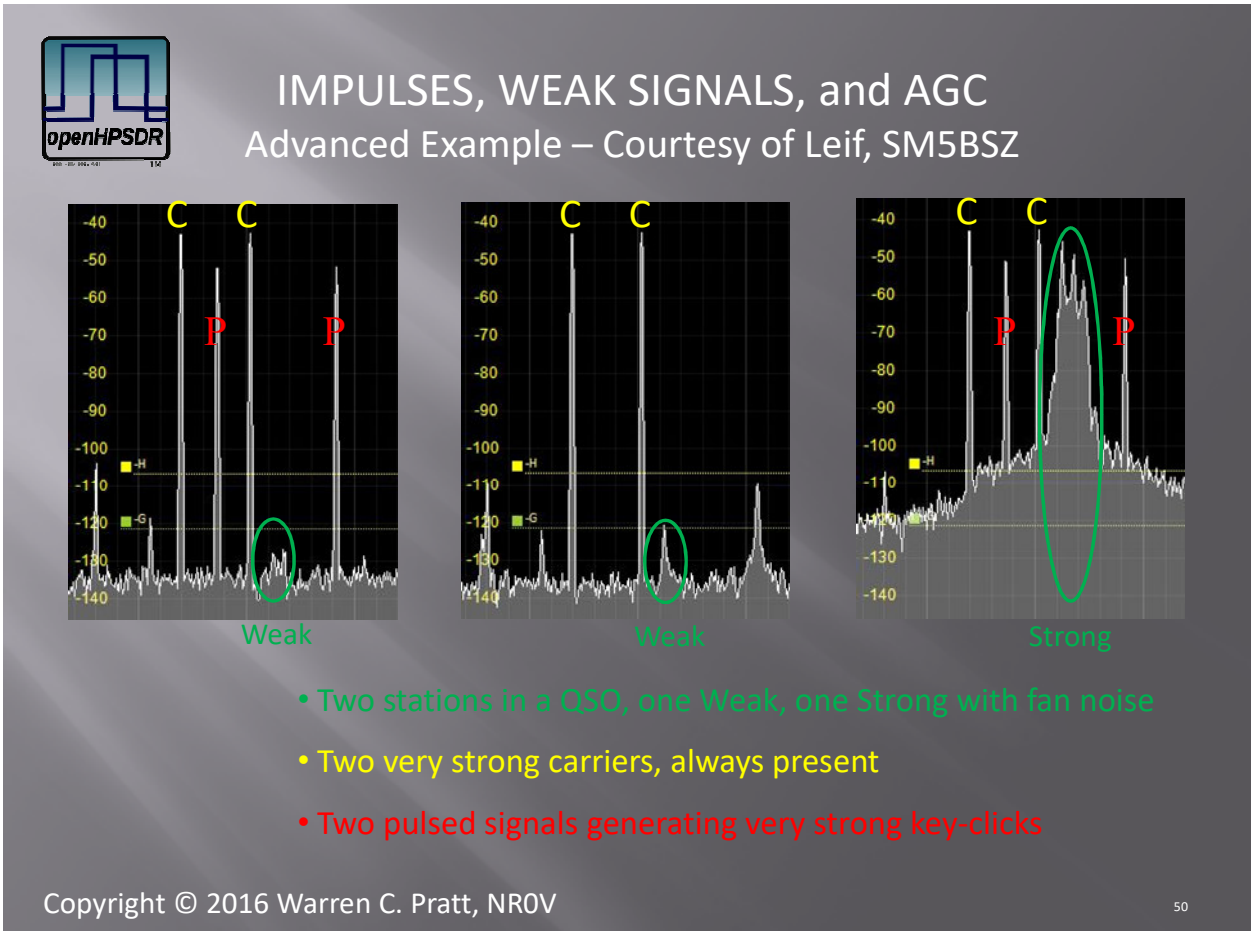


Figure 11: AGC Dual-Decay example showing frequency spectrum snapshots in the presence of two strong carriers.

The following figure illustrates the AGC Gain curve with four different values of slope: 0 dB (no slope), 6 dB, 10 dB, and 20 dB. The plot shows AGC Output level versus AGC Input level, both in dBm. The AGC maximum gain was set to 90 dB.

With no slope, all input levels above -90 dBm have the same output level, the AGC adjusts its gain to produce the same output. Since the maximum gain is set to 90 dB, the output drops (-10 dBm out for -10 dBm in) for input values lower than -90 dBm.

At a slope of 10dB, the output level gradually increases from -10 dBm to 0 dBm for input levels of -100 dBm to 0 dBm.

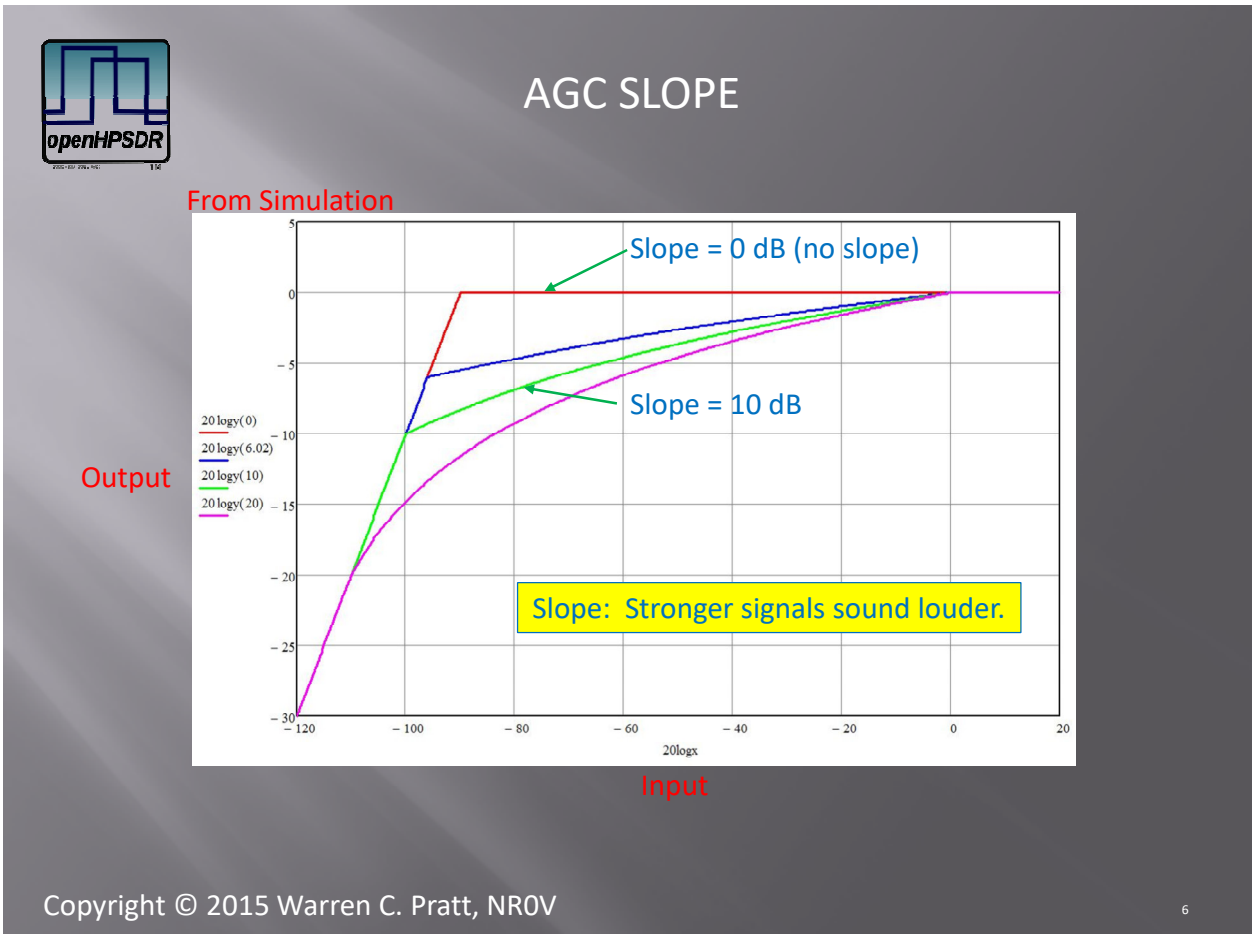


Figure 12: AGC Gain curve with slope values of 0, 6, 10, and 20 dB — output level versus input level.

The following call is used to both (1) turn AGC OFF/ON, and (2) in the event it's ON, set some key parameters:

```
void SetRXAAGCMode (int channel, int mode)
```

channel: number of the receiver channel being accessed

mode: AGC operating mode:

0: AGC_OFF. A fixed gain equal to the AGC fixed gain setting will be applied to the signal.

1: AGC_LONG. Hangtime = 2000 ms. Decay_time_constant = 2000 ms.

2: **AGC_SLOW**. Hangtime = 1000 ms. Decay_time_constant = 500 ms.
3: **AGC_MED**. Hang is turned OFF. Decay_time_constant = 250 ms.
4: **AGC_FAST**. Hang is turned OFF. Decay_time_constant = 50 ms.
5: **AGC_CUSTOM**. AGC is ON; however, no parameters are set by this call. All parameters are set with other calls as needed.
[default = 3]

The following three calls can be used for setting various AGC time-constants or absolute times.

```
void SetRXAAGCAttack (int channel, int attack)
```
channel: number of the receiver channel being accessed
attack: attack time constant (ms). It is recommended to leave this at the default value.Note that "attacks" are along exponential curves and this value is a time-constant as opposed to an absolute amount of time.[default = 1 ms]

```
void SetRXAAGCDecay (int channel, int decay)
```
channel: number of the receiver channel being accessed
decay: decay time constant (ms). Note that "decays" are along exponential curves and this value is a time-constant as opposed to an absolute amount of time.[default = 250 ms]

```
void SetRXAAGCHang (int channel, int hang)
```
channel: number of the receiver channel being accessed
hang: hang-time (ms). Note that this is an absolute amount of time as opposed to a time-constant.[default = 250 ms]

Support is provided for setting the AGC Hang Level either on the bandscope or by use of a simple slider control. When one is changed, the other needs to be adjusted accordingly. Therefore, there are both 'get' and 'set' commands for each type of control. For example, when a 'set' is done on the slider, there is a 'get' that can be used to return the new position required on the bandscope.

```
void GetRXAAGCHangLevel (int channel, double* hanglevel)
```
channel: number of the receiver channel being accessed
hanglevel: pointer to location where the bandscope level (dBm) is to be stored.

```
void SetRXAAGCHangLevel (int channel, double hanglevel)
```
channel: number of the receiver channel being accessed
hanglevel: bandscope level being set as the hang-level (dBm)

```
void GetRXAAGCHangThreshold (int channel, int* hangthreshold)
```
channel: number of the receiver channel being accessed
hangthreshold: pointer to location where the slider setting is to be stored (setting range is 0 to 100)

```
void SetRXAAGCHangThreshold (int channel, int hangthreshold)
```
channel: number of the receiver channel being accessed
hangthreshold: slider setting being set as the hang-level (setting range is 0 to 100)

Support is provided for setting the AGC Maximum-Gain / Threshold both via a slider and by moving a line on the bandscope. When one is changed, the other needs to be adjusted accordingly. Therefore, there are both 'get' and 'set' commands for each type of control. For example, when a 'set' is done on the slider, there is a 'get' that can be used to return the new position required on the bandscope.

On the bandscope, the noise floor moves up and down depending upon the FFT_size and sample_rate of data used to compute the bandscope. Also, it is useful to reference the bandscope line to the noise

floor, typically setting the line a few dB above the noise floor. Therefore, the display FFT_size and display sample_rate parameters must be supplied to achieve the correct bandscope results.

```
void GetRXAAGCThresh (int channel, double* thresh, double size, double rate)
```
channel: number of the receiver channel being accessed
thresh: pointer to location to store the bandscope level (dBm) for AGC Threshold
size: FFT_size being used to compute the bandscope display
rate: sample-rate of data being supplied to compute the bandscope display

```
void SetRXAAGCThresh (int channel, double thresh, double size, double rate)
```
channel: number of the receiver channel being accessed
thresh: the bandscope level (dBm) for AGC Threshold
size: FFT_size being used to compute the bandscope display
rate: sample-rate of data being supplied to compute the bandscope display

The threshold line is computed from the operator's passband. It is not affected by Neural Noise Reduction, and does not move when NNR is turned on or off or the model is changed.

```
void GetRXAAGCTop (int channel, double* max_agc)
```
channel: number of the receiver channel being accessed
max_agc: pointer to location to store the maximum AGC Gain, i.e., the AGC Threshold (range is -20 dB to +120dB)

```
void SetRXAAGCTop (int channel, double max_agc)
```
channel: number of the receiver channel being accessed
max_agc: the maximum AGC Gain, i.e., the AGC Threshold (range is -20 dB to +120dB)[default = +80 dB]

```
void SetRXAAGCSlope (int channel, int slope)
```
channel: number of the receiver channel being accessed
slope: Ten times the desired AGC slope value (dB). The typical range for slope would be 0 to 200, corresponding to 0dB to 20dB.[default = 35, corresponding to 3.5dB]

```
void SetRXAAGCFixed (int channel, double fixed_agc)
```
channel: number of the receiver channel being accessed
fixed_agc: fixed gain to use when the AGC is OFF, i.e., the 'mode' is set to 0. (dB)[default = +60 dB]

5.3.17 Automatic Notch Filter

The WDSP Automatic Notch Filter is used primarily to "notch-out" carriers and similar signals.

The Least-Mean-Squares (LMS) algorithm used here dynamically computes the coefficients of an FIR filter such that the filter passes the carrier and rejects other signals. The carrier is then subtracted from the original signal to produce an output free of the carrier.

The filter coefficients are computed in such a way as to maximize the autocorrelation of the carrier. The delay, Δ, assists in the comparison of two different time periods of the signal.

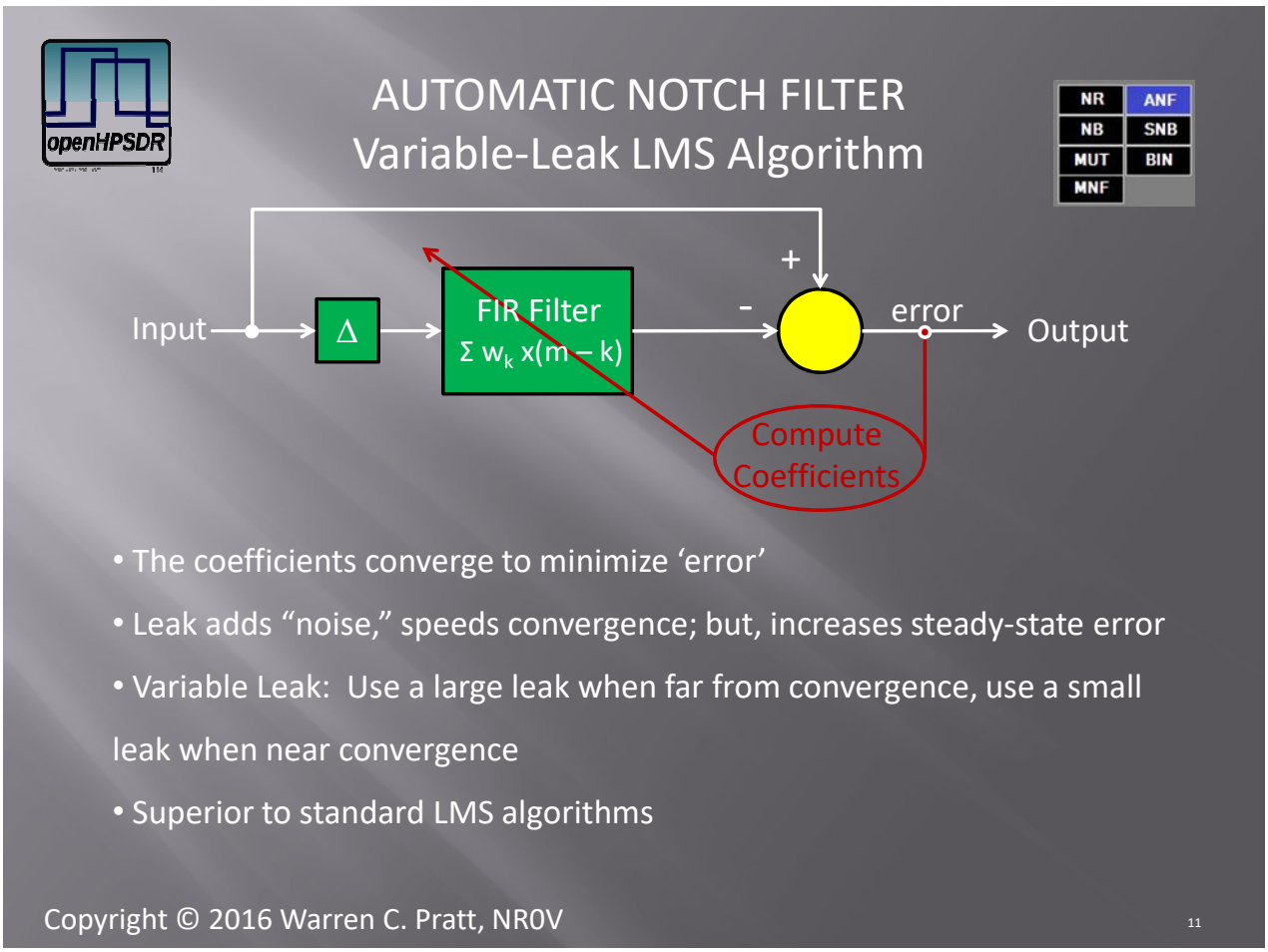


Figure 13: Automatic Notch Filter (ANF) — LMS adaptive filter tracking and removing a carrier from the passband.

The Automatic Notch Filter is turned OFF/ON using the call:

```
void SetRXAANFRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

The number of FIR filter taps is set with the call:

```
void SetRXAANFTaps (int channel, int taps)
```
channel: number of the receiver channel being accessed
taps: number of filter taps (coefficients) to be used[default = 64]

The delay is set using the call:

```
void SetRXAANFDelay (int channel, int delay)
```
channel: number of the receiver channel being accessed
delay: delay, specified in number of samples[default = 16]

```
void SetRXAANFGain (int channel, double gain)
```
channel: number of the receiver channel being accessed
gain: gain value. Typical range is 1.0e-06 to 1.0e-03.[default = 1.0e-04]

> Note that increasing gain allows the algorithm to more effectively lock-onto and notch carriers. HOWEVER, at higher gains the algorithm will begin to generate distortion on speech. I.e., there is a tradeoff between notching effectiveness and speech distortion.

```
void SetRXAANFLeakage (int channel, double leakage)
```
channel: number of the receiver channel being accessed
leakage: leak value. Typical range is 0.001 to 1.000.[default = 0.100]

The following call can be used to set taps, delay, gain, and leakage using only a single call:

```
void SetRXAANFVals (int channel, int taps, int delay, double gain, double
leakage)
```
channel: number of the receiver channel being accessed
taps: number of filter taps (coefficients) to be used[default = 64]
delay: delay, specified in number of samples[default = 16]
gain: gain value. Typical range is 1.0e-06 to 1.0e-03.[default = 1.0e-04]
leakage: leak value. Typical range is 0.001 to 1.000.[default = 0.100]

```
void SetRXAANFPosition (int channel, int position)
```
channel: number of the receiver channel being accessed
position: 0 = execute ANF before AGC; 1 = execute ANF after AGC.

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, this block is not run if it is set to run before the AGC; it does run if set to run after the AGC (§ 5.3.20).

5.3.18 LMS Noise Reduction

Setting your expectations: LMS Noise Reduction algorithms work best when there is already a good signal- to-noise ratio; they can be very effective in providing a quiet background under these circumstances. This is really their purpose. It is unlikely that you will be able to pull an otherwise unreadable signal out of the noise with this type algorithm.

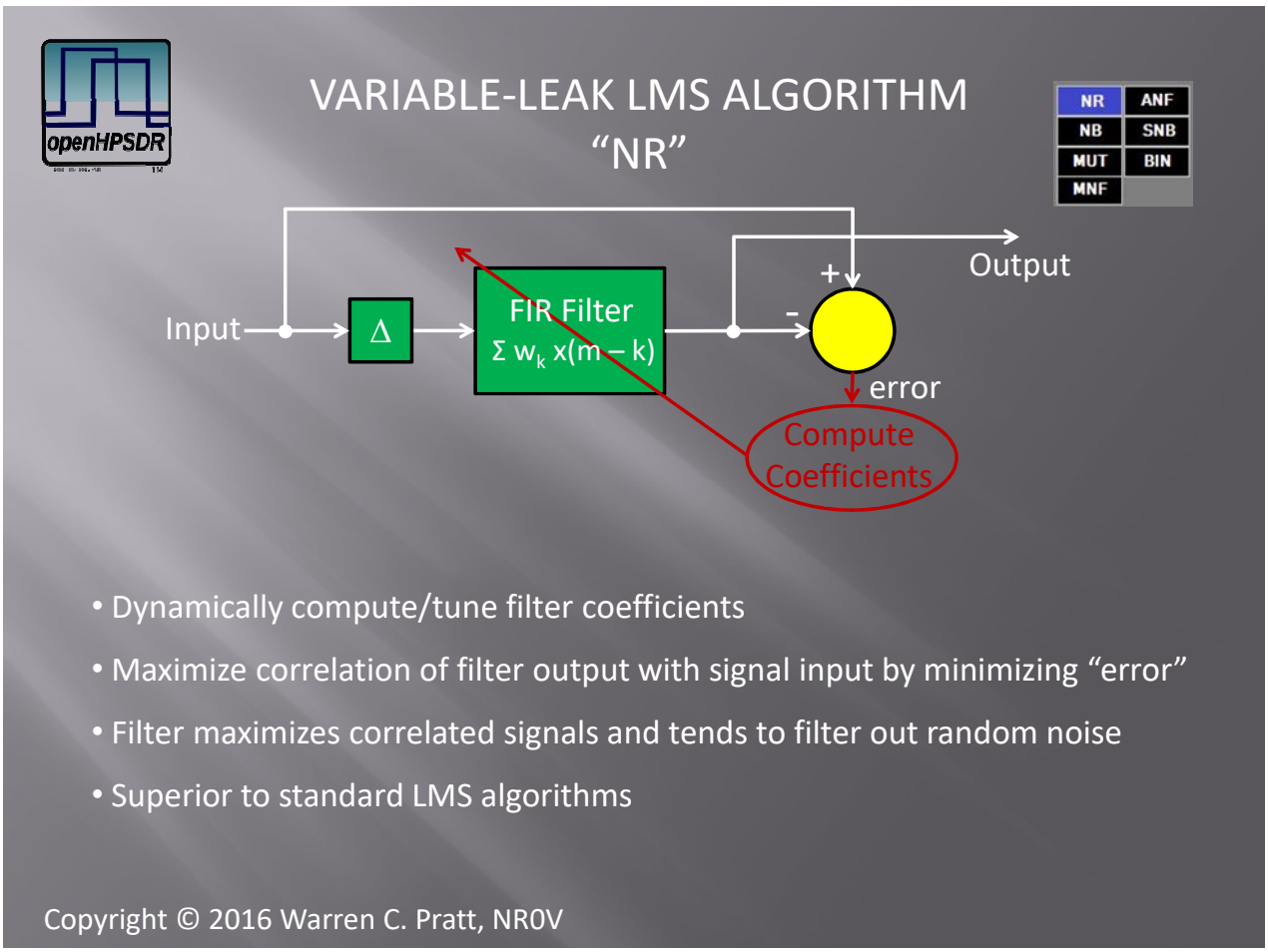


Figure 14: LMS Noise Reduction — adaptive filter structure separating noise-correlated and signal-correlated components.

The LMS Noise Reduction is turned OFF/ON using the call:

```
void SetRXAANRRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

The number of FIR filter taps is set with the call:

```
void SetRXAANRTaps (int channel, int taps)
```
channel: number of the receiver channel being accessed
taps: number of filter taps (coefficients) to be used[default = 64]

The delay is set using the call:

```
void SetRXAANRDelay (int channel, int delay)
```
channel: number of the receiver channel being accessed
delay: delay, specified in number of samples[default = 16]

```
void SetRXAANRGain (int channel, double gain)
```
channel: number of the receiver channel being accessed
gain: gain value. Typical range is 1.0e-06 to 1.0e-03.[default = 1.0e-04]

```
void SetRXAANRLeakage (int channel, double leakage)
```
channel: number of the receiver channel being accessed
leakage: leak value. Typical range is 0.001 to 1.000.[default = 0.100]

The following call can be used to set taps, delay, gain, and leakage using only a single call:

```
void SetRXAANRVals (int channel, int taps, int delay, double gain, double
leakage)
```
channel: number of the receiver channel being accessed
taps: number of filter taps (coefficients) to be used[default = 64]
delay: delay, specified in number of samples[default = 16]
gain: gain value. Typical range is 1.0e-06 to 1.0e-03.[default = 1.0e-04]
leakage: leak value. Typical range is 0.001 to 1.000.[default = 0.100]

```
void SetRXAANRPosition (int channel, int position)
```
channel: number of the receiver channel being accessed
position: 0 = execute Noise Reduction before AGC; 1 = execute Noise Reduction after AGC.

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, this block is not run if it is set to run before the AGC; it does run if set to run after the AGC (§ 5.3.20).

5.3.19 Spectral Noise Reduction

The seminal work for the class of algorithm implemented here for the Spectral Noise Reduction block was published in 1984[1]. At a basic level, an FFT is done to convert to the frequency domain, a gain is applied to each of the frequency bins based upon estimates of the amplitudes of speech and noise within the bins, and an inverse FFT is done to get back to time-domain samples. Statistical models of speech and noise amplitudes are used as "a priori" knowledge to enhance the accuracy of the estimates.

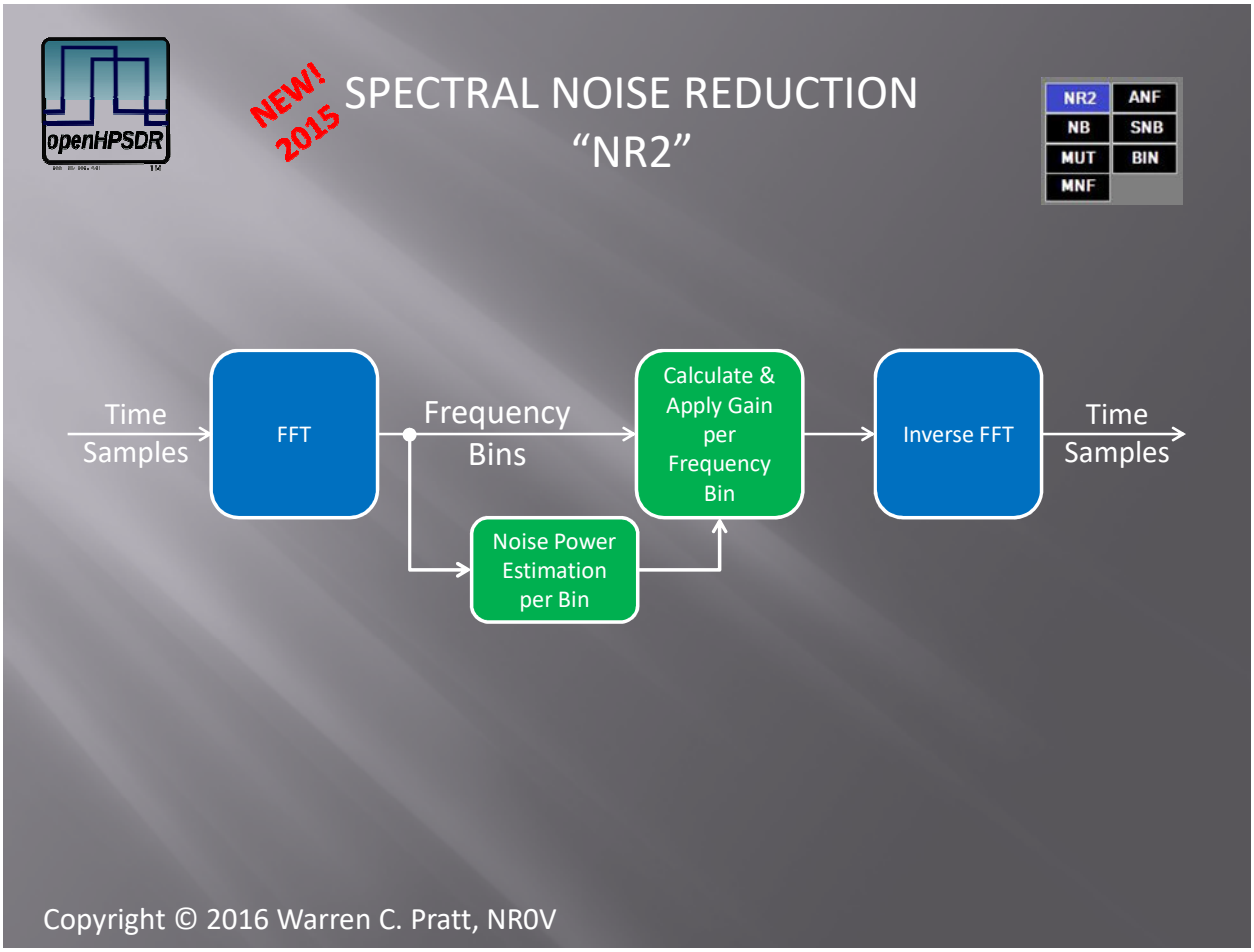


Figure 15: Spectral Noise Reduction — an implementation view.

Over the years, various alternatives and improvements to the speech estimation algorithms and corresponding gain calculations have been developed. Four variants are included in this code and are selectable. Also, noise estimation is an important part of the process and three different noise estimation algorithms have been included.

[1] Yariv Ephraim and David Malah, "Speech Enhancement Using a Minimum Mean-Square Error Short-Time Spectral Amplitude Estimator," 1984.

A Word About Using Noise Reduction

For optimal operation, noise reduction algorithms should be provided with the best possible environment within the radio's processing chain. If using Post-AGC NR, the AGC Threshold should be as far as practical above the noise floor. A Pre-AGC position will generally offer the best signal-to-noise ratio going into the NR, which is beneficial; however, a Post-AGC position may offer better immunity to sudden noise changes. These factors should be taken into consideration anytime any of these noise reduction algorithms are in use.

The Spectral Noise Reduction is turned OFF/ON using the call:

```
void SetRXAEMNRRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

The gain-per-bin calculation method is selected using the following call. The first three choices will produce somewhat similar results with the default Gamma speech distribution being somewhat preferred.

The fourth method, a *Trained* algorithm, is distinct and can produce more dramatic noise reduction; albeit, at the price of producing the human perception of being harsh – more on post-processing for this later. This Trained algorithm was trained to accurately determine, by FFT bin, the signal-to-noise ratio by evaluating multiple "views" of the incoming signal and noise. About seventy-two hours of signal and noise were used for training.

```
void SetRXAEMNRgainMethod (int channel, int method)
```
channel: number of the receiver channel being accessed
method: 0 = Gaussian speech distribution, linear amplitude scale; 1 = Gaussian speech distribution, log amplitude scale; 2 = Gamma speech distribution; 3 = Trained algorithm[default = 2]

The noise-power-estimation (per bin) method is selected using the following call. All three choices will produce somewhat similar results. The OSMS method has been long-proven and is preferred by many. The MMSE method is more responsive to changing noise than OSMS. The *NSTAT* method is the most responsive of all and may be preferred with rapidly changing noise – it is *especially recommended with the "Trained" gain method*.

```
void SetRXAEMNRnpeMethod (int channel, int method)
```
channel: number of the receiver channel being accessed
method: 0 = Optimal Smoothing Minimum Statistics (OSMS); 1 = Minimum Mean-Square Error (MMSE); 2 = Non-stationary Noise Method (NSTAT)[default = 0]

The position in the processing pipeline where this algorithm is applied is selected by:

```
void SetRXAEMNRPosition (int channel, int position)
```
channel: number of the receiver channel being accessed
position: 0 = execute noise reduction before AGC; 1 = execute noise reduction after AGC[default = 0]

While Neural Noise Reduction runs with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU, this block is not run if it is set to run before the AGC; it does run if set to run after the AGC (§ 5.3.20).

Specifically for the "Trained" mode, there are Threshold adjustments that determines at what level and how the training data is invoked. Said differently, the thresholds determine at what SNR the signal will be allowed to pass. The higher the "T1" threshold, the better the signal-to-noise ratio must be for the signal to pass through. Therefore, if you want weaker signals and perhaps a little noise, choose a lower value.

```
void SetRXAEMNRtrainZetaThresh (int channel, double thresh)
```
channel: number of the receiver channel being accessed
thresh: Suggested range is -5.0 to +5.0.[default = -0.5]

In its quest to eliminate all noise, the "Trained" mode can sometimes also eliminate very weak signals that it does not separate from the noise. By lowering the value of an additional threshold, called "T2", those signals may be allowed to pass at the cost of also allowing some additional noise.

```
void SetRXAEMNRtrainT2 (int channel, double t2)
```
channel: number of the receiver channel being accessed
thresh: Suggested range is 0.02 to 0.30.[default = 0.20]

POST-PROCESSING

Following the basic algorithms, as described above, there are a number of post-processing steps that may be applied to improve the human perception of the processed speech.

This first technique, an Artifact-Elimination Post-Filter (AEPF), should ALWAYS be applied with these types of algorithms.

Noise reduction algorithms that operate in the frequency domain and that individually modify FFT frequency bins are well-known for producing artifacts often called "musical tones." This filter is a dynamically-self-adjusting filter to largely remove those tones. There is no reason to turn it off, other than for development test purposes.

```
void SetRXAEMNRaeRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default =1]

The psychoacoustics of audio processed by various NR algorithms can also often be improved by injecting various types of noise and weak-signals and then tapering the frequency response of the signal. Several capabilities are provided to do this.

This following call turns ON/OFF the following set of noise-related post-processing functions. The default is set to zero which directs this set of functions to do absolutely nothing in the event the console controls for them are not implemented.

```
void SetRXAEMNRpost2Run (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default =0]

For psychoacoustic reasons, noise (of a couple types) and signals can be injected into the audio stream coming from the fundamental noise reduction algorithms discussed above. This can have the effect of

allowing very weak signals through and it creates the perception of “softening” the audio sometimes providing more natural-sounding speech.

The level of the noise/signals to be mixed into the audio stream is adjusted by a control:

```
void SetRXAEMNRpost2Nlevel (int channel, double nlevel)
```
channel: number of the receiver channel being accessed
nlevel: range is 0.0 (minimum injection) to 100.0 (maximum injection)[default = 15]

The composition of the noise/signals can also be varied, depending upon personal preference of the operator. At the two extremes are (1) a sample of the noise/signal before the noise reduction algorithm is applied, and (2) white noise with some “speech shaping”. Any mix between the two extremes is possible, i.e., all from the original signal OR all white OR any mix in between. The amplitude of the white noise is also automatically adjusted based upon the strength of the received signal.

```
void SetRXAEMNRpost2Factor (int channel, double factor)
```
channel: number of the receiver channel being accessed
factor: range is 0.0 (sample of original signal/noise) to 100.0 (white noise)[default = 15]

The white noise, if injected, has no signal-value; although, as mentioned, it may serve to “soften” the audio and enhance the human listening experience. That said, it is of no value at all if there is no signal present and may become tiring to the listener. Hence, there is a means to allow it to be present only during speech or to fade-away over the longer-term in the absence of speech.

This control expresses a time-constant in seconds and determines the fade-rate for specifically any white noise that has been injected. At a setting of 0.0, the noise will basically ONLY exist during the syllables of the speech to soften the audio. At a setting of 100.0, specifically the white noise will decline to 37% (one time-constant) of its original value after 100 seconds. Hence, during speech, it remains present.

```
void SetRXAEMNRpost2Rate (int channel, double tc)
```
channel: number of the receiver channel being accessed
tc: range is 0.0 (fastest noise decline) to >=100.0 (slowest noise decline)[default = 5.0, operator may like it or find it very annoying]

Finally, to remove residual artifacts and soften the effects of noise, a “tapering” of the frequency response of the audio has been found to be desirable. This is not a “bandpass” in the usual sense in that the frequency response is “rolled-off” along a specific response curve. The upper-frequency cut-off of this curve can be adjusted.

```
void SetRXAEMNRpost2Taper (int channel, int taper)
```
channel: number of the receiver channel being accessed
taper: range is 0 (EVERYTHING is cut off) to 100 (maximum audio bandwidth, normally 24 KHz)[default =12, good for normal SSB, use higher for ESSB or wideband AM]

ADDITIONAL OPTIONS & REQUIREMENTS:

- At initialization, the Spectral Noise Reduction algorithm looks for the file "calculus" located in the same directory as the WDSP executable. "calculus" is a large data file containing

information required for the best (also the default) gain-computation algorithm choice. This data can be read from a file to enable future experimentation with different and/or improved data.

If the file is not found, a substitute set of data that is stored in the executable will be used. This data is the same as that of the file as of the writing of version 1.27 of this document.

- Specifically for the “Trained” gain mode, the algorithm looks for the file “zetaHat.bin”. This file contains the essential data from the training of the algorithm. This file would normally be copied into the same directory as the WDSP executable.

  If the file is not found, a substitute set of data that is stored in the executable will be used. This data is the same as that of the file as of the writing of version 1.27 of this document.

5.3.20 Neural Noise Reduction

The Spectral Noise Reduction block described above estimates the noise power in each frequency bin and computes a gain from statistical models of how speech and noise behave. Neural Noise Reduction takes a different approach. Rather than being told what speech and noise look like, a neural network was trained on real signals until it learned to tell them apart.

The distinction matters more than it may appear. A statistical model must assume something about the noise — typically that it is slowly varying and roughly stationary — and HF noise frequently is neither. A trained network makes no such assumption. It has simply seen a great deal of what actually arrives at a receiver on the amateur bands and has learned what part of it is somebody talking.

Four trained networks are supplied: two for voice and two for CW. The CW models were added after the voice models and differ from them in ways that matter to a console. These are described under "How the CW Models Were Trained" and "Where NNR Runs in the Receiver" below. In brief, a console offers the operator a choice of Standard or Premium, and uses the voice or the CW version according to the operating mode.

How the Voice Models Were Trained

Two aspects of the training are worth describing, because they are what adapt this algorithm to amateur radio rather than to speech enhancement in general.

First, the noise is real. Over a hundred recordings were collected off-air from receivers around the world and from the author's own station, covering atmospheric noise, static crashes, power-line and switching-supply hash, carriers, and the ordinary congestion of a busy band. None of it is synthetic. Eighty-five of those recordings were specifically chosen for training.

Second, the speech was passed through an actual SSB transmit chain before being mixed with that noise — filtered, compressed, and processed exactly as a transmitting station would process it. The network therefore learned to recognize speech as it arrives at a receiver, rather than speech as it sounds in a quiet room. A model trained on clean studio recordings has, in effect, never heard the signal it will be asked to clean up.

The result is a network that examines the incoming signal and, for every frequency bin of every frame, computes how much of what it sees is speech. It differs from a simple gain in two ways: the coefficients are complex, so phase is corrected as well as amplitude, and each bin is computed from several successive frames rather than from the current one alone. This is known as deep filtering, and it is what allows the algorithm to recover speech that is partly buried rather than merely to turn down the bins that look noisy.

Comparison With Other Current Solutions

Just before release of the voice models, codes from DeepFilterNet3 (official released binary) and RNNoise (released demo code) were downloaded and compared with NNR both in objective numerical measures and listening tests with off-air HF signals. NNR compares very favorably in both numbers and

sound – this can be attributed to the combination of a strong architecture and training on material specifically geared toward amateur radio.

To illustrate an architectural contrast, RNNoise computes one gain for each of twenty-two frequency bands and was trained on the noise of offices, streets, and telephone calls. Neural Noise Reduction computes complex coefficients for each of 257 bins across several frames, and every hour of noise it learned from came off the HF bands.

One specific difference is worth mentioning, because operators who have tried RNNoise on the air on a noisy band will have noticed it. RNNoise pairs its neural network with a pitch filter, which must first track the pitch of the voice it is hearing; until that estimate settles, the audio can be audibly distorted, and a new speaker starts the process again. Neural Noise Reduction has no pitch tracking and no such settling period. It treats each frame on the evidence in front of it, so a station that comes back after a long call is handled identically to one already in progress.

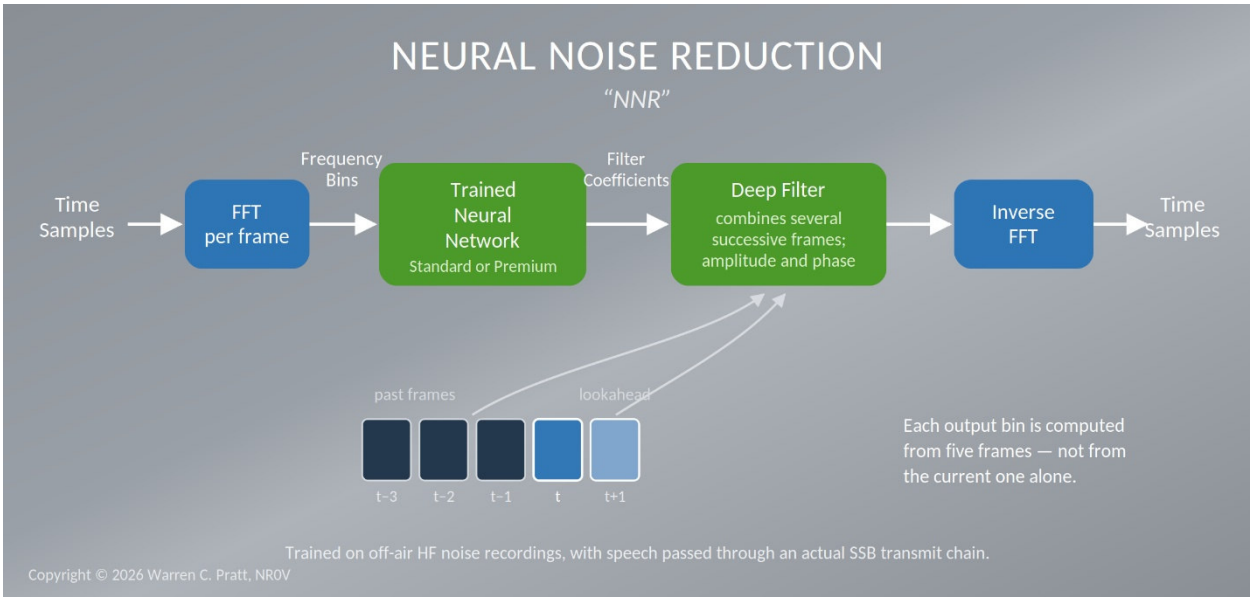


Figure 16: Neural Noise Reduction — an implementation view.

How the CW Models Were Trained

CW is not simply narrow voice, and the voice models are not suitable for it: CW was among the interference they were trained to remove. The two CW models use the same network design, but were trained separately on material prepared for the purpose.

The CW itself is generated rather than recorded: keyed tones over a range of speeds, pitches and rise times, often several signals at once, and some examples with no signal at all so that silence is sometimes the right answer. The noise is the same off-air collection used for the voice models, screened to remove

every segment containing CW, carriers or digital signals, so that the models were never taught to remove what they are meant to keep.

The most important choice was the bandwidth the models see. An operator usually listens to CW through a filter of 250 Hz or less, but a network makes much better decisions when it sees a wide, fixed passband with the signal somewhere inside it. Most of what it sees is then plainly noise, which gives it a reference. The CW models were therefore trained on a fixed passband of 200 to 3000 Hz, with signal pitches between 500 and 1050 Hz. This is the model's "training band." It is recorded in the model file, and the receiver arranges for NNR to be given that band while the operator's own filter still decides what is heard (see "Where NNR Runs in the Receiver" below).

The exact pitch is not critical. In testing the trained models, pitches outside the 500 to 1050 Hz training range were neither rejected nor attenuated. Unwanted pitches are best removed by the operator's filter.

The Models

The quality of noise reduction delivered by NNR does not come without a cost – that cost is CPU cycles. The code has been instrumented, hot-spots examined and some re-written to try to minimize this; however, there is simply a lot of computation to do. Some incremental reduction in CPU requirements may be possible in the future.

Two trained voice networks are supplied with this release. They share the same architecture, the same training corpus and the same latency; they differ in size, and therefore in what they cost to run and how well they perform.

The Standard voice model is the default. The Premium model is larger, measurably better, and costs about two to three times as much processor time. All four models, voice and CW, are compiled into the library and all are loaded when a receiver is created, so switching between them is immediate.

	Standard	Premium
Model slot	0 (default)	1
Parameters [1]	261,834	584,746
Deep filter	5 taps, 1 lookahead	5 taps, 1 lookahead
Added delay	51.17 ms	51.17 ms
Processor [2]	~ 10%	~ 32%
Improvement [3]	+4.3 dB	+4.9 dB
Override file [4]	wdsp_nnr_0.bin	wdsp_nnr_1.bin

Table 1: The two voice models supplied with this release.

[1] After the training-time normalization layers are folded into the convolutions they precede, which is how the model is stored.

[2] Share of one processor hardware thread needed to keep up with real time, measured in an optimized build on an AMD 9950X3D desktop processor. It scales with clock speed and with what vector instructions the compiler was allowed to use, so treat it as an indication of the ratio between the two models rather than as a figure for any particular machine. A console application targeting modest hardware should select the Standard model or allow the operator to try both.

[3] Mean improvement in signal-to-distortion ratio over the unprocessed signal, measured on eight recordings held out of training entirely, across signal-to-noise ratios from −10 to +15 dB. It is a useful summary, but a single number of this kind does not capture everything an operator hears. In listening tests the models are very similar; however, there are several subtle differences, especially when there are very poor signal-to-noise ratios.

[4] Optional. See "Additional Options & Requirements" below.

Two CW models are also supplied (Table 2). Here the Standard and Premium models are almost the same size. The difference is that the Premium CW model examines the signal twice as often, 125 times a second instead of 62.5, which gives crisper keying edges and a steadier tone on long elements. It costs a little more than twice the processor time of the Standard CW model, and has slightly less delay.

	Standard	Premium
Model slot	2	3
Parameters [1]	266,328	274,032
Deep filter	12 taps, 1 lookahead	24 taps, 1 lookahead
Analysis rate	62.5 frames / s	125 frames / s
Added delay [5]	51.17 ms	43.17 ms
Processor [6]	~ 11%	~ 25%
Training band	200 – 3000 Hz	200 – 3000 Hz
Override file [4]	wdsp_nnr_2.bin	wdsp_nnr_3.bin

Table 2: The two CW models supplied with this release.

[5] NNR's own delay, as in Table 1. For the delay of the receive chain as a whole, see "Latency" below.

[6] Estimated from the amount of computation relative to the Standard voice model; not yet measured. Otherwise as note [2]. The Premium CW model is unlikely to keep up on slower processors such as the Raspberry Pi 4.

What To Expect

In listening comparisons against the Spectral Noise Reduction algorithms, Neural Noise Reduction does not necessarily remove more noise. What it does is remove noise without leaving a signature. There is none of the residual character — the faint musical quality, or the sense of the audio having been processed — that frequency-domain algorithms are prone to, and the response to transients such as static crashes is noticeably better. Strong signals pass through with no audible degradation.

It also requires no post-processing. The Spectral Noise Reduction block offers a set of controls for mixing residual signal and white noise back into the output, because completely silent gaps between syllables are unpleasant to listen to. Neural Noise Reduction achieves the same end with a single control, described below, that simply leaves some of the genuine received noise in place. There is nothing to inject, and nothing to configure.

With a voice model, the block is placed immediately after Spectral Noise Reduction in the receive chain and operates post-AGC. The two are not intended to be used together — one or the other should be enabled, not both. Because a voice model runs after the AGC, the guidance given above under "A Word About Using Noise Reduction" applies equally here: the AGC threshold should be set as far above the noise floor as is practical, so that the algorithm is presented with the best signal it can be given.

Its weakness is the same as that of every noise reduction algorithm: when the signal-to-noise ratio is very poor, speech and noise are not reliably separable, and anything the algorithm does is a guess. Under those conditions the operator may find a weak signal more intelligible with noise reduction turned off, or with the mask floor raised. That is not a defect to be worked around — it is the correct response, and the reason the floor control is exposed.

With the CW models, the noise between elements is reduced by about 15 to 20 dB, depending on the mask floor, judged as an operator judges it: with the CW held at the same level. This holds across a wide range of signal-to-noise ratios, and elements keep their keying edges. The Premium CW model keeps the edges crisper and the tone of long elements steadier. The same caution applies to very weak signals: at very low signal-to-noise ratios a weak element may be reduced along with the noise, and raising the mask floor returns some of it.

Where NNR Runs in the Receiver

Where the block runs in the receive chain depends on the model and the mode, and the library arranges it automatically; a console does not set it. The details are given here so that a console developer can explain the receiver's behavior to operators, and because the console should set the passband in the recommended way (using the recommended call) for the CW models to work as intended.

With a voice model, NNR runs after the AGC, on the operator's passband, as described above.

In contrast, a CW model must be given its 200 to 3000 Hz training band, which is far wider than an operator's CW filter. When the model includes a training band specification, the receiver takes a second copy of the signal ahead of the Notched Bandpass Filter, filters it to the training band on the correct sideband, and gives that to NNR. The operator's passband is then applied to NNR's output by the second Bandpass Filter (§ 5.3.21), before the signal reaches the AGC. Figure 17 shows both arrangements. As a result:

- What is heard is set by the operator's passband, just as without NNR, and the S-meter reads that same passband.
- The AGC acts on what is heard rather than on the wider training band, so a strong signal elsewhere in the 200 to 3000 Hz band cannot make the AGC reduce the gain on a weak signal in the passband.
- Because NNR now precedes the AGC, the advice given for post-AGC noise reduction, to set the AGC threshold well above the noise floor, does not apply to the CW models. The AGC threshold line remains correct, and does not move when NNR is turned on or off or the model is changed.

While NNR runs with a CW model in these modes, the following have no effect on what is heard: the fixed notches of the Notched Bandpass Filter, the Spectral Noise Blanker, the Equalizer, and the Automatic Notch Filter, LMS Noise Reduction and Spectral Noise Reduction when they are set to run before the AGC. Apart from the notches, these blocks are not computed at all, which saves processor time. Their settings are kept, and they resume when NNR is turned off or a voice model is selected. A console may wish to disable these controls while a CW model is running. The same three noise reduction

blocks set to run after the AGC still operate, on NNR's output, but are not recommended in combination with NNR.

In AM, SAM, DSB, FM, FM broadcast, SPEC and DRM there is no training-band path, because the second copy of the signal would bypass the demodulator. A CW model selected in one of these modes runs after the AGC on the demodulated audio, as a voice model does. That combination has not been evaluated and is not recommended.

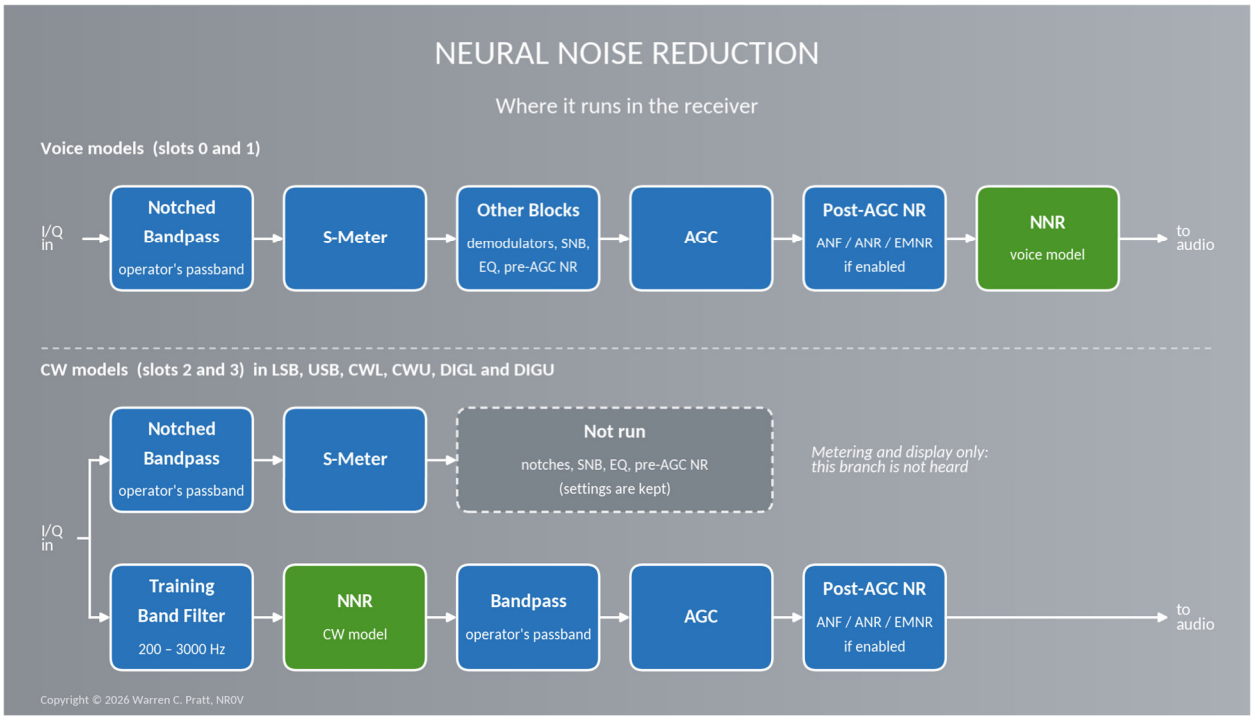


Figure 17: Where Neural Noise Reduction runs — voice models (top) and CW models (bottom).

A console needs to do one thing: make sure the operator's passband reaches the second Bandpass Filter as well as the Notched Bandpass Filter. The collective call RXASetPassband (§ 5.3.27) does this, and with a CW model it is the second filter that decides what is heard. A console that sets only the Notched Bandpass Filter, with the deprecated RXANBPSetFreqs, will narrow the S-meter but not the audio. No other call is needed to set up the path: SetRXAMode, SetRXANNRModel and SetRXANNRRun each re-arrange the path as required, in any order; for example, a change from CWU to CWL moves the training band to the other sideband automatically.

Latency

With a voice model, Neural Noise Reduction adds 51.17 ms of delay to the receive chain. Both voice models have the same latency: the Premium model is larger but does not look any further ahead, so it costs processor time rather than delay. Switching between the voice models therefore does not change the timing of the receive chain.

The figure includes one frame of lookahead, which allows the algorithm to see slightly into the future of the signal it is processing and materially improves its handling of syllable onsets and impulsive noise.

It is worth noting that this is lower than the Spectral Noise Reduction block, which adds 64.00 ms. Neural Noise Reduction operates at a 16 kHz internal sample rate with a 512-point transform, so its analysis window is considerably shorter, and that saving more than pays for the resampling filters and the lookahead frame.

With a CW model, NNR itself adds 51.17 ms with the Standard CW model and 43.17 ms with the Premium: the Premium analyses the signal twice as often, and part of the delay is proportional to the interval between analyses. The filters around NNR change too, but they hardly change the total. The second Bandpass Filter that restores the operator's passband takes the place of the Notched Bandpass Filter in the audio path, so its delay replaces that filter's rather than adding to it. The two have the same length and phase by default, and when set with RXASetNC and RXASetMP (§ 5.3.27). The filter that forms the training band is always minimum phase and adds little. The timing of the receive chain is therefore much the same with the Standard CW model as with a voice model, and about 8 ms shorter with the Premium CW model.

Controls

Neural Noise Reduction is turned OFF/ON using the call:

```
void SetRXANNRRun (int channel, int setit)
```

channel: number of the receiver channel being accessed
setit: 0 = OFF; 1 = ON[default = 0]

The single adjustment provided to the operator is the mask floor. This sets a limit on how far the algorithm is permitted to attenuate any frequency bin, and therefore how much of the received noise is allowed through.

At the most negative settings the algorithm is given a free hand and the result is very quiet. As the value is raised, progressively more of the original noise remains. What comes through is the genuine received noise, not a synthetic substitute, and many operators prefer a little of it: it gives a sense of the band, it makes weak signals easier to follow, and it avoids the disconcerting silence of a fully suppressed channel. Raising the floor is also the appropriate response to a very weak signal, since it returns some of the decision about what is speech to the listener.

```
void SetRXANNRMaskFloor (int channel, double floor_db)
```

channel: number of the receiver channel being accessed
floor_db: suggested range is -10.0 (most noise passed) to -50.0 (maximum suppression), in dB[default = -25.0]

A console may switch between the supplied models at any time, including while audio is running. All are already loaded, so the change takes effect within a frame and there is no pause. The newly selected model starts with its internal state cleared, which is inaudible in practice. A switch between a voice model and a CW model also re-arranges the receive path, as described above.

```
int SetRXANNRModel (int channel, int slot)
```
channel: number of the receiver channel being accessed
slot: 0 = Voice Standard; 1 = Voice Premium; 2 = CW Standard; 3 = CW Premium [default = 0]

The return value is the slot actually in use afterwards. It will differ from the requested slot if that slot holds no model — which is how a console can discover what a given build of the library actually contains, rather than assuming. A control offered to the operator can be enabled or hidden on that basis. In the same way, a console can confirm that slots 2 and 3 hold models before offering them.

```
int GetRXANNRModel (int channel)
```
channel: number of the receiver channel being accessed

Returns the slot currently in use.

It is recommended that a console offer the operator only two choices, Standard and Premium, and choose the slot from the operating mode: slots 2 and 3 in CWL and CWU, and slots 0 and 1 in the voice modes. Whether DIGL and DIGU use the voice or the CW models is the console's choice. When the mode changes, the console calls SetRXANNRModel with the slot for the new mode; whether it does so before or after SetRXAMode does not matter.

```
void SetRXANNRPosition (int channel, int position)   — DEPRECATED
```
channel: number of the receiver channel being accessed
position: ignored

The position of the block is set automatically from the model: after the AGC for the voice models, and before it for the CW models in LSB, USB, CWL, CWU, DIGL and DIGU. This call remains so that existing consoles continue to build and run, but it has no effect.

Additional Options & Requirements

All four trained networks are compiled into the library, so nothing needs to be distributed alongside the executable and the algorithm works as supplied.

At initialization the library looks in the current working directory (normally the directory containing the executables) for a file named after each slot — wdsp_nnr_0.bin through wdsp_nnr_3.bin for slots 0 to 3. If such a file is present it is used in preference to the copy compiled in for that slot; if it is absent, which is the normal case, the built-in copy is used.

This is how an updated or experimental model reaches a tester: the file is dropped into the directory with the executables and nothing else needs to change. Each slot falls back only to its own built-in model, never to another slot's, so a slot whose file is missing and which has no model compiled into it reports itself empty, and switching to it is refused.

A model file also describes itself: how the model divides the signal into frames and, for the CW models, its training band. An updated or experimental model can therefore change these without any change to the library or the console. The one requirement is that all models in a receiver use the same internal sample rate; all of those supplied use 16 kHz. A model whose rate differs from that of the lowest-numbered slot holding a model is refused, and its slot then reports itself empty.

The default file names are usually sufficient. A different path may be given if required, but it must be set before the receiver channel is created — models are loaded when the channel is created, and reloaded only if its DSP sample rate or buffer size changes.

```
void SetNNRModelPathSlot (int slot, const char* path)
```

slot: model slot to set, 0 to 3
path: path to the model file; an empty string leaves the slot to its built-in model

5.3.21 Bandpass Filter

For certain modes and features, a second bandpass filter is needed. It is automatically activated when required. Response curves are the same as the Notched Bandpass Filter discussed above.

It runs in AM and SAM, and whenever the Spectral Noise Blanker, the Automatic Notch Filter, LMS Noise Reduction, Spectral Noise Reduction, or Neural Noise Reduction with a CW model in LSB, USB, CWL, CWU, DIGL or DIGU is in use. Its position is also set automatically. With a CW model in those modes it is always ahead of the AGC. Otherwise it is ahead of the AGC unless one or more of the Automatic Notch Filter, LMS Noise Reduction and Spectral Noise Reduction are running after the AGC and none of them before it. With a CW model this filter sets the bandwidth the operator hears (§ 5.3.20), so its passband must be set, most simply with RXASetPassband (§ 5.3.27).

```
void SetRXABandpassFreqs (int channel, double f_low, double f_high)  – DEPRECATED
```

channel: number of the receiver channel being accessed
f_low: lower of the two frequencies (Hertz). May be negative or positive.[default = -4150.0]
f_high: higher of the two frequencies (Hertz). May be negative or positive.[default = -150.0]

The window function used in designing the filter can be selected with the following call. See the discussion (preceding) on the Notched Bandpass Filter for more information on this selection.

```
void SetRXABandpassWindow (int channel, int wintype)
```

channel: number of the receiver channel being accessed
wintype: 0 for 4-term Blackman-Harris window; 1 for 7-term Blackman-Harris window.[default = 1]

```
void SetRXABandpassNC (int channel, int nc)  – DEPRECATED
```

channel: number of the receiver channel being accessed
nc: length of the desired filter impulse response. This must be a power of two AND must be greater than or equal to the DSP Buffer Size in use. [default = 2048]

```
void SetRXABandpassMP (int channel, int mp)  – DEPRECATED
```

channel: number of the receiver channel being accessed
mp: 0 for Linear Phase; 1 for Low Latency (Minimum Phase)[default = 0]

5.3.22 Scope/Phase Display Sender

The console can call for buffers of processed samples to use for things like an oscilloscope or phase display. Two calls are provided for this purpose. One returns only the 'I' samples while the other returns interleaved I/Q samples.

```
void RXAGetaSipF (int channel, float* out, int size)
```

channel: number of the receiver channel being accessed
out: pointer to buffer into which the I-only samples (float format) are to be placed
size: number of I-only samples to retrieve. 'size' must be <= 4096 per an internal setting.

```
void RXAGetaSipF1 (int channel, float* out, int size)
```

channel: number of the receiver channel being accessed
out: pointer to buffer into which the interleaved I/Q samples (float format) are to be placed
size: number of I/Q samples to retrieve. 'size' must be <= 4096 per an internal setting.

5.3.23 AM Carrier Block

The AM carrier, a DC value, is left in the signal through the AGC. This is desirable such that the typical "AGC-quieting" of an AM receiver will be preserved. However, there are situations where it is desirable to remove this carrier before the WDSP audio output. One example would be when an AM signal is being recorded with the intention of playing it back through the transmitter. In that case, if the carrier were left in the signal, the carrier would be recorded also and the carrier level would be too high on playback through the transmitter since the AM transmitter would add carrier again.

This is a simple filter which removes this carrier, i.e., DC component.

The AM Carrier Block is turned OFF/ON using the call:

```
void SetRXACBLRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

5.3.24 CW Peaking Filter

The CW Peaking Filter is used to peak a single frequency, the frequency of a desired CW signal. Gain, bandwidth, and tune frequency are all selectable. There are four different filter types available:

(1) Bi-Quad: A four-stage bi-quad IIR filter. This was the original APF filter in WDSP and has low-latency. Examples (from measurements) of the frequency response are shown in the figure below.

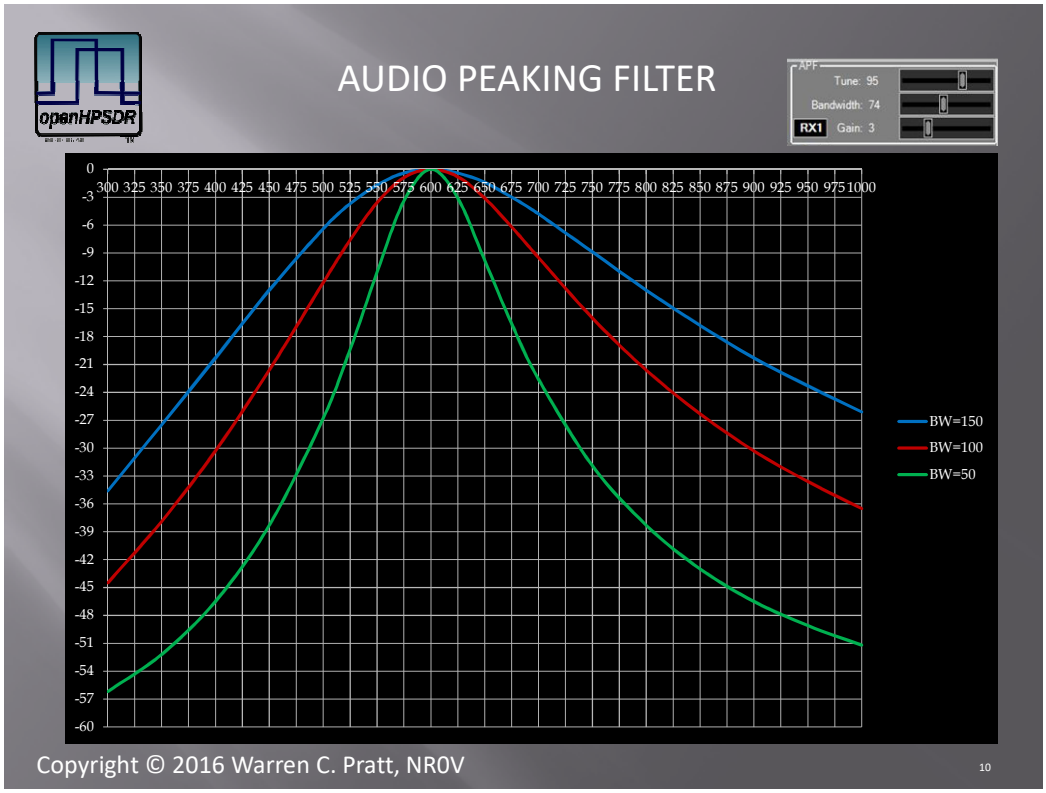


Figure 18: CW Peaking Filter — Bi-Quad (APF) frequency response centered at the CW signal frequency.

(2) Double-Pole: This filter is modelled from a two-pole low-pass filter with very high Q. The frequency response of such a filter has a sharp peak at its cut-off frequency. The CW signal is tuned to be within this peak. Listening tests have shown it to be very desirable in terms or response and listening comfort. Shown below is the response of a Double-Pole filter tuned to 400Hz and set for a 20Hz -6dB bandwidth.

This filter is implemented as a FIR minimum phase filter with a variable number of coefficients, the number being decided based upon the selected bandwidth – smaller bandwidth implies more coefficients.

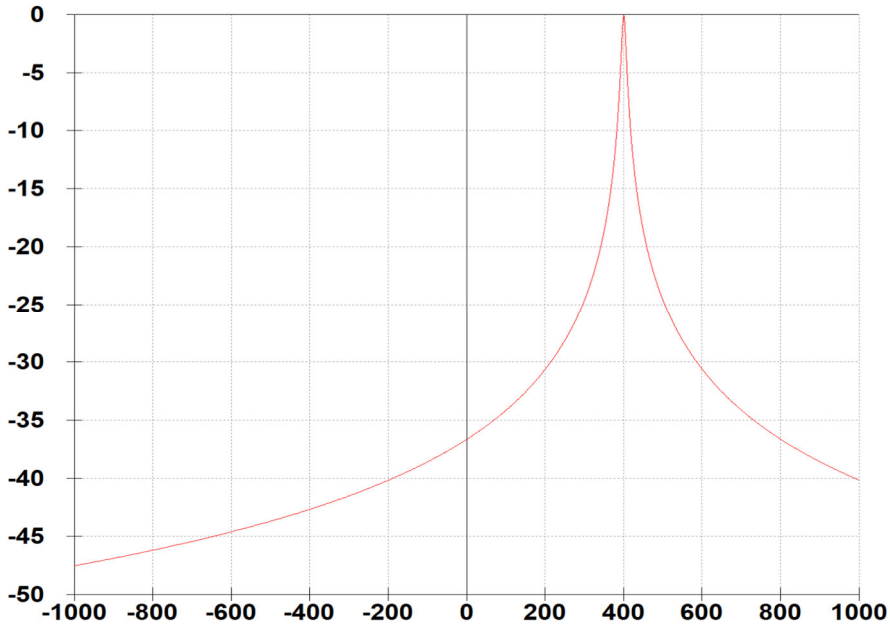


Figure 19: CW Peaking Filter — Double-Pole frequency response showing the sharper resonant peak.

(3) Gaussian: Gaussian filters have the unique property that they have zero ringing. They also have a desirable shape-factor for selection of a single CW signal. Therefore, this option is included. The frequency response is, of course, Gaussian in shape. Shown here is the response of a Gaussian filter tuned to 400Hz and with a -6dB bandwidth of 10Hz.

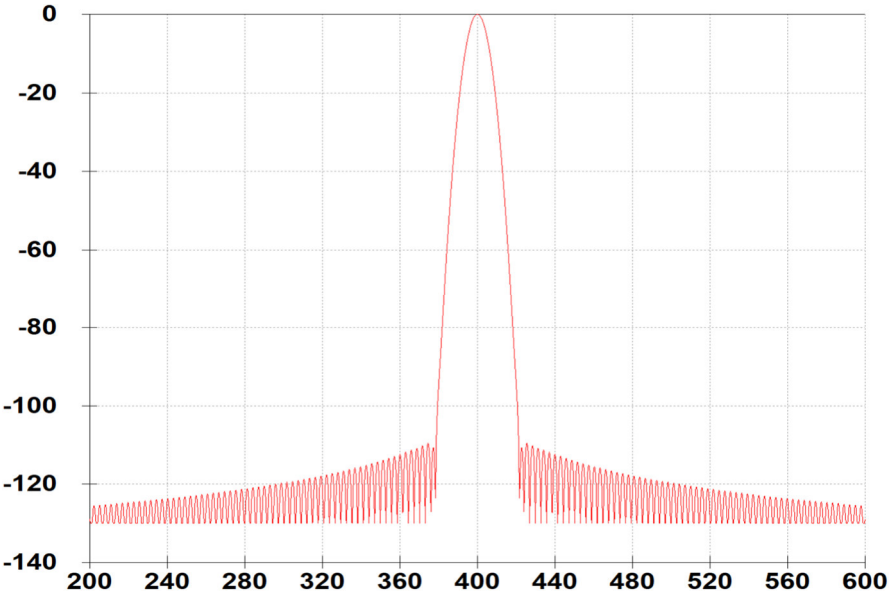


Figure 20: CW Peaking Filter — Gaussian frequency response showing zero-ringing rolloff.

This is implemented as a linear phase FIR filter to preserve its originally-intended phase characteristics. The number of coefficients is variable based upon selected bandwidth.

(4) Matched Filter: The matched filter is a very special type of filter often used for weak-signal communications. It has the potential to provide the best signal-to-noise ratio. However, tuning is critical, i.e., the tuned frequency of the filter must match the pitch of the received tone. Its frequency response appears rather unusual because, in the time-domain, it essentially represents the convolution of the CW signal with itself. The frequency response of a Matched Filter with a center frequency of 400Hz and a -6dB bandwidth of 10Hz is shown below.

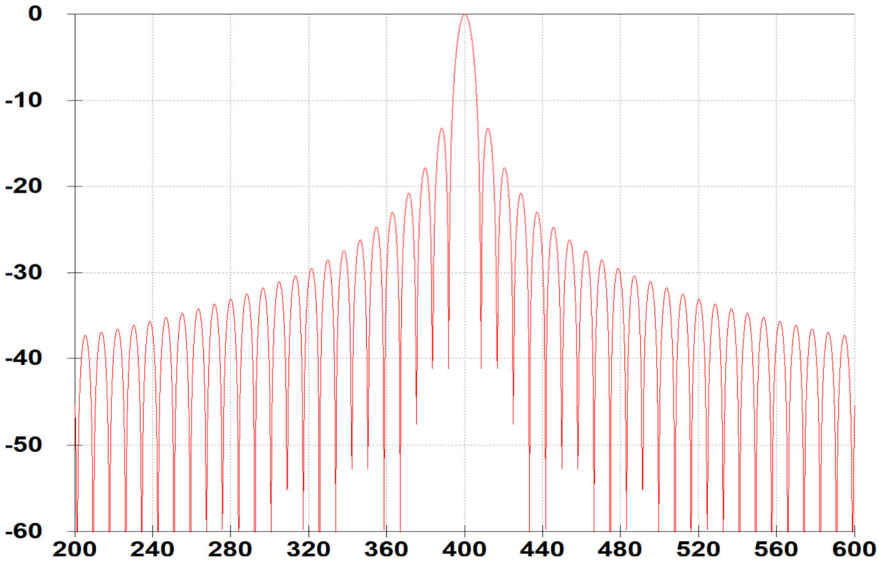


Figure 21: CW Peaking Filter — Matched Filter frequency response, optimized for maximum SNR.

This filter is implemented as a linear phase FIR filter. It cannot be converted to minimum phase without changing its fundamental characteristics and disqualifying its intended purpose. The number of coefficients is determined dynamically based upon the selected filter bandwidth.

About using these filters:

- Many operators will find it desirable to set the bandwidth to something in the range of 50Hz. Setting the filters to very narrow bandwidths may prevent hearing/detecting nearby signals.
- For situation where the CW signal is nearly buried in the band-noise, it may be desirable to reduce the bandwidth to 20Hz or even below. However, with such narrow settings, there is not adequate bandwidth to receive fast CW with the CW symbols being “smeared” together. Therefore, this is best for slow speeds, e.g., 12 WPM.

The CW Peaking Filter is turned OFF/ON with the call:

```
void SetRXASPCWRun (int channel, int run)
```

channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

Filter Selection is set with the following call:

```
void SetRXASPCWSelection (int channel, int selection)
```
channel: number of the receiver channel being accessed
selection: 0 = Double-Pole; 1 = Matched; 2 = Gaussian; 3 = Bi-Quad

Center frequency, bandwidth, and gain are set using the following three calls, respectively.

```
void SetRXASPCWFreq (int channel, double freq)
```
channel: number of the receiver channel being accessed
freq: center frequency in Hertz[default = 600.0]

```
void SetRXASPCWBandwidth (int channel, double bw)
```
channel: number of the receiver channel being accessed
bw: approximate -6dB bandwidth in Hertz[default = 100.0]

```
void SetRXASPCWGain (int channel, double gain)
```
channel: number of the receiver channel being accessed
gain: this is an absolute gain factor (multiplier) -- NOT dB.[default = 2.0]

5.3.25 Dolly Filter

The Dolly Filter is a dual-lobe filter intended for use in RTTY reception. It comprises a pair of CW peaking filters whose inputs are the same and whose outputs are summed. Since it is set up for RTTY operation, only two peaks are desired; however, the underlying filter structure supports an arbitrary number of peaks, each with its own Enable, Center_Frequency, Bandwidth, and Gain.

```
void SetRXAmpeakRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

```
void SetRXAmpeakNpeaks (int channel, int npeaks)
```
channel: number of the receiver channel being accessed
npeaks: number of peaks (peaking filters) desired[default = 2]

```
void SetRXAmpeakFilEnable (int channel, int fil, int enable)
```
channel: number of the receiver channel being accessed
fil: identifier of the filter being enabled. If npeaks is 2, the two filter identifiers will be 0 and 1.
enable: 0 = Disabled; 1 = Enabled[defaults = { 1 , 1 }]

```
void SetRXAmpeakFilFreq (int channel, int fil, double freq)
```
channel: number of the receiver channel being accessed
fil: identifier of the filter being enabled. If npeaks is 2, the two filter identifiers will be 0 and 1.
freq: filter center frequency in Hertz[defaults = { 2125.0, 2295.0 }]

```
void SetRXAmpeakFilBw (int channel, int fil, double bw)
```
channel: number of the receiver channel being accessed
fil: identifier of the filter being enabled. If npeaks is 2, the two filter identifiers will be 0 and 1.
bw: filter bandwidth in Hertz[defaults = {75.0, 75.0}]

```
void SetRXAmpeakFilGain (int channel, int fil, double gain)
```
channel: number of the receiver channel being accessed
fil: identifier of the filter being enabled. If npeaks is 2, the two filter identifiers will be 0 and 1.
gain: gain of the filter. Note that this is an absolute gain --- NOT dB.[defaults = { 1.0, 1.0 }]

5.3.26 Patch Panel - Audio Output Configuration

The PatchPanel block is used to configure the audio output. Operations such as volume control, binaural (I/Q) versus monaural output, audio-pan, I/Q swap, etc., can be performed using the provided controls.

```
void SetRXAPanelRun (int channel, int run)
```
channel: number of the receiver channel being accessed
run: 0 = OFF; 1 = ON[default = 1]

```
void SetRXAPanelSelect (int channel, int select)
```
channel: number of the receiver channel being accessed
select: 0 = no input used; 1 = use only Q input; 2 = use only I input; 3 = Use both I and Q input[default = 3]

```
void SetRXAPanelGain1 (int channel, double gain)
```
channel: number of the receiver channel being accessed
gain: gain to be applied to both I and Q outputs. This is an absolute gain, not dB.[default = 4.0]

```
void SetRXAPanelGain2 (int channel, double gainI, double gainQ)
```
channel: number of the receiver channel being accessed
gainI: gain to be applied to the I output. This is an absolute gain, not dB.[default = 1.0]
gainQ: gain to be applied to the Q output. This is an absolute gain, not dB.[default = 1.0]

The following call provides an alternative way to set gainI and gainQ.

```
void SetRXAPanelPan (int channel, double pan)
```
channel: number of the receiver channel being accessed
pan: range is 0.0 to 1.0. If pan = 0.0, gainI = 1.0 and gainQ = 0.0. If pan = 1.0, gainI = 0.0 and gainQ = 1.0. If pan = 0.5, gainI = 1.0 and gainQ = 1.0. There are smooth transitions between these reference points as pan varies from 0.0 to 1.0, smoothly panning the sound.

```
void SetRXAPanelCopy (int channel, int copy)
```
channel: number of the receiver channel being accessed
copy: 0 = no copy; 1 = copy I to Q; 2 = copy Q to I; 3 = reverse I and Q.[default = 0]

The following call provides an alternative way to set the 'copy' flag.

```
void SetRXAPanelBinaural (int channel, int bin)
```
channel: number of the receiver channel being accessed
bin: 0 = copy I to Q (monaural mode); 1 = no copy (binaural mode).

5.3.27 RXA Collectives & General Controls

The following call sets the reception MODE for RXA. This single call turns OFF/ON and provides certain required settings to blocks such as the AM and FM demodulators.

```
void SetRXAMode (int channel, int mode)
```

channel: number of the receiver channel being accessed
mode: mode assignments are: 0 = LSB; 1 = USB; 2 = DSB; 3 = CWL; 4 = CWU; 5 = FM; 6 = AM; 7 = DIGU; 8 = SPEC; 9 = DIGL; 10 = SAM; 11 = DRM; 12 = Broadcast_FM_Stereo.

A mode change also re-evaluates how Neural Noise Reduction is connected. For example, CWU and CWL place a CW model's training band on opposite sidebands (§ 5.3.20).

The following Collective calls combine the calls to multiple RXA blocks in cases where several blocks need the same information. This avoids having the console code make multiple calls, one per block. It also simplifies any future changes where new blocks might be added that require this same information. It is recommended to use these calls rather than individual calls to the blocks when possible.

The upper and lower pass-band frequency limits are set for all RXA blocks needing this information using the call:

```
void RXASetPassband (int channel, double f_low, double f_high)
```

channel: number of the receiver channel being accessed
f_low: lower of the two frequencies (Hertz). May be negative or positive. Must be lower (more negative) than f_high.
f_high: higher of the two frequencies (Hertz). May be negative or positive. Must be higher (more positive) than f_low.

This matters particularly with Neural Noise Reduction's CW models, because this call also sets the second Bandpass Filter, which then decides what is heard (§ 5.3.20).

The length of all RXA filter impulse responses for all RXA filters are simultaneously set using the following call. Note that when these filters were created, their 'nc' was set to "max (2048, dsp_size)". A minimum of 2048 is recommended; although, 1024 may be acceptable in some cases.

```
void RXASetNC (int channel, int nc)
```

channel: number of the receiver channel being accessed
nc: length of the desired filter impulse response. This must be a power of two AND must be greater than or equal to the DSP Buffer Size in use.

The choice of Linear Phase or Low Latency for all filters accepting such a choice is set by the call:

```
void RXASetMP (int channel, int mp)
```

channel: number of the receiver channel being accessed
mp: 0 for Linear Phase; 1 for Low Latency (Minimum Phase)

This includes the second Bandpass Filter, which with Neural Noise Reduction's CW models takes the place of the Notched Bandpass Filter in the audio path (§ 5.3.20). Neither this call nor RXASetNC changes the filter that forms their training band, which is always minimum phase with at least 2048 coefficients.

6 The TXA Transmitter Unit

6.1 Channel Parameters for TXA

The channel code serves as the interface between the unit it houses and the outside world. Considering this, some of the channel parameters relate only to the outside world. However, others impact the operation or perceived operation of the unit. It is this latter category of parameters that we discuss again here.

dsp_size: The unit only processes data when dsp_size samples have been accumulated. This parameter therefore impacts the latency incurred in DSP processing. Generally, using a lower dsp_size will reduce latency. However, these lower values mean a larger number of buffers are processed per unit time and each buffer requires some amount of processing overhead. Therefore, lower dsp_size implies some increase in the CPU load. For TXA, for typical dsp_rate, dsp_size ranging from a minimum of 64 to a maximum of 1024 offers a reasonable range of trade-offs -- 64 for minimum latency and 1024 to minimize CPU load.

Note, however, that there is an important relationship among dsp_size, dsp_rate, in_size, and input_samplerate. For example, consider the case where dsp_size = 64, dsp_rate = 96000, input_samplerate = 48000 and in_size = 64. As discussed below, a resampler will be automatically invoked to resample the input from 48000 up to 96000. For each one sample at the 48000 input rate, two samples at the 96000 rate will be produced. Therefore, 64 input samples (in_size) will produce 128 samples internally. This is 2 times the 64 samples (dsp_size) required for the unit to process data. Therefore, the unit will process twice before the channel will produce output. This implies that making dsp_size less than 128 will not speed-up DSP processing, given the other values of this example.

dsp_rate: dsp_rate not only impacts latency, as described just above, it also impacts the results in filtering operations. Doubling dsp_rate doubles the width of filter transition bands. This can be precisely compensated by doubling the number of filter coefficients. Of course, increasing the rate and increasing the complexity of the filter, both require more CPU cycles. TXA has been designed to run optimally at a dsp_rate of 96000 for all modes.

tdelayup, tslewup, tdelaydown, tslewdown: Typical settings for these parameters are given here. Note that, to minimize CW TX latency, the CW mode is intended to be implemented in firmware rather than through the DSP library. Therefore, lower times for CW are not considered here.

All modes: tdelayup = 0.010; tslewup = 0.025; tdelaydown = 0.000; tslewdown = 0.010.

6.2 Signal Characteristics & Levels for TXA

The input to TXA is the audio MIC signal. Although, consistent with other channel input and output signals, this is delivered in an interleaved I-Q packet, this is an audio (NOT an I-Q signal). Hence, only the I values or the Q values will be used. See the "Patchpanel" for selection of whether I or Q data is to be used.

The peak amplitude of the MIC signal is limited only by the range of double-precision floating point calculations. However, it is expected that the product of the peak MIC signal and the MIC Gain setting will be roughly 1.0. See the "Patchpanel" for setting MIC Gain.

The TXA output is an I-Q signal with a peak magnitude of 1.0.

6.3 Block Diagram & Controls

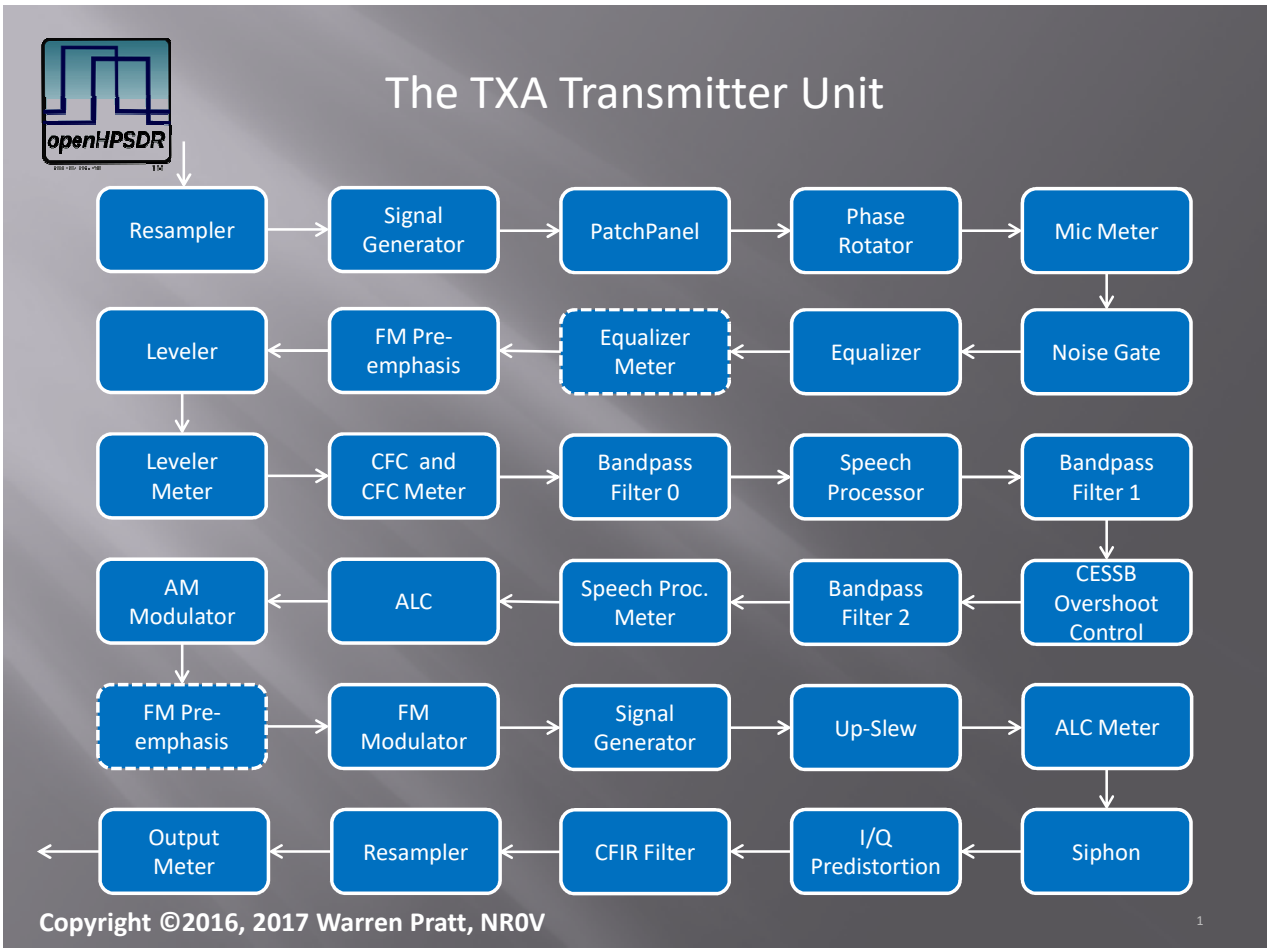


Figure 22: Block diagram of the TXA Transmitter Unit showing the transmitter processing chain.

The above figures displays the arrangement of blocks in the TXA transmitter unit. We will now discuss the operation and control of each block.

6.3.1 Resamplers

In the discussion above of channel parameters, three distinct rates are mentioned:

input_samplerate: samplerate of data entering the channel
dsp_rate: internal rate at which the data is processed within the unit
output_samplerate: samplerate of data exiting the channel

Two resamplers are provided within the TXA unit to convert among these rates. The first of these, the "Input Resampler," converts from the input_samplerate to the dsp_rate. The second of these, the "Output Resampler," converts from the dsp_rate to the output_samplerate.

No external calls are provided to control these resamplers. The three rates are set by the channel parameters and the resamplers are automatically activated when rates differ and therefore resampling is required.

6.3.2 Signal Generators

The TXA unit includes two signal generators. The first of these can replace the incoming MIC signal with a variety of waveforms that are useful for testing TXA setup and operation. The second of these generators is near the end of the processing chain and is intended for generating the TUNE signal, a Sweep Carrier, or a Two-Tone Test signal. Calls to set up the first of these generators contain the term "PreGen" while calls to set up the second of these two generators contain the term "PostGen".

The set of output modes currently supported for TXA "PreGen" comprises: Tone, Noise, Sweep, Sawtooth, Triangle, Pulse, and Silence.

```
void SetTXAPreGenRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 to turn the generator OFF; 1 to turn the generator ON [default = 0]

```
void SetTXAPreGenMode (int channel, int mode)
```
channel: number of the channel being accessed
mode: Tone = 0; Noise = 2; Sweep = 3; Sawtooth = 4; Triangle = 5; Pulse = 6; Silence = 99 [default = 2]

```
void SetTXAPreGenToneMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: peak magnitude value of the single-tone (mode = 0). Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPreGenToneFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: frequency (Hertz) of the single-tone (mode = 0). Can be either positive or negative. [default = +1000.0]

```
void SetTXAPreGenNoiseMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: magnitude value of the noise (mode = 2). Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPreGenSweepMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: magnitude of the swept single tone (mode = 3). Typical range is 0.0 to 1.0.[default = 1.0]

```
void SetTXAPreGenSweepFreq (int channel, double freq1, double freq2)
```
channel: number of the channel being accessed
freq1, freq2: frequency limits of the sweep range (Hertz). Sweep progresses from freq1 to freq2 and then repeats.[defaults: freq1 = -20000.0, freq2 = +20000.0]

```
void SetTXAPreGenSweepRate (int channel, double rate)
```
channel: number of the channel being accessed
rate: rate at which the sweep progresses (Hertz / Second).[default = +4000.0]

```
void SetTXAPreGenSawtoothMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: peak magnitude value of the sawtooth. Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPreGenSawtoothFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: frequency (Hertz) of the sawtooth.[default = +500.0]

```
void SetTXAPreGenTriangleMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: peak magnitude value of the triangle. Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPreGenTriangleFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: frequency (Hertz) of the triangle.[default = +500.0]

> Note that the "Pulse" generator mode can produce a pulsed-tone, really a "burst mode", if the tone frequency is set to a non-zero value.

```
void SetTXAPreGenPulseMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: peak magnitude value of the pulse. Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPreGenPulseFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: frequency (Hertz) of occurrence of the pulse. [default = 0.25]

```
void SetTXAPreGenPulseDutyCycle (int channel, double dc)
```
channel: number of the channel being accessed
dc: Duty-cycle of the pulse = fraction of the time that the pulse is ON.[default = 0.25]

```
void SetTXAPreGenPulseToneFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: Frequency of a pulsed tone (Hertz).[default = 1000.0]

```
void SetTXAPreGenPulseTransition (int channel, double transtime)
```
channel: number of the channel being accessed
transtime: OFF-ON or ON-OFF transition time (seconds).[default = 0.002]

The set of output modes currently supported for TXA "PostGen" comprises: Tone, Two-tone, Sweep, and Silence.

```
void SetTXAPostGenRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 to turn the generator OFF; 1 to turn the generator ON [default = 0]

```
void SetTXAPostGenMode (int channel, int mode)
```
channel: number of the channel being accessed
mode: Tone = 0; Two-Tone = 1; Sweep = 3; Pulsed tone = 6; Pulsed two-tone = 7; Silence = 99.[default = 0]

```
void SetTXAPostGenToneMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: peak magnitude value of the single-tone (mode = 0). Typical range is 0.0 to 1.0. [default = 1.0]

```
void SetTXAPostGenToneFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: frequency (Hertz) of the single-tone (mode = 0). Can be either positive or negative. [default = +1000.0]

```
void SetTXAPostGenTTMag (int channel, double mag1, double mag2)
```
channel: number of the channel being accessed
mag1: peak magnitude value of the first tone (mode = 0). Typical range is 0.0 to 1.0. [default = 0.5]
mag2: peak magnitude value of the second tone (mode = 0). Typical range is 0.0 to 1.0. [default = 0.5]
NOTE: With both mag1 and mag2 set to 0.5, the peak magnitude of the generator output will be 1.0. I.e., the peak magnitude of the generator output is mag1 + mag2.

```
void SetTXAPostGenTTFreq (int channel, double freq1, double freq2)
```
channel: number of the channel being accessed
freq1: frequency (Hertz) of the first tone (mode = 0). Can be either positive or negative. [default = + 900.0]

freq2: frequency (Hertz) of the second tone (mode = 0). Can be either positive or negative. [default = +1700.0]

```
void SetTXAPostGenSweepMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: magnitude of swept single tone (mode = 3). Typical range is 0.0 to 1.0.[default = 1.0]

```
void SetTXAPostGenSweepFreq (int channel, double freq1, double freq2)
```
channel: number of the channel being accessed
freq1, freq2: frequency limits of the sweep range (Hertz). Sweep progresses from freq1 to freq2 and then repeats.[defaults: freq1 = -20000.0, freq2 = +20000.0]

```
void SetTXAPostGenSweepRate (int channel, double rate)
```
channel: number of the channel being accessed
rate: rate at which the sweep progresses (Hertz / Second).[default = +4000.0]

```
void SetTXAPostGenPulseMag (int channel, double mag)
```
channel: number of the channel being accessed
mag: magnitude of pulsed single tone (mode = 6). Range is 0.0 to 1.0.[default = 1.0]

```
void SetTXAPostGenPulseFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: pulsing frequency (not tone frequency) of pulsed single tone (mode = 6), in Hertz.[default = 2.0 (Hz)]

```
void SetTXAPostGenPulseDutyCycle (int channel, double dc)
```
channel: number of the channel being accessed
dc: duty-cycle of pulse (mode = 6), 0.0 < dc < 1.0.[default = 0.25]

```
void SetTXAPostGenPulseToneFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: tone frequency in Hertz. (mode = 6).[default = 600.0]

```
void SetTXAPostGenPulseTransition (int channel, double transtime)
```
channel: number of the channel being accessed
transtime: time used for a pulse transition from OFF to ON or from ON to OFF, in seconds. This is a raised-cosine transition. (mode = 6).[default = 0.005]

```
void SetTXAPostGenPulseIQout (int channel, int IQout)
```
channel: number of the channel being accessed
IQout: specifies either audio output (I only, Q = 0) or I-Q output ('0' => I-only; '1' => IQ). (mode = 6). For use in the TX chain, 'IQout' should normally be set to '1'![default = 0]

```
void SetTXAPostGenTTPulseMag (int channel, double mag1, double mag2)
```
channel: number of the channel being accessed
mag1, mag2: magnitude of each of the two tones (mode = 7). Range for each is 0.0 to 1.0. However, the sum of the two should not exceed 1.0![default = 0.5, 0.5]

```
void SetTXAPostGenTTPulseFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: pulsing frequency (not tone frequencies) of pulsed two-tone (mode = 7), in Hertz.[default = 2.0 (Hz)]

```
void SetTXAPostGenTTPulseDutyCycle (int channel, double dc)
```
channel: number of the channel being accessed
dc: duty-cycle of pulse (mode = 7), 0.0 < dc < 1.0.[default = 0.25]

```
void SetTXAPostGenTTPulseToneFreq (int channel, double freq1, double freq2)
```
channel: number of the channel being accessed
freq1, freq2: tone frequencies in Hertz. (mode = 7).[default = 900.0, 1700.0]

```
void SetTXAPostGenTTPulseTransition (int channel, double transtime)
```

channel: number of the channel being accessed

f: time used for a pulse transition from OFF to ON or from ON to OFF, in seconds. This is a raised-cosine transition. (mode = 7).[default = 0.005]

```
void SetTXAPostGenTTPulseIQout (int channel, int IQout)
```

channel: number of the channel being accessed

IQout: specifies either audio output (I only, Q = 0) or I-Q output ('0' => I-only; '1' => IQ). (mode = 7). For use in the TX chain, 'IQout' should normally be set to '1'![default = 0]

6.3.3 PatchPanel

The TXA patchpanel is used to select which channel (I or Q) is used for MIC input and to set the MIC gain. Available calls are:

```
void SetTXAPanelRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 = this patchpanel is not used; 1 = this patchpanel is active[default = 1]

```
void SetTXAPanelSelect (int channel, int select)
```
channel: number of the channel being accessed
select: 1 for Q; 2 for I[default = 2]

```
void SetTXAPanelGain1 (int channel, double gain)
```
channel: number of the channel being accessed
gain: MIC Gain to be applied (this is a linear value of gain, NOT a dB value)[default = 1.0]

6.3.4 Meters

As shown in the block diagram, metering outputs from TXA are the Mic Meter, Equalizer Meter, Leveler Meter, CFC Meter, Speech Processor Meter, ALC Meter, and the Output Meter. Taken together, these provide visibility of internal TXA processing levels that is sufficient to optimize many TXA settings.

Meter values are retrieved using the single call:

```
double GetTXAMeter (int channel, int mt)
```

return value: requested meter value (dBV, except dB for Leveler_Gain and ALC_Gain). This will return a value of 0.0 dBV for a signal magnitude level of 1.0.
channel: number of the channel being accessed
mt: type of meter data being requested. mt values are:

MIC_Peak = 0;
MIC_Average = 1;
EQ_Peak = 2;
EQ_Average = 3;
Leveler_Peak = 4;
Leveler_Average = 5;
Leveler_Gain = 6;
CFC_Peak = 7;
CFC_AV = 8;
CFC_Gain = 9;
Compressor_Peak = 10;
Compressor_Average = 11;
ALC_Peak = 12;
ALC_Average = 13;
ALC_Gain = 14;
Output_Peak = 15;
Output_Average = 16;

6.3.5 Noise Gate

The Noise Gate can be set to lower the microphone audio gain if the input signal for a period of time is below a specified level. This can be used to lower transmitted background noise (for example fan noise) during pauses in speech.

An instance of the same "AM Squelch" used in the RXA receiver is used for this purpose. Therefore, many of the same features, like raised-cosine transitions between gain levels, are available.

To activate this feature:

```
void SetTXAAMSQRun (int channel, int run)
```

channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

To set the gain when "muted", i.e., the low gain level:

```
void SetTXAAMSQMutedGain (int channel, double dBlevel)
```

channel: number of the channel being accessed
dBlevel: gain (dB) to be used when the noise gate is MUTED. This is a negative value, relative to the UN-MUTED level.[default = -13.98 dB]

To set the threshold for "un-mute":

```
void SetTXAAMSQThreshold (int channel, double threshold)
```

channel: number of the channel being accessed
threshold: threshold (dB) at which to un-mute the noise gate.[default = -13.98 dB]

6.3.6 Equalizer

The equalizer uses a continuous frequency response algorithm meaning that the gain is specified at any number of frequencies and the response is interpolated between the specified points. This is in contrast to a frequency "band" approach. The TX Equalizer uses Minimum Phase (Low Latency) mode, eliminating the latency of linear phase while producing results that are audibly indistinguishable over HF propagation paths. For a full discussion, see Appendix B.

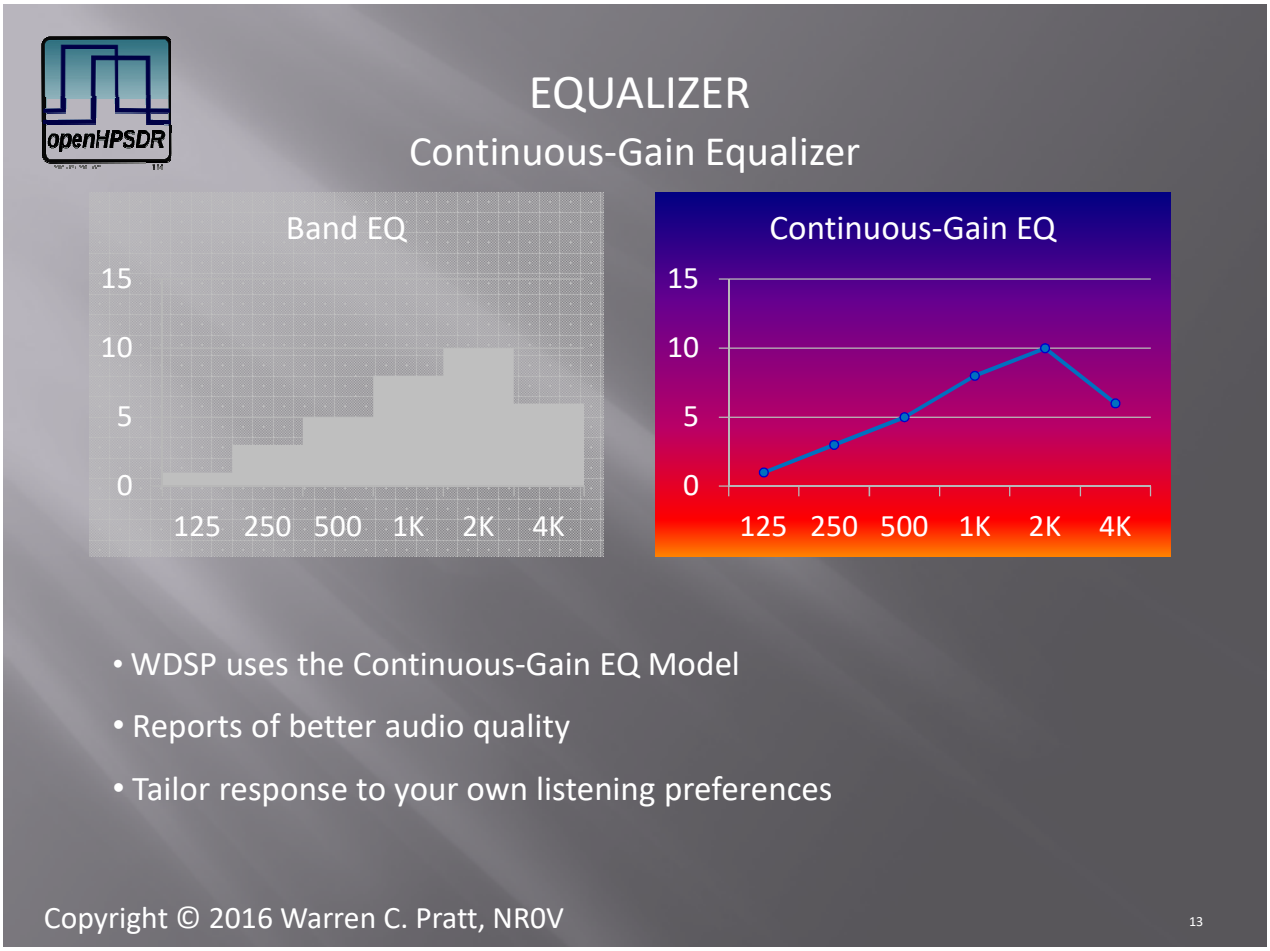


Figure 23: TXA Continuous-Gain Equalizer panel comparing Band EQ and Continuous-Gain EQ response models.

The equalizer is turned OFF/ON using calls to:

```
void SetTXAEQRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 for OFF; 1 for ON[default = 0]

```
void SetTXAEQNC (int channel, int nc)  — DEPRECATED
```
channel: number of the receiver channel being accessed
nc: length of the EQ filter impulse response. 'nc' must be a power-of-two and must be greater than or equal to the DSP Buffer Size in use. [default = AUTOMATIC]

```
void SetTXAEQMP (int channel, int mp)  – DEPRECATED
```
channel: number of the channel being accessed
mp: 0 for Linear-Phase; 1 for Low-Latency (Minimum Phase) [default = 1 (Minimum Phase) = AUTOMATIC]

The window-type used in the generation of the EQ filter can be selected. The Kaiser window (wintype = 2) provides slightly better stopband attenuation than the Blackman-Harris options. This setting should normally be left at the default value.

```
void SetTXAEQWintype (int channel, int wintype)
```
channel: number of the channel being accessed
wintype: 0 = 4-term Blackman-Harris; 1 = 7-term Blackman-Harris; 2 = Kaiser[default = 2 (Kaiser)]

The cutoff-mode used in the generation of the EQ filter can be selected. This setting should normally just be left at the default value.

```
void SetTXAEQCtfmode (int channel, int mode)
```
channel: number of the channel being accessed
mode: 0 = 4th-power roll-off on each side of the specified passband; 1 = no roll-off outside the specified passband[default = 0]

The following call is the primary and preferred method of setting the equalizer response.

```
void SetTXAEQProfile (int channel, int nfreqs, double* F, double* G)
```
channel: number of the channel being accessed
nfreqs: number of frequency/gain pairs to be specified
F: pointer to an array of frequencies (Hertz). NOTE: The 0th element of this vector is NOT USED. So, for example, if nfreqs is 5, F[0] is not used and the five frequencies will be stored in F[1] ... F[5]. The F[] array has (nfreqs + 1) values.
Note that acceptable values are between 0.0 Hz and dsp_rate/2.0 Hz. It is acceptable to specify the same frequency multiple times; however, if different gains are specified for the same frequency, it is indeterminate as to which of the gains will be applied at that frequency. Frequency/Gain pairs do not necessarily have to be specified in order of increasing frequency. If they are not, they will be sorted into the appropriate order prior to application.
G: pointer to a corresponding array of gain values (dB). The 0th element is used to specify a "preamp gain" value to be applied to all frequencies. The G[] array has (nfreqs + 1) values.

```
void SetTXAGrphEQ (int channel, int* rxeq)  – DEPRECATED
```
channel: number of the channel being accessed

```
void SetTXAGrphEQ10 (int channel, int* rxeq)  – DEPRECATED
```
channel: number of the channel being accessed

6.3.7 Equalizer – Using Higher-Degree Curves

The TX Equalizer supports higher-degree B-spline / NURBS curve interpolation between control points. For a complete description — including degrees, weights, knot placement, and graphical output — see Appendix A. The following calls provide access to this functionality.

> Note that the Control Points are set by calls above, they are specified by the values in the 'F' and 'G' arrays.

```
void SetTXAEQCurve (int channel, int degree, int r, int umethod)
```
channel: number of the receiver channel being accessed
degree: degree of the curve segments [default = 0]

0 – Use previous method, linear, no graphical output.

1 – Linear, high-degree method with graphical output.

2, 3, … , 16 – Degree of curve segments.

r:'0' – do NOT use Weights; '1' – use Weights [default = 0]
umethod: method used to position knots. [default = 0, recommended]

0 – calculated positions based upon control point activity.

1 – equally-spaced in 'u' parameter space.

```
void SetTXAEQWeights (int channel, int ncp, double* weights)
```

channel: number of the receiver channel being accessed
ncp: number of control points (note: one weight per control point)
weights: vector of weights [defaults = 1.0]

> NOTE: There must be a weight, for each control point, satisfying these conditions when weights are used:

- weights[i] > 0.0
- weights[i] < 100.0

```
void GetTXAEQDraw (int channel, double* X, double* Y)
```

channel: number of the receiver channel being accessed
X, Y: pointers to vectors into which X (frequency) and Y (gain) values will be copied.

6.3.8 FM Pre-emphasis

FM transmit Pre-emphasis, along with companion receive de-emphasis, are used to improve the signal-to-noise ratio of FM transmission-reception.

The standard bandwidth for narrowband FM communications is 300 Hz to 3000 Hz. Pre-emphasis follows a 6 dB per octave (20 dB per decade) slope. When implemented with the default impulse response length (2048), the TXA pre-emphasis curve is as shown in the following figure. Note that the frequency scale shown here is linear; therefore the curve appears to be more rounded than the straight-line one would see if this were plotted with a log-domain frequency axis. Also, note that no attempt has been made here to make the cutoff extremely sharp as this will be controlled by the bandpass filter.

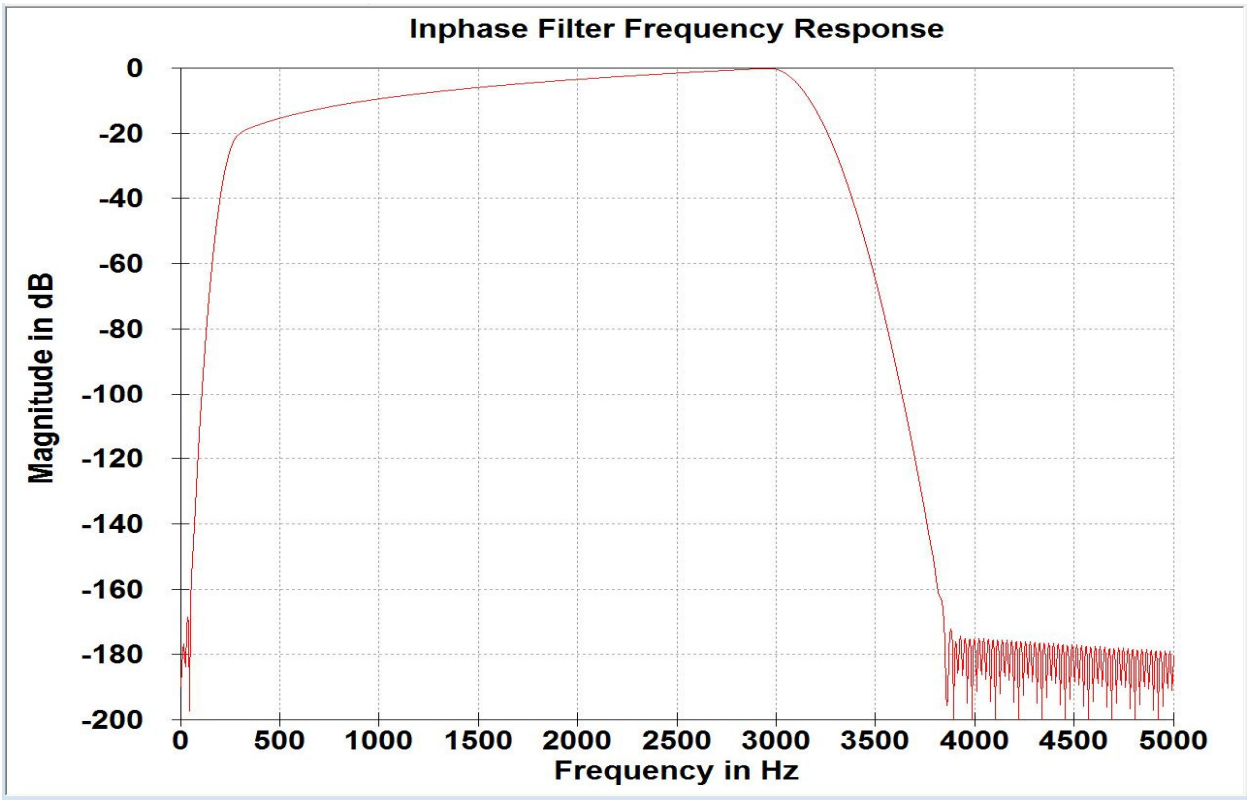


Figure 24: TXA FM Pre-emphasis filter frequency response — 6 dB/octave rising slope from 300 Hz to 3000 Hz.

Referring back to the overall TXA block diagram, note that FM Pre-emphasis is shown twice, each time in a block with a dashed outside line. The meaning is that the FM pre-emphasis can be located in either position, controllable by a "switch":

```
void SetTXAFMEmphPosition (int channel, int position)
```

channel: number of the channel being accessed
position: 0 = BEFORE pre-processing and ALC; 1 = AFTER pre-processing and ALC[default = 1]

The latter (default) position will absolutely limit the transmitted magnitude of a particular frequency, i.e., a 1000 Hz could never be transmitted with the same power as a 2000 Hz signal. The former position

has a different behavior which generally may allow the signal to sound louder. The correct positioning has been the subject of some discussion and so both options are provided via the switch.

```
void SetTXAFMEmphNC (int channel, int nc)  — DEPRECATED
```

channel: number of the channel being accessed

nc: length of the Pre-emphasis filter impulse response. 'nc' must be a power-of-two AND must be greater than or equal to the DSP Buffer Size in use.[default = 2048]

```
void SetTXAFMEmphMP (int channel, int mp)  — DEPRECATED
```

channel: number of the channel being accessed

mp: 0 for Linear-Phase; 1 for Low-Latency (Minimum Phase)[default = 0]

6.3.9 Leveler

The audio leveler can be invoked to provide additional gain when the audio signal is low and then to reduce gain as the audio level increases. This can be useful, for example, to provide a little extra gain if the operator momentarily turns away from the microphone or to reduce gain if the operator momentarily gets closer to the microphone, in other words, it "levels" the audio signal.

The Leveler also helps provide a known audio amplitude level for the phase rotator, CFC, and speech processor (compressor) that follow. This helps ensure that the requested compression level is the level delivered. In the absence of this assistance from the Leveler, it's solely up to the operator to ensure that the audio level before the following stages is peaking at about 0 dB.

The Leveler is implemented with an instance of AGC/ALC code; however, the settings are considerably different for the Leveler application. You will, however, notice similarity in some of the settings. Settings available for the Leveler are:

```
void SetTXALevelerSt (int channel, int state)
```
channel: number of the channel being accessed
state: 0 = OFF; 1 = ON[default = 0]

```
void SetTXALevelerAttack (int channel, int attack)
```
channel: number of the channel being accessed
attack: attack time-constant in ms. Note that this is a time-constant for an exponential curve, NOT an absolute time. It is recommended to leave this at the default value.[default = 1 ms]

```
void SetTXALevelerDecay (int channel, int decay)
```
channel: number of the channel being accessed
decay: decay time-constant in ms. Note that this is a time-constant for an exponential curve, NOT an absolute time. The default value works well for normal operating conditions.[default = 500 ms]

```
void SetTXALevelerTop (int channel, double maxgain)
```
channel: number of the channel being accessed
maxgain: maximum gain (dB) that the leveler is to apply.[default = 5.0 dB]

6.3.10 Phase Rotator

In some AM transmitters, it is possible to boost peak output power by having an asymmetrical audio waveform (positive peaks greater than negative peaks) and modulating to greater than 100% on positive peaks while restricting to ≤100% on negative peaks to avoid “pinch-off.”

In other AM transmitters and in the case of digital-up-conversion (DUC) SDRs, this approach is incorrect. Hard limits exist that cannot be exceeded, such as the dynamic range of the DAC. An asymmetrical waveform with positive peaks exceeding negative peaks cannot increase modulation in the positive direction; it instead REDUCES average power. The correct approach is to make the audio waveform as symmetrical as possible, i.e., equal positive and negative peaks.

The Phase Rotator makes the audio waveform more symmetrical by shifting the phase of different audio frequencies by varying amounts, changing the shape of the waveform away from the typically asymmetric shape of human speech. Multiple identical stages are cascaded; the total phase shift is the sum of the contributions from each stage. Each stage produces 0° of phase shift at DC, 90° at the corner frequency fc, and 180° at Nyquist. Frequencies above fc receive more total phase shift than frequencies below fc; this differential phase rotation across the audio band is what reshapes the waveform.

The result is a lower peak-to-average ratio, allowing the ALC to operate at a higher average level for the same peak limiting threshold — effectively more average transmitted power.

While the above discussion focused on AM, the phase rotator can also improve average power for any speech mode. Note, however, that it may be detrimental for digital modes that require a coherent phase-versus-frequency relationship.

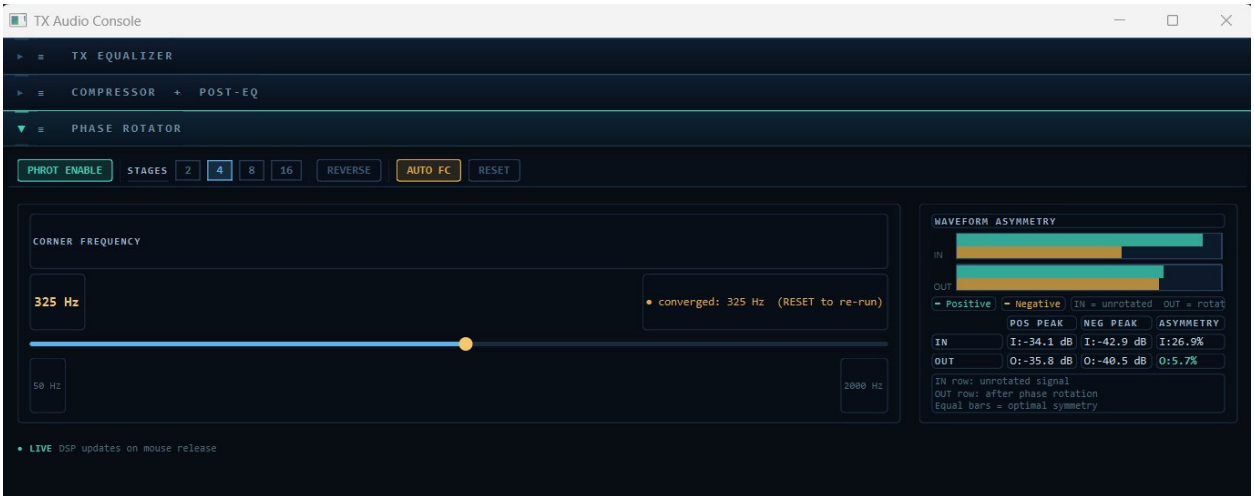


Figure 25: Phase Rotator panel in the TX Audio Console, showing the corner frequency slider, optimizer status (converged at 325 Hz), and waveform asymmetry display.

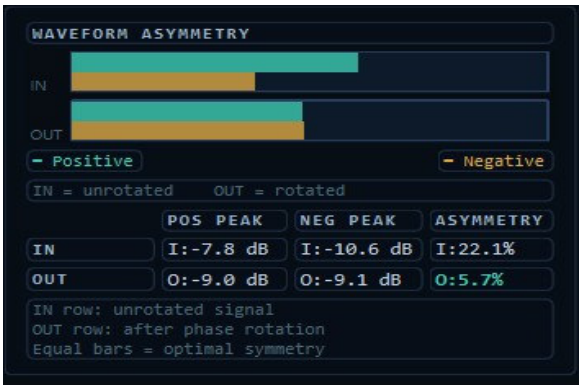


Figure 26: Close-up of the Waveform Asymmetry display. IN row (unrotated): 22.1% asymmetry. OUT row (phase-rotated): 5.7% asymmetry — highlighted in teal when meaningfully better than the input.

Operator Guidance

Sensitivity to the exact corner frequency is low in the region of the optimum. Typical voice material shows a broad minimum in the 200–400 Hz range, with noticeably worse results only at the extremes (below ~100 Hz or above ~1000 Hz for 4 stages). This behaviour is consistent with commercial broadcast phase rotators such as the Orban Optimod, which commonly use a fixed corner frequency around 250–350 Hz.

The AUTO FC optimizer finds the optimal corner frequency automatically. Each A/B comparison holds the filter at a candidate frequency for approximately 10–15 seconds before comparing against an adjacent frequency; total convergence from a cold start is typically 3–5 minutes. Once converged, the corner frequency is locked and the optimizer stops until RESET is pressed. Typical asymmetry improvement: 2–10 percentage points, depending on voice and microphone.

Settings for the Phase Rotator are:

```
void SetTXAPHROTRun (int channel, int run)
```

channel: number of the channel being accessed
run: 0 = OFF; 1 = ON
[default = 0]

```
void SetTXAPHROTCorner (int channel, double frequency)
```

channel: number of the channel being accessed
frequency: Corner frequency in Hz. Each stage produces 90° of phase shift at this frequency. Range: 50–2000 Hz. If the auto-optimizer (SetTXAPHROTAutoMode) is active, calling SetTXAPHROTCorner resets the optimizer to re-evaluate from the new frequency.
[default = 338.0]

```
void SetTXAPHROTNstages (int channel, int nstages)
```

channel: number of the channel being accessed
nstages: Number of cascaded all-pass stages. More stages produce more total phase rotation at a given corner frequency. This is an internal/developer parameter and is not normally exposed to the end operator.
[default = 4]

```
void SetTXAPHROTReverse (int channel, int reverse)
```

channel: number of the channel being accessed

reverse: Inverts the polarity of the input signal before filtering. Can improve symmetry in some signal chains. [default = 0]

```
void SetTXAPHROTAutoMode (int channel, int autoMode)
```

channel: number of the channel being accessed

autoMode: 1 = enable automatic corner frequency optimisation; 0 = disable. When enabled, the optimizer adjusts the corner frequency every ~10–15 seconds using an A/B comparison algorithm that minimises waveform asymmetry. Enabling resets the optimizer and starts a fresh search from the current corner frequency. Turning auto mode OFF then ON resumes the search from the current corner frequency. Use SetTXAPHROTAutoReset to restart from the default frequency (338 Hz). [default = 0]

```
void SetTXAPHROTAutoReset (int channel)
```

channel: number of the channel being accessed

Resets the corner frequency to the default (338 Hz) and restarts the optimizer search from that point, without disabling auto mode. Use after a significant change in voice, microphone, or room acoustics, or when successive auto-optimize runs appear to drift away from the optimum.

```
void GetTXAPHROTAsymmetry (int channel, double* in_pos, double* in_neg,
double* in_ratio, double* out_pos, double* out_neg, double* out_ratio,
double* current_fc, double* auto_step)
```

channel: number of the channel being accessed

in_pos: smoothed positive peak envelope of the input signal (pre-filter), linear scale

in_neg: smoothed negative peak envelope of the input signal (pre-filter), linear scale

in_ratio: smoothed asymmetry ratio of the input signal: |pos−neg| / (pos+neg). 0.0 = perfectly symmetric; 1.0 = maximally asymmetric

out_pos: smoothed positive peak envelope of the output signal (post-filter), linear scale

out_neg: smoothed negative peak envelope of the output signal (post-filter), linear scale

out_ratio: smoothed asymmetry ratio of the output signal. Compare with in_ratio to verify improvement

current_fc: current corner frequency in Hz (reflects manual or auto-optimizer setting)

auto_step: current optimizer step size in Hz when searching; −1.0 when converged (fc locked); 0.0 when auto mode is disabled. Intended for display: the UI can show “searching” vs “converged” status.

6.3.11 Continuous Frequency Compressor (CFC)

Many audio racks and Digital Audio Workstations provide "Multiband Compressors" which allow specifying different amounts of compression for different audio frequency bands. The CFC offers a superset of that concept where, instead of utilizing multiple bands with constant compression in each, the compression varies smoothly between frequency points at which it is specified. This concept is similar to the WDSP Equalizer function discussed above; however, this time, we are varying the compression level across frequencies rather than gain. A Post-CFC Equalizer is also provided as an integral part of this function.

Use of this function for speech modes can significantly increase the "density" and average power of the signal. Note also that use for digital modes may be detrimental, depending upon the nature of the mode.

Settings for the CFC are:

```
void SetTXACFCOMPRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

```
void SetTXACFCOMPprofile (int channel, int nfreqs, double* F, double* G,
double* E)
```
channel: number of the channel being accessed
nfreqs: number of frequencies at which compression and post-CFC equalizer gain are specified
F: pointer to the array of frequency points (values in Hertz)
G: pointer to the array of compression at each frequency point (values in dB)
E: pointer to the array of post-CFC equalizer gains (values in dB)

> NOTE: The above function (SetTXACFCOMPprofile) is retained for backward compatibility with interfaces where the Compressor and Post-EQ share the same frequency array. For new implementations, the following two functions are preferred as they allow independent frequency arrays for the Compressor and Post-EQ.

```
void SetTXACFCOMPGprofile (int channel, int nfreqs, double* F, double* G)
```
channel: number of the channel being accessed
nfreqs: number of frequency points at which compression gain is specified
F: pointer to the array of frequency points for the Compressor (values in Hertz)
G: pointer to the array of compression gain at each frequency point (values in dB)

```
void SetTXACFCOMPEprofile (int channel, int nfreqs, double* F, double* E)
```
channel: number of the channel being accessed
nfreqs: number of frequency points at which Post-EQ gain is specified
F: pointer to the array of frequency points for the Post-EQ (values in Hertz)
E: pointer to the array of Post-EQ gain at each frequency point (values in dB)

The Compressor and Post-EQ each support higher-degree curve interpolation between control points, allowing smooth transitions between specified compression and gain values. See Appendix A for a complete description of the curve system including degrees, weights, and knot methods. The following calls provide this functionality for both the Compressor and Post-EQ independently.

```
void SetTXACFCOMPCompCurve (int channel, int deg, int r, int umethod)
```
channel: number of the channel being accessed
deg: degree of the curve segments [default = 0]

0 – Linear, legacy mode (piecewise linear, no graphical output) 1 – Linear, B-spline method with graphical output 3, 5, 7, … – Higher-degree NURBS/B-spline (see Appendix A)

r: 0 = B-spline (weights ignored); 1 = rational NURBS (weights active) [default = 0]
umethod: knot placement method: 0 = based on control point positions; 1 = uniform [default = 0]

```
void SetTXACFCOMPCompWeights (int channel, int nfreq, double* weights)
```
channel: number of the channel being accessed
nfreq: number of weights (must equal the number of Compressor control points)
weights: pointer to the array of weights, one per control point [defaults = 1.0]

> NOTE: SetTXACFCOMPCompCurve must be called with r=1 before weights take effect. All weights must satisfy: 0.0 < weight < 100.0.

```
void GetTXACFCOMPCompDraw (int channel, double* X, double* Y)
```
channel: number of the channel being accessed

X: pointer to storage for the x-coordinates of the 1024 curve points. (frequency as a fraction of the Nyquist rate, e.g., if the sample-rate is 96000, the Nyquist rate is 48000 – multiply values by 48000 to get Hertz)
Y: pointer to storage for the y-coordinates (compression gain in dB) of the 1024 curve points

The returned (X, Y) pairs form a polyline for displaying the current Compressor curve. X values are in Hz (not normalized) and are NOT equally spaced; polyline plotting must be used.

```
void SetTXACFCOMPPeqCurve (int channel, int deg, int r, int umethod)
```
channel: number of the channel being accessed
deg: degree of the curve segments [default = 0] (see SetTXACFCOMPCompCurve)
r: 0 = B-spline (weights ignored); 1 = rational NURBS (weights active) [default = 0]
umethod: knot placement method: 0 = based on control point positions; 1 = uniform [default = 0]

```
void SetTXACFCOMPPeqWeights (int channel, int nfreq, double* weights)
```
channel: number of the channel being accessed
nfreq: number of weights (must equal the number of Post-EQ control points)
weights: pointer to the array of weights, one per control point [defaults = 1.0]

> NOTE: SetTXACFCOMPPeqCurve must be called with r=1 before weights take effect. All weights must satisfy: 0.0 < weight < 100.0.

```
void GetTXACFCOMPPeqDraw (int channel, double* X, double* Y)
```
channel: number of the channel being accessed
X: pointer to storage for the x-coordinates of the 1024 curve points. (frequency as a fraction of the Nyquist rate, e.g., if the sample-rate is 96000, the Nyquist rate is 48000 – multiply values by 48000 to get Hertz)
Y: pointer to storage for the y-coordinates (Post-EQ gain in dB) of the 1024 curve points

The returned (X, Y) pairs form a polyline for displaying the current Post-EQ curve. X values are in Hz (not normalized) and are NOT equally spaced; polyline plotting must be used.

```
void SetTXACFCOMPPrecomp (int channel, double precomp)
```
channel: number of the channel being accessed
precomp: an additional amount of compression to be applied equally across the frequency spectrum (dB) [default = 0.0]

```
void SetTXACFCOMPPeqRun (int channel, int run)
```
channel: number of the channel being accessed
run: Turn OFF/ON the post-CFC Equalizer: 0 = OFF; 1 = ON [default = 0]

```
void SetTXACFCOMPPrePeq (int channel, double prepeq)
```
channel: number of the channel being accessed
prepeq: an additional amount of gain to be applied equally across the frequency spectrum in the post-CFC equalizer (dB) [default = 0.0]

If it is desired to display the compression depth versus frequency, a function has been provided to supply that information. This function polls the DSP for new data; if data is available it is decimated to always return exactly 1025 values covering DC to fs/2, regardless of the internal FFT size.

```
void GetTXACFCOMPDisplayCompression (int channel, double* comp_values, int* ready)
```
channel: number of the channel being accessed

comp_values: pointer to storage for 1025 compression values (in dB), one per display bin. The internal FFT may have more bins than this; the values are decimated to always return exactly 1025 values covering the range from DC to fs/2.

ready: pointer to storage for a flag indicating whether or not new values are available since the last call was made to this function. If no new values are available, there would be no reason to re-draw the compression versus frequency display.

6.3.12 Bandpass Filters

There are three bandpass filters:

- The first of these, Bandpass Filter 0, is always ON. It "squares up" the pass-band and converts the audio to a complex I-Q signal. This is the primary bandpass filter.
- The second, Bandpass Filter 1, is used if and only if the Speech Processor is used. The speech processor is a non-linear device and therefore creates out-of-band frequency components. This bandpass filter removes those components. This filter is also a prerequisite for the CESSB Overshoot Control. The ON/OFF state of this filter is automatically set inside WDSP.
- The third, Bandpass Filter 2, is used if and only if the CESSB Overshoot Control is activated and serves as the final overshoot correction step for this algorithm. The ON/OFF state is automatically set inside WDSP.

For AM, AM_LSB, AM_USB, FM, and DSB modes, it is expected that the filter frequencies will be set by the console to cover an equal-sided range, e.g., f_low = -5000.0 and f_high = +5000.0. Any other necessary adjustments are handled internally in the WDSP code.

For other single-sided modes, such as LSB and USB, filter frequencies should be set as desired for the mode. For example, for LSB, f_low = -3000.0 and f_high = -200.0.

The "sharpness" of these filters, when running TXA at a dsp_rate of 96000, is shown in the following two figures. The first of these figures shows the response curves for different impulse response lengths, all using the 4-term Blackman-Harris window. The second figure is the same except using the 7-term Blackman-Harris window. The 4-term yields slightly sharper cutoff; however, less cutoff depth. The 4-term would be preferred in most situations since its cutoff depth is generally more than sufficient; the 7-term is overkill on cutoff depth for most situations.

> Note that the transition bands (regions between the pass-band and the cutoff bands) are the most important part to observe in these figures. These transition bands will maintain the same width independent of the width of the filter pass-band.

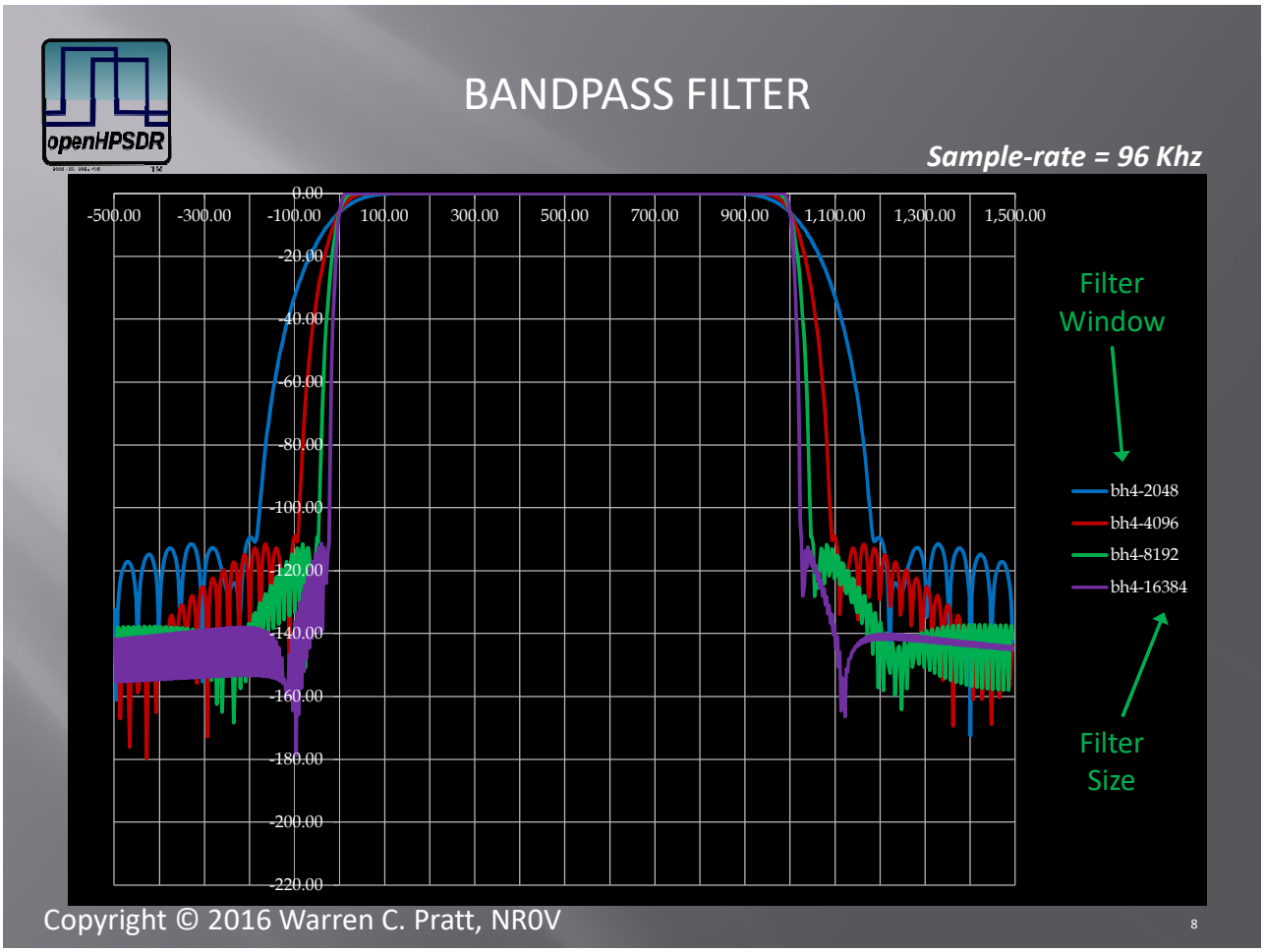


Figure 27: TXA Bandpass Filter frequency response, 4-term Blackman-Harris window, 96 kHz sample rate.

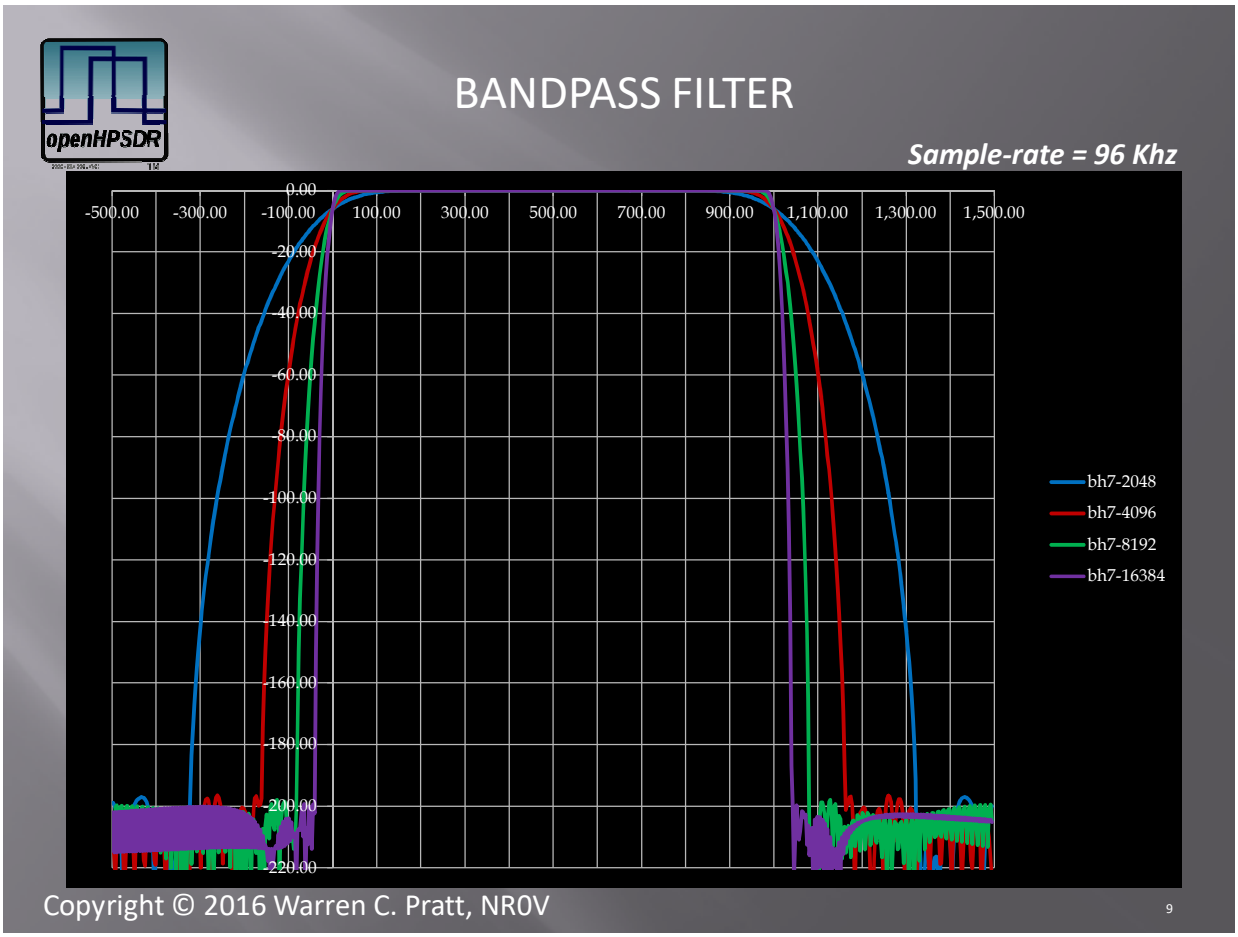


Figure 28: TXA Bandpass Filter frequency response, 7-term Blackman-Harris window, 96 kHz sample rate.

The lower and upper frequency cutoffs for the three bandpass filters are set using the call:

```
void SetTXABandpassFreqs (int channel, double f_low, double f_high)
```

channel: number of the channel being accessed
f_low: low cutoff frequency (Hz). May be either positive or negative. Must be less (more negative) than f_high.[default = -5000.0 Hz]
f_high: high cutoff frequency (Hz). May be either positive or negative. Must be greater (more positive) than f_low.[default = -100.0 Hz]

The window function to be used in generating the bandpass filters is set using:

```
void SetTXABandpassWindow (int channel, int wintype)
```

channel: number of the channel being accessed
wintype: 0 = Blackman-Harris 4-term; 1 = Blackman-Harris 7-term

```
void SetTXABandpassNC (int channel, int nc)  – DEPRECATED
```

channel: number of the channel being accessed
nc: length of the filter impulse response. Must be a power of two and must be greater than or equal to the DSP buffer size in use.[default = 2048]

```
void SetTXABandpassMP (int channel, int mp)  – DEPRECATED
```

channel: number of the channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency (Minimum Phase)

6.3.13 Speech Processor

The Speech Processor is the equivalent of an RF Speech Clipper; however, implemented at baseband rather than at RF frequencies.

> Note that the specified compression level is accurate only when the input signal is achieving a peak magnitude of exactly 1.0. At lower peak input levels, less compression will be achieved and at higher peak input levels more compression will be achieved. As previously discussed (see "Leveler"), the Leveler can provide assistance in regulating the input level. Whether the Leveler is used or not, the metering functionality provides excellent feedback to the operator concerning the signal levels in the audio chain.

The Speech Processor is turned OFF/ON using the following call:

```
void SetTXACompressorRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

The compression level is set using:

```
void SetTXACompressorGain (int channel, double gain)
```
channel: number of the channel being accessed
gain: compression level in dB.[default = 3.0 dB]

6.3.14 CESSB Overshoot Control

The concept of CESSB (Controlled Envelope Single Sideband) was provided to the Amateur Radio community by Dave Hershberger, W9GR.[2] In brief, this is an algorithm to increase the average transmitted power of an SSB signal while introducing only minimal distortion. To understand more detail, please see Dave's article.

Only one control is needed, the OFF/ON control:

```
void SetTXAosctrlRun (int channel, int run)
```

channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

HOWEVER, note that it is the responsibility of the console application to ensure that the Speech Processor is turned ON as a prerequisite for this block! This block will NOT operate properly in the absence of the Speech Processor also being active!

> Note that for optimum performance, Linear Phase bandpass filters should be used with this feature. The amount of performance degradation when using Low Latency filters has not been measured and will vary with voice characteristics.

[2] David L. Hershberger, W9GR, "Controlled Envelope Single Sideband," QEX Magazine, November/December 2014.

6.3.15 ALC

Unlike conventional radios, the Automatic Level Control function for TXA is accomplished in a single block; it does not include any kind of feedback from later blocks in the pipeline. ALC systems with feedback loops generally "drive by looking in the rear-view mirror," i.e., they react to an over-drive event in a later stage by reducing the gain in an earlier stage. However, that gain reduction is after-the-fact ... the over-drive and initial distortion have already occurred. The ALC system in TXA contains a short delay line which allows it to "see" events coming on the horizon before it has to provide output for them. Therefore, it can gradually reduce the gain along an exponential curve BEFORE the event happens rather than after.

In a Digital Up-Conversion (DUC) SDR, it is EXTREMELY important to insure that peak magnitudes NEVER exceed a specific level. Even though larger values would be no problem in WDSP floating-point calculations, they would overflow the (finite-range, integer-input) DAC, and, perhaps the integer calculations preceding the DAC. These overflows produce extreme distortion and very broadband splatter. The WDSP ALC system is designed to avoid this happening. For this reason, the ALC should ALWAYS be active.

Also, per this type ALC action, there is no ALC overshoot produced by this system.

For testing purposes, the ALC can be turned OFF/ON using the following call. HOWEVER, per the discussion above, it is expected that the ALC will ALWAYS BE ON.

```
void SetTXAALCSt (int channel, int state)
```
channel: number of the channel being accessed
state: 0 = OFF; 1 = ON[default = 1]

The ALC Attack time-constant can be set using the following call. It is recommended to leave this at the default value.

```
void SetTXAALCAttack (int channel, int attack)
```
channel: number of the channel being accessed
attack: attack time-constant (ms). Note that this is a time-constant for an exponential curve, not an absolute time.[default = 1 ms]

The ALC Decay time-constant is set using the following call.

```
void SetTXAALCDecay (int channel, int decay)
```
channel: number of the channel being accessed
decay: decay time-constant (ms). Note that this is a time-constant for an exponential curve, not an absolute time.[default = 10 ms]

A maximum gain for the ALC block is specified with the following call. This is "maximum gain" since the ALC will reduce the gain (compress) as needed to deliver the targeted output level.

```
void SetTXAALCMaxGain (int channel, double maxgain)
```
channel: number of the channel being accessed
maxgain: maximum gain (dB)[default = 0dB]

6.3.16 AM Modulator

The AM Modulator is automatically activated/deactivated based upon the **SetTXAMode(...)** setting. (See the section on "TXA Collectives & General Controls.")

The only setting for the AM Modulator is the Carrier Level setting:

```
void SetTXAAMCarrierLevel (int channel, double c_level)
```

channel: number of the channel being accessed
c_level: 0.0 - NO CARRIER (DSB); 0.5 - 100% modulation; >0.5 - less than 100% modulation. [default = 0.5]

> Note that the carrier level setting applies to the modes AM_LSB and AM_USB as well as the AM mode.

6.3.17 FM Modulator

The FM Modulator is automatically activated/deactivated based upon the **SetTXAMode(...)** setting. (See the section on "TXA Collectives & General Controls.")

The peak deviation for the FM signal can be set using the call:

```
void SetTXAFMDeviation (int channel, double deviation)
```
channel: number of the channel being accessed
deviation: peak deviation of the FM modulation (Hz).[default = 5000.0]

A CTCSS tone can optionally be transmitted on the FM signal. There are controls to turn the tone OFF/ON and to set its frequency.

```
void SetTXACTCSSRun (int channel, int run)
```
channel: number of the channel being accessed
run: 0 = tone OFF; 1 = tone ON.[default = 1]

```
void SetTXACTCSSFreq (int channel, double freq)
```
channel: number of the channel being accessed
freq: tone frequency (Hz).[default = 100.0]

The highest audio modulating frequency for the FM mode is 3000 Hz. Carson's Rule states that about 98% of the power of the resulting FM signal is contained within a bandwidth ranging from minus(deviation+highest_audio_frequency) to plus(deviation+highest_audio_frequency). While, at first glance, that may seem significant, that means that the power outside this range is only 17 dB below the power within the range. Therefore, the FM Modulator includes an output bandpass filter that limits the transmitted power to be within this specified range.

```
void SetTXAFMNC (int channel, int nc)  – DEPRECATED
```
channel: number of the channel being accessed

```
void SetTXAFMMP (int channel, int mp)  – DEPRECATED
```
channel: number of the channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency (Minimum Phase).

The normal audio frequency range for narrow-band FM communication is 300 Hz to 3000 Hz. This can be adjusted with the call:

```
void SetTXAFMAFFilter (int channel, double low, double high)
```
channel: number of the channel being accessed
low: low-cut frequency (Hz)
high: high-cut frequency (Hz)

However, when setting these frequencies to wider limits, it is also necessary to open-up other TX bandpass limits which can be done with a call to **SetTXABandpassFreqs (…).** Note also that when going lower in frequency, you'll may be overlapping the range in which CTCSS tones are transmitted. When going higher in frequency, you will be increasing your transmit bandwidth which, by Carson's Rule is equal to 2 * (maximum_deviation + highest_audio_frequency).

6.3.18 Up-Slew

> Note that the Up-Slew block is located after (1) the Signal Generators, (2) the AM Modulator, and (3) the FM Modulator. This block gradually increases the signal level from zero to maximum, along a raised-cosine curve, when one of these blocks is active and the TXA channel state transitions from OFF to ON.

In the case of SSB or DSB, the RF output is proportional to the audio input and there is no RF output until audio is present inside and processed through the TXA unit. The audio input to TXA must pass through the channel's initial up-slew. Hence, the RF output will in-turn up-slew also. However, with the three special cases mentioned above, signal is created internal to the TXA unit independently of audio being present. This signal requires an up-slew to avoid an abrupt transition, consequent "pop" in the audio, and consequent disturbance in the frequency spectrum. In the case of the signal generator, the signal referenced is whatever signal the generator is set to produce. In the case of the AM Modulator and FM Modulator, it is the carrier they generate whose up-slew must be controlled.

There is a control for the length of this Up-Slew. For AM and FM modes, it should be set to a value LESS THAN the Up-Slew specified for the TXA channel – this is so that the carrier will rise faster than the modulating audio.

```
void SetTXAuSlewTime (int channel, double time)
```

channel: identifier/number of the channel being accessed
time: length of the up-slew (seconds)

6.3.19 Siphon

In the block diagram for TXA, after the Siphon block, there are two blocks that can alter the time-domain and frequency-domain content of the signal: (1) PureSignal Predistortion, which, by definition, is intended to predistort the signal to compensate for later non-linearities in the processing chain, and (2) the CFIR which intentionally distorts the frequency response to compensate for frequency-response distortion in later CIC filters in FPGA firmware. It is often desirable to monitor the content of the signal BEFORE these intentional distortions are applied. The Siphon provides the data output for this monitoring. It "siphons" copies of the data from the TXA processing pipeline as the data flows through and either (1) sends the data directly to a WDSP display unit, or (2) makes it available to the console through calls that the console can make to the siphon.

> NOTE: Per the setting of an internal parameter, the number of contiguous samples being internally buffered is 4096. Therefore, retrievals of data are limited to this size.

```
void TXASetSipMode (int channel, int mode)
```
channel: number of the channel being accessed
mode: 0 = data is to be retained awaiting a retrieval call from the console; 1 = data is to be sent directly to a specified display unit.[default = 0]

```
void TXASetSipDisplay (int channel, int disp)
```
channel: number of the channel being accessed
disp: identifier of the display unit to which data is to be sent.[default = 0]

There are currently three calls the console can make to retrieve data. For historical reasons, at the time of this writing, all return 'float' data. At some future time, calls will likely be added to provide data in the 'double' type.

The following function returns a block of consecutive raw sample values (I-values only) in an array.

```
void TXAGetaSipF (int channel, float* out, int size)
```
channel: number of the channel being accessed
out: pointer to the array where the values are to be stored
size: number of values to be returned

The following function returns a block of consecutive raw sample values (complex interleaved I-Q samples) in an array.

```
void TXAGetaSipF1 (int channel, float* out, int size)
```
channel: number of the channel being accessed
out: pointer to the array where the interleaved I-Q values are to be stored
size: number of I-Q samples to be returned

The following call returns a set of values of the frequency-domain magnitudes to be used for a frequency-domain display. Values are in dBV, i.e., a sample magnitude of 1.0 yields an output value of 0.0 dB. Per the setting of an internal parameter, fftsize, 4096 magnitude values are returned; hence, there is no 'size' parameter.

```
void TXAGetSpecF1 (int channel, float* out)
```
channel: number of the channel being accessed

out: pointer to the array where the magnitude values are to be stored

There are two possible orderings in which the values from **TXAGetaSpecF1(...)** can be returned. The choice of which to return is determined by the setting of the internal variable specmode which is set by the following call.

```
void TXASetSipSpecmode (int channel, int mode)
```

channel: number of the channel being accessed

mode: 0 => the "Normal Order" of the FFTW[3] library output is re-arranged such that the two halves of the spectrum are swapped; 1 => the "Normal Order" of the FFTW[4] library output is re-arranged to mirror each half of the FFTW output in place.[default = 1]

[3] FFTW is the FFT library used for WDSP. See <www.fftw.org> for more information.

[4] FFTW is the FFT library used for WDSP. See <www.fftw.org> for more information.

6.3.20 PureSignal Adaptive Predistortion

PureSignal 3.0 (Released in WDSP 2.0)

PureSignal 3.0 is a major revision and step-forward for PureSignal Adaptive Predistortion. Most of the internal code has been re-written, informed from the learning experiences of previous versions. The key focus of this new release is on robustness of the algorithm. It often operates in an imperfect environment – an environment with very noisy feedback at low signal levels, sometimes gaps in the returned sample stream across the network, feedback data errors which could cause correction errors to propagate, and sometimes dramatically over-driven amplifier chains.

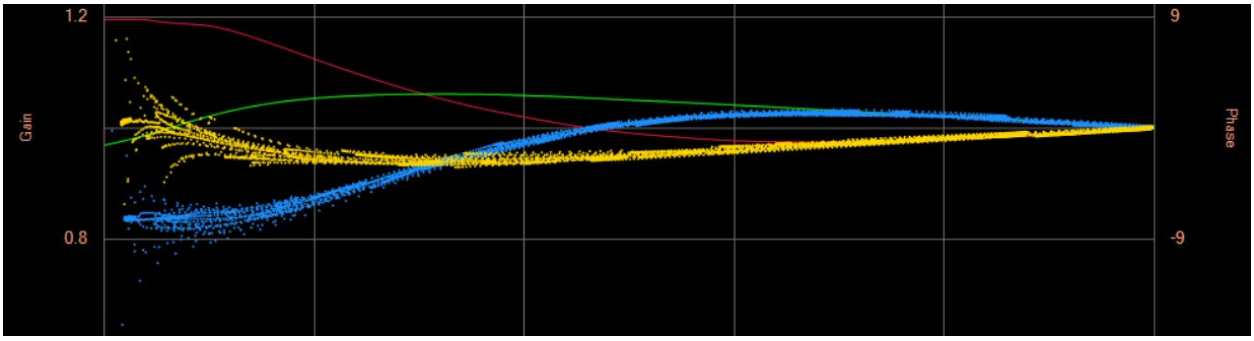


Figure 29: AmpView of amplifier with significant memory effects and a system with noisy data return. Blue – Magnitude input; Yellow – Phase input; Red – PureSignal magnitude correction; Green – PureSignal phase correction.

With an eye toward increasing tolerance and performing in difficult environments, PureSignal 3.0 was created. While the internal algorithms have changed very significantly, the functions to call to control and monitor are almost entirely the same. One exception is a couple added parameters in **GetPSDisp()** that assist in setting up and using the AmpView display.

The previous PureSignal internal algorithms did least-squares fits of cubic splines for gain and phase to the "inverse" of collected feedback samples. This works well; however, those curves have their limitations. PureSignal 3.0 begins by least-squares fitting of NURBS (Non-uniform Rational B-Splines) to the relevant data. Taking advantage of some inherent properties of NURBS and combined with other algorithms such as LOWESS estimation, this solution can natively handle things like near output power flattening and fold-down in extremely over-driven cases, very noisy input data at low signal levels and uncertain peak levels. Once properly created and checked for quality, the NURBS curves are converted to equivalent simpler splines for fast evaluation in the real-time correction path.

The downside: Most good enhancements require more CPU cycles. Fortunately, CPUs are getting a lot faster! However, the other good news is that this doesn't require more cycles on the real-time path, the extra cycles are for calibration which proceeds at its own pace. The 'calibration thread' can be throttled per your own system needs.

A Word about Overdriven Amplifiers

While PureSignal attempts to be very tolerant of the way you are operating your amplifier, there are practical limitations. The more you over-drive your amplifier into compression, the less data PureSignal can extract for correction! For the best operation and correction, you should drive your amplifier at or near the appropriate level. PureSignal measures the percentage of normal total data that is available for calibration at highest signal levels. At the time of PureSignal 3.0 release, it is set to calibrate with as little as 6% of the data that it would ideally get in the upper amplitude range. It will refuse to calibrate at less than that because the calibration might not achieve a reasonable level of accuracy. Alternatively to outright refusal, it may turn ON and OFF as it searches. An indicator of over-drive is available in the feedback from PureSignal; that indicator is discussed below. The bottom line: For best results, operate your amplifier correctly!

PureSignal – The Basics

The following figure displays the operation of PureSignal in the complete SDR environment including software, FPGA firmware, and hardware. The software block "Apply Correction" is where the I-Q signal is modified, applying amplitude and phase correction. The block "Compare Signals & Calculate Correction" executes asynchronously with the dataflow through the TXA unit and uses as its input synchronized samples from the amplifier's input and the amplifier's output. The fact that two sample streams (amplifier input and amplifier output) are required, and the fact that they must be synchronous (each output sample must correspond to the input sample that drove the amplifier to that output), means that some mechanism other than normal channel input and dataflow must be used to provide these samples since the channel structure only supports input of a single sample stream.

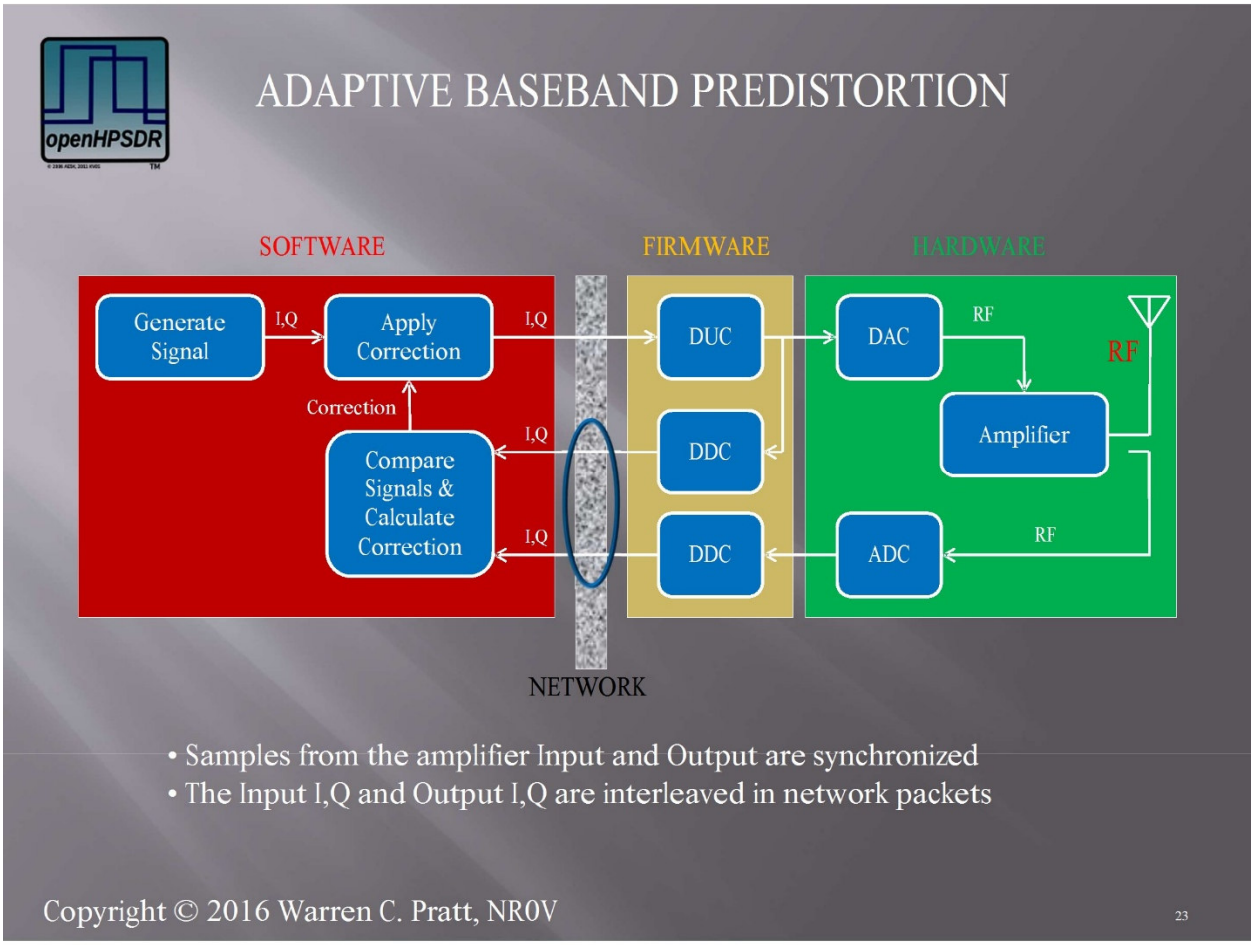


Figure 30: PureSignal adaptive predistortion — SDR transmit/receive feedback loop for PA linearization.

Data Input

Buffers of the two sample streams are provided as input data using the following call.

```
void pscc (int channel, int size, double* tx, double* rx)
```

channel: number of the TXA channel being accessed
size: number of I-Q samples provided for each sample stream
tx: pointer to buffer containing the output-bound sample stream, i.e., the sample stream to drive the amplifier input
rx: pointer to buffer containing the sample stream received from the amplifier's output

Control

Considerable information about the state of affairs in PureSignal can be obtained using the following call. This information is often useful in software/firmware debugging and in understanding issues with specific hardware configurations. Some of the information is also used for normal monitoring, for example monitoring of the amplifier output feedback level.

```
void GetPSInfo (int channel, int* info)
```

channel: number of the channel being accessed

info: pointer to an array to hold 16 integer values. Assignment of these 16 values is as follows:

- 0 - status of correction curve builder for rx_scale
- 1 - status of correction curve builder for magnitude
- 2 - status of correction curve builder for cos(phase)
- 3 - status of correction curve builder for sin(phase)
- 4 - amplifier output feedback level
- 5 - count of completed calibrations
- 6 – solution checks – Including Over-Drive (see Note below)
- 7 – count of attempted calibrations
- . . . . . UNUSED . . . . .
- 14 - indicates the state-machine says that iqcRun = 1, i.e., the state-machine says that correction values are being applied
- 15 – 'state' of the control state-machine

> NOTE: An info[6] value of '2' indicates that the operator is likely severely over-driving the amplifier. As discussed above, PureSignal will refuse to calibrate (or remain in calibration) under these conditions. Detection of info[6]=2 may warrant a visible message/warning to the operator of the form "WARNING – PROBABLE SEVERE OVER-DRIVE. CHECK YOUR DRIVE LEVEL!"

Data collection and state machine operation is turned OFF/ON by the following call. Nothing happens unless run is set to 1. Note that the default is 1 and there are other ways to turn OFF/ON PureSignal operation; hence, this call may be unnecessary in your application.

```
void SetPSRunCal (int channel, int run)
```

channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 1]

The sample-rate of the two streams provided to **pscc(...)** must be set using the following call.

```
void SetPSFeedbackRate (int channel, int rate)
```

channel: number of the channel being accessed
rate: the sample rate (samples / second)

The primary mode of operation is set by the following call. This call provides four control variables that affect the progression of the control state-machine. Note that often the call is made with only one of the last four parameters as '1' and the other three as zero. An example of where multiple parameters may be set to '1' would be in the event that Automatic Calibration Mode is active but we want to change to Single (Manual) Calibration mode. In that case, we can make the call **SetPSControl**(channel, 1, 1, 0, 0); . This call will reset PureSignal, turning off correction. It will also specify the Single/Manual Calibration mode and turn OFF the Automatic Calibration Mode.

```
void SetPSControl (int channel, int reset, int mancal, int automode, int
turnon)
```
channel: number of the channel being accessed
reset: 0 = PS is free to operate as directed; 1 = turn-off PS and reset the control state-machine. To turn OFF PureSignal calibration and correction, call with parameters (1, 0, 0, 0).
mancal: Call with parameters (0, 1, 0, 0) to perform a single calibration run and then turn-on correction using that calibration. In this mode, the algorithm is not "adaptive"; it just continues to use this same correction.
automode: Call with parameters (0, 0, 1, 0) to turn on auto-calibration mode and correction. In this mode, the algorithm continues looping and re-calibrating, i.e., it is adaptive.
turnon: Call with parameters (0, 0, 0, 1) to tell the internal state-machine to continue indefinitely to use whatever correction has been loaded into the correction buffers. This variable is set in situations such as restoring calibration information from a file.

Calibration values can be saved in files and then restored to active correction. These calls save the current calibration and then restore that to active correction.

```
void PSSaveCorr (int channel, char* filename)
```
channel: number of the channel being accessed
filename: the file to which the calibration is to be stored

```
void PSRestoreCorr (int channel, char* filename)
```
channel: number of the channel being accessed
filename: the file from which the calibration is to be restored

When in automatic-calibration mode, it is possible to add some delay in the re-calibration (adaptation) loop. The benefit would be to slightly reduce CPU utilization. Such delay can be added using the following call.

```
void SetPSLoopDelay (int channel, double delay)
```
channel: number of the channel being accessed
delay: delay to insert after each calibration (seconds)[default = 0.0]

Valid data for calibration can only be collected during transmit, i.e., when MOX is active. The following call conveys the state of MOX to PureSignal.

> NOTE: SetPSMox should set 'mox = 0' BEFORE setting the TXA Channel State to zero/OFF. This will allow certain internal operations to complete during the slew-down of the TXA channel.

```
void SetPSMox (int channel, int mox)
```
channel: number of the channel being accessed
mox: 0 = not transmitting; 1 = transmitting

After MOX goes true, there is some delay required before valid samples are flowing back to the calibration functions. This delay can be adjusted, if desired, using the following call.

```
void SetPSMoxDelay (int channel, double delay)
```
channel: number of the channel being accessed
delay: delay to insert before beginning a calibration and after MOX is detected as being active (seconds)[default = 0.1]

There is some delay between the two sample streams that must be compared to compute the calibration. This delay is partially due to time delay through the DAC and ADC; however, most of it is

due to delay through the analog amplifier circuitry. This delay must be compensated in software to achieve optimum correction. For a solid-state amplifier, it will generally be in the range of 100ns to 1us and will depend upon the band of operation. The desired value of software delay compensation must be provided to the PureSignal software. That is done via the following call.

```
double SetPSTXDelay (int channel, double delay)
```
return value: the actual delay the PureSignal provides. The exact delay requested may not be available; so, the actual delay provided is returned for comparison.
channel: number of the channel being accessed
delay: desired delay in seconds

The peak magnitude value of the *returned* outbound sample stream (samples returned in the tx buffer for the **pscc(...)** call), i.e., the value when the samples have a value of 1.0 when leaving the TXA software, is a key parameter in computing the correction solution. This value will depend upon scaling in the DUC and DDC firmware/software. The following are calls to "set" this value, meaning to tell the PureSignal software what it is, and to "get" this value which is useful when the console needs to know the default value saved in TXA.

```
void SetPSHWPeak (int channel, double peak)
```
channel: number of the channel being accessed
peak: peak magnitude value[default = 0.4072] Note: This is the value for the openHPSDR "old protocol" firmware.

```
void GetPSHWPeak (int channel, double* peak)
```
channel: number of the channel being accessed
peak: pointer to location to store the peak magnitude value

When testing new DUC/DDC firmware or software, it is also useful to be able to detect the measured peak magnitude. That is done using the following call. You may wish to call this repeatedly and display the result as it will increase until the maximum is found.

```
void GetPSMaxTX (int channel, double* maxtx)
```
channel: number of the channel being accessed
maxtx: pointer to location to store the maximum measured peak magnitude value

Data points can be retrieved from PureSignal to produce a real-time display of the amplifier gain and phase curves (versus signal magnitude) and of the gain and phase correction curves that are being applied. This is done for the AmpView display available in the Thetis console application. This is accomplished with the following call.

```
void GetPSDisp (int channel, double* x, double* ym, double* yc, double* ys,
double* xm_cor, double* ym_cor, double* xa_cor, double* ya_cor, int*
nsamps_out, int* cpts_out, double* phs_ref_deg_out)
```
channel: number of the channel being accessed
x: pointer to vector for magnitude value of the driving signal (0.0 to 1.0 range). These points are used as reference points to plot ym, yc, and ys data.
ym: pointer to vector for amplifier output magnitude
yc: pointer to vector for amplifier output cos (phase)
ys: pointer to vector for amplifier output sin (phase)
xm_cor: pointer to vector for x-values of magnitude correction curve

ym_cor: pointer to vector for y-values of magnitude correction curve
xa_cor: pointer to vector for x-values of angle correction curve
ya_cor: pointer to vector for y-values of angle correction curve [degrees]
nsamps_out: number of point values in x, ym, yc, and ys. Variable, currently 4096.
cpts_out: number of point values in xm_cor, ym_cor, xa_cor, ya_cor. Currently 512.
phs_ref_deg_out: phase reference/offset for plotting phase of points of driving signal.

> NOTE: There were certain additional controls that were exposed in the PureSignal implementation in Thetis. These were things such as PIN, MAP, and TINT. Having these adjustable was for experimentation purposes. They were not documented in the "WDSP Guide". These controls are non-existent in the new PureSignal implementation – there simply is nothing there of that sort to adjust. If these were somehow included in your specific implementation, you should remove them.

6.3.21 CFIR Filter

A Compensating FIR (CFIR) filter is generally used in SDR situations where Cascaded Integrator Comb (CIC) filters are used in the Digital Up-Conversion (DUC) or Digital Down-Conversion (DDC) process. CIC filters are commonly used when the DUC or DDC is done in an FPGA as they require very few gates to implement. The CFIR is needed because the CIC filter has a downside that it does not have a flat passband. The CFIR offsets the "droop" in the CIC passband producing a flat frequency response for the combination of the CIC and CFIR. The CFIR can be implemented in either the FPGA or in software. A software implementation is provided here.

> Note: For the openHPSDR "USB Protocol"/"Old Protocol", this software function is not used in TXA as it is performed in the FPGA firmware. However, for the openHPSDR "New Protocol", it is used in TXA.
>
> NOTE: At this point, the only call provided for this CFIR in TXA is to turn it OFF/ON. However, there are a number of internal parameters used to specify the shape and characteristics of the filter. These parameters can be made externally available if we have different compensation needs in the future. The internal parameters are currently set to compensate for the CIC filters in the FPGA firmware of the openHPSDR "New Protocol". Key parameters are as follows:

- Input Sample Rate to this filter: dsp_rate (channel parameter)
- CIC Input Sample Rate: output_samplerate (channel parameter)
- CIC Differential Delay: 1
- CIC Interpolation Factor: 640
- CIC Integrator-Comb Pairs: 5

The following call is used to turn the CFIR OFF/ON.

```
void SetTXACFIRRun (int channel, int run)
```

channel: number of the channel being accessed
run: 0 = OFF; 1 = ON[default = 0]

6.3.22 TXA Collectives & General Controls

The following call sets the transmission MODE for TXA. This single call turns OFF/ON and provides certain required settings to blocks such as the AM and FM modulators.

```
void SetTXAMode (int channel, int mode)
```

channel: number of the channel being accessed
mode:
0 = LSB;
1 = USB;
2 = DSB;
5 = FM;
6 = AM;
7 = DIGU;
8 = SPEC;
9 = DIGL;
12 = AM_LSB;
13 = AM_USB.
[default = 0]

The following collective is used to set the length of filter impulse responses within TXA. This is the PREFERRED method, compared to setting the filters individually. Note that when these filters were created, their 'nc' was set to "max (2048, dsp_size)". A minimum of 2048 is recommended; although, 1024 may be acceptable in some cases.

```
void TXASetNC (int channel, int nc)
```

channel: number of the channel being accessed
nc: length of filter impulse responses. This must be a power-of-two and must be greater than or equal to the dsp_size currently in use.

The following collective is used to choose Linear Phase versus Low Latency for filters within TXA. This is the PREFERRED method, compared to setting the filters individually.

```
void TXASetMP (int channel, int mp)
```

channel: number of the channel being accessed
mp: 0 = Linear Phase; 1 = Low Latency (Minimum Phase).

7 Panadapter & Other Frequency-Domain Displays

WDSP contains functionality to accept raw wide-bandwidth time-domain samples and, from them, repetitively create the data ("pixel values") for panadapter displays and other frequency-domain displays. This full-featured implementation includes support for:

- combining or interpolating FFT output bins, as needed, to create a requested number of pixel values across the screen,
- symmetrical and asymmetrical clipping to support pan and zoom functionality,
- multiple averaging modes,
- multiple detectors (peak, average, rosenfell, and sample),
- normalization to a one Hertz bandwidth for noise measurements,
- multiple pixel outputs for the same input data set, for example, the panadapter and waterfall can be using different averaging modes,
- software spur elimination for spectrum analyzer applications,
- stitched display for viewing a spectrum exceeding the bandwidth of a single sample stream,
- both real and complex input samples,
- a selection of window functions used to window the input data, and
- curve-fitted amplitude calibration facilities.

Instances of such "Displays" are not created within RXA or TXA. These Displays must be separately instantiated. A typical scenario would be to create one Display for each RXA and for each TXA channel and also perhaps one Display for each ADC such that a view can simultaneously be provided for the entire frequency spectrum digitized by the ADC. However, if computing resources are constrained, it is also possible to create less Displays and share them among multiple views. For example, a single Display could be shared between being a panadapter during receive and displaying the frequency spectrum of the out-going transmitted signal during transmit.

The display code was originally written for a different project with different objectives and was then later included within WDSP. Therefore, there are some style differences between this Display code and the rest of WDSP. Expect that this code is likely to become more similar to the rest of WDSP over time, as development time permits.

The Display unit receives calls to set various parameters, to provide it with input data, and to retrieve "pixel values" from it. These calls to a Display unit may be made from either within an RXA or TXA channel or from outside WDSP entirely, depending upon the needs. For example, one of the supported features of the Display is to "stitch" multiple sub-spectra together to create a view of a wider received spectrum. However, a WDSP channel only supports the receipt of one continuous sample stream. Since stitching requires multiple continuous sample streams, calls to supply the input data to the Display would generally be made outside of the WDSP channel. On the other hand, for a view of the Spectrum data after the bandpass filter, the call to supply input data to the Display would need to be made within the WDSP channel.

7.1 Creating and Destroying a Display

Creating a display unit consists of allocating all the resources it requires. These resources are a function of things like the maximum FFT size to be used, the maximum number of LO positions to be used for spur elimination (a spectrum analyzer feature), and the maximum number of sub-spans to be stitched together to form the complete spectrum. A Display is created using the call:

```
void XCreateAnalyzer (int disp, int* success, int m_size, int m_LO, int
m_stitch, char* app_data_path)
```

disp: number/identifier of the Display being created. This value currently ranges from 0 through 63. If one display is to be created per DSP channel, it may prove convenient to have the 'disp' value match the channel identifier.

success: a pointer to a return value that will be zero if the display has been successfully created.

m_size: the maximum FFT size to be used. The WDSP Wisdom calculations must include any selected values and currently calculate power-of-two values through 262144. This value must be a power-of-two.

m_LO: the maximum number of Local Oscillator positions to be used for spur elimination. This is a spectrum analyzer feature that provides comparison of spectra produced from multiple LO frequencies to automatically eliminate spurious responses. For non-spectrum-analyzer use, set this to 1.

m_stitch: the maximum number of sub-spans that will be stitched together to create a single spectrum to be displayed. For example, if no more than three firmware receiver sample streams are to be used to create a single display, set this to 3.

app_data_path: not used.

When the display is no longer needed, for example when the host application is preparing to close, its resources are freed using this call:

```
void DestroyAnalyzer (int disp)
```

disp: the identifier of the Display whose resources are to be freed.

7.2 Setting Display Parameters

The SetAnalyzer(...) call is used to set many of the parameters for the display unit.

```
void SetAnalyzer (int disp, int n_pixout, int n_fft, int typ, int* flp, int
sz, int bf_sz, int win_type, double pi, int ovrlp, int clp, double fscLin,
double fscHin, int n_pix, int n_stch, int calset, double fmin, double fmax,
int max_w)
```

disp: the identifier of the Display whose parameters are to be set.

n_pixout: the number of pixel outputs being used. For example, this could be 1 if the same pixel-value data is to be used for the panadapter and waterfall; or, it could be 2 if different detectors or averaging modes are desired for the panadapter and waterfall.

n_fft: the number of LO positions to be used for spur elimination (a spectrum analyzer feature). This should be set to 1 if spur elimination is not being used.

typ: specifies whether REAL or COMPLEX sample data is to be provided. 0 for REAL which must be delivered in the 'I' channel; 1 for COMPLEX which will be delivered in I and Q channels.

flp: pointer to a vector with one element per LO frequency. The elements are used to specify whether the LO frequencies are on the low side or high side of the operating range. For non-SA use, this will be a single element with a value of 0. For example:

```
int[] flp = { 0 };
```

sz: size of the FFT to be used. This must be a power of two.

bf_sz: number of samples to be provided each time a call to provide input samples is made.

win_type: is used to specify the type of window function to be used for processing. Beginning with 0, assignments are: Rectangular, 4-term Blackman-Harris, Hann, Flat-top, Hamming, Kaiser, 7-term Blackman-Harris. For general purpose use, 5 (Kaiser) and 6 (7-term Blackman-Harris) are good choices. For best amplitude accuracy, use 3 (Flat-top).

pi: parameter used to shape the Kaiser window function. 14.0 is a reasonable default value.

ovrlp: the number of samples to be re-used from the previous FFT. For SA use, overlap can reveal transient signals that may otherwise not be displayed. For general purpose radio use, overlap allows the display frame-rate to be independent of the sample-rate. For example, at an FFT size of 262144 and a sample rate of 192000, it takes over one second to capture enough samples for an entirely new display frame. However, by re-using some number of samples from the previous FFT, we can create new frames at a much faster rate and have an interactive display. Overlap can be calculated as:

```
//set overlap as needed to achieve the desired frame_rate
overlap = (int)Math.Max(0.0, Math.Ceiling(fft_size -
      (double)sample_rate / (double)frame_rate));
```

where 'frame_rate' is in frames/second.

clp: the number of FFT bins to be (symmetrically) clipped from each side of each sub-span. Preceding firmware decimation filters (in either FPGA firmware or Direct Fourier Conversion code) are not perfectly rectangular -- they have a roll-off on each side of the spectrum. It is generally not desirable to display the roll-off area, especially if stitching is to be used. A primary use of this capability is to clip off those bins. The value for 'clp' can generally be set by specifying some fraction of the spectrum to be clipped on each side. For example:

```
//fraction of the spectrum to clip off each side of each sub-span
const double CLIP_FRACTION = 0.017;
//clip is the number of bins to clip off each side of each sub-span
clp = (int)Math.Floor(CLIP_FRACTION * fft_size);
```

fscLin: the number of bins to clip from low side of the entire (stitched) span. This parameter can be used along with fscHin to PAN and ZOOM within the spectrum represented by the entire stitched span. Based upon PAN and ZOOM settings, these parameters can be set as follows:

```
//the amount of frequency in each fft bin is given by:
```

```
//   this is also equal to the interval width!
double bin_width = (double)sample_rate / (double)fft_size;

//the number of useable bins per subspan is
//   the '-1' is due to clipping the Nyquist bin
int bins_per_subspan = fft_size - 1 - 2 * clip;

//the amount of useable bandwidth we get from each subspan is:
//  we'd subtract '1' from 'bins_per_subspan' if we wanted the
//  interval_width_per_subspan
bw_per_subspan = bins_per_subspan * bin_width;

//the total number of bins available to display is:
int bins = stitches * bins_per_subspan;

//the number of intervals among all the bins equals 'bins - 1'
double intervals = (double)(bins - 1);

//apply log function to zoom slider value
double zoom_slider = Math.Log10(9.0 * z_slider + 1.0);

//limits how much you can zoom in; higher value means you zoom more
const double zoom_limit = 100;

//calculate the width in intervals after applying zoom
double width = intervals * (1.0 - (1.0 - 1.0 / zoom_limit) *
     zoom_slider);

//fscLin is 0 if pan_slider is 0; it's 'intervals - width' if
//pan_slider is 1
//fscHin is 'intervals - width' if pan_slider is 0; it's 0 if
//pan_slider is 1
double fscLin = pan_slider * (intervals - width);
double fscHin = intervals - width - span_clip_l;
```

fscHin: See above comments about fscLin.
n_pix: number of pixel values to be retrieved upon each call to retrieve pixels, i.e., the width, in pixels, of the window in which the results are to be displayed.
n_stch: number of sub-spans to be stitched. If only one DDC is to be used, this is 1. If, for example, the sample streams from 3 DDCs are to be stitched, this is 3.
calset: specifies which of multiple sets of calibration data is to be used to adjust signal amplitude across the range of the spectrum. If the calibration feature is not in use, set this value to 0.
fmin: the minimum frequency of the span (used for calibration). If calibration is not in use, set this value to 0.0.
fmax: the maximum frequency of the span (used for calibration). If calibration is not in use, set this value to 0.0.
max_w: a parameter used to determine how much data will be held in display buffers. It is calculated as:

```
const double KEEP_TIME = 0.1;
max_w = fft_size + (int)Math.Min(KEEP_TIME * sample_rate, KEEP_TIME *
            fft_size * frame_rate);
```

When the calibration feature is used, the following call is to set the calibration data. A table is to be provided with two columns, (0) frequency and (1) the calibration multipliers corresponding to those frequencies. The 'calibration multipliers' are NOT dB values to be added, they are linear multipliers. This table is interpolated using a cubic spline to create a smooth calibration curve.

```
void SetCalibration (int disp, int set_num, int n_points, double(* cal)[
dMAX_M+1])
```
disp: the identifier of the Display whose calibration is being set.
set_num: the identifier for this calibration data set.
n_points: the number of calibration points in the set; i.e., the number of rows in the following table.
cal: pointer to the table of calibration data; dMAX_M+1 should be 2.

The following call is used to select the detector-type for a particular Display and Pixel Output from that Display. The positive-peak detector is the best selection for measuring the amplitude of a signal. The average and sample detectors are provided specifically for noise measurements and the output is adjusted for the chosen window's equivalent noise bandwidth when using these two detectors. The rosenfell detector is common in spectrum analyzers and gives an indication of both signal and noise.

```
void SetDisplayDetectorMode (int disp, int pixout, int mode)
```
disp: identifier for the Display.
pixout: identifier of the pixel output for which the detector is being set.
mode: detector mode. 0 = positive peak; 1 = rosenfell; 2 = average; 3 = sample; 4 = rms (across fft bins within the pixel).

The following call is used to select the averaging-mode for a particular Display and Pixel Output from that Display.

```
void SetDisplayAverageMode (int disp, int pixout, int mode)
```
disp: identifier for the Display.
pixout: identifier of the pixel output for which the averaging mode is being set.
mode: averaging mode. -1 = peak-hold; 0 = no averaging; 1 = weighted averaging of linear data; 2 = time-window averaging of linear data; 3 = weighted averaging of log data. This last mode, weighted averaging of log data, looks nice on the panadapter log scale; however, it is not mathematically accurate for time-varying signals or noise measurements.

When time-window averaging of data is used, the number of display frames to average needs to be specified. The following call performs that operation.

```
void SetDisplayNumAverage (int disp, int pixout, int num)
```
disp: identifier for the Display.
pixout: identifier of the pixel output for which the value is being set.
num: number of frames to average.

When weighted averaging is used, a multiplier value must be supplied. Averaging then proceeds as: new_display_value = mult * old_display_value + (1.0 - mult) * new_pixel_value;

```
void SetDisplayAvBackmult (int disp, int pixout, double mult)
```
disp: identifier for the Display.
pixout: identifier of the pixel output for which the value is being set.
mult: multiplier value.

The display code must know the sample-rate of the input data to be able to normalize measurements to a 1.0 Hz bandwidth, when desired. The bandwidth is set by:

```
void SetDisplaySampleRate (int disp, int rate)
```
disp: identifier for the Display.
rate: sample-rate (samples per second).

Pixel values will be normalized to a 1.0 Hz bandwidth if 'norm' is set to 1.

```
void SetDisplayNormOneHz (int disp, int pixout, int norm)
```

disp: identifier for the Display.
pixout: identifier of the pixel output for which the parameter is being set.
norm: 0 = do not normalize; 1 = normalize to one Hz bandwidth.

7.3 Supplying Input Data

Several input functions have been developed over time to meet varying console needs.

When I and Q are kept in separate buffers, as opposed to being interleaved in a single buffer, use the Spectrum(...) call. Note that the type of the I/Q data is specified as **dINREAL** which is defined as either **double** or **float** in the file "comm.h".

```
void Spectrum (int disp, int ss, int LO, dINREAL* pI, dINREAL* pQ)
```

disp: identifier for the Display.
ss: sub-span (of a stitched display) for which data is being supplied. If stitching is not being used, ss = 0.
LO: LO position for which data is being supplied. (for Spectrum Analyzer software spur removal) If spur removal is not being used, LO = 0.
pI: pointer to the array of I values
pQ: pointer to array of Q values

When I and Q are interleaved in a single buffer, one generally uses the Spectrum2(...) call. Note that the type of the I/Q data is specified as **dINREAL** which is defined as either **double** or **float** in the file "comm.h".

```
void Spectrum2 (int run, int disp, int ss, int LO, dINREAL* pbuff)
```

run: 0 = ignore input data and return; 1 = copy input data from pbuff and process.
disp: identifier for the Display.
ss: sub-span (of a stitched display) for which data is being supplied. If stitching is not being used, ss = 0.
LO: LO position for which data is being supplied. (for Spectrum Analyzer software spur removal) If spur removal is not being used, LO = 0.
pbuff: pointer to the array of interleaved I/Q values

In the event interleaved I/Q data is to be supplied as **double**, but because float data is at times being supplied from elsewhere **dINREAL** is defined as **float**, the Spectrum0(...) call may be used.

```
void Spectrum0 (int run, int disp, int ss, int LO, double* pbuff)
```

run: 0 = ignore input data and return; 1 = copy input data from pbuff and process.
disp: identifier for the Display.
ss: sub-span (of a stitched display) for which data is being supplied. If stitching is not being used, ss = 0.
LO: LO position for which data is being supplied. (for Spectrum Analyzer software spur removal) If spur removal is not being used, LO = 0.
pbuff: pointer to the array of interleaved I/Q values

7.4 Retrieving Pixel Values

As previously discussed, new frames of data are calculated periodically based upon several settings. Pixel values should be periodically retrieved, at this same desired frame-rate, using GetPixels(...). If new data is available (indicated by flag = 1), it should be drawn. If not, check again later -- nothing new to draw now.

```
void GetPixels (int disp, int pixout, dOUTREAL* pix, int* flag)
```

disp: identifier for the Display.
pixout: identifier of the pixel output from which pixel values are to be retrieved.
pix: pointer to buffer into which the pixel values (dBm) will be placed.
flag: pointer to an integer that will be set to 1 if new data was available and will be set to 0 if no new data was available.

7.5 Clearing the Display Pipeline

If it is desired to clear the in-process data from the display pipeline, the following call can be used. This might be used, for example, on a TX/RX transition for a continuously running display.

```
void ResetPixelBuffers (int disp)
```

disp: identifier for the Display.

7.6 Retrieving Maximum Bin Value (dB) Within a Frequency Range

A frequency range, relative to the center_frequency of the DDC driving the display, can be specified and the maximum value of any display FFT bin within that range will be returned, in dB. Note that if adjustments are normally applied to display 'pixel' values, those same adjustments should be applied to this value.

This functionality must be declared for each 'disp' for which the functionality is desired. Call this function for initial setup AND anytime one of the specified parameters changes.

```
void SetupDetectMaxBin (int run, int disp, int ss, int LO, double rate,
double fLow, double fHigh, double tau, int frame_rate)
```

run: Set to '1' if this feature is in use, '0' otherwise. Note that CPU cycles can be saved by setting this to '0' when not in use.
disp: identifier for the Display.
ss: sub-span. Set to 0 if stitching is not in use. Note that only one ss/LO pair is permitted, per display, at a time.
LO: LO position. Set to 0 if spur-removal is not in use. Note that only one ss/LO pair is permitted, per display, at a time.
rate: current sample-rate of display data, e.g., 192000.0.
fLow: Lowest frequency of the frequency range to evaluate, referenced to the center_frequency to which the DDC is tuned. For example, for LSB, not using CTUN, a typical value would be -3000.0.
fHigh: Highest frequency of the frequency range to evaluate, referenced to the center_frequency to which the DDC is tuned. For example, for LSB, not using CTUN, a typical value would be -300.0.
tau: Metering decay time-constant, in seconds. Useful when the bin-value is used to drive a meter. Typical value might be 0.5.

frame_rate: Display frame_rate current in use, e.g., 60.

The following function returns the maximum bin value (dB) for the selected display. It may be called at any time and will return the value from the most recent FFT.

```
double GetDetectMaxBin (int disp)
```

return value: maximum bin value (dB).
disp: identifier for the Display.

8 Blocks Used Outside Channels

There are several blocks included with the WDSP library that are often utilized OUTSIDE of the WDSP channels. Instantiating and accessing those blocks is discussed here.

8.1 Preemptive Wideband Noise Blanker

Noise blankers attempt to remove "impulse noise", brief time-domain impulses imposed upon the I-Q sample data. These impulses are often caused by things like power line arcing and may or may not be repetitive in nature. To remove these impulses, noise blankers must do two things: (1) detect the samples that have been corrupted by the impulses, and (2) replace those samples with some estimate of the original signal.

In the case of the Preemptive Wideband Noise Blanker, detection is done by comparing the power contained in each sample with the time-averaged power across the wide bandwidth seen by the blanker. "Wideband" implies that the sample stream has a high enough sample-rate to support the bandwidth of roughly the size of a typical amateur band, for example, a few hundred Khz for HF operation. Processing a wider bandwidth in the blanker has little if any benefit since the input bandwidth is usually restricted anyway by input bandpass filters or by the bandwidth of the antenna itself.

Correction, in this blanker, is done by replacing a sequence of samples believed to be corrupt with an estimate value of zero. However, there are several other considerations.

- We should not transition instantaneously from the uncorrupted signal level to zero and from zero back to the uncorrupted signal level; this would create a disturbance in the frequency spectrum and audible clicks.
- The leading and trailing edges of the impulse may not be detected even though the higher magnitude center is. Therefore, we may wish to "advance" the blanking to before the detected impulse and "hang" onto the blanking slightly after the impulse appears to have passed.

The following figure illustrates the operation of the Preemptive Wideband Noise Blanker. Signal amplitude slewing to zero and back, as well as advance and hang times are illustrated. Note also that the threshold at which a sample is judged to be part of an impulse is adjustable.

It is desirable to have the wideband blankers also affect the panadapter and waterfall displays. Since the data input for these occurs outside an RXA channel, the wideband blanker must be called from outside an RXA channel also.

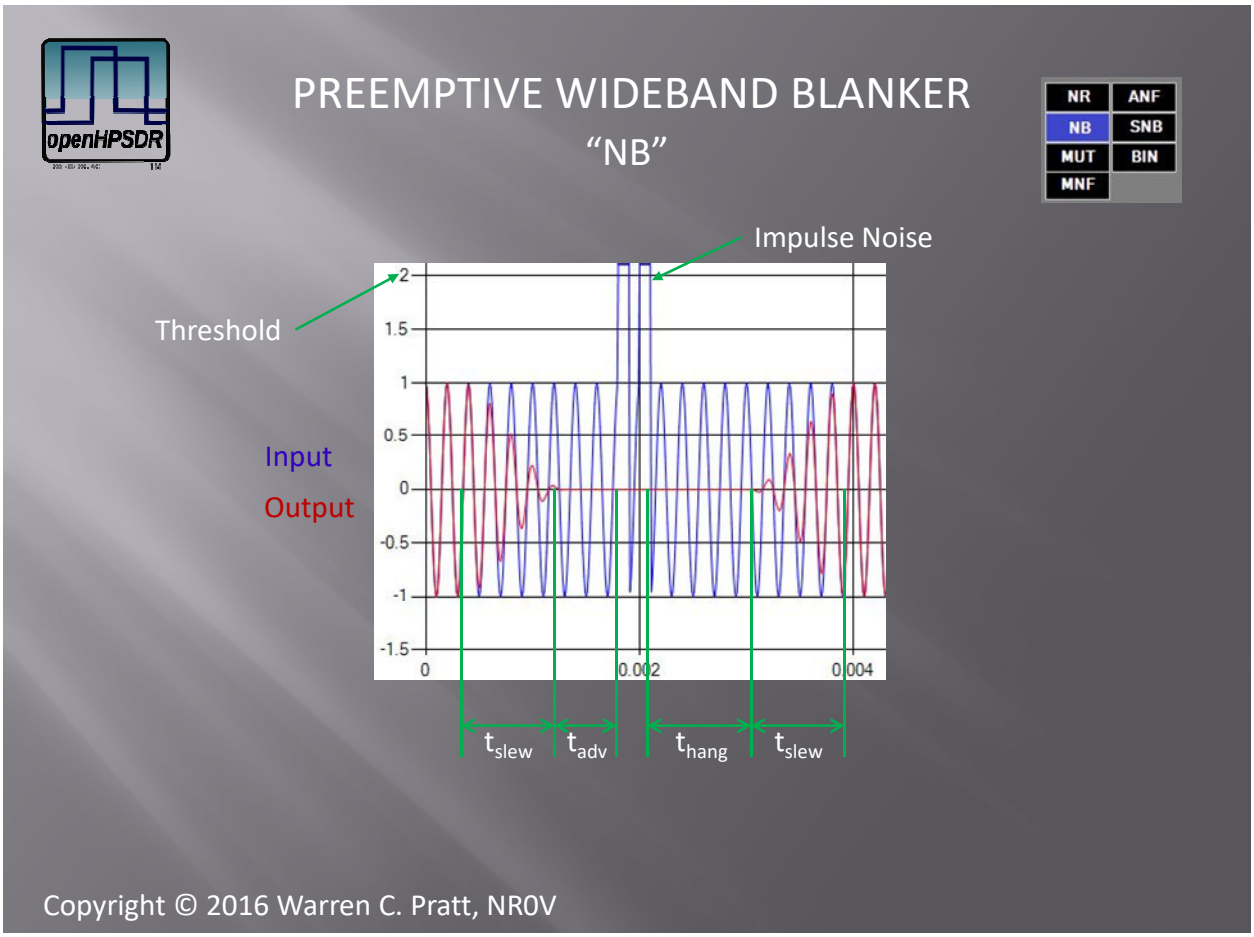


Figure 31: Preemptive Wideband Noise Blanker (NB) — impulse detection and blanking waveform.

There are two sets of calls that can be used to access the Preemptive Wideband Blanker:

1) When coding in C, one may include the "nob.h" header file and use calls that directly reference the pointer to the noise blanker data structure which is of type ANB. The 'create' function, creating an instance of the blanker, returns a pointer of type ANB and then this pointer is subsequently used in all communications with this instance of the blanker.
2) The more general set of calls requires the specification of an integer 'identifier' in the call to the create function. This identifier is then subsequently used in all communications with this same instance of the blanker.

Pointer-Based Calls

A blanker is instantiated (its resources are allocated and default values set) using the following call.

```
ANB create_anb (int run, int buffsize, double* in, double* out, double
samplerate, double tau, double hangtime, double advtime, double backtau,
double threshold)
```

run: default value of the run variable. 0 = OFF; 1 = ON. When this block is turned OFF, the input buffer is copied to the output buffer in the event the buffers are not the same.
buffsize: default number of complex I-Q samples in the input buffer.

in: default pointer to the input buffer.
out: default pointer to the output buffer which may be the same as the input buffer.
samplerate: default sample-rate.
tau: default duration (seconds) of the raised-cosine transitions between normal signal levels and the zero estimate during an impulse, t_{slew}. 0.0001 is a reasonable starting value, depending upon sample-rate.
hangtime: default hang-time, t_{hang} (seconds). 0.0001 is a reasonable starting value, depending upon sample-rate.
advtime: default advance-time, t_{adv} (seconds). 0.0001 is a reasonable starting value, depending upon sample-rate.
backtau: default time-constant for averaging power across the wide bandwidth. 0.05 is a reasonable starting value.
threshold: default threshold. A sample is judged to be part of an impulse if its magnitude is greater than threshold*average_power. An appropriate range for threshold is approximately 15.0 to 500.0. If threshold is too low, normal signals may be wrongly identified as impulse noise. If it is too high, successful detection will not occur.

Resources are freed using the following call.

```
void destroy_anb (ANB a)
```
a: pointer to the blanker data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

Buffers are flushed of stale data and indexes are re-initialized using the following call.

```
void flush_anb (ANB a)
```
a: pointer to the blanker data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

A buffer of input data is processed and a buffer of output data written using the following call.

```
void xanb (ANB a)
```
a: pointer to the blanker data structure.

Various parameters are reset using the following calls. Unless otherwise noted, these following calls may be made at any time including while data is flowing.

```
void setBuffers_anb (ANB a, double* in, double* out)  — NOT EXPORTED FROM DLL
```
a: pointer to the blanker data structure.
in: pointer to the input buffer.
out: pointer to the output buffer which may be the same as the input buffer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void setSamplerate_anb (ANB a, int rate)  — NOT EXPORTED FROM DLL
```
a: pointer to the blanker data structure.
rate: sample-rate.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void setSize_anb (ANB a, int size)  — NOT EXPORTED FROM DLL
```
a: pointer to the blanker data structure.
size: number of complex I-Q samples in the input buffer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void pSetRCVRANBRun (ANB a, int run)
```
a: pointer to the blanker data structure.
run: 0 = OFF, 1 = ON. When OFF, the input buffer will be copied to the output buffer if they are not the same.

```
void pSetRCVRANBTau (ANB a, double tau)
```
a: pointer to the blanker data structure.
tau: duration (seconds) of the raised-cosine transitions between normal signal levels and the zero estimate during an impulse, t_{slew}.

```
void pSetRCVRANBHangtime (ANB a, double time)
```
a: pointer to the blanker data structure.
time: hang-time, t_{hang} (seconds).

```
void pSetRCVRANBAdvtime (ANB a, double time)
```
a: pointer to the blanker data structure.
time: advance-time, t_{adv} (seconds).

```
void pSetRCVRANBBacktau (ANB a, double tau)
```
a: pointer to the blanker data structure.
tau: time-constant for averaging power across the wide bandwidth.

```
void pSetRCVRANBThreshold (ANB a, double thresh)
```
a: pointer to the blanker data structure.
thresh: detection threshold. (See explanation above in description of 'create' call.)

Identifier-Based Calls

A blanker is instantiated (its resources are allocated and default values set) using the following call.

```
void create_anbEXT (int id, int run, int buffsize, double samplerate, double
tau, double hangtime, double advtime, double backtau, double threshold)
```
id: an integer value used as the identifier for this instance of the blanker. The number of blankers is currently restricted (by the setting of an internal variable) to 32. Correspondingly, identifiers must be in the range 0 through 31.
run: default value of the run variable. 0 = OFF; 1 = ON. When this block is turned OFF, the input buffer is copied to the output buffer in the event they are not the same.
buffsize: default number of complex I-Q samples in the input buffer.
samplerate: default sample-rate.
tau: default duration (seconds) of the raised-cosine transitions between normal signal levels and the zero estimate during an impulse, t_{slew}. 0.0001 is a reasonable starting value, depending upon sample-rate.
hangtime: default hang-time, t_{hang} (seconds). 0.0001 is a reasonable starting value, depending upon sample-rate.
advtime: default advance-time, t_{adv} (seconds). 0.0001 is a reasonable starting value, depending upon sample-rate.
backtau: default time-constant for averaging power across the wide bandwidth. 0.05 is a reasonable starting value.
threshold: default threshold. A sample is judged to be part of an impulse if its magnitude is greater than threshold*average_power. An appropriate range for threshold is approximately 15.0 to 500.0. If threshold is too low, normal signals may be wrongly identified as impulse noise. If it is too high, successful detection will not occur.

Resources are freed using the following call.

```
void destroy_anbEXT (int id)
```
id: identifier for this instance of the blanker.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

Buffers are flushed of stale data and indexes are re-initialized using the following call.

```
void flush_anbEXT (int id)
```
id: identifier for this instance of the blanker.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

A buffer of input data is processed and a buffer of output data written using the following call.

```
void xanbEXT (int id, double* in, double* out)
```
id: identifier for this instance of the blanker.
in: pointer to the input buffer.
out: pointer to the output buffer which may be the same as the input buffer.

Various parameters are reset using the following calls. Unless otherwise noted, these following calls may be made at any time including while data is flowing.

```
void SetEXTANBRun (int id, int run)
```
id: identifier for this instance of the blanker.
run: 0 = OFF, 1 = ON. When OFF, the input buffer will be copied to the output buffer if they are not the same.

```
void SetEXTANBSamplerate (int id, int rate)
```
id: identifier for this instance of the blanker.
rate: sample-rate.

```
void SetEXTANBBuffsize (int id, int size)
```
id: identifier for this instance of the blanker.
size: number of complex I-Q samples in the input buffer.

```
void SetEXTANBTau (int id, double tau)
```
id: identifier for this instance of the blanker.
tau: duration (seconds) of the raised-cosine transitions between normal signal levels and the zero estimate during an impulse, t_{slew}.

```
void SetEXTANBHangtime (int id, double time)
```
id: identifier for this instance of the blanker.
time: hang-time, t_{hang} (seconds).

```
void SetEXTANBAdvtime (int id, double time)
```
id: identifier for this instance of the blanker.
time: advance-time, t_{adv} (seconds).

```
void SetEXTANBBacktau (int id, double tau)
```
id: identifier for this instance of the blanker.
tau: time-constant for averaging power across the wide bandwidth.

```
void SetEXTANBThreshold (int id, double thresh)
```
id: identifier for this instance of the blanker.
thresh: detection threshold. (See explanation above in description of 'create' call.)

8.2 Interpolating Wideband Noise Blanker

The Interpolating Wideband Noise Blanker is similar to the Preemptive Wideband Noise Blanker except that it has additional modes to estimate the original signal, rather than just estimating it as zero as is done by the Preemptive Wideband Noise Blanker. As shown in the following figure, it can do things like interpolate across impulses as its estimate of the original signal.

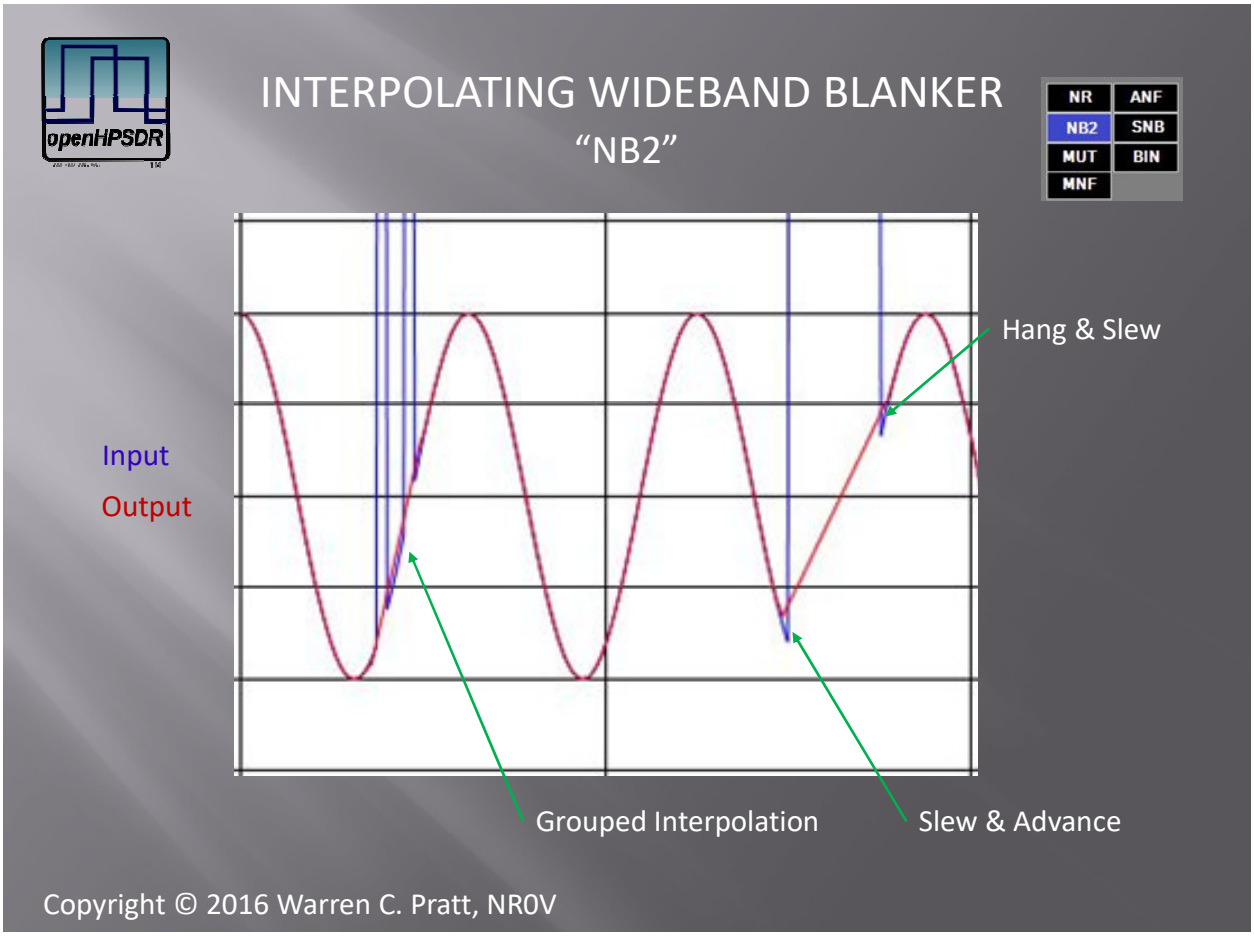


Figure 32: Interpolating Wideband Noise Blanker — impulse correction by interpolation across blanked samples.

There are two sets of calls that can be used to access the Interpolating Wideband Blanker:

1) When coding in C, one may include the "nobII.h" header file and use calls that directly reference the pointer to the noise blanker data structure which is of type NOB. The 'create' function, creating an instance of the blanker, returns a pointer of type NOB and then this pointer is subsequently used in all communications with this instance of the blanker.
2) The more general set of calls requires the specification of an integer 'identifier' in the call to the create function. This identifier is then subsequently used in all communications with this same instance of the blanker.

Pointer-Based Calls

A blanker is instantiated (its resources are allocated and default values set) using the following call.

```
NOB create_nob (int run, int buffsize, double* in, double* out, double
samplerate, int mode,  double advslewtime, double advtime, double
hangslewtime, double hangtime, double max_imp_seq_time, double backtau,
double threshold)
```

run: default value of the run variable. 0 = OFF; 1 = ON. When this block is turned OFF, the input buffer is copied to the output buffer in the event they are not the same.
buffsize: default number of complex I-Q samples in the input buffer.
in: default pointer to the input buffer.
out: default pointer to the output buffer which may be the same as the input buffer.
samplerate: default sample-rate.
mode: default mode. mode determines what estimate values are used for the sequence of corrupt samples to be replaced. 0 = zero mode (estimate as zero); 1 = sample-hold (take the value of non-corrupt signal at the beginning of the impulse and hold that throughout the corrupt sequence); 2 = mean-hold (average the non-corrupt values at the beginning and end of the corrupt sequence and use that as the estimate during the corrupt sequence); 3 = hold-sample (take the value of non-corrupt signal at the end of the impulse and hold that throughout the corrupt sequence); 4 = linearly interpolate across the corrupt sequence.
advslewtime: default time (seconds) to slew from the non-corrupt signal before the advance time. 0.0001 is a reasonable initial value for typical sample rates.
advtime: default advance time (seconds). 0.0001 is a reasonable initial value for typical sample rates.
hangslewtime: default time (seconds) to slew after the hang time back to the non-corrupt signal. 0.0001 is a reasonable initial value for typical sample rates.
hangtime: default hang time (seconds). 0.0001 is a reasonable initial value for typical sample rates.
max_imp_seq_time: default maximum amount of time (seconds) across which a closely spaced sequence of impulses will be evaluated. 0.025 is a reasonably conservative value.
backtau: default time-constant for averaging power across the wide bandwidth. 0.05 is a reasonable starting value.
threshold: default threshold. A sample is judged to be part of an impulse if its magnitude is greater than threshold*average_power. An appropriate range for threshold is approximately 15.0 to 500.0. If threshold is too low, normal signals may be wrongly identified as impulse noise. If it is too high, successful detection will not occur.

Resources are freed using the following call.

```
void destroy_nob (NOB a)
```

a: pointer to the blanker data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

Buffers are flushed of stale data and indexes are re-initialized using the following call.

```
void flush_nob (NOB a)
```

a: pointer to the blanker data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

A buffer of input data is processed and a buffer of output data written using the following call.

```
void xnob (NOB a)
```

a: pointer to the blanker data structure.

Various parameters are reset using the following calls. Unless otherwise noted, these following calls may be made at any time including while data is flowing.

```
void setBuffers_nob (NOB a, double* in, double* out)  — NOT EXPORTED FROM DLL
```

a: pointer to the blanker data structure.
in: pointer to the input buffer.
out: pointer to the output buffer which may be the same as the input buffer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void setSamplerate_nob (NOB a, int rate)  — NOT EXPORTED FROM DLL
```

a: pointer to the blanker data structure.
rate: sample-rate.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void setSize_nob (NOB a, int size)  — NOT EXPORTED FROM DLL
```

a: pointer to the blanker data structure.
size: number of complex I-Q samples in the input buffer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void pSetRCVRNOBRun (NOB a, int run)
```

a: pointer to the blanker data structure.
run: 0 = OFF, 1 = ON. When OFF, the input buffer will be copied to the output buffer if they are not the same.

```
void pSetRCVRNOBMode (NOB a, int mode)
```

a: pointer to the blanker data structure.
mode: mode determines what estimate values are used for the sequence of corrupt samples to be replaced. 0 = zero mode (estimate as zero); 1 = sample-hold (take the value of non-corrupt signal at the beginning of the impulse and hold that throughout the corrupt sequence); 2 = mean-hold (average the non-corrupt values at the beginning and end of the corrupt sequence and use that as the estimate during the corrupt sequence); 3 = hold-sample (take the value of non-corrupt signal at the end of the impulse and hold that throughout the corrupt sequence); 4 = linearly interpolate across the corrupt sequence.

```
void pSetRCVRNOBTau (NOB a, double tau)
```

a: pointer to the blanker data structure.
tau: the duration (seconds) of both the advslewtime and hangslewtime are set to tau.

```
void pSetRCVRNOBHangtime (NOB a, double time)
```

a: pointer to the blanker data structure.
time: hang-time, t_{hang} (seconds).

```
void pSetRCVRNOBAdvtime (NOB a, double time)
```

a: pointer to the blanker data structure.
time: advance-time, t_{adv} (seconds).

```
void pSetRCVRNOBBacktau (NOB a, double tau)
```

a: pointer to the blanker data structure.
tau: time-constant for averaging power across the wide bandwidth.

```
void pSetRCVRNOBThreshold (NOB a, double thresh)
```

a: pointer to the blanker data structure.
thresh: detection threshold. (See explanation above in description of 'create' call.)

Identifier-Based Calls

A blanker is instantiated (its resources are allocated and default values set) using the following call.

```
void create_nobEXT (int id, int run, int mode, int buffsize, double
samplerate, double slewtime, double hangtime, double advtime, double backtau,
double threshold)
```

id: an integer value used as the identifier for this instance of the blanker. The number of blankers is currently restricted (by the setting of an internal variable) to 32. Correspondingly, identifiers must be in the range 0 through 31.
run: default value of the run variable. 0 = OFF; 1 = ON. When this block is turned OFF, the input buffer is copied to the output buffer in the event they are not the same.
mode: default mode. mode determines what estimate values are used for the sequence of corrupt samples to be replaced. 0 = zero mode (estimate as zero); 1 = sample-hold (take the value of non-corrupt signal at the beginning of the impulse and hold that throughout the corrupt sequence); 2 = mean-hold (average the non-corrupt values at the beginning and end of the corrupt sequence and use that as the estimate during the corrupt sequence); 3 = hold-sample (take the value of non-corrupt signal at the end of the impulse and hold that throughout the corrupt sequence); 4 = linearly interpolate across the corrupt sequence.
buffsize: default number of complex I-Q samples in the input buffer.
samplerate: default sample-rate.
slewtime: default duration (seconds) of both the slew before advance time and the slew after hang time.
hangtime: default hang-time, t_{hang} (seconds).
advtime: default advance-time, t_{adv} (seconds).
backtau: default time-constant for averaging power across the wide bandwidth.
threshold: default threshold. A sample is judged to be part of an impulse if its magnitude is greater than threshold*average_power.

Resources are freed using the following call.

```
void destroy_nobEXT (int id)
```

id: identifier for this instance of the blanker.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

Buffers are flushed of stale data and indexes are re-initialized using the following call.

```
void flush_nobEXT (int id)
```

id: identifier for this instance of the blanker.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

A buffer of input data is processed and a buffer of output data written using the following call.

```
void xnobEXT (int id, double* in, double* out)
```

id: identifier for this instance of the blanker.
in: pointer to the input buffer.
out: pointer to the output buffer which may be the same as the input buffer.

Various parameters are reset using the following calls. Unless otherwise noted, these following calls may be made at any time including while data is flowing.

```
void SetEXTNOBRun (int id, int run)
```

id: identifier for this instance of the blanker.
run: 0 = OFF, 1 = ON. When OFF, the input buffer will be copied to the output buffer if they are not the same.

```
void SetEXTNOBMode (int id, int mode)
```
id: identifier for this instance of the blanker.
mode: mode determines what estimate values are used for the sequence of corrupt samples to be replaced. 0 = zero mode (estimate as zero); 1 = sample-hold (take the value of non-corrupt signal at the beginning of the impulse and hold that throughout the corrupt sequence); 2 = mean-hold (average the non-corrupt values at the beginning and end of the corrupt sequence and use that as the estimate during the corrupt sequence); 3 = hold-sample (take the value of non-corrupt signal at the end of the impulse and hold that throughout the corrupt sequence); 4 = linearly interpolate across the corrupt sequence.

```
void SetEXTNOBSamplerate (int id, int rate)
```
id: identifier for this instance of the blanker.
rate: sample-rate.

```
void SetEXTNOBBuffsize (int id, int size)
```
id: identifier for this instance of the blanker.
size: number of complex I-Q samples in the input buffer.

```
void SetEXTNOBTau (int id, double tau)
```
id: identifier for this instance of the blanker.
tau: the duration (seconds) of both the advslewtime and hangslewtime are set to tau.

```
void SetEXTNOBHangtime (int id, double time)
```
id: identifier for this instance of the blanker.
time: hang-time, t_{hang} (seconds).

```
void SetEXTNOBAdvtime (int id, double time)
```
id: identifier for this instance of the blanker.
time: advance-time, t_{adv} (seconds).

```
void SetEXTNOBBacktau (int id, double tau)
```
id: identifier for this instance of the blanker.
tau: time-constant for averaging power across the wide bandwidth.

```
void SetEXTNOBThreshold (int id, double thresh)
```
id: identifier for this instance of the blanker.
thresh: detection threshold. (See explanation above in description of 'create' call.)

8.3 Resampler

Resamplers are used to convert from one sample-rate to another sample-rate. Conceptually, the resamplers provided here accomplish this by finding a least common multiple of the input and output frequencies, interpolating up to this least common multiple, and then decimating down from this least common multiple. In the actual computations, care was taken not to perform operations for samples that, in the end, will be discarded anyway.

Resamplers are used within the RXA and TXA channels. However, they are also often useful within various other parts of the application to convert either complex I-Q or simple audio samples from one rate to another.

> PROGRAMMING NOTE: There are two sets of calls that can be used to each of the below-described resamplers.

1) When calling from the C language, one can include the "resample.h" header file, use a 'create' call that returns a pointer of the native RESAMPLE (for the Complex Double-Precision Resampler) or RESAMPLEF (Non-Complex Single-Precision Resampler) pointer type, and then use that pointer to identify the resampler data structure in future references.
2) When calling from another language, or when it is not desirable to include the "resample.h" header file, there are calls that return and use pointers that have been cast to type void*. This was originally done for historical reasons and this author acknowledges that there are potential problems with this approach. With limited exceptions, the C standard does NOT guarantee that pointers to different types will be of the same length or structure. Therefore, pointer type conversions back and forth cannot be guaranteed. This situation is even more questionable when multiple languages are involved. Practically, on common architectures in use and using common tools, this generally works out OK. However, please beware!

8.3.1 Complex Double-Precision Resampler

As the title implies, this resampler is for complex I-Q samples of double-precision floating-point values.

Native Pointers

The resampler is created (its resources allocated), destroyed (its resources freed), flushed (data cleared from buffers and indexes reset), and executed using the following four calls, respectively.

```
RESAMPLE create_resample (int run, int size, double* in, double* out, int
in_rate, int out_rate, double fc, int ncoef, double gain)
```

return value: The return value is a pointer of type RESAMPLE, i.e., a pointer to the resampler data structure that has been created.
run: 0 = resampler does not run, it simply copies input buffer (length size) to output buffer; 1 = resampler runs. Normally, this parameter should be entered as 1.
size: number of complex I-Q samples in the input buffer.
in: pointer to the input buffer.
out: pointer to the output buffer.
in_rate: input sample-rate (samples / second)
out_rate: output sample-rate (samples / second)

fc: cut-off frequency for bandpass filtering. If entered as 0.0, this will be automatically set to +/-0.45 * min_rate, where min_rate = min (in_rate, out_rate). Setting this parameter to 0.0 is generally recommended.
ncoef: number of filter coefficients to use. If set to 0, this will be automatically calculated to deliver an excellent filter. Setting this parameter to 0 is generally recommended.
gain: a gain factor may be applied in the resampling process, if desired. Otherwise, set this gain to 1.0.

```
void destroy_resample (RESAMPLE a)
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void flush_resample (RESAMPLE a)
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
int xresample (RESAMPLE a)
```
a: pointer to the resampler data structure.

void Pointers*

These are simplified wrappers around the above calls. The create function returns a void pointer and the void pointer is used to reference the resampler in future calls. Calls to create, destroy, and execute the resampler are as follows, respectively.

```
void* create_resampleV (int in_rate, int out_rate)
```
return value: a pointer to the resampler data structure that has been created.
in_rate: input sample-rate (samples / second)
out_rate: output sample-rate (samples / second)

> Considering other parameters that are now internal, note that using this simplified approach, fc = 0.0, ncoef = 0, and gain = 1.0 in the creation of the resampling filters.

```
void destroy_resampleV (void* ptr)
```
ptr: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void xresampleV (double* in, double* out, int size, int* outsamps, void* ptr)
```
in: pointer to the input buffer.
out: pointer to the output buffer.
size: number of complex I-Q samples in the input buffer.
outsamps: pointer to location where the number of output samples written to the output buffer is to be stored.
ptr: pointer to the resampler data structure.

8.3.2 Non-Complex (Audio) Single-Precision Resampler

As the title implies, this resampler is for non-complex (I-only) samples of single-precision floating-point values.

Native Pointers

The resampler is created (its resources allocated), destroyed (its resources freed), flushed (data cleared from buffers and indexes reset), and executed using the following four calls, respectively.

```
RESAMPLEF create_resampleF (int run, int size, float* in, float* out, int
in_rate, int out_rate)  — NOT EXPORTED FROM DLL
```
return value: The return value is a pointer of type RESAMPLEF; a pointer to the resampler data structure that has been created.
run: 0 = resampler does not run, simply copies input buffer (length size) to output buffer; 1 = resampler runs. Normally, this parameter should be entered as 1.
size: number of audio (I-only) samples in the input buffer.
in: pointer to the input buffer.
out: pointer to the output buffer.
in_rate: input sample-rate (samples / second)
out_rate: output sample-rate (samples / second)

```
void destroy_resampleF (RESAMPLEF a)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void flush_resampleF (RESAMPLEF a)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
int xresampleF (RESAMPLEF a)  — NOT EXPORTED FROM DLL
```
return value: number of output samples.
a: pointer to the resampler data structure.

void Pointers*

These are simplified wrappers around the above calls. The create function returns a void pointer and the void pointer is used to reference the resampler in future calls. Calls to create, destroy, and execute the resampler are as follows, respectively.

```
void* create_resampleFV (int in_rate, int out_rate)
```
return value: a pointer to the resampler data structure that has been created.
in_rate: input sample-rate (samples / second)
out_rate: output sample-rate (samples / second)

```
void destroy_resampleFV (void* ptr)
```
ptr: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void xresampleFV (float* in, float* out, int size, int* outsamps, void* ptr)
```
in: pointer to the input buffer.
out: pointer to the output buffer.
size: number of audio (I-only) samples in the input buffer.
outsamps: pointer to location where the number of output samples written to the output buffer is to be stored.
ptr: pointer to the resampler data structure.

8.4 Variable Ratio Resampler

When passing sampled signals between clock domains, it is generally necessary to resample with a ratio that is adjustable to a high-level of precision and which cannot necessarily be expressed as the ratio of two integers. An example of this would be audio generated by the radio hardware and subsequent signal processing being passed to a computer sound card to be output to a speaker. The radio has one hardware clock and the sound card has another. Even if the radio output sample-rate is set to, for example, 48Khz, and the sound card input is also set to 48Khz, since the hardware clocks are different, buffer underflows or overflows will eventually result. Resampling with a ratio of, for example 1.0056 or 0.99974 might be necessary to match the rates of the two domains. The Variable Ratio Resampler supports this type resampling where the ratio can be entered as a floating point value and can be changed over time to respond to clock drift.

The Variable Ratio Resampler is instantiated as follows:

```
VARSAMP create_varsamp (int run, int size, double* in, double* out, int
in_rate, int out_rate, double fc_high, double fc_low, int R, double gain,
double var, int varmode)  – NOT EXPORTED FROM DLL
```

return value: The return value is a pointer of type VARSAMP; a pointer to the variable ratio resampler datastructure that has been created.
run: 0 = resampler does not run, simply copies input buffer (length size) to output buffer; 1 = resampler runs. Normally, this parameter should be entered as 1.
size: number of double-precision complex (I-Q) samples in the input buffer.
in: pointer to the input buffer.
out: pointer to the output buffer.
in_rate: nominal input rate, e.g., 48000.
out_rate: nominal output rate, e.g., 192000.
fc_high: cut-off frequency for bandpass filtering. If entered as 0.0, this will be automatically set to +/-0.4275 * min_rate, where min_rate = min (in_rate, out_rate). Setting this parameter to 0.0 is generally recommended.
fc_low: cut-off frequency to create a "notch" around zero Hertz. If set to a negative value, e.g., -1.0, there is NO "notch" around zero Hertz and frequency coverage is continuous from -fc_high to + fc_high. However, if this is set to a positive value, e.g., 500.0, the spectrum will be "notched" from -500.0 Hertz to +500 Hertz.
R: This is the density factor for a table of filter coefficients that is precomputed when this function is called. The "density" determines the ratio of the number of table coefficients to the number of coefficients that are used when the filter executes. For example, if set to 1024, there will be 1024 times as many coefficients in the table as are used for filter execution. Simulation has shown that 1024 is a good value producing very low spur levels but yet not requiring an excessive amount of memory to store the table.
gain: a gain factor may be applied in the resampling process, if desired. Otherwise, set this parameter to 1.0.
var: initial value for the variable portion of the resampling ratio. This value should be near 1.0, normally in the range 0.95 to 1.05. Note that the TOTAL resampling ratio is var * (out_rate / in_rate).
varmode: If set to 0, the same value of var will be used for each entire buffer of data. If set to 1, the value of var will be interpolated for each sample beginning with the ending value for the previous buffer of data and ending at the value for the current buffer of data. Setting to 1 therefore produces "smoother" transitions in sample-rate.

Functions to free varsamp resources, flush the current data, and execute processing of a new input buffer of data are as follows:

```
void destroy_varsamp (VARSAMP a)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void flush_varsamp (VARSAMP a)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
int xvarsamp (VARSAMP a, double var)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
var: variable portion of the resampling ratio. A new value can be given for processing each buffer of input data. Note also the comments above regarding varmode.

The following four functions would generally be used if the variable rate resampler were integrated into a WDSP unit. Note that data must NOT be flowing when these are executed.

```
void setBuffers_varsamp (VARSAMP a, double* in, double* out)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
in: pointer to the input buffer.
out: pointer to the output buffer.

```
void setSize_varsamp (VARSAMP a, int size)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
size: size (number of complex samples) of the input buffer.

```
void setInRate_varsamp (VARSAMP a, int rate)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
rate: input nominal sample-rate.

```
void setOutRate_varsamp (VARSAMP a, int rate)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
rate: output nominal sample-rate.

The following two calls are also available to facilitate reconfiguring cutoff frequencies for the variable ratio resampler. Note that data must NOT be flowing when these are executed.

```
void setFCLow_varsamp (VARSAMP a, double fc_low)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
fc_low: cut-off frequency to create a "notch" around zero Hertz. If set to a negative value, e.g., -1.0, there is NO "notch" around zero Hertz and frequency coverage is continuous from -fc_high to + fc_high. However, if this is set to a positive value, e.g., 500.0, the spectrum will be "notched" from -500.0 Hertz to +500 Hertz.

```
void setBandwidth_varsamp (VARSAMP a, double fc_low, double fc_high)  — NOT EXPORTED FROM DLL
```
a: pointer to the resampler data structure.
fc_low: cut-off frequency to create a "notch" around zero Hertz. If set to a negative value, e.g., -1.0, there is NO "notch" around zero Hertz and frequency coverage is continuous from -fc_high to + fc_high. However, if this is set to a positive value, e.g., 500.0, the spectrum will be "notched" from -500.0 Hertz to +500 Hertz.
fc_high: cut-off frequency for bandpass filtering. If entered as 0.0, this will be automatically set to +/-0.4275 * min_rate, where min_rate = min (in_rate, out_rate). Setting this parameter to 0.0 is generally recommended.

A simplified version of **create_varsamp(...)** is provided. Internally, this calls the full version of the function; however, certain parameters have either not been provided at this point or have fixed values. This may be especially useful when the variable ratio resampler is being instantiated and called from another language such as C#.

```
void* create_varsampV (int in_rate, int out_rate, int R)
```

return value: The return value is a pointer of type void*; a pointer to the variable ratio resampler datastructure that has been created.
in_rate: input nominal sample-rate.
out_rate: output nominal sample-rate.
R: This is the density factor for a table of filter coefficients that is precomputed when this function is called. The "density" determines the ratio of the number of table coefficients to the number of coefficients that are used when the filter executes. For example, if set to 1024, there will be 1024 times as many coefficients in the table as are used for filter execution. Simulation has shown that 1024 is a good value producing very low spur levels but yet not requiring an excessive amount of memory to store the table.

> Note that the input buffer size and pointers to the input and output buffers have NOT been specified. They are specified in the subsequent calls to execute the resampler:

```
void xvarsampV (double* input, double* output, int numsamps, double var, int*
outsamps, void* ptr)
```

input: pointer to the input buffer.
output: pointer to the output buffer.
numsamps: number of double-precision complex (I-Q) samples in the input buffer.
var: variable portion of the resampling ratio. A new value can be given for processing each buffer of input data. Note also the comments above regarding varmode.
outsamps: pointer to a location where the number of samples written to the output buffer is to be stored.
ptr: pointer to the resampler data structure.

varsamp resources can also be freed using the void* pointer returned from **create_varsampV(...)** :

```
void destroy varsampV (void* ptr)
```

ptr: pointer to the resampler data structure.

8.5 Sample-Rate Matcher

While the Variable Rate Resampler (discussed above) is a key component is "matching" sample-rates between clock domains, it does not solve the entire problem. We must still determine the value of "var", the variable component of the resampling ratio, and we must still buffer data being transferred from one domain to another. The Sample-Rate Matcher comprises an instance of the Variable Rate Resampler and, additionally, these other required functions. Once instantiated, the Sample-Rate Matcher is simply called with data from one domain and called to take out data from the other domain. By buffering the data and measuring the ratio of inputs to outputs it determines "var" and operates the Variable Ratio Resampler.

Due to system distractions/disturbances, occasionally the O/S does not schedule tasks for execution in a timely fashion. In those cases, it may not be practical to eliminate a buffer underflow or overflow. However, the Sample-Rate Matcher contains some additional functionality to mitigate the unpleasantness of those experiences.

In an underflow situation, the internal ring buffer is filled half-full of zeros to avoid a second underflow if there is a successive read before enough data has been written. In this case, the sample-rate matcher "slews" (along a raised-cosine) to zero before the zeros are inserted and then slews back to full volume after the zeros. The waveform looks like this:

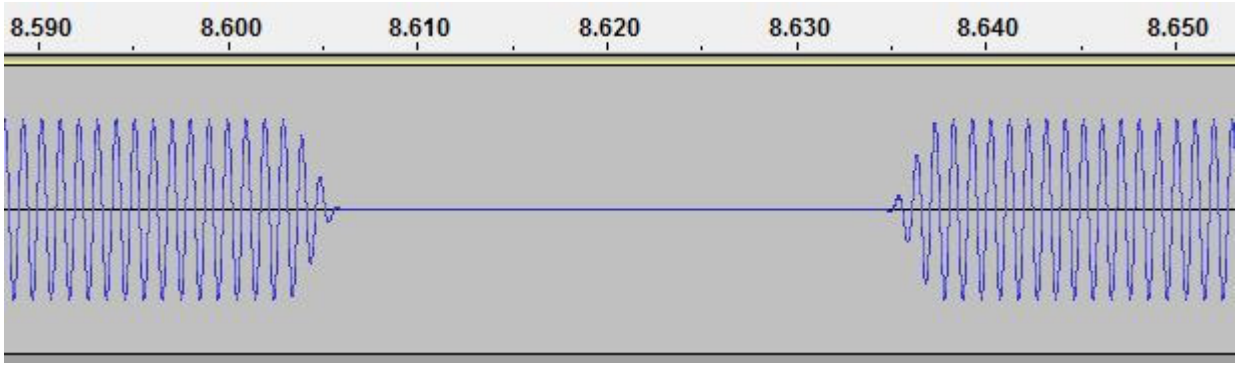


Figure 33: Sample-Rate Matcher underflow recovery — ring buffer half-filled with zeros.

In an overflow situation, half of the samples in the internal ring are discarded so that another overflow will not occur if the next write occurs before a sufficient number of samples are read. In that case, the waveform before the discarded samples is "blended" with the waveform after the discarded samples.

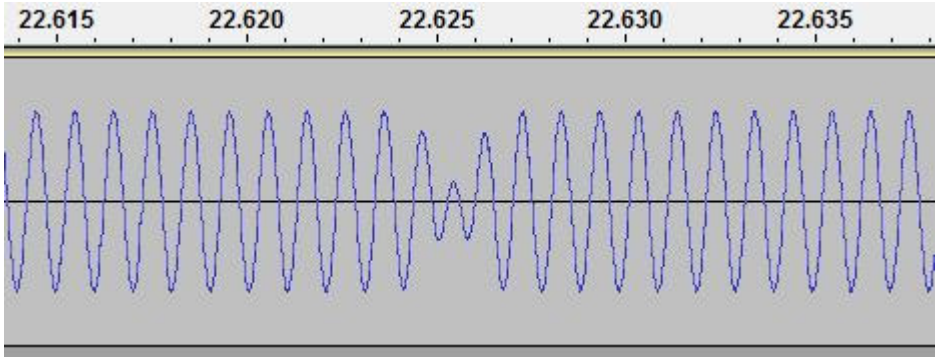


Figure 34: Sample-Rate Matcher overflow recovery — oldest half of ring buffer discarded.

Since these actions involve half the samples in the ring buffer, they are substantial and the sound of the disturbance is not totally masked. However, in all cases, discontinuities in the sample stream have been avoided and the high-frequency "pop" energy caused by such discontinuities is eliminated.

The Sample-Rate Matcher is instantiated as follows:

```
RMATCH create_rmatch (int run, double* in, double* out, int insize,  int
outsize,  int nom_inrate, int nom_outrate, double fc_high, double fc_low,
double gain, double startup_delay, int auto_ringsize, int ringsize, int R,
double var, int ffmav_min, int ffmav_max, double ff_alpha, int prop_ringmin,
int prop_ringmax, double prop_gain, int varmode, double tslew)  — NOT
EXPORTED FROM DLL
```

return value: The return value is a pointer of type RMATCH; a pointer to the sample-rate matcher data structure that has been created.

run: 0 = matcher does not run, it does not read from the input buffer nor does it write to the output buffer. Normally, this parameter should be entered as 1.

in: pointer to the input buffer.

out: pointer to the output buffer.

insize: number of samples to read from the input buffer.

outsize: number of samples to write to the output buffer.

nom_inrate: nominal input rate, e.g., 48000.

nom_outrate: nominal output rate, e.g., 192000.

fc_high: cut-off frequency for bandpass filtering. If entered as 0.0, this will be automatically set to +/-0.4275 * min_rate, where min_rate = min (in_rate, out_rate). Setting this parameter to 0.0 is generally recommended.

fc_low: cut-off frequency to create a "notch" around zero Hertz. If set to a negative value, e.g., -1.0, there is NO "notch" around zero Hertz and frequency coverage is continuous from -fc_high to + fc_high. However, if this is set to a positive value, e.g., 500.0, the spectrum will be "notched" from -500.0 Hertz to +500 Hertz.

gain: a gain factor may be applied in the resampling process, if desired. Otherwise, set this parameter to 1.0.

startup_delay: At system startup, if often takes a short time for data to begin flowing. Any control data collected during this time is likely to be erroneous. A non-zero value specified here delays the collection of control data by the amount of time specified (seconds).

auto-ringsize: THIS FEATURE HAS NOT BEEN IMPLEMENTED. Set this value to zero.

ringsize: the size of the ring buffer (complex samples).

R: This is the density factor for a table of filter coefficients that is precomputed when this function is called. The "density" determines the ratio of the number of table coefficients to the number of coefficients that are used when the filter executes. For example, if set to 1024, there will be 1024 times as many coefficients in the table as are used for filter execution. Simulation has shown that 1024 is a good value producing very low spur levels but yet not requiring an excessive amount of memory to store the table.

var: initial value for the variable portion of the resampling ratio. This value should be near 1.0, normally in the range 0.95 to 1.05. Note that the TOTAL resampling ratio is var * (nom_outrate / nom_inrate).
ffmav_min: minimum value for the size of the moving average filter used for the feed-forward control component. For usual operating conditions, 4096 is a good value.
ffmav_max: maximum number of data points used for the feed-forward control component. For usual operating conditions, 262144 is a good value. NOTE: This value MUST be a power of two.
ff_alpha: exponential smoothing parameter for the feed-forward control. For usual operating conditions, 0.01 is a good value.
prop_ringmin: minimum value for the size of the moving average filter used for the proportional control component. For usual operating conditions, 4096 is a good value.
prop_ringmax: maximum number of data points used for the proportional control component. For usual operating conditions, 163844 is a good value. NOTE: This value MUST be a power of two.
prop_gain: gain for the proportional feedback term. For usual operating conditions, 4.0e-06 is a good value.
varmode: If set to 0, the same value of var will be used for each resampler buffer of data. If set to 1, the value of var will be interpolated for each sample beginning with the ending value for the previous buffer of data and ending at the value for the current buffer of data. Setting to 1 therefore produces "smoother" transitions in sample-rate.
slew_time: time (seconds) to be used in slewing in the event of underflow or overflow.

Functions to free matcher resources, and to reset the current state are as follows:

```
void destroy_rmatch (RMATCH a)   — NOT EXPORTED FROM DLL
```
a: pointer to the matcher data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

```
void reset_rmatch (RMATCH a)   — NOT EXPORTED FROM DLL
```
a: pointer to the matcher data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

Simplified functions where many parameters are automatically set, and using void* pointers, are also available to instantiate the Rate Matcher and to free its resources. These may be especially useful when the sample-rate matcher is being instantiated and called from another language such as C#.

```
void* create_rmatchV (int in_size, int out_size, int nom_inrate, int
nom_outrate, int ringsize, double var)
```
return value: The return value is a pointer of type void*; a pointer to the sample-rate matcher data structure that has been created.
in_size: number of samples to read from the input buffer.
out_size: number of samples to write to the output buffer.
nom_inrate: nominal input rate, e.g., 48000.
nom_outrate: nominal output rate, e.g., 192000.
ringsize: the size of the ring buffer (complex samples).
var: initial value for the variable portion of the resampling ratio.

```
void destroy_rmatchV (void* a)
```
a: pointer to the matcher data structure.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

The following functions are to, respectively, write data to the Sample-Rate Matcher and read data from the Sample-Rate matcher.

```
void xrmatchIN (void* b,  double* in)
```
b: pointer to the matcher data structure.
in: pointer to the input data buffer.

```
void xrmatchOUT (void* b,  double* out)
```
b: pointer to the matcher data structure.
out: pointer to the output data buffer.

Diagnostic information from the Sample-Rate Matcher can be obtained, if desired, using the following function.

```
void getRMatchDiags (void* b,  int* underflows,  int* overflows,  double*
var,  int* ringsize, int* nring)
```
b: pointer to the matcher data structure.
underflows: pointer to the location into which the cumulative number of underflows should be stored.
overflows: pointer to the location into which the cumulative number of overflows should be stored.
var: pointer to the location into which the current value of 'var' should be stored.
ringsize: pointer to the location into which the current ring-buffer size should be stored.
nring: the current number of samples in the ring buffer.

The 'underflows' and 'overflows' counts can be reset to zero by calling:

```
void resetRMatchDiags (void* b)
```
b: pointer to the matcher data structure.

For testing purposes, it may also be desirable to "force" the variable ratio resampler to a particular 'var' ratio. That can be done by calling this function.

```
void forceRMatchVar (void* b,  int force,  double fvar)
```
b: pointer to the matcher data structure.
force: 0 = DON'T force; 1 = FORCE.
fvar: value of 'var' to force.

As of Revision 1.19, nine additional functions were added to enable dynamic setting and/or checking of various internal parameters for experimentation and tuning purposes. These are:

```
void setRMatchFeedbackGain (void* b, double feedback_gain)
```
b: pointer to the matcher data structure.
feedback_gain: this is the 'prop_gain' value as described above in create_rmatch(…).

```
void setRMatchSlewTime (void* b, double slew_time)
```
b: pointer to the matcher data structure.
slew_time: this is the 'slew_time' value as described above in create_rmatch(…).

```
void setRMatchSlewTime1 (void* b, double slew_time)
```
b: pointer to the matcher data structure.
slew_time: this is the 'slew_time' value as described above in create_rmatch(…).

```
void setRMatchFFAlpha (void* b, double ff_alpha)
```
b: pointer to the matcher data structure.
ff_alpha: this is the 'ff_alpha' value as described above in create_rmatch(…).

```
void setRMatchPropRingMin (void* b, int prop_min)
```
b: pointer to the matcher data structure.
prop_min: this is the 'prop_ringmin' value as described above in create_rmatch(…).

```
void setRMatchPropRingMax (void* b, int prop_max)
```
b: pointer to the matcher data structure.
prop_max: this is the 'prop_ringmax' value as described above in create_rmatch(…).

```
void setRMatchFFRingMin (void* b, int ff_ringmin)
```
b: pointer to the matcher data structure.
ff_ringmin: this is the 'ffmax_min' value as described above in create_rmatch(…).

```
void setRMatchFFRingMax (void* b, int ff_ringmax)
```
b: pointer to the matcher data structure.
ff_ringmax: this is the 'ffmax_max' value as described above in create_rmatch(…).

```
void getControlFlag (void* b, int* control_flag)
```
b: pointer to the matcher data structure.
control_flag: pointer to location to store 'control_flag'.

8.6 Diversity Mixer

The Diversity Mixer is used to implement diversity reception, an important and unique interference reduction technique. Two ADCs, each connected to a separate antenna, are required for this technique. After the signals are digitized by the ADCs, they can be mixed, while applying amplitude and phase adjustments, to null a signal (or noise) that is arriving from a particular direction.

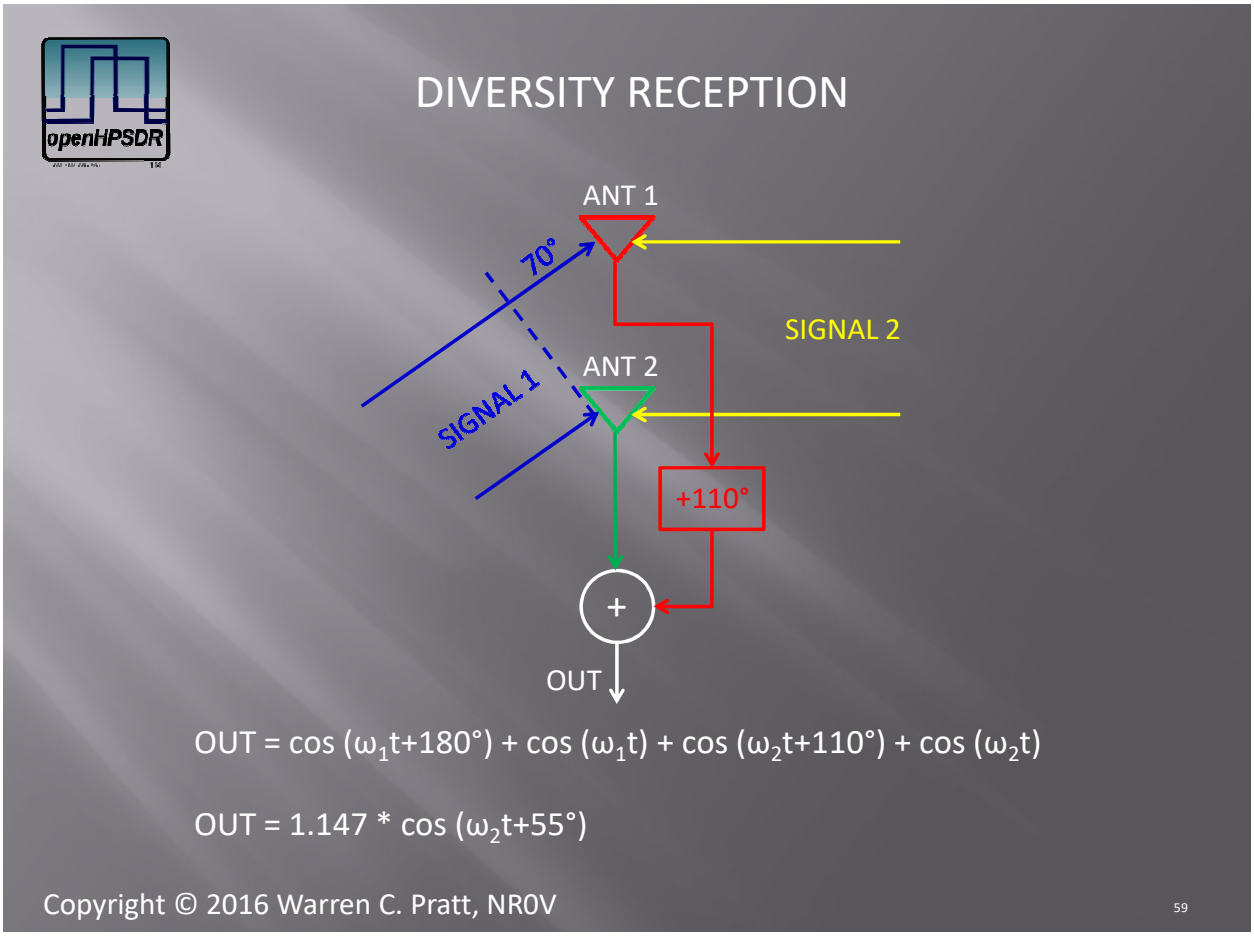


Figure 35: Diversity Mixer — two-antenna reception with adjustable phase and gain for signal combining.

Consider the example shown in the above figure. Signals 1 and 2 are arriving at the antennas from different angles. The path-length (and therefore delay) for Signal 2 is the same to both antennas. However, for Signal 1, the path-length is longer to Antenna 1 which results in an added 70 degrees of phase shift of that signal. Now, suppose we digitize the signals (not shown) and then mix them applying an additional 110 degrees of phase shift to Signal 1. For simplicity, we also assume the signals are of the same magnitude. Looking at the output signal resulting from the mixing (summing), we see that Signal 2 has been slightly strengthened and that Signal 1 is completely absent!

Diversity mixing requires multiple data streams from multiple DDC units. Therefore, it does not fit in the DSP channel model. Diversity mixing is instead performed BEFORE data enters an RXA channel and then

the single resulting stream from the mixer is provided as the input to an RXA channel. Note that we could mix more than the two signals shown using more antennas, ADCs, and DDCs.

A diversity mixer is created (its resources are allocated) using the following call.

```
void create_divEXT (int id, int run, int nr, int size)
```

id: an identifier assigned to this diversity mixer. The number of diversity mixers is currently limited to 2 by an internal parameter; so, this must currently be either 0 or 1. This identifier is used to refer to the mixer in subsequent calls.
run: default value of run variable. 0 => mixer does not operate, it does copy from in[0] to out; 1 => inputs are mixed if (nr == output), otherwise, in[output] is copied to out. (See below for setting output.)
nr: default number of streams to be mixed. The maximum is limited to 8 by an internal parameter.
size: default number of I-Q samples in each input buffer.

Resources for the mixer are freed using the following call.

```
void destroy_divEXT (int id)
```

id: identifier for the mixer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

The mixer is flushed using the following call.

```
void flush_divEXT (int id)
```

id: identifier for the mixer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

The mixer executes on the buffers of input data and writes a buffer of output data each time the following call is made.

```
void xdivEXT (int id, int size, double** in, double* out)
```

id: identifier for the mixer.
size: number of I-Q samples in each input buffer (and the number that will be written to the output buffer.
in: pointer to an array of pointers (nr of them) to the input buffers.
out: pointer to the output buffer.

The diversity mixer can be turned OFF/ON using:

```
void SetEXTDIVRun (int id, int run)
```

id: identifier for the mixer.
run: 0 => mixer does not operate, it does copy from in[0] to out; 1 => inputs are mixed if (nr == output), otherwise, in[output] is copied to out. (See below for setting output.)

The size of the input buffers can be changed using this call:

```
void SetEXTDIVBuffsize (int id, int size)
```

id: identifier for the mixer.
size: number of complex I-Q samples in each input buffer.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

The number of streams to be mixed can be changed using:

```
void SetEXTDIVNr (int id, int nr)
```
id: identifier for the mixer.
nr: number of streams to be mixed. The maximum is limited to 8 by an internal parameter.

> Note: This call must not be executed while data is flowing, i.e., while the execute function is being called.

The output value provides a means of having a selected input copied to the out buffer rather than running the mixer. if (output == nr), the mixer runs. However, if (output != nr), in[output] is copied to out. output is set using the following call.

```
void SetEXTDIVOutput (int id, int output)
```
id: identifier for the mixer.
output: number of the input stream to copy to the output. if (output == nr), the mixer runs.

The I and Q gain/phase-rotation multipliers must be specified for each input stream. Note that I and Q multipliers of 1.0 and 0.0, respectively, may be specified for the "reference receiver."

```
void SetEXTDIVRotate (int id, int nr, double* Irotate, double* Qrotate)
```
id: identifier for the mixer.
nr: number of input streams
Irotate: pointer to vector of I gain/phase-rotation multipliers, one element per input stream.
Qrotate: pointer to vector of Q gain/phase-rotation multipliers, one element per input stream.

Before summing, each sample of each stream is modified by its respective I and Q multipliers as follows:

I_modified = Irotate[this_stream] * I - Qrotate[this_stream] * Q;

Q_modified = Irotate[this_stream] * Q + Qrotate[this_stream] * I;

8.7 EER

The EER module is present to support power amplifiers operating with modes such as "Envelope Elimination and Restoration" and "Envelope Tracking." Specifically, the EER block accepts the normal I-Q output of the TXA transmitter channel and converts that to two different output streams:

(1) the original I-Q data multiplied by a scale factor, mgain, and

(2) a choice of one of the following three:

a) Phase information only: I-Q phase information multiplied by a scale factor pgain. The overall magnitude of the output, sqrt(I^2 + Q^2), is constant at a value of pgain; however, I and Q vary in relationship to each other, thereby expressing the phase.
b) Magnitude and Phase Information: The original I-Q data multiplied by pgain.
c) Envelope only: The magnitude of the I-Q envelope, multiplied by pgain, and sent only as 'I' values.

When operating in EER/ETR type modes, compensation must be provided for delays in the analog circuitry. Adjustable delays are also included in this software for that purpose.

An EER unit is created (its resources are allocated) using the following call.

```
void create_eerEXT (int id, int run, int size, int rate, double mgain, double
pgain, int rundelays, double mdelay, double pdelay, int amiq)
```

id: an identifier assigned to this EER unit.
run: default value of run variable. 0 => EER unit does not operate, it does copy from in to out; 1 => EER unit operates per the settings specified.
size: default number of I-Q samples in each input buffer.
rate: the sample-rate of the data.
mgain: default multiplier for the outM output.
pgain: default multiplier for the out output.
rundelays: 0 => delays will not be used in operation of the EER unit; 1 => delays will be activated in operation of the EER unit.
mdelay: default delay (us) for outM output.
pdelay: default delay (us) for out output.
amiq: default for what is to be provided on the out output.
0: Phase information only: I-Q phase information multiplied by a scale factor pgain. The overall magnitude of the output, sqrt(I^2 + Q^2), is constant at a value of pgain; however, I and Q vary in relationship to each other, thereby expressing the phase.
1: Magnitude and Phase Information: The original I-Q data multiplied by pgain.
2: Envelope only: The magnitude of the I-Q envelope, multiplied by pgain, and sent only as 'I' values.

The EER unit can be turned OFF/ON using the call:

```
void pSetEERRun (EER a, int run)
```

a: pointer of type EER to the EER unit.
run: 0 => EER does not operate, it does copy from in[0] to out; 1 => EER is operational.

The size of the input buffers can be changed using this call:

```
void pSetEERSize (EER a, int size)
```

a: pointer of type EER to the EER unit.

size: number of complex I-Q samples in each input buffer.

The samplerate in use for input and output data can be changed using this call:

```
void pSetEERSamplerate (EER a, int rate)
```
a: pointer of type EER to the EER unit.
rate: the samplerate of the input and output data.

The gain to be applied to the outM output can be changed using this call:

```
void pSetEERMgain (EER a, double gain)
```
a: pointer of type EER to the EER unit.
gain: the gain (linear scale) to be applied to the outM output.

The gain to be applied to the out output can be changed using this call:

```
void pSetEERPgain (EER a, double gain)
```
a: pointer of type EER to the EER unit.
gain: the gain (linear scale) to be applied to the out output.

The use of delays can be turned OFF/ON using the call:

```
void pSetEERRunDelays (EER a, int run)
```
a: pointer of type EER to the EER unit.
run: 0 => delays do not operate; 1 => delays are operational.

The following call is used to set the delay to be used for the outM output data:

```
void pSetEERMdelay (EER a, double delay)
```
a: pointer of type EER to the EER unit.
delay: delay (us). Note that fractional us may be used.

The following call is used to set the delay to be used for the out output data:

```
void pSetEERPdelay (EER a, double delay)
```
a: pointer of type EER to the EER unit.
delay: delay (us). Note that fractional us may be used.

The following call is used to set the mode for the out output. See descriptions above for the various options.

```
void pSetEERAMIQ (EER a, int amiq)
```
a: pointer of type EER to the EER unit.
amiq: output type to be used, per above description of options.

8.8 Combined VOX & Downward Expander

This block is an integrated implementation of VOX and a Downward Expander. They are integrated in that the same triggering is used for both and their operation is synchronized in time. Several advanced features are incorporated such as a 'side-channel filter' to select a range of speech frequencies for triggering and a 'look ahead algorithm' to avoid clipping the first sounds of a speech segment. Anti-VOX functionality, allowing received audio to reduce VOX sensitivity, is also provided.

The software author wishes to thank Scott, WU2O, for his input in defining the functionality of this block.

This block is instantiated using the following call:

```
void create_dexp (int id, int run_dexp, int size, double* in, double* out,
int rate, double dettau, double tattack, double tdecay, double thold, double
exp_ratio, double hyst_ratio, double attack_thresh, int nc, int wtype, double
lowcut, double highcut, int run_filt, int run_vox, int run_audelay, double
audelay, void (__stdcall *pushvox)(int id, int active), int antivox_run, int
antivox_size, int antivox_rate, double antivox_gain, double antivox_tau)
```

id: The identifier for this instance of the block. A value from 0 to 3.
run_dexp: 0 to turn OFF the downward expander; 1 to turn ON the downward expander.
size: Buffer size (complex samples) of the input and output buffers. This value must be a power-of-two, must be less than or equal to 'nc', and must be less than 'rate'.
in: Pointer to the input buffer.
out: Pointer to the output buffer which may be the same as the input buffer if desired.
rate: Sample-rate.
dettau: Time-constant used for low-pass filtering of the input samples to the detector (seconds).
tattack: Attack time. This is the raised-cosine length (in seconds) from low-gain to high-gain of the downward expander.
tdecay: Release/decay time. This is the raised-cosine length (in seconds) from high-gain to low-gain of the downward expander.
thold: Hold time. Time (in seconds) from when the trigger signal drops below the threshold to initiation of the release raised-cosine gain curve.
exp_ratio: Expansion ratio. The ratio between full-gain (gain = 1.0) and the reduced gain level for the downward expander. Note that this is on a linear scale, i.e., if you want to express this in dB in your User Interface, you must then convert from dB to linear scale.
hyst_ratio: Hysteresis ratio. This is a ratio (< 1.0) between (1) the level at which VOX/DEXP trigger and (2) the level at which hold-time commences for VOX/DEXP.
attack_thresh: The trigger value at which VOX is triggered and DEXP attack commences.
nc: Number of coefficients of the side-channel filter. This must be a power-of-two and an integral multiple of the buffer size, 'size'.
wtype: Window type to be used in generation of the side-channel filter. 0 for 4-term Blackman-Harris and 1 for 7-term Blackman-Harris.
lowcut: Low-end cutoff frequency (Hertz) of the side-channel trigger filter.
highcut: High-end cutoff frequency (Hertz) of the side-channel trigger filter.
run_filt: 0 to turn OFF the side-channel trigger filter; 1 to turn ON the side-channel trigger filter.
run_vox: 0 to turn OFF VOX operation; 1 to turn ON VOX operation.
run_audelay: 0 to turn OFF the 'lookahead' delay; 1 to turn ON the 'lookahead' delay.
audelay: Lookahead delay (seconds).

Copyright © 2016–2026, Warren C. Pratt, NR0V

(__stdcall *pushvox)(int id, int active): Pointer to the function to call to change the state of the transmitter based upon VOX information. 'Active' is 1 to turn ON the transmitter; 'Active' is 0 to turn OFF the transmitter.

antivox_run: 0 to turn OFF anti-vox and 1 to turn ON anti-vox.
antivox_size: Size (complex samples) of each buffer of anti-vox data.
antivox_rate: Sample-rate of anti-vox data.
antivox_gain: Gain value to be applied to anti-vox data.
antivox_tau: Time-constant to be applied in smoothing anti-vox data.

To release all resources of the block, use the call:

```
void destroy_dexp (int id)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).

To flush stale data from an instance of this block, call:

```
void flush_dexp (int id)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).

To execute the block when a new buffer of input data is available, call:

```
void xdexp (int id)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).

If the pointer to the VOX function was not known when 'create_dexp' was called, or if the function to call needs to be changed after creation, the following function can be called to do so:

```
void SendCBPushDexpVox (int id, void (__stdcall *pushvox)( int id, int
active))
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).

(__stdcall *pushvox)(int id, int active): Pointer to the function to call to change the state of the transmitter based upon VOX information. 'Active' is 1 to turn ON the transmitter; 'Active' is 0 to turn OFF the transmitter.

The downward expander can be turned OFF/ON using the following call. Note that VOX can be used with or without the downward expander running.

```
void SetDEXPRun (int id, int run)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
run: 0 to turn OFF the downward expander; 1 to turn ON the downward expander.

Use the following call to set/reset the input/output buffer size. This value must be a power-of-two, must be less than or equal to 'nc', and must be less than the value of 'rate'.

```
void SetDEXPSize (int id, int size)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
size: Buffer size (complex samples) of the input and output buffers.

The following call can be used to set/reset the input and output buffer pointers. Note that the output buffer can be the same as the input buffer if desired.

```
void SetDEXPIOBuffers (int id, double* in, double* out)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
in: Pointer to the input buffer.
out: Pointer to the output buffer.

The sample rate in use can be changed using the following call:

```
void SetDEXPRate (int id, double rate)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
rate: Sample-rate.

The following call is used to set/reset the time-constant of the trigger detector:

```
void SetDEXPDetectorTau (int id, double tau)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
tau: Time-constant used for low-pass filtering of the input samples to the detector (seconds).

The DEXP Attack Time can be adjusted using the following call:

```
void SetDEXPAttackTime (int id, double time)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
time: Attack time. This is the raised-cosine length (in seconds) from low-gain to high-gain of the downward expander.

The DEXP Release Time is adjusted using the following call:

```
void SetDEXPReleaseTime (int id, double time)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
time: Release/decay time. This is the raised-cosine length (in seconds) from high-gain to low-gain of the downward expander.

The DEXP/VOX Hold Time can be adjusted as follows:

```
void SetDEXPHoldTime (int id, double time)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
time: Hold time. Time (in seconds) from when the trigger signal drops below the threshold to initiation of the release raised-cosine gain curve.

The DEXP Expansion Ratio is adjusted as follows:

```
void SetDEXPExpansionRatio (int id, double ratio)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
ratio: Expansion ratio. The ratio between full-gain (gain = 1.0) and the reduced gain level for the downward expander. Note that this is on a linear scale, i.e., if you want to express this in dB in your User Interface, you must then convert from dB to linear scale.

To adjust the DEXP Hysteresis Ratio:

```
void SetDEXPHysteresisRatio (int id, double ratio)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
ratio: Hysteresis ratio. This is a ratio (< 1.0) between (1) the level at which VOX/DEXP trigger and (2) the level at which hold-time commences for VOX/DEXP.

The Attack Threshold is adjusted as follows:

```
void SetDEXPAttackThreshold (int id, double thresh)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
thresh: The trigger value at which VOX is triggered and DEXP attack commences.

The number of filter taps used for the side-channel filter can be changed after creation using the following call:

```
void SetDEXPFilterTaps (int id, int taps)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
taps: Number of coefficients of the side-channel filter. This must be a power-of-two and an integral multiple of the buffer size, 'size'.

To set the window-type to be used in generation of the side-channel filter:

```
void SetDEXPWindowType (int id, int type)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
type: Window type to be used in generation of the side-channel filter. 0 for 4-term Blackman-Harris and 1 for 7-term Blackman-Harris.

Use the following calls to set the 'low cut' and 'high cut for the side-channel trigger filter:

```
void SetDEXPLowCut (int id, double lowcut)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
lowcut: Low-end cutoff frequency (Hertz) of the side-channel trigger filter.

```
void SetDEXPHighCut (int id, double highcut)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
highcut: High-end cutoff frequency (Hertz) of the side-channel trigger filter.

The side-channel trigger bandpass filter can be turned OFF/ON using the following call. Note that VOX and DEXP can be used with or without this filter.

```
void SetDEXPRunSideChannelFilter (int id, int run)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
run: '0' turns the side-channel filter OFF; '1' turns the side channel filter ON.

VOX can be turned OFF/ON using the following call. Note that the DEXP can be used with or without VOX being active.

```
void SetDEXPRunVox (int id, int run)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
run: '0' turns VOX OFF; '1' turns VOX ON.

The following call turns the audio look-ahead OFF/ON. This can be turned OFF/ON independent of whether VOX or DEXP are active.

```
void SetDEXPRunAudioDelay (int id, int run)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
run: 0 to turn OFF the 'lookahead' delay; 1 to turn ON the 'lookahead' delay.

To set the amount of audio delay (look-ahead), use the following call:

```
void SetDEXPAudioDelay (int id, double delay)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).

delay: Lookahead delay (seconds).

For console monitoring purposes, the peak signal value can be obtained using the following call:

```
void GetDEXPPeakSignal (int id, double* peak)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
peak: Pointer to where the peak signal value is to be stored.

Anti-vox functionality is turned OFF/ON with the following call:

```
void SetAntiVOXRun (int id, int run)
```
id: The identifier initially specified to create this instance of the VOX/DEXP block. (An integer of 0 through 3).
run: 0 to turn OFF anti-vox functionality; 1 to turn ON anti-vox functionality.

The size (in complex samples) of each buffer of anti-vox data to be passed to the anti-vox algorithm is set as follows:

```
void SetAntiVOXSize (int id, int size)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
size: The number of complex samples per buffer of anti-vox data.

The sample-rate of the anti-vox data is specified using the following call:

```
void SetAntiVOXRate (int id, double rate)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
rate: The sample-rate of the anti-vox data.

The gain to be applied to the anti-vox data is set as follows:

```
void SetAntiVOXGain (int id, double gain)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
gain: Gain (linear scale) to be applied to the anti-vox data.

The time-constant (seconds) used for smoothing the anti-vox data is specified using this call:

```
void SetAntiVOXDetectorTau (int id, double tau)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
tau: The time constant (seconds) used for smoothing anti-vox data.

Each buffer of anti-vox data is sent to the anti-vox algorithm using the following call. Note that the parameter 'nsamples' is specified, for compatibility reasons. HOWEVER, this parameter is NOT used. The number of samples being transferred is specified in create_dexp(...) and/or in SetAntiVOXSize(...).

```
void SendAntiVOXData (int id, int nsamples, double* data)
```
id: The identifier initially specified to create this instance of the block. (An integer of 0 through 3).
size: The number of complex samples per buffer of anti-vox data.

8.9 Bandpass Filter Characterization Utility

This block provides the ability to get a magnitude characterization of a bandpass filter. This characterization can be used for graphical, to illustrate the filter, or otherwise.

Resources for a BFCU (bandpass filter characterization utility) must be allocated before its use. Note that up to four instances may be allocated and in use at a time. They are given IDs of 0 – 3. The available number could be extended in the code if needed.

```
int create_bfcu (int id, int min_size, int max_size, double rate, double
corner, int points)
```

id: The identifier specified to create this instance of the block. (An integer of 0 – 3.)
min_size: The minimum impulse response size that will be used for the filter, i.e., minimum filter size. This MUST be a power of 2.
max_size: The maximum impulse response size that will be used for the filter, i.e., maximum filter size. This MUST be a power of 2.
rate: The sample-rate at which the filter operates.
corner: The -6dB corner frequency. NOTE: As created, the bandpass filter will be symmetrical about zero, i.e., the corners will be at ‘+corner’ and ‘-corner’.
points: The number of points to generate for each filter response. This value (1) must be a power of 2, and (2) must be >=max_size.

After use, the resources consumed by the BFCU should be released as follows:

```
void destroy_bfcu (int id)
```

id: The identifier specified to create this instance of the block. (An integer of 0 – 3.)

For convenience is using the results of this utility, the following returns the indexes (indexes within the array to be returned) of the upper and lower corners of the filter characterization. The values are stored at 'lower-index' and 'upper-index').

```
void getFilterCorners (int id, int* lower_index, int* upper_index)
```

id: The identifier specified to create this instance of the block. (An integer of 0 – 3.)
lower_index: Pointer to location for lower index value.
upper_index: Pointer to location for upper index value.

Finally, a call to retrieve the actual curve/results of the filter characterization.

```
void getFilterCurve (int id, int size, int w_type, int index_low, int
index_high, double* segment)
```

id: The identifier specified to create this instance of the block. (An integer of 0 – 3.)
size: The current impulse response size (filter size).
w_type: The window-type used for the filter. Currently supported are 0 for Blackman-Harris 4 and 1 for Blackman-Harris 7.
index_low: The lower index of the segment you want returned. The range is 0 through points-1.
index_high: The upper index of the segment you want returned. The range is 0 through points-1.
segment: Pointer to location where the results are to be stored.

8.10 Filter Impulse Response Cache

Storing impulse responses for filters that have been previously used is a mechanism that can be used to speed-up things such as mode changes or changes in filter parameters. When available, a filter response stored in the cache will be used rather than re-creating it. Note that the memory used to store impulse responses, and optionally the file-size used when the cache is saved between application uses, can become quite large.

Call this function to initialize the cache.

```
void init_impulse_cache (int use)
```
use: Set to 1 to use the cache, 0 otherwise.

After the impulse cache is initialized, call this to turn it ON or OFF.

```
void use_impulse_cache (int use)
```
use: Set to 1 to use the cache, 0 otherwise.

Call to free resources used by the impulse cache.

```
void destroy_impulse_cache (void)
```
No parameters.

Call to write impulse cache to a file.

```
int save_impulse_cache (const char* path)
```
return value: 0 for success.
path: File path, including filename.

Call to read impulse cache from a file.

```
int read_impulse_cache (const char* path)
```
return value: 0 for success.
path: File path, including filename.

9 WDSP Version Information

The calling application may wish to query WDSP for its version number. This allows enabling or disabling certain console features based upon which version of WDSP is in use. It also enables checking to be sure that an appropriate version of WDSP is found.

The following function is available for this purpose.

```
int GetWDSPVersion ()
```
return value: integer version number, as described

As written in text, WDSP version numbers always contain exactly two digits to the right of the decimal point. However, for easy comparison with expected version values, the value returned by this function is version_number*100. E.g., for Version 1.14, the function will return 114.

10 Appendix A: Higher-Degree Curve Interpolation (NURBS / B-Spline)

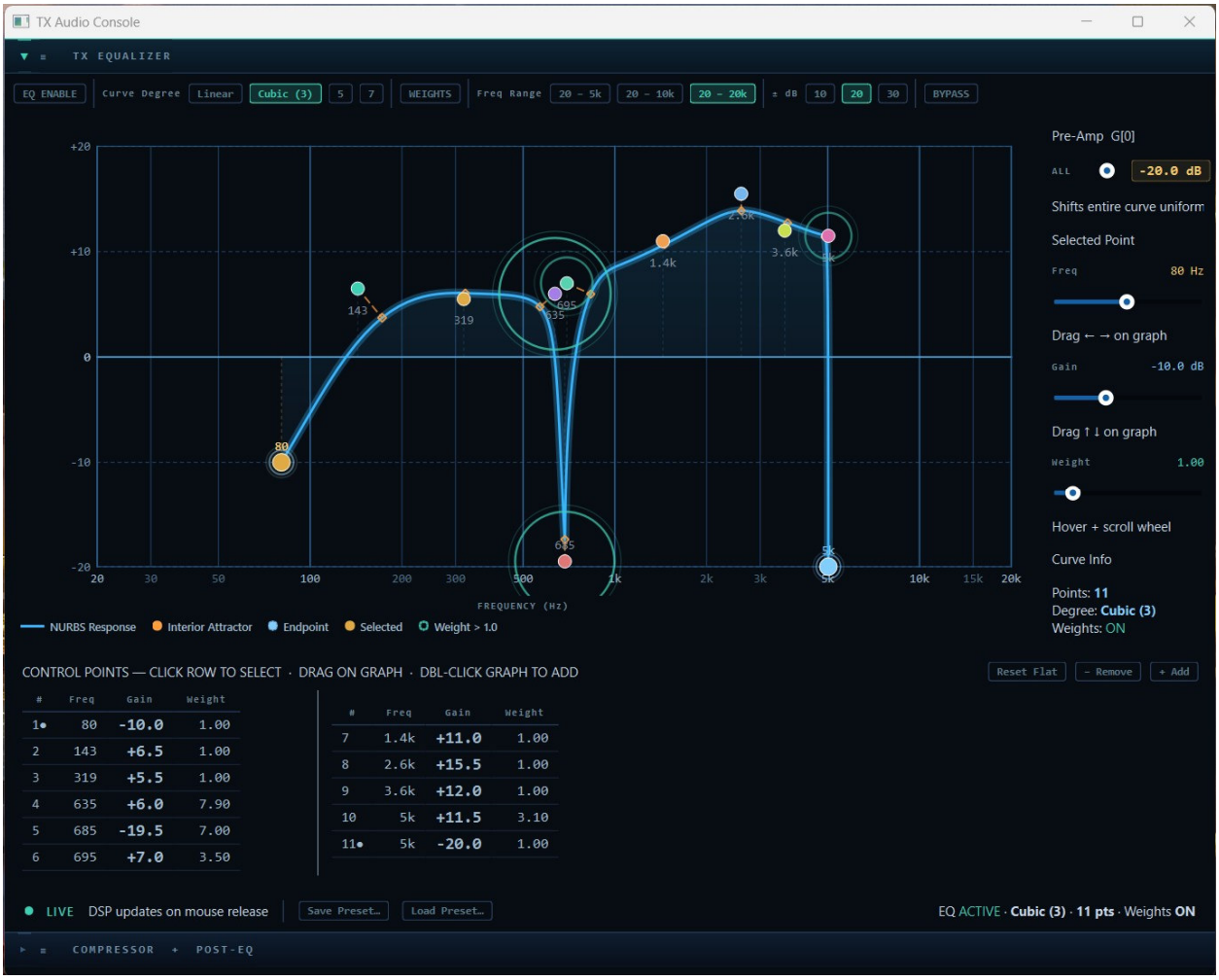


Figure A-1: TX Audio Console — Equalizer panel showing 11 control points, Cubic (3) degree, Weights ON. This is a live User Interface developed to implement and test the NURBS curve functionality described in this appendix. The blue curve represents the actual computed frequency response of the EQ filter. Control points are draggable; hovering and using the scroll wheel adjusts the weight of each point.

Several WDSP functions support higher-degree B-spline / NURBS curve interpolation between specified control points, including the Transmit Equalizer, the RX Equalizer, the Continuous Frequency Compressor (CFC), and the Post-CFC Equalizer. This appendix describes the curve system that underlies all of these functions.

A control point is specified by a frequency (Hz) and a level value (gain in dB for equalizers; compression level in dB for the compressor). The curve system fits a smooth spline through or near these control points and provides 1024 (X, Y) pairs for display in the User Interface as a polyline.

10.1 Control Points

In the linear (degree = 1) case, straight line segments connect the control points. For higher degrees, the curve passes through the control point at each end of the span but is subject to the gravitational attraction of intermediate points without necessarily passing through them. The number of control points must always be at least one greater than the degree (e.g., at least 4 control points for degree 3).

A clear benefit of NURBS, compared to for example a simple cubic spline, is that when there are several control points a perturbation in one portion of the curve does not affect other parts of the curve, even though smoothness and continuity are maintained.

10.2 Degree

The degree parameter controls the polynomial order of each curve segment. Degree 0 uses the legacy piecewise-linear algorithm (it provides no graphical draw output). Degree 1 uses the B-spline linear mode with graphical output. Degree 3 (cubic) is the most commonly useful higher degree, providing smooth transitions without excessive deviation from control points. Degrees 5, 7, and higher are available; as degree increases, the curve becomes smoother but may move farther from intermediate control points. Degrees through 16 are supported; although, in practice 7th degree is usually the maximum degree used.

The degree refers to each curve segment, not the curve as a whole. The segments are connected at knots and the complete curve is first-derivative and second-derivative continuous through the knots.

10.3 Weights

Each control point may optionally be assigned a weight that controls how strongly the curve is attracted toward that point. A higher weight pulls the curve closer to that control point. When all weights are equal, the effect cancels out; the benefit is in unequal weights. Weights must satisfy: 0.0 < weight < 100.0.

One natural User Interface design is to use the mouse scroll wheel while hovering over a control point to adjust its weight interactively. To avoid audio glitches during rapid scrolling, DSP updates should be debounced and deferred until scrolling pauses.

10.4 Knot Placement (umethod)

The umethod parameter controls how the internal knot vector is positioned. umethod = 0: knot positions are derived from the control point x-coordinates (recommended; preserves the visual correspondence between control points and curve shape). umethod = 1: knots are uniformly distributed.

10.5 Graphical Output

When using degree >= 1, the GetXXXEQDraw / GetTXACFCOMPCompDraw / GetTXACFCOMPPeqDraw functions returns 1024 (X, Y) pairs that can be plotted as a polyline in the User Interface to display the current curve shape. The X values are in Hz (not normalized) and are NOT equally spaced; polyline plotting must be used.

10.6 User Interface Guidance

The following UI patterns have been found effective in practice:

Drag on the graph to position control points. Defer DSP updates until mouse button release to avoid audio glitches during dragging.

Double-click anywhere on the curve to add a new control point at that location.

Hover over a control point and use the scroll wheel to adjust its weight. Show a visual indicator (such as a ring sized proportionally to the weight) to distinguish high-weight points.

Plot the 1024 GetDraw output points as a polyline, not as equally-spaced data. X values are in Hz and not uniformly distributed.

For degree selection, offer Linear (1), Cubic (3), and at least one higher degree (5 or 7) as options. Cubic is a practical default.

11 Appendix B: Phase Response in Audio Filters

Linear Phase and Minimum Phase: A Guide for the Technically Curious Operator

---

11.1 Introduction: What Phase Response Actually Is

When you adjust a filter — whether a bandpass filter shaping your received signal or a transmit equalizer sculpting your audio — you are primarily thinking about frequency response: which frequencies pass through and at what level. This is the visible, audible, easily understood part of what a filter does.

What is less obvious is that any filter that changes the amplitude of a signal at some frequencies may also change the timing of those frequencies relative to each other. This is an inescapable consequence of the mathematics of filtering, not a design flaw. A filter that passes low frequencies and attenuates high frequencies will often delay low frequencies more than high frequencies. The gain response and the phase response are coupled together.

This timing effect is called phase response, and it is the source of the difference between two filtering approaches available in this software.

11.1.1 Group Delay: The Practical Measure

The most useful way to think about phase response is through a derived quantity called group delay — measured in milliseconds, it tells you how much a filter delays signals at each frequency. A filter with flat group delay delays all frequencies equally, so the relative timing and shape of the signal is preserved. A filter with varying group delay delays some frequencies more than others, which alters the shape of rapidly-changing signals.

11.1.2 The Two Approaches

Linear phase filtering achieves flat group delay across the passband — all frequencies are delayed by exactly the same amount. The shape of the waveform passing through the filter is preserved perfectly. The cost is that the filter must introduce a fixed time delay (latency) in order to accomplish this, typically on the order of 20 to 85 milliseconds in this application, depending on filter length.

Minimum phase filtering – which we sometimes call “Low-latency” in WDSP documentation – introduces the minimum possible additional delay while still achieving the specified frequency response. In practice this means essentially zero added latency. The tradeoff is that group delay is no longer flat — it varies with frequency, with the most variation occurring near the filter's transition regions. This variation is mathematically determined by the gain response; you cannot independently specify both.

The amount of *incremental* latency for linear phase over minimum phase can easily be approximated for a given Filter_Size that is selected. The approximations given here are specifically for the RXA receiver and TXA transmitter as implemented in WDSP and each running at their recommended sample rates

which are 48K samples / second for the receiver in non-FM modes and 96K samples / second for the transmitter in all modes. These are also per filter. If you have a configuration, for example CESSB, that requires multiple filters, each will add a delay.

RXA_Incremental_Delay = (Filter_Size / 96) ms
TXA_Incremental_Delay = (Filter_Size / 192) ms

11.1.3 What the Research Says About Audibility

Whether group delay distortion is audible has been studied for decades, and the answer is nuanced.

The foundational work by Lipshitz, Pocock, and Vanderkooy (1982) found that phase distortion alone — without any change in frequency response — is generally not detectable by listeners using broadband program material. This finding has been replicated in many subsequent studies. For the vast majority of listeners under typical conditions, linear phase and minimum phase processing sound identical.

However, Blauert and Laws (1978) established that group delay distortion can become audible when it exceeds frequency-dependent thresholds: roughly 1.6 milliseconds at 1 kHz, more at higher frequencies, and less at lower frequencies. Whether a given filter exceeds these thresholds depends on how aggressive the filter is, where its transition regions fall, and what kind of signal is passing through it.

The effect is most noticeable on transient-rich material — sharp consonants in speech, the onset of a CW dit, the leading edge of a digital symbol. The auditory system is particularly sensitive to rapid changes in amplitude, and group delay distortion can subtly shape these events. For steady-state signals, the effect is even less perceptible.

With this foundation established, we can look at how these considerations apply to specific filter types in radio operation.

11.2 Bandpass Filters

A bandpass filter defines the slice of spectrum that you hear — it passes frequencies within a chosen range and rejects everything outside. Every mode of operation has different characteristics that determine how much the filter's phase response matters in practice.

11.2.1 SSB Voice

In single-sideband voice operation, operators typically set their receive passband somewhere in the range of 100–300 Hz on the low end to 2.5–3.0 kHz on the high end. (Wider ranges apply for ESSB operation; however, the concepts discussed here are the same.) The transmitted signal approximately fills this passband, with a great deal of energy in the lower portion of the spectrum — male voices in particular are energy-dense in the 100–600 Hz region — while the higher components, though lower in energy, remain essential for intelligibility.

The critical observation for SSB is that the signal energy fills the passband, including near the edges. This means the group delay distortion that minimum phase filtering introduces near the filter transitions is occurring where significant signal energy lives — not in some distant stop band.

Does this matter in practice? For SSB voice, almost certainly not for most listeners. The group delay variation introduced by a typical SSB bandpass filter is well within the thresholds established by Blauert and Laws. The filter transitions are usually relatively gradual, so the group delay variation is spread over a wider frequency range rather than concentrated sharply. Research on speech intelligibility going back to French and Steinberg (1947) has consistently found that intelligibility is dominated by amplitude response — the correct frequencies at the correct relative levels — not by phase response.

The practical recommendation for SSB voice: either mode works well. Linear phase provides the theoretical assurance of perfect waveform preservation; minimum phase eliminates any added latency. Most operators will not perceive a difference.

11.2.2 AM

Amplitude modulation presents a symmetric passband — a typical receive filter might span ±4 to ±5 kHz around the carrier, passing both sidebands equally. The audio information in an AM signal is encoded in the relationship between the upper and lower sidebands: they are mirror images of each other, and when combined in demodulation they reconstruct the original audio. This symmetry is the key to understanding why phase response matters for AM in a way that it does not for SSB.

AM transmit is straightforward. The transmitted carrier can be placed exactly at the center of the filter passband by design. With the carrier centered, both sidebands are equidistant from the filter edges and experience identical amplitude and group delay treatment. Minimum phase filtering introduces no asymmetry between sidebands in this case, and the transmitted signal is undistorted. Either phase mode is appropriate for AM transmission.

AM receive is a fundamentally different situation. In practice, the received carrier will never be perfectly centered in the receive passband. Transmitter frequency drift, receiver oscillator error, and Doppler effects all conspire to place the carrier slightly off-center — by anywhere from a few Hz to several hundred Hz in typical operation. This is not a deficiency in the receiver; it is an inescapable physical reality.

When the carrier is off-center in a minimum phase filter, the upper and lower sidebands no longer experience identical group delay. The upper sideband, which is closer to the upper filter edge, sees a different group delay profile than the lower sideband, which is closer to the lower edge. When the demodulator combines the sidebands to reconstruct audio, this timing mismatch manifests as harmonic and intermodulation distortion. The magnitude of the effect depends on how far off-center the carrier is and how steep the filter transitions are, but it is always present to some degree with minimum phase filtering.

A common question at this point is whether synchronous AM (SAM) demodulation addresses this problem. SAM locks a locally-generated carrier to the received carrier and uses it for demodulation,

which provides excellent immunity to carrier fading and can recover audio from very weak signals. However, SAM does not solve the sideband group delay asymmetry problem, because that asymmetry is introduced by the filter before the signal reaches the demodulator. The carrier lock loop compensates for carrier amplitude and phase variations at the carrier frequency itself, but it cannot undo the differential group delay that has already been applied to the sideband frequencies.

The practical recommendation for AM receive is unambiguous: use linear phase filtering. A linear phase filter has flat group delay across its entire passband, so both sidebands experience identical delay regardless of where the carrier sits within the passband. This applies equally to standard AM envelope detection and to SAM demodulation.

11.2.3 CW

CW filtering presents an interesting case because operators typically use very narrow filters — 200–500 Hz bandwidth for casual copy, as narrow as 10–100 Hz for contesting or weak-signal work. Narrow filters have steep transitions, and steep minimum phase filter transitions can produce significant group delay variation concentrated over a narrow frequency range.

For the main CW bandpass filter, despite reasonably narrow bandwidth, the situation is better than it might appear. The CW signal itself — a single tone being keyed — normally sits somewhere near the center of the filter's passband, well away from both transition regions. The group delay variation is greatest near the transitions and smallest at the center, precisely where the signal energy is concentrated. For most CW operating speeds and filter bandwidths, minimum phase is entirely appropriate for the main bandpass filter.

Audio Peak Filters (APF): The APF is a resonant bandpass filter applied to the demodulated audio, typically only 10–150 Hz wide, creating a sharp peak at the desired audio tone frequency. Because the APF is so narrow, the group delay at its center frequency can be quite significant even for a minimum phase design. For most operators at typical keying speeds, this is below the threshold of perception. For very high-speed CW operation — above perhaps 35–40 WPM — a linear phase APF might produce a marginally cleaner sound, though the practical benefit is likely small.

11.2.4 Digital Modes

Digital modes present the clearest case among all the filter applications considered here: the demodulation algorithms used for HF digital modes (PSK31, FT8, JS8Call, Winlink, and their contemporaries) are designed from the ground up to cope with the amplitude and phase distortions introduced by HF propagation, which are orders of magnitude larger than anything a receive filter introduces. Phase response of the receive filter is essentially irrelevant for digital modes. Minimum phase is the appropriate default and involves no significant tradeoffs.

11.3 The TX Audio Console: Equalizer, Compressor, and Post-EQ

The transmit audio processing chain in the WDSP library has three components that we will discuss here in terms of their phase response: a standalone transmit equalizer, a continuous frequency compressor, and a post-compression equalizer. Each has a different internal architecture, and that architecture determines how phase response is handled.

11.3.1 The Transmit Equalizer

The transmit equalizer is implemented as a FIR filter. The operator's control point settings define a frequency response curve, and a filter is computed that implements that curve.

The TX Equalizer uses Minimum Phase (Low Latency) filtering in this application, for two complementary reasons.

The first is latency. The transmit equalizer requires large filter lengths to achieve fine frequency resolution at low frequencies. With the large N values required — up to N = 16,384 at a sample rate of 96,000 Hz — a linear phase filter would introduce a latency of N/2 samples, corresponding to 85 milliseconds. While this latency is not usually significant for the signal received at the other station, it can be objectionable in the transmitting operator's monitoring path, in fast turn-around DX operation, and sometimes in other situations. Minimum phase eliminates this added latency, which is particularly valuable when the operator is listening to their own transmitted audio during adjustment.

The second is on-air audibility. Even in a controlled listening environment, the difference between linear and minimum phase equalization is subtle and detectable only by some listeners under favorable conditions. In actual radio communication, the transmitted signal passes through HF propagation — multipath delays of several milliseconds, frequency-selective fading that changes the amplitude response moment to moment, ionospheric dispersion, and Doppler spreading — before reaching the receiving station's own filters, AGC, speaker, and room acoustics. This chain of real-world imperfections is orders of magnitude larger than the phase response difference between linear and minimum phase equalization at the transmitter. The physics of HF propagation makes it essentially impossible for the receiving operator to distinguish linear from minimum phase TX equalization under actual operating conditions.

The transmit equalizer therefore uses Minimum Phase (Low Latency) mode. The operator's effort is better spent on the equalization curve itself.

11.3.2 The Compressor and Post-EQ

The continuous frequency compressor and its associated post-compression equalizer operate on a fundamentally different architecture: Weighted OverLap-Add (WOLA) processing in the frequency domain. The audio is transformed into the frequency domain via FFT, each frequency bin is processed independently, and the result is transformed back to the time domain.

In this architecture, compression is applied as a per-bin magnitude scaling — each FFT bin's amplitude is multiplied by a gain factor derived from the compression and equalization profiles, while the bin's phase

is left unchanged. This is not merely a convenient implementation choice; it reflects a body of research on optimal signal processing in the short-time Fourier transform domain. The foundational work by Ephraim and Malah (1984) on noise reduction established that the optimal estimator for signal recovery in the STFT domain modifies only the magnitude of each bin and preserves the phase — because phase carries fine timing information that is both difficult to estimate reliably and important to preserve.

The practical consequence for the compressor and post-EQ is significant: neither component introduces any phase distortion whatsoever, regardless of how aggressive the gain curve is. The phase of each frequency bin entering the processor exits unchanged. The compressor can apply dramatic frequency-selective gain reduction — boosting some frequency ranges and leaving others alone — without affecting the phase relationships between those frequencies at all. This is a genuine advantage of frequency-domain processing over time-domain approaches.

The post-EQ operates on exactly the same principle. Applying an equalization gain to each bin is a pure magnitude operation; the phase passes through untouched. From a phase response standpoint, the post-EQ is transparent even when applying substantial boosts or cuts.

This means the question of linear phase versus minimum phase simply does not arise for the compressor or post-EQ. The architecture answers it automatically and optimally.

11.3.3 Latency Considerations

The transmit equalizer's minimum phase implementation introduces essentially zero additional latency. This is a significant practical advantage compared to linear phase at large filter lengths used for fine frequency resolution.

The WOLA-based compressor and post-EQ introduce a processing latency determined by the FFT size and overlap factor, typically on the order of 10–20 milliseconds. This is inherent to any block-based frequency-domain processor and is independent of the phase mode question. From strictly a latency perspective, WOLA presents a middle-of-the-road alternative – more than minimum phase but less than typical linear phase.

11.4 Summary and Recommendations

Application	Linear Phase	Minimum Phase	Recommendation
SSB bandpass	Perfect transient preservation; 20–85 ms latency	Essentially zero latency; imperceptible phase distortion	Either; minimum phase is a practical default
AM transmit	No advantage over minimum phase	No drawback; signal centered by design	Either

AM receive	Preserves sideband symmetry regardless of carrier offset	Sideband group delay asymmetry causes distortion when carrier is off-center	Linear phase recommended
CW main filter	No benefit at typical bandwidths	No drawback	Minimum phase
CW APF (narrow)	Flat group delay through resonance	Concentrated group delay at center	Linear phase if keying speed is very high
Digital modes	No benefit	No drawback	Minimum phase
TX Equalizer	Larger latency at large N; on-air difference imperceptible	No latency; audibly identical on HF paths	Minimum phase used; no user choice needed
Compressor	Not applicable — frequency-domain architecture	Not applicable — frequency-domain architecture	Phase-neutral by design
Post-EQ	Not applicable — frequency-domain architecture	Not applicable — frequency-domain architecture	Phase-neutral by design

The honest summary is that for most operators in most situations, the choice between linear phase and minimum phase is a matter of personal preference rather than operational necessity. Both modes produce a correct and identical frequency response. The differences in phase behavior are real but typically below the threshold of perception for voice communication and irrelevant for digital modes.

The one clear exception is AM reception: because a received AM carrier will always drift somewhat off-center in the passband, minimum phase filtering introduces an asymmetric group delay between the upper and lower sidebands that degrades demodulation. Linear phase filtering eliminates this problem entirely and is the recommended choice for AM reception regardless of which demodulator is used.

For the transmit audio processing chain, the phase mode question is resolved by the architecture of each component: the equalizer uses minimum phase for practical reasons of latency and because on-air audibility over HF is essentially zero; the compressor and post-EQ are phase-neutral by virtue of their frequency-domain design. No user choice is required or offered.

11.5 References

Lipshitz, S.P., Pocock, M., and Vanderkooy, J. (1982). “On the Audibility of Midrange Phase Distortion in Audio Systems.” Journal of the Audio Engineering Society, 30(9), 580–595.

Blauert, J. and Laws, P. (1978). “Group Delay Distortions in Electroacoustical Systems.” Journal of the Acoustical Society of America, 63(5), 1478–1483.

Ephraim, Y. and Malah, D. (1984). “Speech Enhancement Using a Minimum Mean-Square Error Short-Time Spectral Amplitude Estimator.” IEEE Transactions on Acoustics, Speech, and Signal Processing, 32(6), 1109–1121. The foundational work establishing phase preservation as the optimal strategy in STFT-domain signal processing.

French, N.R. and Steinberg, J.C. (1947). “Factors Governing the Intelligibility of Speech Sounds.” Journal of the Acoustical Society of America, 19(1), 90–119.

ITU-R BS.1116-3 (2015). “Methods for the Subjective Assessment of Small Impairments in Audio Systems.” International Telecommunication Union.

12 Appendix C: The Transmit Audio Chain

Technical Operation and Operator Guidance

The transmit audio processing chain in WDSP transforms a microphone signal into a conditioned, band-limited signal ready for final up-conversion and transmission in the radio's firmware and hardware. The main body of this document describes each processing block from a developer's perspective: how to create and configure instances, what each function parameter controls, and how to wire blocks together. This appendix serves a different purpose. It describes what each block actually does to the audio, why it is designed the way it is, and how an operator can make informed choices to get the most from it.

The blocks described here are those most relevant to SSB voice operation and most likely to benefit from deeper understanding. Blocks that are essentially transparent to the operator — resamplers, bandpass filters, modulators — are omitted to simplify this discussion.

The figure below shows the signal flow through the audio processing chain for SSB voice operation, following the path from microphone input, through the software, and to the transmitter firmware and hardware. Metering taps, shown to the right, provide visibility into the signal at key points in the chain and are essential for setting levels correctly across the processing stages.

It is understood that "Microphone Input" may be accomplished through the CODEC in the radio hardware, or through VAC or some other interface such as a fast ASIO within the computer. If present, a WAV player in the radio console is also a common input for testing and on-air recording playback. All the same comments apply, no matter which is the source.

The output of these processing blocks is shown as going "To transmitter". This output is a fully modulated and prepared stream of complex (I and Q components) samples at the specified output rate for the WDSP transmitter channel. The signal is at "baseband", meaning it is centered on a frequency of zero Hertz, with positive and negative frequencies above and below zero. The signal has been adjusted and controlled in amplitude by the preceding stages and the ALC, it is been pre-distorted if PureSignal is enabled, and it has been frequency-compensated for any non-flat frequency responses anticipated in the firmware up-conversion process that is to follow.

The transmit signal is then converted to an integer data format and transferred via a network to the firmware/hardware. Two key high-speed functions are required of the firmware: (1) Interpolate to convert the sample rate up to that of the DAC, and (2) After interpolation, mix the signal with an oscillator tuned to the transmit frequency to bring it from baseband up to that transmit frequency. Following that, it's off to hardware – the DAC and the following amplifier chain.

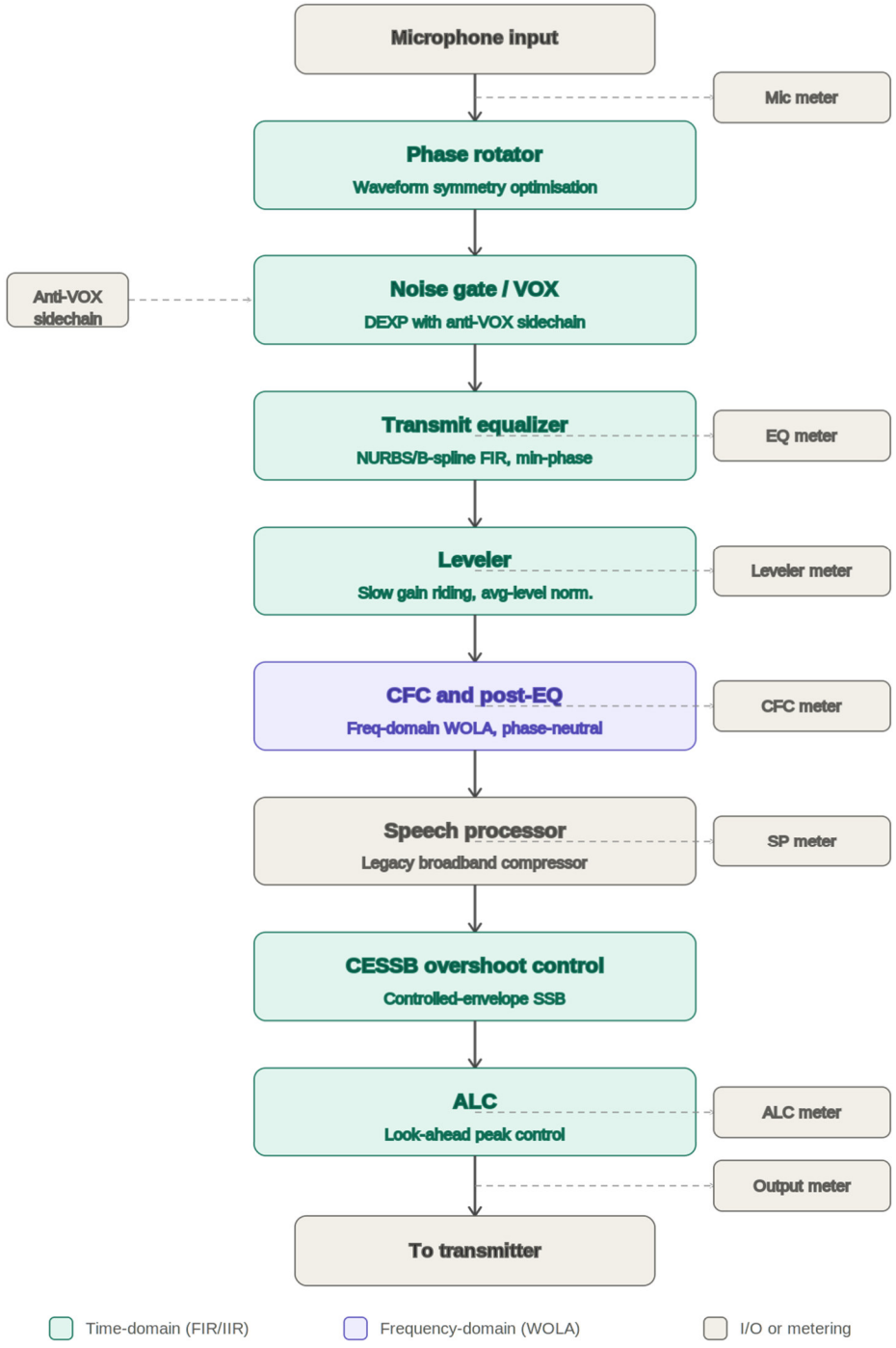


Figure C-1: Signal flow through the TXA transmit audio processing chain for SSB voice operation.

Two processing architectures are represented in this chain. The time-domain blocks — Phase Rotator, Noise Gate, Equalizer, Leveler, CESSB, and ALC — operate using finite impulse response (FIR) or infinite

impulse response (IIR) filters whose phase behavior is determined by the filter design, as discussed in Appendix B. The frequency-domain block — the CFC and its associated Post-EQ — operates using Weighted-Over-Lap-Add (WOLA) processing, which makes it phase-neutral by architectural design regardless of the gain profile applied.

12.1 Microphone / Audio Input

Audio input as monitored on the associated MIC Meter and controlled by gain adjustments is expected to peak in the vicinity of 0 dB. On a linear scale, inside the software, that equates to a magnitude value of 1.0. It's clear that levels vary continuously given the nature of speech and that your readings will fluctuate. Considering this, the following stages of the audio pipeline are designed to be somewhat tolerant of this reality. Some additional information on the nature and requirements of the input signal can be found in § 6.2 of this document.

12.2 Phase Rotator

Speech waveforms are inherently asymmetric: the positive and negative peaks of a typical voice signal are not equal in magnitude. How this asymmetry affects transmitted power depends on the type of transmitter, and the effects can be either positive or negative. The effects are sufficiently meaningful that it is worth taking time to examine and contrast how they can affect different types of transmitters and how we best handle them in SDRs.

Consider for the moment plate-modulated AM transmitters — the vacuum-tube designs still enjoyed by many operators for their vintage character — the modulator varies the plate voltage of the final amplifier to create the AM envelope. These transmitters can sustain positive modulation peaks beyond 100%: the modulator boosts the plate voltage above its quiescent value on positive audio peaks, extracting additional power from the supply. In this context, an asymmetric audio waveform with larger positive peaks than negative is sometimes deliberately cultivated, because those enhanced positive peaks increase average modulation depth and therefore average received signal strength. Both commercial broadcast and amateur transmitters have used this technique. The asymmetric speech waveform is beneficial.

Modern SDR transmitters operate on a fundamentally different principle. The audio signal passes through a digital-to-analog converter whose output range is fixed and cannot be exceeded. There is no mechanism by which a positive peak can draw additional power from the supply; the DAC simply clips at its maximum value. In this environment, waveform asymmetry is a liability. When the larger polarity reaches the DAC ceiling first, the opposite polarity still has headroom to spare but cannot use it. The transmitter is being peak-limited on one polarity while leaving average power on the table on the other. The correct approach is the opposite of the vintage AM case: make the waveform as symmetric as possible, so that both polarities approach the peak limit equally and average power is maximized.

The Phase Rotator addresses this by passing the audio through a cascade of all-pass filter stages. Unlike an equalizer, an all-pass filter does not boost or cut any frequency — the amplitude response is flat

across the entire audio band. What it changes is the phase relationship between frequencies: different frequencies are delayed by different amounts, which alters the shape of the time-domain waveform without affecting what a spectrum analyzer would show. Each all-pass stage produces 0° of phase shift at DC, 90° at the corner frequency fc, and 180° at its Nyquist frequency. Frequencies above fc receive more total phase shift than frequencies below fc; this differential rotation across the audio band reshapes the waveform toward greater symmetry. For our implementation, four all-pass stages is the default and provides a good balance for typical voice signals; the number of stages is usually held as an internal parameter and not normally exposed to the operator.

The result from our phase rotator is a lower peak-to-average ratio. The transmitter can be driven to a higher average level before the peak limit is reached, effectively delivering more average power on the air. Typical asymmetry improvement is 2–15 percentage points, depending on voice, microphone, and acoustic environment.

> Note that the phase rotator alters phase relationships between frequencies and should be disabled for digital modes that require a coherent phase-versus-frequency relationship.

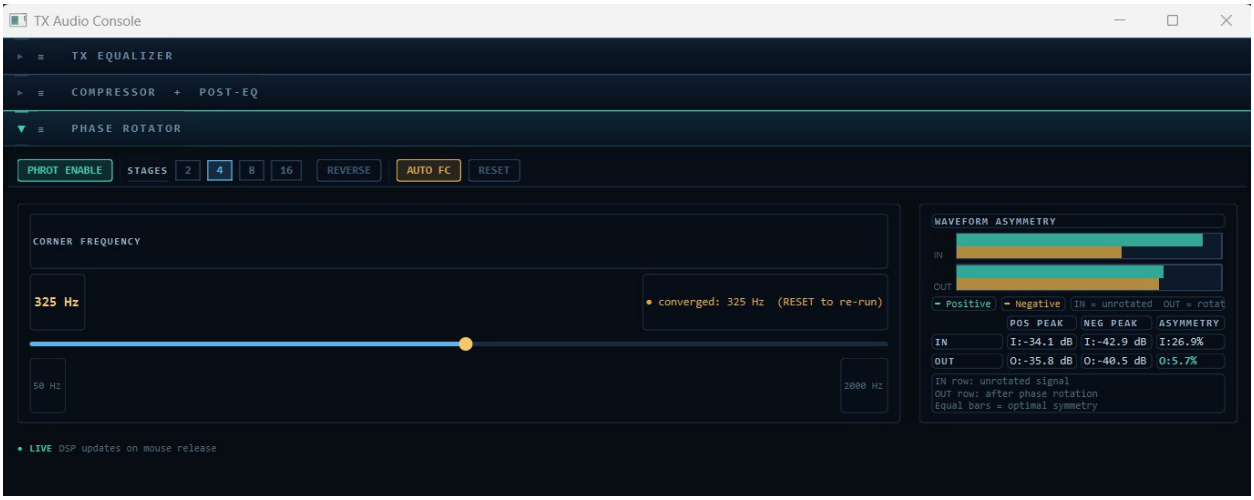


Figure C-2: Phase Rotator panel in the TX Audio Console, showing the corner frequency slider, optimizer status (converged at 325 Hz), and waveform asymmetry display.

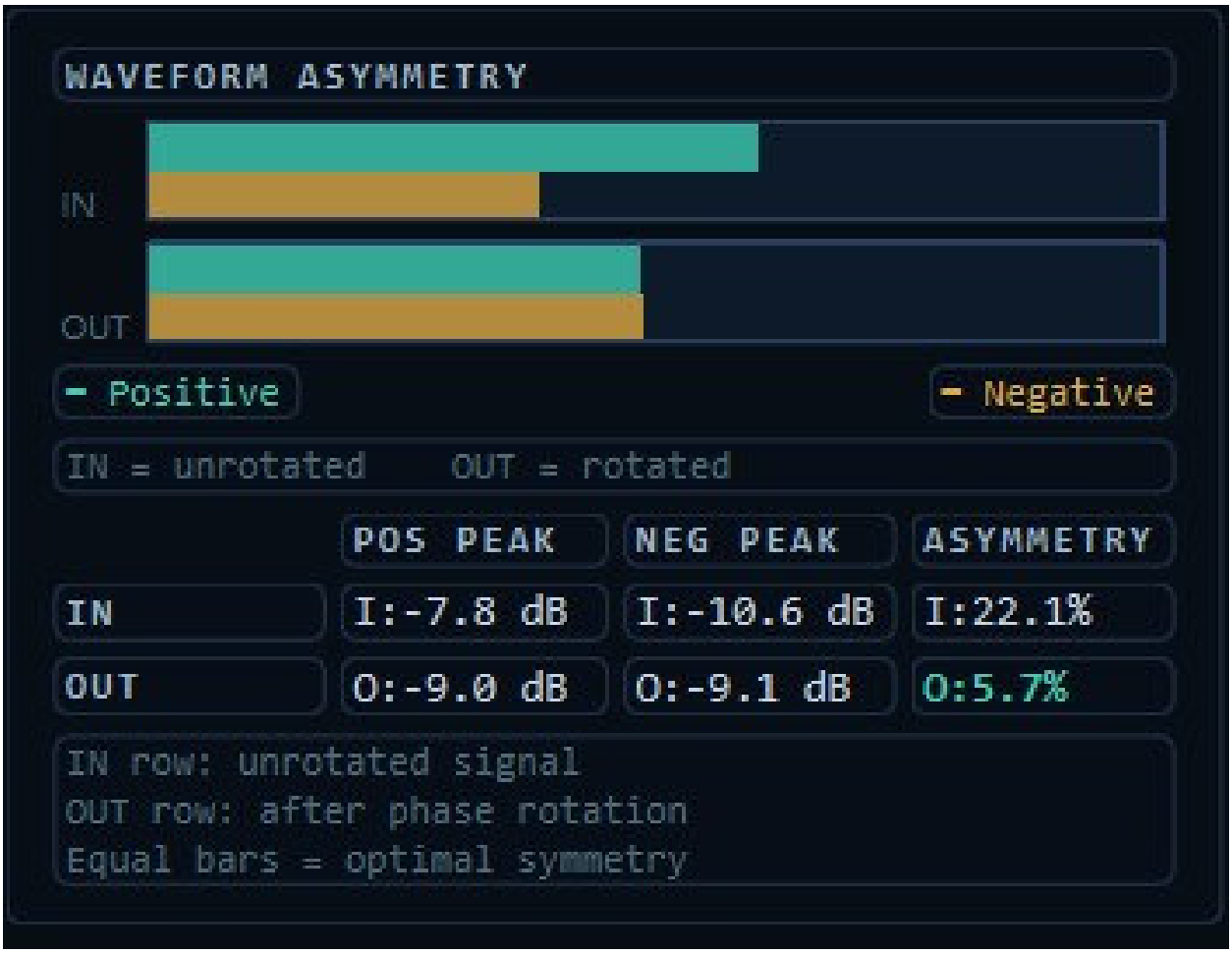


Figure C-3: Close-up of the Waveform Asymmetry display. IN row (unrotated): 22.1% asymmetry. OUT row (phase-rotated): 5.7% asymmetry — highlighted in teal when meaningfully better than the input.

Corner frequency and sensitivity

The corner frequency fc determines where the all-pass network centers its phase shift, and the optimal value varies with the individual voice, microphone characteristics, and room acoustics. This is why the control is adjustable: the best setting is personal, and what works well for one operator's voice and setup may not be optimal for another's.

That said, sensitivity to the exact corner frequency is low across a broad range. Typical voice material shows a wide, shallow minimum of asymmetry when it’s in the 200–400 Hz region: corner frequencies anywhere in this range produce similar asymmetry improvement, with noticeably worse results only at the extremes — below roughly 100 Hz or above roughly 1000 Hz for four stages. This behavior is consistent with commercial broadcast phase rotators; the Orban Optimod, for example, commonly uses a fixed corner frequency in the 250–350 Hz range. An operator who sets the corner frequency anywhere in the 200–400 Hz region by ear or by judgment is unlikely to be far from optimal.

The auto-optimizer

Rather than requiring manual adjustment, the Phase Rotator includes an automatic optimizer that adjusts the corner frequency to minimize waveform asymmetry. The optimizer uses an A/B comparison approach: it holds the filter at a candidate frequency for approximately 10–15 seconds, measuring the resulting asymmetry ratio, then compares against an adjacent candidate frequency. Convergence from a cold start typically takes 3–5 minutes of sustained voice activity. Once converged, the corner frequency is locked and the optimizer stops unless RESET is pressed to do a new calibration.

Because the asymmetry minimum is broad and flat, and because speech varies, successive optimizer runs may not converge to exactly the same corner frequency. This is normal behavior, not a deficiency: any frequency in the shallow minimum region produces similar performance, and the optimizer is finding one of many acceptable solutions rather than a single sharp optimum. The asymmetry display allows the operator to verify that the rotated output is genuinely more symmetric than the input regardless of where the optimizer settles.

The RESET function restarts the optimizer search from the default corner frequency (338 Hz). This is appropriate after a significant change in voice, microphone, or room acoustics.

Using the asymmetry display

The Waveform Asymmetry display shows two pairs of bars, one pair for input positive and negative peaks and the other for output positive and negative peaks. It also shows, below that, numerical values for input and output. The IN indicators reflect the unrotated input signal, and the OUT indicators reflect the phase-rotated output. The bars displaying positive and negative peak envelopes side by side, make the degree of asymmetry immediately visible as a difference in bar lengths. In the numerical values, the OUT in is highlighted in teal when the rotated output is meaningfully more symmetric than the input.

Watching this display during normal speech is the most direct way to verify that the phase rotator is operating correctly and providing a useful benefit. If the IN and OUT indicators show similar asymmetry, the rotator is having little effect for this particular voice and setup — which can happen, since some voices are naturally more symmetric than others. In that case the rotator can be left enabled (it does no harm) or disabled if simplicity is preferred.

Placement in the chain

The phase rotator acts on the microphone signal before equalization, compression, or any other processing. This placement ensures that the downstream processing acts on a waveform that is already as symmetric as the rotator can make it, and that the optimizer is working with the unmodified voice signal rather than a spectrally or dynamically altered version of it.

12.3 Noise Gate, VOX, and Anti-VOX

Most operators are familiar with VOX — voice-operated transmit switching — and with anti-VOX, which prevents received audio from falsely triggering the transmitter. These functions are generally well

understood and need little introduction. What is less familiar, because it has rarely appeared in conventional radio equipment, is the downward expander (DEXP) that is integrated with the VOX system in this implementation. Understanding the state machine that drives the DEXP is the key to getting the most from this block.

> Note that a simpler noise gate is also available. This earlier implementation provides straightforward gating between a muted and unmuted level with raised-cosine transitions between the two. The DEXP described here supersedes it for most voice applications, offering VOX integration, the side-channel trigger filter, lookahead delay, and hysteresis — all of which produces much more natural-sounding speech for SSB VOX operations. Without VOX, the DEXP provides graceful suppression of background noise during SSB and AM transmissions. Anecdotally, many operators report that listeners are unable to distinguish between PTT and VOX operation using this DEXP/VOX implementation.

The state machine

The DEXP operates as a five-state machine. The diagram of Figure C-4 shows signal level over time (upper track) and the corresponding gain state (lower track).

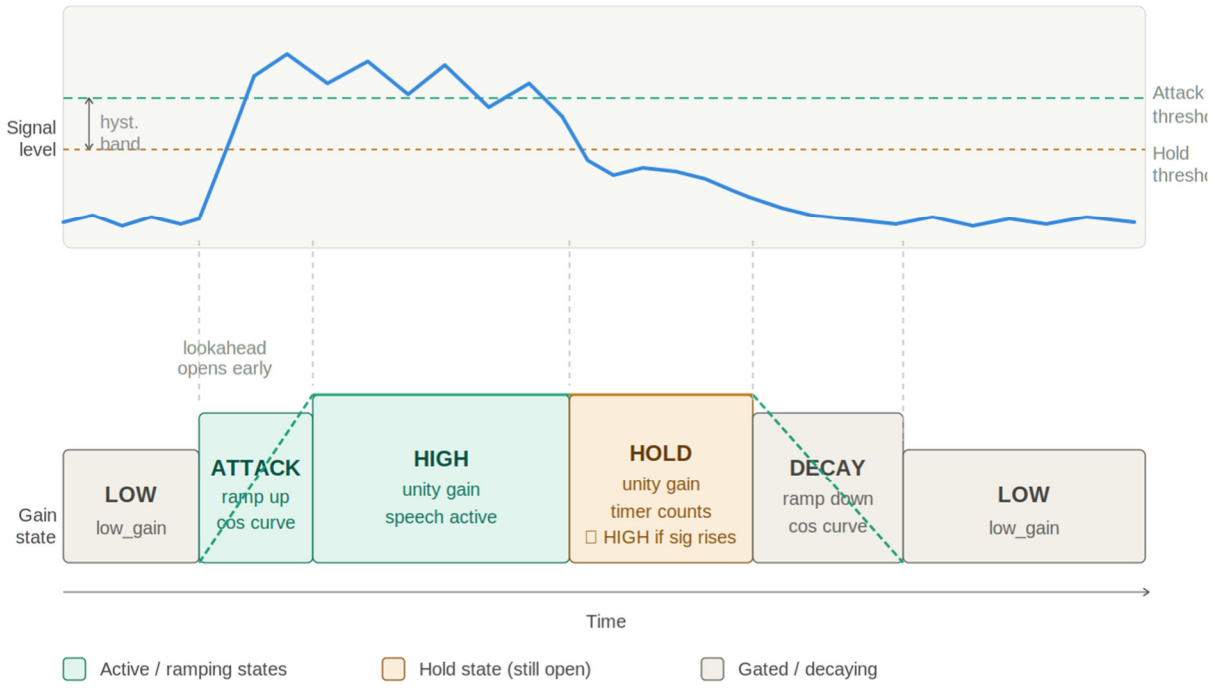


Figure C-4: DEXP state machine timeline. Upper track: input signal level versus the attack threshold (teal dashed) and hold threshold (amber dashed). Lower track: the five gain states and the raised-cosine gain transitions between them.

The five states are:

LOW: The gate is closed. The signal is below the attack threshold and the expander applies low_gain (= 1.0 / exp_ratio) to the audio, attenuating background noise by the expansion ratio. Anti-VOX is

calculated only in this state, since it is only relevant when the transmitter is not already triggered. If VOX is enabled, the LOW state equates to the radio being in the Receive state – from a listener standpoint downward expansion is infinite.

ATTACK: The signal has crossed the attack threshold. The expander ramps gain from low_gain up to unity using a raised-cosine curve over the attack time. This smooth, S-shaped ramp avoids the click or thump that an abrupt transition would produce. VOX transmit-on is triggered immediately when ATTACK begins.

HIGH: The signal is above the hold threshold and the expander applies unity gain. The audio passes through completely unattenuated. This is the normal state during active speech.

HOLD: The signal has dropped below the hold threshold (= attack_thresh × hysteresis_ratio), initiating a hold timer. The expander remains at unity gain for the entire hold period — the audio is still fully open. If the signal rises above the attack threshold again during HOLD, the state returns immediately to HIGH, preventing the gate from closing between words. When the hold timer expires, the state transitions to DECAY.

DECAY: The expander ramps gain from unity back down to low_gain using a raised-cosine curve over the decay (release) time, producing a smooth, natural fade-out of any trailing audio before the gate closes.

The expansion ratio and low_gain floor

The expansion ratio sets the attenuation applied in the LOW state: low_gain = 1.0 / exp_ratio. An expansion ratio of 10 dB (linear ≈ 3.16) gives a low_gain of about 0.316, attenuating background noise by 10 dB while the gate is closed. Higher ratios produce deeper noise suppression but also make the ATTACK transition more audible if the attack time is very short, since the gain change is larger. The Thetis default of 10.0 dB is a reasonable starting point for most environments.

Hysteresis and the hold threshold

Hysteresis separates the level at which the expander opens (attack_thresh) from the level at which hold-time begins (hold_thresh = hysteresis_ratio × attack_thresh). The hysteresis band between these two levels stabilizes operation when the signal is near the threshold. Without hysteresis, a signal hovering at the attack threshold would cause rapid, audible oscillation between ATTACK and the beginning of HOLD. With hysteresis, the signal must drop a defined amount below the attack threshold before the hold timer begins. The Thetis default of 2.0 dB provides a practical band for most operating environments.

The look-ahead delay

A conventional gate reacts after the fact: it detects a signal crossing the threshold and then begins the ATTACK ramp, but the very first samples of the triggering audio have already passed through at low_gain. This clips the onset of words beginning with soft consonants — an effect that is immediately audible as a cut-off on the first sound of each transmission.

The look-ahead delay solves this by routing a copy of the signal to the detector without delay, while the audio path itself is delayed by the look-ahead time. The detector therefore sees the signal arriving slightly before it reaches the gate, allowing ATTACK to begin before the leading edge of the audio actually needs to pass through. When the look-ahead is set correctly, the first sound of each word is transmitted fully intact. The Thetis default of 60 ms provides ample look-ahead for typical speech transients.

The side-channel trigger filter

The trigger detector can optionally be bandpass filtered before use, responding only to frequencies in a specified range — typically the core speech band. The Thetis default of 500 Hz to 1500 Hz focuses the detector on frequencies that are highly characteristic of voice while rejecting low-frequency thumps, high-frequency hiss, and broadband noise that might cause false triggers. A gain compensation factor is applied to the filtered trigger signal to account for the energy reduction of bandpass filtering, keeping the effective threshold consistent whether the filter is enabled or not.

Importantly, only the trigger detector uses the filtered signal. The audio path uses a separately compensated delay to maintain time alignment with the detector; the operator hears the full-bandwidth audio.

Detection averaging (Det. Tau)

The trigger detector does not respond instantaneously to the raw signal level. Instead, it applies a first-order low-pass filter with time constant dettau before comparing against the threshold. This averaging prevents brief transients — a single loud consonant, a room reflection — from triggering a full ATTACK/DECAY cycle. The Thetis default of 20 ms provides moderate smoothing; shorter values make the detector more responsive, longer values make it more immune to transients.

VOX and anti-VOX integration

When VOX operation is enabled, VOX uses the same detector and threshold as the DEXP, so a single threshold setting controls both functions simultaneously. When ATTACK begins, the ‘pushvox’ callback fires immediately to switch the transmitter on. The transmitter stays on for the duration of HIGH and HOLD states, and switches off only after the DECAY completes — ensuring that the trailing edge of audio is transmitted before PTT releases.

Anti-VOX feeds a smoothed, gain-weighted sample of the received audio into the detector as a subtracting signal, but only in the LOW state. Once the transmitter is already active, anti-VOX has no effect. The anti-VOX gain and tau settings determine how strongly and how smoothly the received audio raises the effective threshold, preventing received speaker audio from leaking into the microphone and falsely triggering VOX, without making it impossible for the operator to break in intentionally.

Practical setup

Threshold (dBV): Set just above the ambient noise floor. Observe the detector level during a quiet pause — room noise present, no voice — and set the threshold just above that level. Too low and background noise will trigger the gate; too high and soft speech or word endings will be clipped.

Exp. Ratio: Start with the default (10 dB). Increase only if background noise is particularly difficult to suppress. Very high ratios make the ATTACK transition more audible because the gain change from LOW to HIGH is larger.

Hyst. Ratio: The default (2.0 dB) is appropriate for most environments. Increase slightly if the gate chatters near the threshold on normal speech.

Attack: Short (2–10 ms) for tight, responsive gating. The raised-cosine shape keeps even short attack times from sounding abrupt.

Hold: Long enough that the gate does not close between words during normal speech. The default (250 ms) handles typical conversational pauses well.

Release: The default (100 ms) produces a natural fade. Longer values give a more gradual tail but may leave the transmitter open slightly longer at the end of each over.

Det. Tau: The default (20 ms) is a good starting point. Reduce if the gate feels sluggish on fast speech; increase if brief transients are causing false triggers.

Side-channel filter: Enable with default frequencies (500–1500 Hz) unless operating in an acoustically clean environment. The filter improves threshold stability in the presence of broadband noise.

Look-ahead: Enable with the default (60 ms). There is rarely a reason to disable this.

12.4 Transmit Equalizer

The transmit equalizer allows the operator to shape the frequency response of the audio before it reaches the compression and modulation stages. Unlike a receive equalizer, which compensates for deficiencies in the listening path, the transmit equalizer shapes what is actually transmitted — its effect is heard by every station receiving the signal, not just locally.

The equalizer uses a continuous frequency response curve rather than a fixed set of frequency bands. Gain is specified at any number of control points, and the response is smoothly interpolated between them using B-spline or NURBS curves; the full curve system is described in Appendix A. The practical consequence for the operator is that the response can be shaped with whatever precision the situation requires — a gentle broad lift across the upper midrange, a sharp notch at a specific problem frequency, or an elaborate multi-region profile — all without the stepped, band-constrained behavior of conventional graphic equalizers.

The filter is implemented in minimum-phase mode. As discussed in Appendix B, this means the filter introduces the minimum phase shift consistent with the specified amplitude response, eliminating the latency of linear-phase designs while producing results that are audibly indistinguishable over HF propagation paths. For monitoring purposes, the equalizer meter shows the signal level after the EQ

filter has been applied, giving a representative view of what is being sent downstream to the leveler and compressor stages.

What the transmit equalizer is for

Voice signals from a microphone vary considerably in spectral character depending on the microphone type, the acoustic environment, the operator's voice, and the distance and angle to the microphone. The transmit equalizer gives the operator control over this spectral character before it is locked in by the transmitter.

The goals vary by operator and application. Some operators use the equalizer primarily for correction — flattening a microphone with a peaked response, reducing proximity-effect bass boost from close-talking technique, or taming a harsh room resonance. Others use it for deliberate shaping — adding presence in the 2–3 kHz region to improve intelligibility on marginal paths, rolling off low frequencies below 200–300 Hz to reduce low-frequency noise transmission and focus energy in the speech band, or adding a subtle high-frequency lift to compensate for the gentle roll-off that HF propagation tends to introduce on longer paths. Many operators combine both: correct first, then shape.

Thinking about the frequency regions

For SSB voice, the transmitted passband is typically 100–3000 Hz or similar, set by the bandpass filters surrounding the audio chain. Within this passband, the spectral character of the voice determines intelligibility and tonal quality at the receiving end. ESSB operation expands this range for additional character and intelligibility.

The low frequencies — roughly 50–300 Hz — carry the fundamental pitch and body of the voice. Too much energy here makes the signal sound thick or muddy, and on a congested band it can reduce intelligibility. Too little removes warmth and makes the voice sound thin. Most operators benefit from a gentle low-frequency roll-off if the microphone or room is contributing energy in this region. However, consider that the importance of this range must not be underestimated and it should be carefully adjusted. Proper low frequency content can make a voice much more distinctive and pleasant and perhaps minimize listening fatigue.

The midrange — roughly 300 Hz to 1 kHz — contains the core of vowel intelligibility and the power of the voice. This region should generally be left relatively flat; dramatic cuts here thin the voice noticeably while dramatic boosts add congestion.

The upper midrange — roughly 1–3 kHz — is the most important region for intelligibility. Consonants, which carry the linguistic information that distinguishes words, are concentrated here. A moderate lift in this region, perhaps 3–6 dB centered around 2–2.5 kHz, is one of the most effective things an operator can do to improve copy on difficult paths. This is sometimes called the “presence” region and its effect is most audible on signals with fast, articulate speech.

The high frequencies — above 3 kHz — contribute definition but diminish rapidly with HF propagation. Within the normal SSB passband, energy above 3 kHz has limited intelligibility value on transoceanic or noisy band conditions, though it adds brightness on local or regional paths.

Figure C-5 shows a practical starting-point profile using seven control points and cubic NURBS interpolation. The curve provides a gentle high-pass character below 300 Hz, a relatively flat midrange through 1 kHz, and a smooth presence lift peaking at approximately +5 dB near 2.5 kHz before rolling off to the right endpoint. This is not a prescription — it is an illustration of the kind of gentle, purposeful shaping the equalizer is well-suited to produce. Each operator's voice, microphone, and acoustic environment will call for a different curve.



Figure C-5: TX Equalizer panel showing a seven-point cubic NURBS presence-lift profile. Low-frequency rolloff below 300 Hz, flat midrange, +5 dB presence peak near 2.5 kHz. Weights OFF; Cubic (3) degree.

Interaction with downstream processing

Because the equalizer sits upstream of the leveler and compressor stages, its effect on those stages matters. A large low-frequency boost increases the peak level of the signal entering the leveler, potentially causing more gain reduction in the leveler than intended — and since compression is non-linear, a spectrally uneven input produces spectrally uneven compression. An operator who shapes the equalizer with the following stages in mind will generally get cleaner results than one who treats the

equalizer as a totally independent correction tool. This interaction can be used very deliberately, for example rolling-off low-frequency energy before the following stages means that they are reacting primarily to the midrange and upper-midrange content — the frequencies that matter most for intelligibility — rather than to the overall amplitude including the bass. This is one reason why a transmit chain with a high-pass character in the equalizer often sounds more intelligible under heavy compression than one without.

Practical approach

The most reliable way to set the transmit equalizer is on-air monitoring with a cooperative receiving station, ideally using a known-quality receiver on a path that is representative of the operator's typical operating conditions. A flat or corrected starting point — compensating for known microphone and room anomalies — is a better foundation than starting from a heavily shaped profile.

Another effective technique, particularly when on-air monitoring is not immediately available, is to make a WAV recording of the microphone input and play it back in a loop through the transmitter input while listening to the transmitter monitor output. With the EQ adjustments applied in real time, the operator can hear the effect of each change on the same consistent source material without requiring a second station or a live transmission. Some consoles have built-in WAV recorders and players that make this technique straightforward.

If neither approach is immediately available, the equalizer meter in the metering chain gives a useful view of the spectral energy reaching the downstream stages, and local monitoring through the receiver's sidetone path (where available) can reveal gross problems. However, the final reference must always be how the signal sounds at the receiving end, since local monitoring bypasses the effects of the transmit bandpass filters and the propagation path.

12.5 Leveler

The Leveler is a slow-acting automatic gain control applied to the audio signal after the transmit equalizer. Its purpose is narrow and specific: to normalize the average level of the transmitted audio over time, compensating for variations in microphone technique, vocal effort, and the natural dynamics of speech. It is not a compressor in the conventional sense, and understanding the distinction matters for setting it correctly.

A compressor responds to instantaneous or short-term peaks in the signal, reducing gain quickly when levels spike and then increasing gain when levels subside. The result is increased average power by limiting peaks, but also a characteristic compression sound — sometimes reduced dynamics, or a dense quality that experienced ears can identify. The Leveler operates on a fundamentally different timescale. With a decay time constant of 500 milliseconds, or more, it reacts to the average level of speech over several syllables rather than to individual peaks. The effect is closer to a skilled operator manually riding a gain control than to any form of dynamic processing: it keeps the overall level consistent without significantly affecting the moment-to-moment dynamics of the voice.

The look-ahead ring buffer

The gain-control code used for the Leveler — and for the ALC — employs a ring buffer that introduces a true look-ahead into the gain computation. The output sample being delivered at any moment was written into the ring buffer a short time earlier, while the gain being applied to it is computed from the maximum signal level observed across that interval — including samples that immediately follow the current output sample in the buffer. The gain control therefore responds to what is coming, not only to what has just passed. This eliminates the overshoot that a feedback-based AGC produces on signal onsets: the Leveler has already begun adjusting gain before the peak arrives at the output.

How the Leveler differs from the ALC

Both the Leveler and the ALC use the same underlying code, but their initialization parameters make them behave quite differently. The most important difference is the decay time constant: 500 milliseconds for the Leveler versus 10 milliseconds for the ALC. This fifty-fold difference in decay rate is what gives each block its character. The Leveler rides slowly over the natural shape of speech, averaging out differences between loud and soft passages without audibly pumping. The ALC responds rapidly to prevent the transmitter output from exceeding its ceiling, acting more like a peak limiter.

A second difference is in what is measured. The Leveler measures the maximum of the I and Q signal components, while the ALC measures the true envelope magnitude — the more conservative and precise measurement for protecting the transmitter output.

By default, the maximum gain of the Leveler is limited to approximately +5 dB. This prevents the Leveler from boosting a very quiet passage excessively, which would amplify noise or room ambience to audible levels. The ALC has a maximum gain of unity — it can only attenuate, never boost.

What the Leveler is and is not for

The Leveler also serves a purpose that is easy to overlook: it presents a consistent, known amplitude level to the downstream stages. The CFC and speech processor are calibrated against a signal peaking at approximately 0 dB. Without the Leveler, it falls entirely on the operator to ensure that the audio arriving at those stages is consistently at the right level — a task that requires careful attention to microphone gain and technique on every transmission. With the Leveler active, that normalization happens automatically, and the downstream processing behaves predictably regardless of how the upstream level varies.

The Leveler is appropriate when the operator's microphone level or vocal effort varies significantly from session to session or within a session, and the goal is to keep average transmitted level consistent without resorting to heavy compression. It is particularly useful for operators who move relative to the microphone, or who naturally produce a wide range of vocal intensities across a conversation.

The Leveler is not a substitute for setting the microphone gain correctly in the first place. If the input level is consistently too low or too high, the microphone gain or the transmitter drive should be adjusted directly. The Leveler works best when the signal arriving at it is already at a reasonable level and the task is simply to smooth out variation.

The Leveler is also not intended to increase average power in the way that a compressor or the CFC does. Its slow time constant means it cannot respond to the short-term peaks that limit average power in an SSB signal. Operators seeking to maximize average power should look to the CFC and CESSB rather than to the Leveler.

Practical settings

The attack time (1 millisecond default) is fast enough that the Leveler responds promptly to the onset of a loud passage, preventing a brief loud moment from causing a prolonged gain reduction. The decay time (500 millisecond default) is the primary character-defining parameter and is generally best left at or near its default value. Shortening the decay time moves the Leveler toward compressor behavior; lengthening it makes the gain riding more gradual but means it takes longer to recover from a loud passage.

The maximum gain setting (default +5 dB) is a ceiling on how much the Leveler can boost a quiet signal. Increasing it allows more work on very quiet passages but risks raising the noise floor audibly if the signal drops below useful levels. In Thetis, the controls exposed for the Leveler are Maximum Gain and Decay; the attack time is set at a sensible default and is not normally adjusted.

The Leveler meter shows the signal level after the Leveler has acted, as well as the gain being applied. Watching the gain display during normal speech gives a clear picture of how much work the Leveler is doing. If the gain is frequently at its maximum limit and then relaxing, the input level may be too variable for the Leveler alone to handle gracefully; attention to microphone technique or gain staging upstream may be more effective.

12.6 Continuous Frequency Compressor and Post-EQ

The Continuous Frequency Compressor (CFC) is the most powerful and flexible processing block in the transmit audio chain. It is also the one most likely to reward a deeper understanding of how it works, because the relationship between its controls and their effect on the signal is less intuitive than a conventional compressor. This section describes the underlying architecture, explains what the controls actually do, discusses an important challenge in setting up any multiband compressor and how the Post-EQ addresses it, and offers guidance on using the CFC effectively.

The WOLA architecture and why it matters

The CFC operates in the frequency domain using Weighted-Over-Lap-Add (WOLA) processing. The signal is divided into overlapping frames, each frame is transformed to the frequency domain by an FFT, gain adjustments are applied independently to each frequency bin, and the modified spectrum is converted back to the time domain by an inverse FFT. Successive frames are overlap-added to reconstruct the output signal.

This architecture has two consequences that matter most to the operator.

First, the CFC processes each frequency bin independently. It can apply a different amount of compression to a 500 Hz bin than to a 2 kHz bin, and does so continuously across the entire spectrum, with the compression profile varying as smoothly as the NURBS curve system allows. This is categorically different from a multiband compressor, which divides the spectrum into a small number of fixed bands and applies a single compressor to each. The CFC has no bands — it has a continuous compression profile across as many frequency bins as the FFT size provides.

Second, the WOLA architecture is inherently phase-neutral. Each bin's magnitude is modified independently, but the phase of each bin is preserved exactly. Unlike a time-domain compressor, which changes the signal envelope in ways that introduce phase-related artifacts, the CFC does not alter the phase relationships between frequency components. The spectral balance of the signal changes; the phase structure does not. This is directly analogous to the receive spectral noise reduction and is discussed further in Appendix B.

It is the WOLA architecture that allows the dynamic (frame-by-frame) modification of individual frequency bins and then conversion back to the time-domain without introducing distortion. This is fundamentally different than the Transmit Equalizer which, each time the operator makes a change, designs a new filter implementing the response curve. This architecture is more flexible, at the cost of increased CPU cycles and increased latency.

How the compression decision is made — and what it is not

For each frequency bin in each frame, the CFC computes the product of a programmed gain (derived from the compression profile) and the magnitude of that bin. If the product would exceed unity — meaning the programmed gain would drive that bin's output above the reference level — the gain applied to that bin is reduced to exactly normalize it to unity. If the product is at or below unity, the full programmed gain is applied and no reduction occurs in that bin.

It is important to understand what this process is and what it is not. It is not clipping, and, while all compressors change/distort the signal in some way, this is not distortion in the conventional sense. Consider a pure sine wave at a frequency that falls in a single bin. If the compression stage normalizes that bin to unity, the inverse FFT produces a perfect sine wave at the output — same frequency, same phase, simply at a different amplitude. No harmonics are introduced, no intermodulation products, no waveform distortion. What changes is the amplitude of that bin relative to the others, which changes the spectral balance of the reconstructed signal. The same principle applies to the complex voice signal: the CFC reshapes the spectrum, but the signal in each frequency region remains undistorted.

This per-bin independence also means the CFC does not pump the way a broadband compressor can. A broadband compressor's gain is determined by the loudest components in the signal at any moment; when a loud low-frequency transient occurs, gain drops across the entire spectrum, causing the higher frequencies to be momentarily suppressed even though they contain the speech content the operator most wants to preserve. In the CFC, a loud low-frequency component causes gain reduction only in the low-frequency bins where it is present. The upper-midrange bins, carrying the consonant information most critical for intelligibility, are unaffected.

The compression profile (G curve)

The compression profile specifies, at each of a set of control-point frequencies, how much gain to attempt to apply to that frequency region. Higher values in the profile mean more drive in that region: the programmed gain is more likely to push bin magnitudes above the unity threshold, causing more limiting in those bins. Lower values mean the content in that region reaches the threshold less often and passes through with little or no gain reduction.

The profile is set using the same NURBS/B-spline curve system as the transmit equalizer, so the compression can be specified as smoothly as desired between control points. A typical voice-optimized profile applies moderate compression in the lower frequencies, heavier compression in the 1–3 kHz presence region where intelligibility is concentrated, and tapers off above 3 kHz where HF propagation quickly removes the benefit. Figure C-6 shows an example of such a profile.



Figure C-6: CFC Compressor panel showing a voice-optimized profile tested with a male voice WAV recording. Six control points, Cubic (3), Weights OFF. Pre-Comp G[0] = +16 dB drives meaningful compression across the speech band; the bar graph display shows per-bin compression activity. Control points: 200 Hz +3.0 dB, 500 Hz +5.0 dB, 1k +8.0 dB, 2k +10.0 dB, 3k +10.0 dB, 4k +6.0 dB.

The pre-compression gain

The pre-compression gain is a flat gain applied uniformly across all frequency bins before the compression decision. It functions as a master drive control: increasing it pushes more bins above the unity threshold, increasing overall compression depth. Decreasing it reduces compression across the board.

In practice, the pre-compression gain and the profile shape work together. The profile determines the spectral distribution of compression — which frequency regions are compressed more and which less. The pre-compression gain then determines how aggressively the entire profile is applied. A practical setup approach is to establish the profile shape first, then adjust the pre-compression gain to set the overall compression depth as shown on the compression display.

The multiband reconstruction problem and the role of the Post-EQ

Any multiband compressor — including the CFC — faces a challenge that does not arise with broadband compression: the output amplitude after reconstruction can differ significantly from the input amplitude.

The reason is straightforward. The input time-domain signal has some peak amplitude. After the forward FFT, its energy is distributed across frequency bins. Heavy compression can drive many bins simultaneously toward or at their unity ceiling. When the inverse FFT reconstructs the time-domain signal from those bins, the contributions from all active bins sum together. If many bins are simultaneously at or near their ceiling, the reconstructed peak amplitude can be substantially higher than the input — potentially by several dB under heavy compression settings.

In practice, voice signals are spectrally sparse in the sense that matters here. Voiced speech has energy concentrated at the fundamental and its harmonics, with quieter inter-harmonic content. At any given moment, only a subset of bins carry significant energy and reach the compression ceiling. This makes the reconstruction amplitude problem less severe than a worst-case analysis might suggest, and it is intermittent rather than constant. Nevertheless, it is real, and an operator running aggressive compression settings will find the output level higher than the input level.

The Post-EQ is specifically placed to address this. Because it operates after the compression stage but before the inverse FFT, it can reduce the magnitude of selected bins — or all bins uniformly — while still in the frequency domain, before they recombine in the time-domain reconstruction. By rolling off the Post-EQ gain in the frequency regions where compression is heaviest, the operator can bring the reconstructed output level back into a controlled range without sacrificing the spectral shaping benefit of the compression profile.

The pre-EQ gain parameter — a flat gain applied uniformly across all Post-EQ bins — is the most direct tool for this level management. Setting it to a negative value (in dB) trims the overall output level of the CFC block without affecting the relative spectral shape of the Post-EQ curve. The shaped Post-EQ curve then provides additional frequency-selective adjustment on top of this uniform trim. In the example shown in Figures C-6 and C-7, a Pre-Comp of +16 dB drives aggressive multi-bin compression, and a Pre-

EQ of −20 dB restores the output to a level appropriate for the downstream ALC — illustrating how substantially the multi-bin reconstruction can elevate the output under heavy compression. Operators pushing the pre-compression gain hard should expect to use correspondingly large negative pre-EQ values, and should ensure their console exposes sufficient range on both controls.

A well-matched pair of compression profile and Post-EQ is one where the CFC delivers a full, spectrally shaped signal at a level that does not unnecessarily drive the downstream ALC. The ALC is always present as a safety net, but treating it as a compression stage — letting it work hard because the CFC output is too high — defeats the careful spectral control that the CFC provides. Figure C-7 shows a Post-EQ that complements the compression profile of Figure C-6.



Figure C-7: Post-EQ panel complementing the compression profile of Figure C-6, tested with the same male voice WAV recording. Five control points, Cubic (3), Weights OFF. Pre-EQ G[0] = −20 dB provides the primary level management. Control points: 200 Hz 0.0 dB, 800 Hz +2.0 dB, 2k +1.0 dB, 3.5k −2.0 dB, 4.5k −5.0 dB.

The compression display

The CFC maintains, for each frequency bin, a record of how much compression is being applied — the difference between the programmed gain and the actual gain applied after the limiting decision. This is tracked with a peak-hold and decay, and is available for real-time display. The compression display shows this per-bin compression depth across the audio spectrum, giving the operator a continuous view

of where and how much compression is occurring. The bar graph visible in Figure C-6 shows this display during live voice input: compression is concentrated in the lower speech band where the profile gain is rising, with progressively less activity in the bins above the bulk of the voiced energy. This kind of display confirms that the compressor is working where intended and gives immediate feedback when the pre-compression gain is adjusted.

Practical guidance

The CFC works best when the signal arriving at it is at a consistent level — which is one reason the Leveler upstream is useful. Variable input levels cause variable compression behavior, since each bin's limiting decision depends on the actual signal amplitude in that frame.

The interaction between the CFC and the transmit equalizer upstream is also worth considering. The transmit equalizer shapes the spectrum before it reaches the CFC, so the compression decisions are made on the pre-shaped spectrum. An operator who uses the transmit equalizer to roll off low-frequency content will find that the CFC compresses less in the low end regardless of the compression profile, because the low-frequency bins carry less energy. This is usually desirable: it concentrates compression energy in the speech band.

For digital modes, the CFC should be disabled. While the WOLA processing preserves phase within each bin, the nonlinear amplitude decisions applied per-bin introduce spectral products that can be detrimental to digital modulation schemes.

12.7 Speech Processor

The Speech Processor is a legacy compression block that predates the CFC. It remains in the processing chain and continues to work well for its intended purpose; however, for most operators setting up a new transmit audio chain, the CFC offers substantially more capability and flexibility. This section explains what the Speech Processor does, why it produces the results it does, and under what circumstances an operator might still choose to use it.

Historical context

Before frequency-domain processing was practical in software radio implementations, one standard approach to speech compression for SSB transmitters was the RF speech clipper. The RF clipper operates on the actual modulated SSB carrier at IF or RF: the signal is amplified until its envelope exceeds the clipping threshold, hard-clipped, and then bandpass-filtered to remove the harmonics and intermodulation products that clipping introduces. The bandpass filter is essential — without it, the clipped signal has severe spectral splatter. With it, the result is an SSB signal with increased average power relative to its peak, at the cost of some waveform distortion that the bandpass filter cannot completely remove.

The RF clipper was a practical and effective technique for hardware transmitters, and many experienced operators associate it with a particular “processed” sound that cuts through noise effectively on difficult paths. Its main limitation is that the clipping introduces genuine nonlinear distortion — harmonics and

intermodulation products among the audio frequencies — that the subsequent bandpass filter reduces but cannot eliminate.

The baseband implementation

The Speech Processor in WDSP implements the equivalent of RF clipping at baseband, using the algorithm described by Peter Martinez, G3PLX, in the January 2010 issue of RadCom magazine.[5] It operates on the complex I-Q signal that represents the baseband SSB audio. The processing at each sample is:

1. Compute the envelope magnitude: mag = square_root($I^2 + Q^2$)
2. If gain × mag > 1.0 (above threshold): output I/mag, output Q = 0
3. If gain × mag ≤ 1.0 (below threshold): output I × gain, output Q = 0

The gain parameter sets the compression level: higher gain drives more of the signal above the unity threshold, producing more compression.

The sine wave property

The most striking property of this algorithm — and the one that distinguishes it from naive time-domain clipping — is that a pure sine wave input produces a pure sine wave output with no distortion.

For a single audio tone, the baseband SSB signal is the complex analytic signal I = A·cos(ωt), Q = A·sin(ωt), with envelope mag = A — a constant for a pure tone. When the envelope exceeds the threshold, the output is I/mag = cos(ωt): a perfect unit-amplitude cosine. Dividing by the envelope of a pure tone divides by a constant, which simply scales the waveform without introducing any harmonics or other distortion products. The output is spectrally pure.

For a real voice signal, the envelope varies with time and the algorithm is nonlinear in the general case — different gain is applied at each sample depending on the instantaneous envelope. This does introduce distortion, but because the algorithm operates on the complex envelope rather than directly on a carrier waveform, the character of that distortion is different from simple clipping.

The equivalence to RF speech processing

The baseband Speech Processor is mathematically equivalent to RF clipping followed by bandpass filtering, which explains why it has the characteristic sound and properties of an RF speech processor.

A conventional RF clipper operates on the modulated carrier $s(t) = A(t)\cdot\cos(\omega_t t + \phi(t))$, where A(t) is the audio envelope and ϕ(t) is the instantaneous phase carrying the audio information. Hard-clipping this signal limits its envelope to the clipping threshold but preserves the instantaneous phase ϕ(t), because clipping affects amplitude while zero-crossing positions — which encode the phase — are unchanged. After bandpass filtering to remove harmonics, the result is a signal whose envelope is constrained but whose phase modulation, and therefore whose audio content, is preserved.

The baseband compressor performs the same operation directly on the I-Q representation. The output I/mag is the cosine of the instantaneous phase angle of the complex signal: I/mag = cos(θ(t)) where θ(t)

encodes the audio. The envelope is normalized to unity; the phase is preserved exactly. This is precisely what the RF clipper plus bandpass filter achieves at RF, but performed at baseband where the analytic signal representation makes the envelope and phase explicitly available as separate quantities.

The baseband approach has a practical advantage over the RF approach: the bandpass filtering equivalent is simply the existing transmit bandpass filter already present in the chain — no additional filtering stage is needed. Setting Q to zero at each sample ensures the output is a real signal as needed for some modulation modes, and implicitly removes the quadrature component that the clipping operation would otherwise introduce.

This equivalence is why experienced operators familiar with RF speech processing recognize the character of the Speech Processor's output: it is not a coincidence or an approximation, it is the same operation expressed in a different domain.

When to use the Speech Processor

For a new installation, the CFC is the recommended choice for compression. It offers independent control over compression across the entire frequency spectrum, introduces no distortion in the sense described in Section C.5, and produces a more full, more spectrally controllable output.

The Speech Processor may still be appropriate in certain circumstances. Operators who prefer the specific character of RF-clipper-style processing, and who have tuned their transmit chain around it, have no reason to change. It is also simpler to configure — a single gain parameter — which may suit operators who want effective compression without the setup overhead of the CFC.

One important constraint to note for a later section: the Speech Processor must be active as a prerequisite for CESSB Overshoot Control. Operators who wish to use CESSB must have the Speech Processor enabled regardless of whether they are using it for compression. This dependency is discussed further in Section C.7.

The gain parameter

The gain parameter sets the compression level in dB. The compression is accurate when the input signal peaks at approximately 1.0; at lower input levels less compression is achieved, and at higher input levels more compression occurs. The Leveler upstream can help maintain a consistent input level, improving the predictability of the compression. The default of 3 dB is a conservative starting point; values of 6–10 dB are common for more assertive processing, with higher values producing a denser, more heavily compressed sound at the cost of increased distortion.

12.8 CESSB Overshoot Control

Controlled Envelope Single Sideband (CESSB) is a technique for increasing the average power of an SSB transmission while, if properly applied, introducing only minimal distortion. The concept was brought to the amateur radio community by Dave Hershberger, W9GR, in QEX Magazine.[2] This section describes

the problem CESSB solves, how the algorithm works, what it is and what it is not, why it requires multiple linear-phase bandpass filters, and how to use it effectively.

The problem CESSB solves

An SSB transmitter's average power is determined by the ratio of average envelope power to peak envelope power. The transmitter runs at full power only when the audio signal reaches its peak; the rest of the time it operates at a lower level. For typical speech, peaks are brief and the average-to-peak ratio is poor, meaning the transmitter spends most of its time well below full power.

The Speech Processor addresses this by limiting the envelope directly, increasing average power at the cost of some waveform distortion. The problem is that even after the Speech Processor, the bandpass filter that follows it — which is required to remove the out-of-band products created by the compression — can itself create envelope peaks that exceed unity. This is not a defect in the filter; it is an unavoidable consequence of bandpass filtering a non-flat-envelope signal. The filtered signal's envelope transiently overshoots the level that the Speech Processor established. These overshoots limit the average power that can be achieved, because the transmitter's peak ceiling must accommodate them.

CESSB eliminates these overshoots. It allows the transmitter to be driven closer to its full peak power limit while keeping the envelope below that limit, thereby increasing average transmitted power. Hershberger reports that properly implemented CESSB can recover approximately 2–3 dB of additional average power compared to speech-processed SSB without overshoot control, depending on voice characteristics and the amount of prior compression.

What CESSB is — and what it is not

This distinction is important enough to state plainly before describing the algorithm, because misunderstanding it leads directly to misuse.

CESSB is a filter overshoot controller. Its job is to prevent the brief envelope peaks that bandpass filtering introduces from exceeding the transmitter's peak limit. It is applied to a signal that has already been compressed by the Speech Processor, and its contribution is to suppress the filter-induced transients that sit on top of that compressed signal. Done correctly, CESSB is nearly transparent — the operator at the receiving end hears a signal that sounds like normal processed SSB, at a somewhat higher average level.

CESSB is not a compressor. It does not increase average power by the same mechanism as compression, and it does not behave like one when driven hard. A compressor reduces gain whenever the signal exceeds a threshold, and more drive produces more compression and more average power — up to a point determined by audible artifacts. CESSB reduces gain only in the immediate neighborhood of envelope peaks that exceed unity, and the amount of gain reduction is proportional to the degree of overshoot. If the input to CESSB is a properly compressed signal with only modest filter-induced overshoots, the gain reduction is small, brief, and largely inaudible.

If CESSB were to be over-driven, the algorithm would be forced to apply large, frequent gain reductions across wide portions of the signal. The result of that is not heavy compression with a frequency-full, loud sound; it is distortion. The gain reductions are not shaped to sound natural as a compressor's are, and the signal acquires a harsh, degraded character that has nothing to do with correct CESSB operation. The controversy that surrounds CESSB in amateur radio practice arises largely from this misuse: operators who have heard over-driven CESSB reasonably conclude that it degrades audio quality, when in fact the degradation comes from driving it incorrectly.

How the algorithm works

The CESSB implementation uses a sliding-window peak detector with a short look-ahead. The window spans a fixed number of samples computed from the signal bandwidth — at a sample rate of 96 kHz with a 3 kHz SSB bandwidth, this is approximately 10 samples, or about 0.1 milliseconds. This window width is chosen to match the duration of the shortest envelope fluctuations that the bandpass filter can produce, which is approximately the reciprocal of the filter bandwidth.

For each output sample, the algorithm maintains a running maximum envelope value across the entire window centered on that output sample. The centering is achieved by a delay line: the complex I-Q signal and its envelope are written into circular delay lines, and the output is read from a point half the window length behind the input. Each output sample therefore has equal amounts of past and future envelope data available for the peak detection.

If the maximum envelope across the window exceeds unity, the output sample is ultimately reduced by a factor slightly greater than the maximum envelope, reducing the local gain. This is done by creating a modified waveform and then re-filtering it through the same type filter. If the maximum envelope is at or below unity, the output sample passes through unchanged.

The use of the window, rather than a per-sample check, is what Hershberger calls “peak stretching”: a brief overshoot peak causes gain reduction not just at the precise sample of the peak, but across the entire surrounding window. This ensures the reconstructed SSB envelope cannot exceed unity even in the intervals between samples where a naive instantaneous check might miss a transient overshoot.

Why linear-phase filters are required

CESSB requires that the bandpass filters surrounding the Speech Processor and CESSB block operate in linear-phase mode. This requirement is fundamental to the algorithm's correctness.

The CESSB algorithm computes and controls the envelope of the complex I-Q signal: square-root($I^2 + Q^2$). For SSB, Q is the Hilbert transform of I — a 90° phase-shifted version of the signal at all frequencies. The accuracy of the envelope computation depends entirely on this quadrature relationship being preserved across all frequencies in the passband.

A minimum-phase (low-latency) filter has a phase response that varies nonlinearly with frequency — different frequencies are delayed by different amounts. When an SSB signal passes through such a filter, the phase relationship between I and Q is disturbed: Q is no longer the true Hilbert transform of I at all frequencies. The computed envelope is then an inaccurate representation of the true SSB envelope.

CESSB's gain decisions, based on this corrupted envelope estimate, are wrong — the algorithm may under-correct (allowing overshoots to pass) or over-correct (reducing gain unnecessarily on signals within limits).

A linear-phase filter has constant group delay: every frequency is delayed by exactly the same amount. I and Q emerge from the filter with their quadrature relationship intact, the computed envelope is accurate, and CESSB works correctly.

The cost is latency. Constant group delay means the signal is delayed by a fixed time equal to half the filter's impulse response length — typically a few milliseconds. This latency is inherent; it is the price of the accurate envelope computation that CESSB requires. As discussed in Appendix B, linear-phase filters are the only filter type that can provide this guarantee. When CESSB is enabled, the bandpass filters in the chain automatically switch to linear-phase mode, adding this latency to the processing chain.

The Speech Processor prerequisite

CESSB requires the Speech Processor to be active. The reason is structural: CESSB is an overshoot controller that suppresses the envelope peaks the bandpass filter creates after compression. The Speech Compressor limits the input level to CESSB to precisely 1.0, the exact level expected by CESSB prior to filter overshoots. The bandpass filter, whose overshoot CESSB monitors, is inserted after the Speech Compressor when it is active. Without the Speech Compressor in-line, CESSB could not operate correctly.

In practice, the two stages form a natural pair: the Speech Processor provides compression that increases average power, and CESSB then suppresses the filter-induced overshoots that the bandpass filtering introduces, allowing the transmitter to be driven to full peak power.

Practical use

CESSB is controlled by a single ON/OFF switch. When enabled, the bandpass filters automatically switch to linear-phase mode. No further adjustment is required or available; the window width is fixed for SSB operation, and the input level is controlled by the preceding stages such that CESSB operates correctly at the level it receives.

The most effective use of CESSB is with the Speech Processor set to a moderate gain — enough compression to increase average power meaningfully, but not so much that the signal already sounds processed before CESSB is added. CESSB then suppresses the filter-induced overshoots that this compression produces, allowing the transmitter to be driven to its full peak limit cleanly.

Operators who prioritize audio quality and are skeptical of any processing that introduces distortion may prefer to leave CESSB disabled. At modest Speech Processor gain levels, the distortion CESSB introduces is inaudible on a typical HF path; at higher gain levels it becomes increasingly audible. The choice of whether to use CESSB, and at what Speech Processor gain level, is ultimately a matter of what the operator values: maximum average power within the peak limit, or the cleanest possible audio at somewhat lower average power. Both are entirely valid priorities.

12.9 ALC

The Automatic Level Control (ALC) is the final gain stage in the transmit audio chain. After the ALC, the signal passes to interpolation and up-conversion to the RF frequency before reaching the DAC. The ALC's function is straightforward: ensure that the peak magnitude of the signal never exceeds a defined ceiling. This sounds simple, but the architecture that makes it work reliably — and why that reliability is non-negotiable in a DUC SDR — deserves careful explanation.

Why the ALC ceiling is critical

In most SDR transmitters, the digital signal produced by the audio chain is converted to an analog output by a DAC operating near its full dynamic range. Output power is typically controlled by DAC current or by subsequent stages such as an adjustable attenuator or similar mechanism downstream of the DAC — rather than by reducing the digital signal level fed to the converter. This means the DAC is driven at or near full scale regardless of the selected output power, and the digital signal must stay within the DAC's valid input range at all times.

If the digital signal exceeds the DAC's input ceiling, the converter clips — not gracefully, but catastrophically. A DAC operating in integer arithmetic wraps or saturates at its maximum value, producing extreme nonlinear distortion and very broadband spectral splatter that extends far outside the intended transmit band. Even a single sample exceeding the ceiling, lasting perhaps 10 microseconds, produces an artifact that is visible on a panadapter and audible on adjacent frequencies. Unlike audio peak limiting in a conventional transmitter, where a brief overshoot is merely a minor distortion, a DAC overflow in a DUC SDR is a severe event. The ALC is the last line of defense against this, and it must never be bypassed.

The look-ahead architecture

A conventional ALC system is reactive: it detects that an overshoot has occurred and then reduces gain to prevent the next one. But by the time the gain reduction takes effect, the damaging sample has already been output. For the DAC protection purpose described above, a reactive ALC is not sufficient — the overshoot that triggers it has already caused the problem it was meant to prevent.

The WDSP ALC uses a look-ahead architecture that eliminates this problem entirely. The incoming signal is written into a ring buffer. The output sample delivered at any moment is not the most recent input sample, but a sample that entered the buffer approximately 6 ms earlier. The gain computation, however, is performed on the maximum signal level observed across the entire look-ahead interval — including the samples that have entered the buffer but not yet been output.

The result is that the ALC knows what is coming before it has to deliver it. When a large peak enters the buffer, the gain begins reducing smoothly along an exponential curve before that peak reaches the output. By the time the peak sample is actually delivered, the gain has already been reduced enough to bring it within the ceiling. No overshoot ever reaches the output, and no sample exceeding the ceiling is ever sent to the DAC. This is not merely an improvement over a reactive ALC — it is a qualitatively different guarantee.

How the ALC differs from the Leveler

The ALC and Leveler use the same underlying gain-control code, but their parameters make them behave quite differently, as discussed in Section C.4. The key differences for the ALC are:

The decay time constant is 10 milliseconds — fifty times faster than the Leveler's 500 milliseconds. The ALC reacts to signal level changes over tens of milliseconds, not hundreds. This fast response, and an algorithm that repeatedly scans its delay line, is what allows it to track the rapid amplitude variations of speech without allowing peaks to slip through between gain updates.

The ALC measures the true envelope magnitude — $\text{sqrt}(I^2 + Q^2)$ — rather than the simpler $\max(|I|, |Q|)$ used by the Leveler. For a signal approaching the DAC ceiling, the true envelope is the relevant quantity, and the ALC uses the most accurate measurement available.

The maximum gain of the ALC is unity. Unlike the Leveler, which can boost quiet signals by up to 5 dB, the ALC only attenuates. It is a ceiling, not a normalizer.

The ALC is always on. There is no operating mode in which the ALC should be disabled. The main document provides an OFF control for test purposes only.

No ALC overshoot

A conventional feedback ALC overshoots: the gain reduction lags the signal, the output momentarily exceeds the threshold, and the feedback loop chases it back down. This overshoot is visible as a brief transient at the onset of loud passages and is a well-known artifact of feedback-based gain control.

The look-ahead ALC produces no overshoot. The envelope of the output signal is held smoothly at or below the ceiling with no transient excursions. This is particularly important at the DAC interface, where even a brief overshoot would be harmful, but it also means the ALC does not add the characteristic transient artifact of conventional ALC systems to the transmitted audio.

The ALC as the safety net, not the compressor

A common mistake is to use the ALC as a compression stage — setting upstream levels such that the ALC is regularly reducing gain by several dB during normal speech. The ALC will do this, and it will keep the output within the ceiling, but the result is not the same as proper compression.

The ALC's fast decay time (10 milliseconds) means it releases quickly after each peak, producing an uncharacteristic sound for voice. The Leveler, CFC, and Speech Processor are all designed to shape the signal before it reaches the ALC, so that the ALC sees a signal that is already well-conditioned and rarely needs more than a few dB of gain reduction. When the ALC is working hard, it is usually a sign that something upstream is not properly configured — either the input level is too high, the compression settings are too aggressive, or the CFC's Post-EQ pre-gain is not trimming the output level adequately.

The ALC meter in the metering chain shows the current gain being applied. In normal well-configured operation, this gain should be close to 0 dB most of the time, with brief reductions on the loudest peaks such that the signal is spanning the entire available dynamic range. An ALC meter that is frequently

showing significant gain reduction, e.g., “pinned” at 0dB during normal speech, is a useful diagnostic indicator that the upstream chain needs attention.

The attack and decay time constants

The ALC attack time constant (1 millisecond default) controls how quickly the gain decreases when a peak enters the look-ahead buffer. The decay time constant (10 millisecond default) controls how quickly the gain recovers after the peak passes. These defaults are well-chosen for voice operation and are generally best left at their values. The main document notes that the attack time constant in particular should be left at its default.

The gain recovery after a peak is gradual enough that it does not produce audible pumping on normal voice material at correct levels, but fast enough that the ALC does not hold gain down unnecessarily between words or during pauses.

12.10 Adaptive Predistortion and In-band Distortion

This final section steps back from the audio processing chain to consider a system-level effect that most operators are not aware of: the relationship between PA nonlinearity and distortion within the transmitted passband. The point is not to provide a tutorial on predistortion — that is a subject in its own right — but to explain something that adaptive predistortion does quietly and automatically, whether or not the operator has thought about it.

IMD: the visible and the hidden

When a power amplifier is driven into its nonlinear region, it produces intermodulation distortion (IMD). The most visible consequence is adjacent-channel splatter: IMD products appear on both sides of the transmitted signal, extending outward into the spectrum beyond the passband. These products are what operators check on a panadapter and what regulatory spectral purity requirements address. They are the reason PureSignal predistortion is typically enabled — to clean up the transmitted spectrum and eliminate interference to adjacent channel users.

Figure C-8 shows a transmitted SSB signal with adaptive predistortion disabled. The light-shaded region in the center is the transmitted passband; the bandpass filter edges are sharp and clearly defined. The IMD skirts extending outward from the passband on both sides are clearly visible. The products to the left are not a failure of the bandpass filter but a separate set of IMD products that fall on that side of the signal.

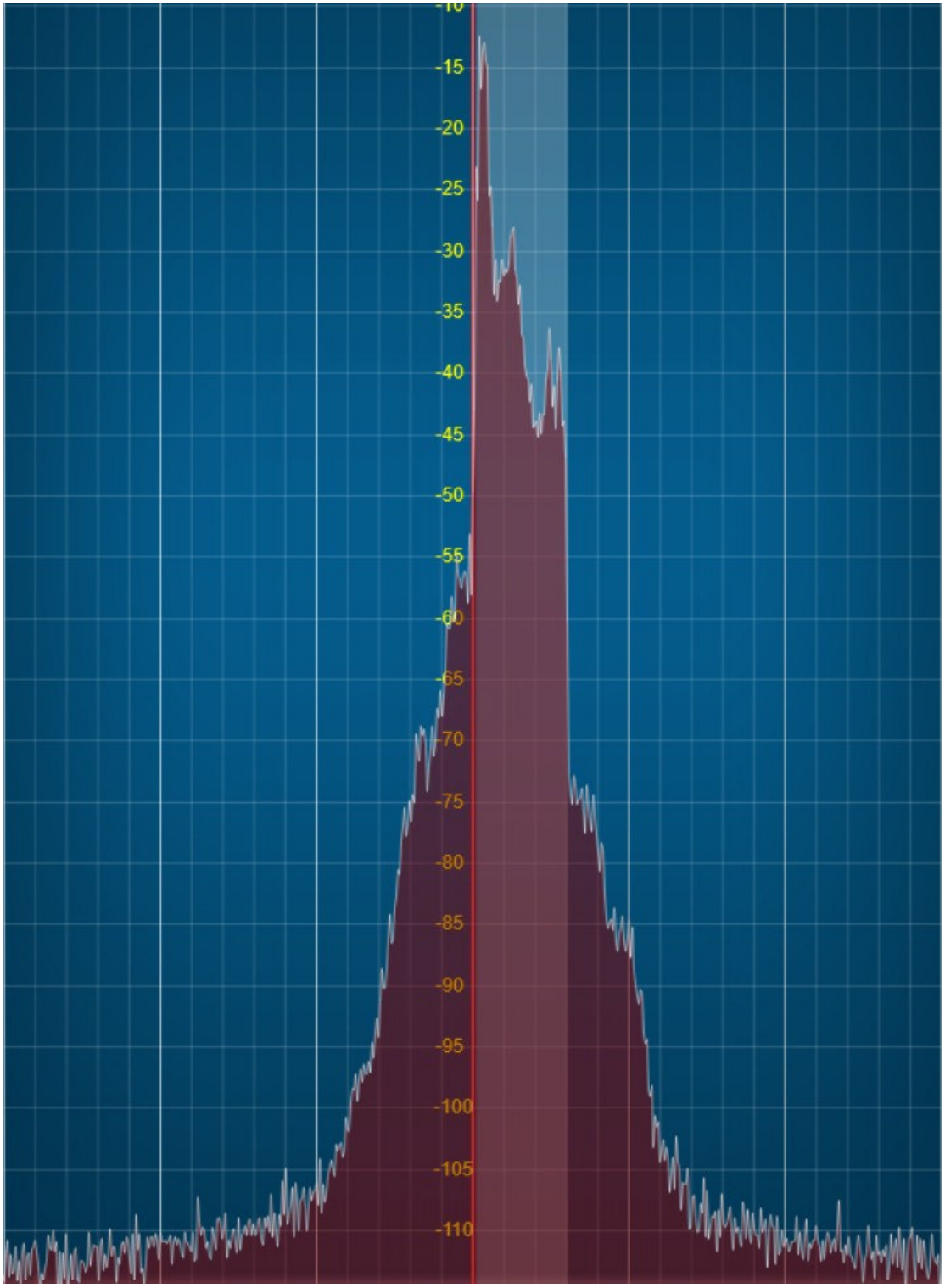


Figure C-8: Panadapter display of a transmitted SSB signal with adaptive predistortion DISABLED. The light-shaded area is the transmitted passband. IMD products are visible on both sides of the passband. The products to the left of the passband are not filter leakage — they are IMD products falling on that side of the signal.

The hidden products: in-band IMD

What Figure C-8 cannot show — because they are masked by the intended signal itself — are the IMD products that fall within the passband. These exist for exactly the same reason as the adjacent-channel products: PA nonlinearity causes audio frequency components to mix with each other, generating sum and difference products at frequencies that may fall anywhere in the spectrum, including within the passband.

For a simple two-tone test signal with tones near the edges of the passband, third-order IMD products fall just outside the passband — those are what dominate the screenshot and what the two-tone IMD specification measures. But a real voice signal has a very different character, and the frequency distribution of its IMD products is correspondingly different.

Voice energy is concentrated in the lower frequencies — fundamentals at 100–300 Hz, strong harmonics extending through 1 kHz, with diminishing energy above that. When two voice components at, say, 300 Hz and 500 Hz above the carrier mix through a nonlinear PA, their third-order IMD products fall at 100 Hz and 700 Hz — both well within the passband. For IMD products to fall outside a 300–3000 Hz passband, the interacting components must be spaced widely enough that their mixing products land beyond the band edges — a condition that requires components near both the upper and lower passband limits simultaneously. Most voice energy does not satisfy this condition.

The fifth-order and higher odd-order products cluster even closer to the originating tones than third-order products do. The net result is that the bulk of the IMD energy generated by a voice signal — dominated as it is by lower-frequency components — falls within the passband rather than outside it. The adjacent-channel splatter visible on the panadapter arises primarily from interactions among the relatively sparse components near the upper passband edge. The much larger population of lower-frequency interactions produces products that are hidden within the signal itself. The panadapter shows the tip of the iceberg; the greater part of the IMD energy is in-band and invisible.

These in-band products add to the intended signal at each frequency, distorting the spectral content of the transmission. For a PA operating moderately into its nonlinear region, the in-band distortion floor is typically 10–20 dB below the adjacent IMD products visible outside the passband, and 40–60 dB below the peak signal. Whether this is audible at the receiving end depends on how hard the PA is being driven: a PA well back from its 1 dB compression point produces little in-band IMD, while one pushed several dB into compression can produce distortion that an experienced receiving operator notices as harshness, muddiness, or reduced intelligibility — qualities that are difficult to attribute to a specific cause because the distortion is hidden within the signal rather than visible beside it.

What predistortion does

Adaptive predistortion — PureSignal in this implementation — works by characterizing the PA's nonlinear transfer function and applying a pre-distortion to the input signal that is the mathematical inverse of the PA's distortion. The distorted input passes through the PA, and the PA's nonlinearity acts on the pre-distorted signal in a way that largely cancels the distortion, producing a more linear output.

The important point for this section is that predistortion corrects PA nonlinearity at its source. It does not filter the output — it linearizes the amplifier. An amplifier that has been linearized by predistortion produces less IMD of all orders, at all frequencies — both outside the passband and within it. The adjacent-channel splatter is reduced, which is what operators see on the panadapter. But simultaneously and automatically, the in-band IMD products are reduced by the same mechanism. The receiving operator hears a cleaner, less distorted signal even if neither operator can see the improvement on any display.

This is the aspect of predistortion that is least understood and least discussed. Operators think of PureSignal as a tool for reducing splatter and meeting spectral purity requirements — which it is. But enabling predistortion also improves the quality of what is inside the passband, reducing the subtle distortion that degrades intelligibility and naturalness on a driven PA. The benefit is real even when the adjacent-channel products look acceptable, and even when the in-band distortion is too low to identify as distortion rather than simply as "not quite as clean as it could be."

A practical note

The magnitude of the in-band IMD benefit from predistortion depends on how nonlinear the PA is to begin with. On a PA operating well back from its compression point, both the adjacent-channel products and the in-band products are already low, and predistortion provides modest additional improvement. On a PA being driven hard — as is common when operators are trying to maximize transmitted power — the nonlinearity is more severe, the in-band IMD is correspondingly higher, and the predistortion benefit is more substantial.

Operators who run their PA at high power levels and have wondered why their signal sounds slightly harsh or processed at the receiving end, even with a well-adjusted audio chain, may find that enabling or retuning adaptive predistortion improves the received audio quality in a way that no adjustment to the transmit audio chain alone can achieve. The audio chain produces a clean signal; predistortion ensures the PA delivers it cleanly.

[5]: Peter Martinez, G3PLX, "A New Processor for SSB," RadCom, January 2010.

13 Appendix D: Intermodulation Distortion (IMD) and PureSignal

Understanding Transmit Distortion, Its Correction, and What Your Measurements Really Mean

This appendix is written for the operator who wants a clean transmitted signal and wants to understand what PureSignal is doing to achieve it. The main body of this reference manual describes the transmitter and its predistortion from a developer's perspective. Here the goal is different: to explain, in plain terms, what intermodulation distortion is, how predistortion reduces it, and — just as important — how to interpret the measurements you make of it. That last point matters because the most familiar bench test, the two-tone test, can report results that seem to contradict what you hear and what a broadband measurement shows.

13.1 What IMD Is, and Why It Matters

No amplifier is perfectly linear. Ideally, an amplifier would multiply its input by exactly the same factor at every instant, regardless of how strong the signal happens to be at that moment. Real amplifiers fall short of this: the gain, and to a lesser extent the phase, change slightly with the instantaneous signal level. At low drive the gain may be one value; as the signal approaches full drive the gain sags, and the phase shifts. These are commonly called the AM-to-AM (amplitude affecting amplitude) and AM-to-PM (amplitude affecting phase) characteristics of the amplifier.

When a signal contains more than one frequency — and every real voice signal contains a great many — this level-dependent gain causes those frequencies to mix with one another. The mixing produces new frequencies that were not present in the original signal, called intermodulation products. The stronger and more troublesome ones fall inside the passband of the original signal and nearby, just outside the passband. On a panadapter they appear as “shoulders” and skirts extending beyond the intended signal width. On the air, they are splatter — energy landing on frequencies used by your neighbors.

This is why IMD matters to an operator. It is not an abstract quality metric; it is the difference between a signal that stays within its intended bandwidth and one that interferes with adjacent stations. A clean signal is a narrow signal, a legal signal, and a courteous one on a crowded band. Reducing IMD is the whole purpose of predistortion.

13.2 Predistortion and PureSignal

The idea behind predistortion is elegant. If the amplifier distorts the signal in a known way, then we can distort the drive signal in advance by exactly the opposite amount, so that the amplifier's own distortion cancels our pre-distortion and the output emerges clean. Where the amplifier's gain sags, we drive it a little harder; where its phase shifts, we shift the drive phase oppositely. Applied continuously and correctly, the two distortions annihilate one another and what leaves the antenna is far closer to the ideal amplified signal than the amplifier could produce on its own.

The challenge is knowing the amplifier's behavior accurately, because it is not fixed — it varies with frequency, temperature, supply voltage, and the amplifier's own aging. PureSignal solves this by

measuring the amplifier continuously. A sample of the transmitted signal is fed back from the amplifier output and compared against what was sent. From the relationship between the two, PureSignal builds a correction that describes how the amplifier's gain and phase vary with drive level, and it updates that correction as conditions change. The mechanics of how this correction is constructed — the receiver feedback path, the calibration, and how the resulting correction is applied in the transmit chain — are described below and in the predistortion discussion of Appendix C. The result is a dramatic reduction in IMD: shoulders that were prominent on the panadapter drop toward the noise floor, and the transmitted signal collapses to its intended width.

13.3 Measuring IMD: Two-Tone, Noise, and Speech

The purpose of all of this is to transmit clean *speech*. That is worth keeping in mind, because it determines which measurement actually tells you what you want to know. Human speech is a broadband, constantly varying, noise-like signal. Its envelope never settles; it sweeps continuously and unpredictably across the full range of levels. A measurement that resembles speech will therefore predict on-air performance far better than one that does not.

A noise test drives the transmitter with band-limited random noise. Because noise is broadband and its envelope varies randomly across all levels, it is a reasonable proxy for speech. The resulting IMD — visible as spectral regrowth, the spreading of energy beyond the intended bandwidth — reflects how the correction will perform on an actual voice signal. When you want to know how clean your signal really is, the noise test is the most reliable measurement to trust.

The two-tone test, by contrast, drives the transmitter with just two steady sinusoidal tones. It is the traditional IMD measurement, developed in the 1930s, and it has some real virtues: it is simple, repeatable, and it places the intermodulation products at easily identified frequencies. But in a modern software-defined radio it must be read with caution, for several reasons that have nothing to do with the quality of the correction and everything to do with the nature of the two-tone stimulus.

The first is periodicity. Two tones of equal amplitude produce an envelope that beats up and down at a steady, perfectly regular rate — the same trajectory, from null to peak and back, repeated endlessly. This is nothing like speech. It means the amplifier is exercised over the identical path again and again, and any feature of the correction curve at a particular level is visited on every single beat, in a fixed and repeatable way. A deterministic stimulus like this can expose — or reward — a specific detail of the correction that a random signal such as speech or noise would average away.

The second is the interaction between tone spacing and sample rate. When the spacing between the two tones is a sub-multiple of the processing sample rate, the digitized envelope repeats its exact sample values in tight clusters rather than spreading them smoothly across all levels. The calibration then sees the amplifier at only a few repeated points instead of a continuous sweep. In some situations, this can both make it difficult for the calibration to find all the data it needs to characterize the amplifier and it can distort the correction. This is of course very different than a more random voice signal. Choosing a tone spacing that is not a simple sub-multiple of the sample rate avoids this particular trap.

The third is the nature of what the analyzer sees. A two-tone signal appears as two narrow spikes, and its products as further narrow spikes at specific frequencies. The measured number may depend on the analyzer's capabilities, setup, adjustment and exactly where those spikes land relative to the analyzer's resolution. A small change in setup can move a product into or out of a measurement bin and/or change its perceived level. A noise test, presenting broadband energy, gives a more robust and representative picture of total spectral regrowth than the height of an individual spike.

> **A practical caution:** a two-tone reading and a noise reading of the same transmitter can differ, and when they disagree the noise reading is the more representative guide to how the signal will actually sound on the air. A difference of a few decibels on a two-tone test, in particular, is well within the range that the test's own quirks can produce, and should not by itself be taken as evidence that one IMD result or correction is better or worse than another.

13.4 Memory Effects

There is one further complication in the amplifier's behavior that deserves its own discussion, because it sets a fundamental limit on how well predistortion can work, and because it lies at the heart of some differences described in the next section. So far we have described the amplifier's gain as depending on the instantaneous signal level. That is very nearly true, but not exactly. The gain also depends, a little, on what the signal was doing just before this instant — the recent history of the envelope. Thermal effects, drain and gate bias supply dynamics, and charge storage within the devices all give the amplifier a short memory.

The practical consequence is that at one and the same instantaneous input level, the amplifier's gain can be slightly different depending on whether the envelope is on its way up or on its way down. Rising through a given level, the gain takes one value; falling back through the same level a moment later, it takes another. This is a memory effect, and the diagram below illustrates it: instead of a single gain curve, the amplifier follows a thin loop, with the rising and falling paths separating most where the memory effect is strongest and coming back together where it is weak.

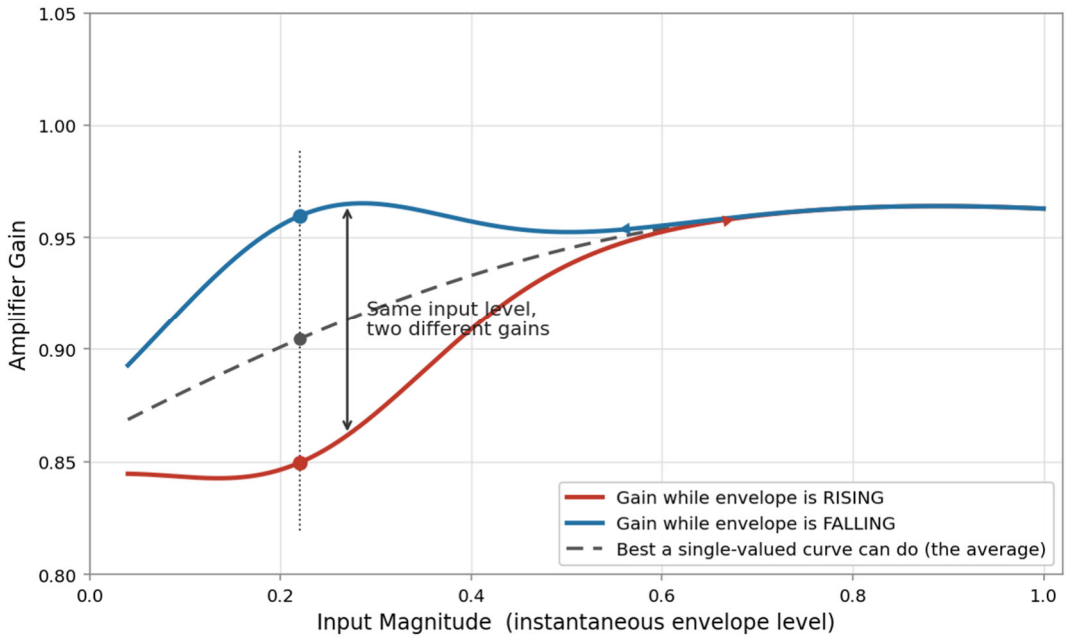


Figure D-1: A memory effect, illustrated. At a single instantaneous input level the amplifier presents two slightly different gains depending on whether the envelope is rising or falling. A single-valued correction curve can only follow the average of the two paths (dashed), so it cannot fully invert an effect that is genuinely two-valued in level. The separation is exaggerated here for clarity.

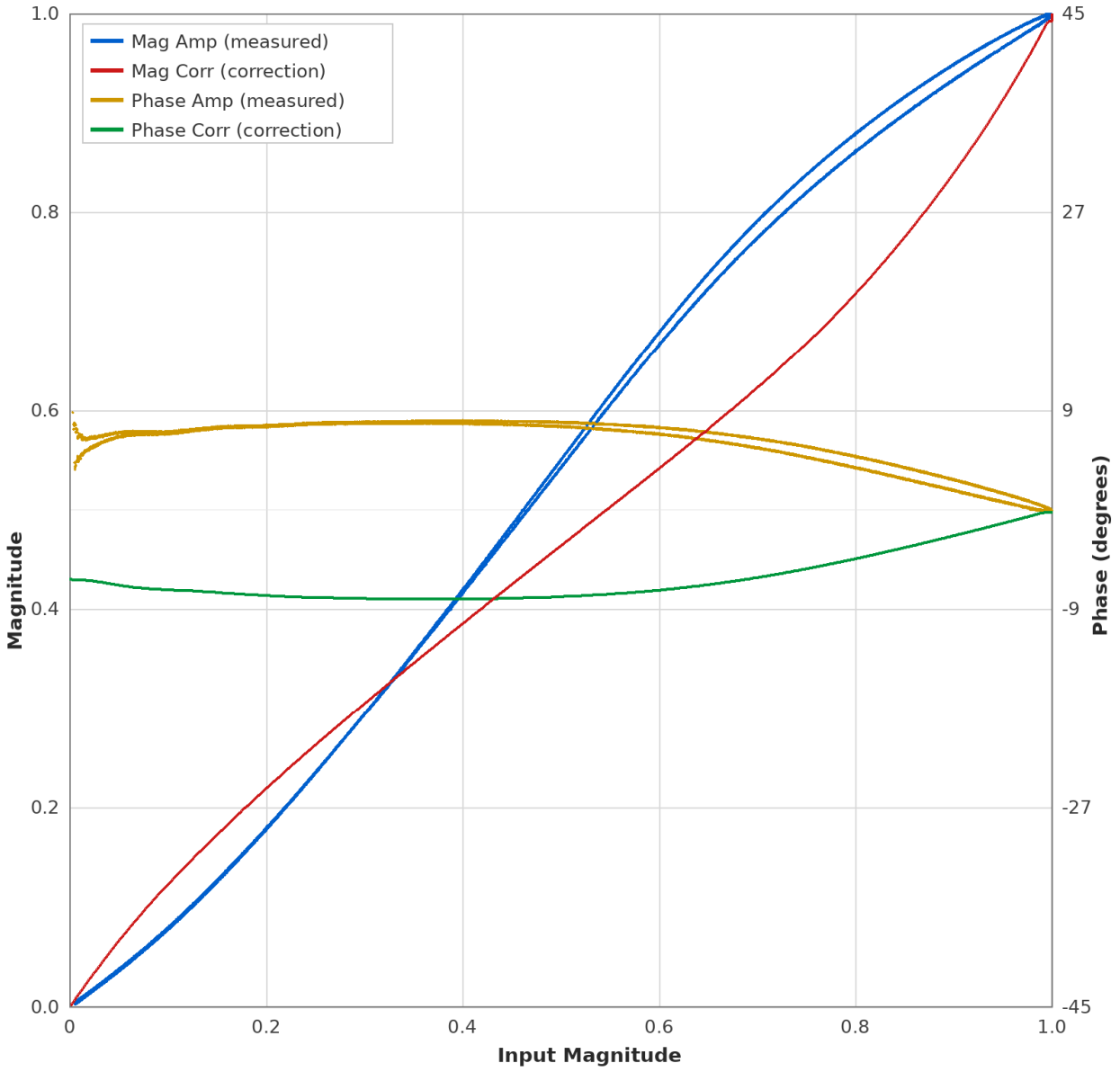


Figure D-2: A memory effect observed directly in a real amplifier.

Here is the crucial point for predistortion. A typical correction curve has exactly one value at each level — it is, a single-valued function of drive. It therefore cannot perfectly undo an effect that has two values at each level. No matter how carefully it is built, the best a single-valued correction can do in the presence of a memory effect is to follow the average of the two paths, leaving a small residual that it is fundamentally unable to remove. This is a physical limit, not a deficiency in the chosen correction algorithm; it is the reason a real transmitted signal, however well corrected, retains a small irreducible amount of distortion in the region where memory effects are present.

Over the years, algorithms have been developed to mimic the time-varying behavior of amplifiers and attempt to cancel memory effects. The latest PureSignal software also contains a novel ability to track the direction of travel along a correction curve and choose different branches — using this has not been

fully developed or tested. These capabilities may be utilized at some point in PureSignal; however, at the moment, for predistortion use in amateur radio, there are more significant issues to address such as feedback resolution and purity.

13.5 PureSignal 3 versus PureSignal 2

PureSignal 3 is a redesign of how the correction is constructed. It shares the core idea of its predecessor — measure the amplifier, build a correction, apply it continuously — but it differs in a way that becomes visible in the correction curves and, under certain tests, in the measured numbers. Understanding the difference explains an apparent paradox that a meticulous operator might encounter.

One essential change concerns when and where smoothing is applied. The earlier version stabilized its correction by smoothing the measured data before fitting its curves to it — the data was averaged first, and the correction was built from the smoothed result. PureSignal 3 instead fits its curves to the raw measured data and applies smoothing afterward, to the finished correction. The distinction sounds subtle but its effect is not: because PureSignal 3 fits the raw data, it sees and represents the amplifier's true behavior faithfully, including the low-drive memory effects. The earlier approach, by smoothing first, averaged that behavior away before it was ever fit or displayed. A related improvement is at the very lowest drive levels: PureSignal 3 extends its correction cleanly down toward zero drive, anchored on the actual low-drive measurements rather than forced through an arbitrary fixed point.

One consequence is visible directly in the correction curves. Figure D-3 compares the two versions on the same amplifier under identical two-tone drive. The lower-drive region is where they most visibly differ: PureSignal 3 produces a smooth gain correction that continues cleanly down toward zero drive, while the earlier version's correction carries a small ripple across the low-drive region — a by-product of how it anchors the curve at the origin. The scattered points in the measured gain at low drive are the amplifier's real memory effect, not noise or instability; PureSignal 3 fits that raw data directly rather than smoothing it away. In short, the PureSignal 3 curve is showing you more of the truth, and carrying less of an artifact of its own.

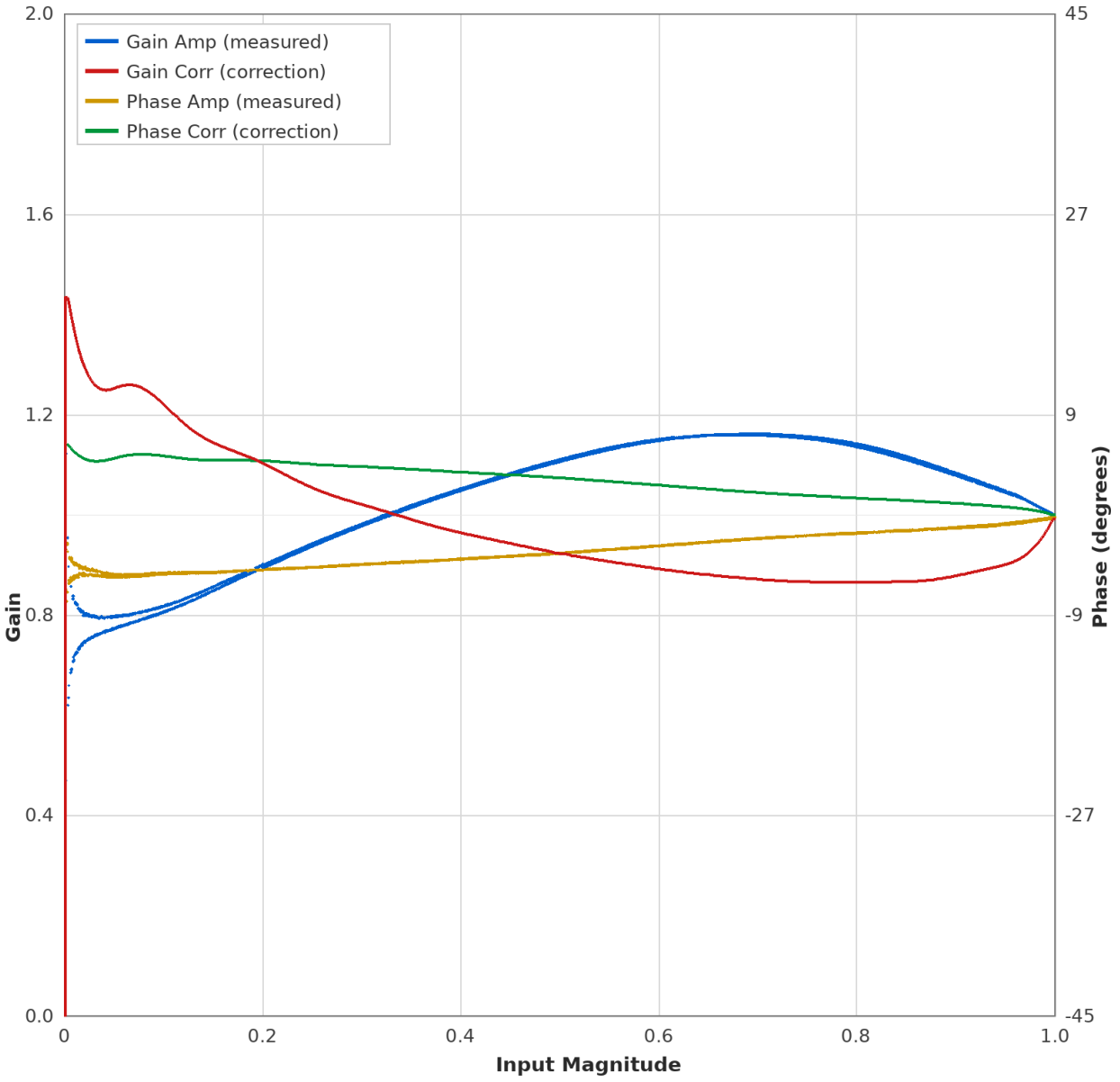
The envelope of a two-tone signal has steeper slopes and sharp cusps at low amplitudes. These tend to generate higher-order (farther out) but lower level IMD. Therefore it turns out that the high-order IMD is very sensitive to the shape of the correction curves on the left side, at low amplitudes.

In contrast, the stronger low-order (closer in) IMD is most affected by the stronger signals controlled by the correction curves on the right side. This is also one of the areas where memory effects are often present, fueled by the stresses of higher currents, more voltage sag and more temperature variation. This makes the stronger low-order IMD products very sensitive to the exact shape of the correction curves as we look toward the right side.

Comparing very carefully the two plots of Figure D-3, there is a difference in the high amplitude region. The smooth curvature of the blue curve, the measured amplifier, is the same in both and we note that the trace widens slightly, indicating a memory effect, in the input magnitude region of 0.8 to 1.0. For PureSignal 3, the red correction curve also has a perfectly smooth curvature, faithfully taking the

memory effect's central path and compensating. However, looking very carefully at PureSignal 2, there is an anomaly. In this region of interest, the curve has an area where is flattens just slightly and then turns upward more abruptly. Although barely visible on these graphs, this is exactly the kind of thing that can cause a couple of dB of difference in specifically a two-tone measurement. Which curve would produce the better two-tone measurement? We cannot determine that from these graphs, it could be either.

PureSignal 2



PureSignal 3

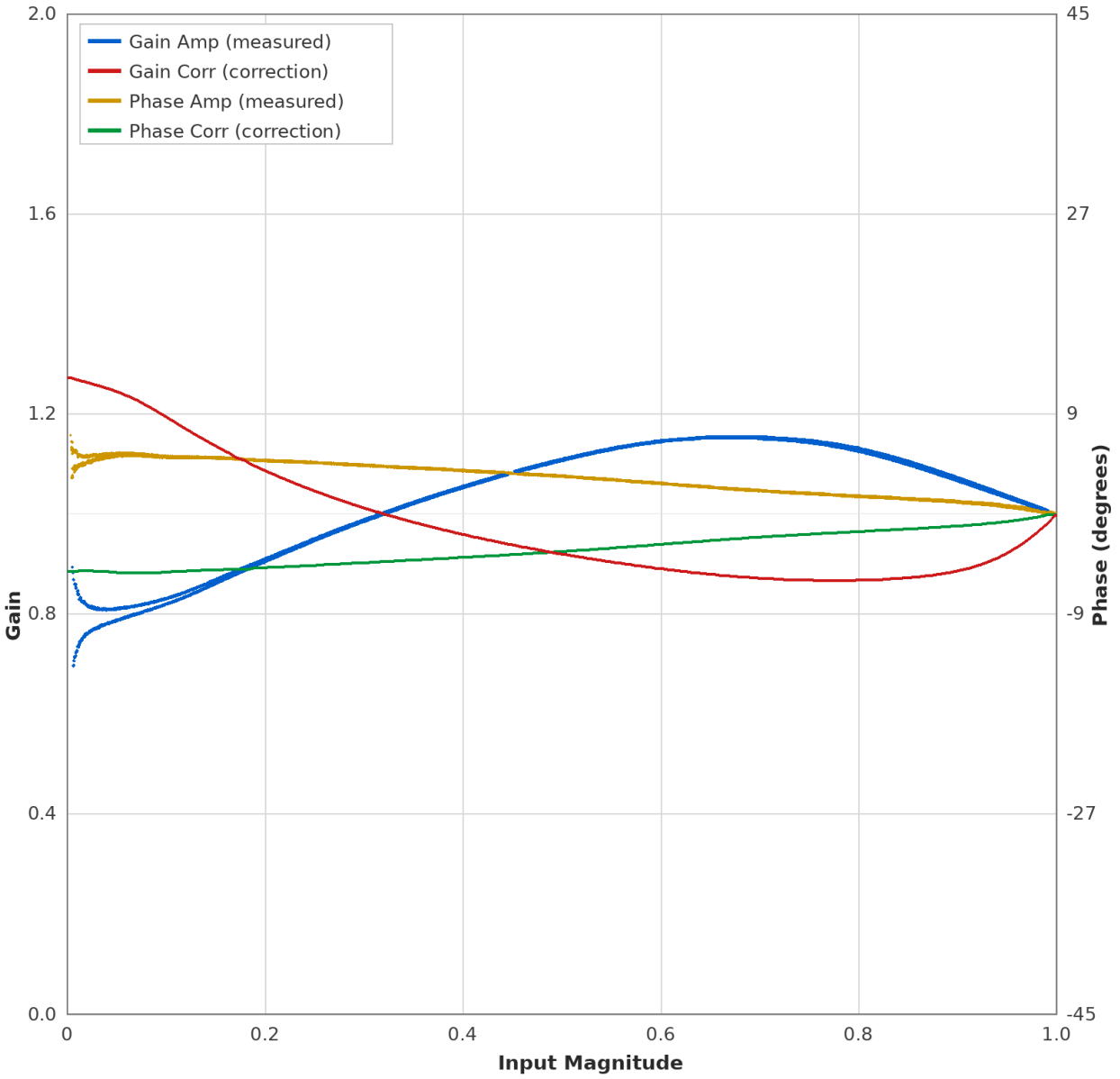


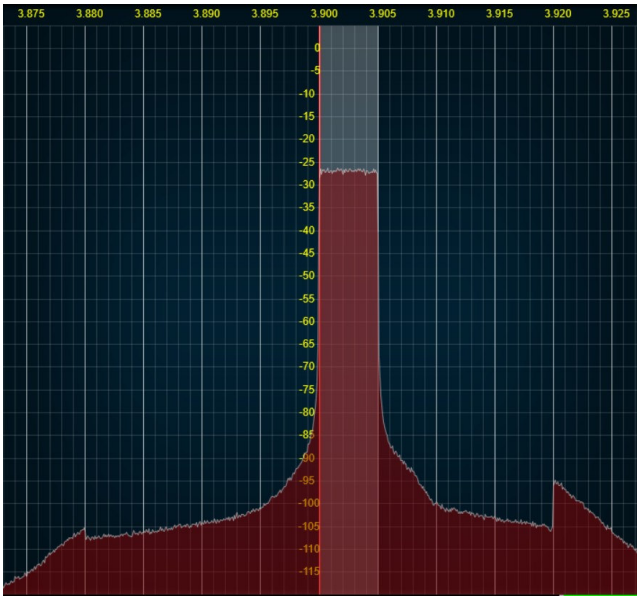
Figure D-3: Correction curves for the same amplifier under identical drive — the earlier version (top) and PureSignal 3 (bottom). Watch the gain-correction curve (red) at low drive: PureSignal 3's is smooth and continues cleanly toward zero, while the earlier version's carries a small ripple. The scattered points at low drive in the measured gain (blue) are the amplifier's real memory effect. This low-drive ripple, absent in PureSignal 3, is a left-side feature whose dominant effect is on the high-order (far-out) IMD; the subtle right-side difference discussed above is the one most likely to shift a close-in two-tone reading.

This brings us to clearly stating the paradox, and it is worth stating plainly because it can be genuinely misleading. On a two-tone test, PureSignal 3 sometimes measures a few decibels worse in close-in (third- and fifth-order) IMD than the earlier version. This does not mean PureSignal 3 is correcting less well. Recall that a memory effect cannot be fully corrected by any single-valued curve, and that the two versions, because they are built differently, arrive at slightly different — but equally legitimate — single-

valued curves through the region where the memory effect lives. PureSignal 3 lands on the honest average of the amplifier's behavior, which is the correct choice for a general, broadband signal. The earlier version's curve, having smoothed the data first, sits at a slightly different point. On the steady, periodic envelope of a two-tone — and only there — that different point can happen to leave a marginally smaller residual in the specific frequency bins the analyzer is watching. It is the periodic, repetitive nature of the two-tone stimulus that makes this possible; a broadband signal offers no such fixed target to flatter.

The evidence that this is a two-tone measurement artifact rather than a real difference in correction quality is consistent across tests. First, the difference does not appear on a noise test: with a broadband stimulus, the close-in IMD of the two versions is effectively the same, as the paired spectra in Figure D-4 show. Second, on that same noise test, PureSignal 3 is the cleaner of the two farther out from the signal — the region governed by the fine, fast structure of the nonlinearity that PureSignal 3 captures and the earlier version smoothed away. Third, the low-drive ripple in the earlier version's correction (Figure D-3) does slightly modulate the outgoing signal, but its dominant effect is on the far-out region, consistent with the sensitivity of high-order IMD to the left side of the curve; it is not the primary driver of the close-in difference. Because the ripple sits at low drive, the two transmitted signals are in any case nearly identical. The close-in difference instead traces to the subtle right-side difference in the high-amplitude region described above — the region that governs low-order products — where, on the steady periodic envelope of a two-tone, a small but repeatable difference in curve shape can shift the measured products by a couple of decibels.

PureSignal 2



PureSignal 3

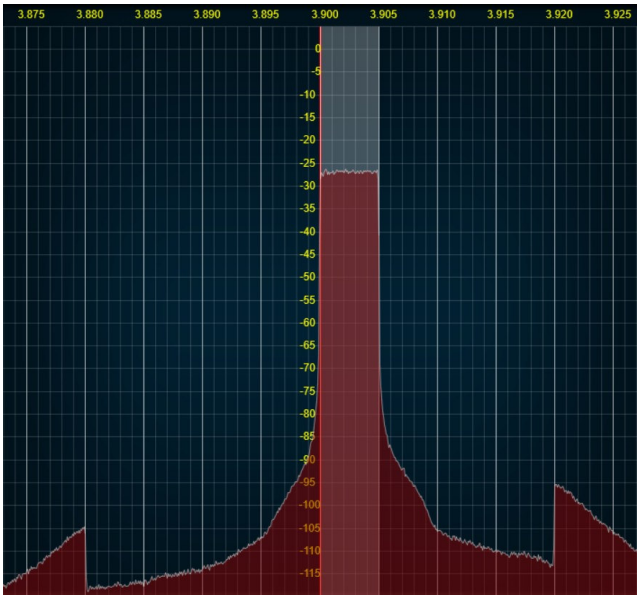


Figure D-4: A broadband (noise) test of the two PureSignal versions on the same transmitter — the earlier version (top) and PureSignal 3 (bottom). Close in to the signal the two are essentially the same; farther out, PureSignal 3 sits somewhat lower, consistent with its more faithful capture of the amplifier's fast nonlinearity. The raised "ears" near ±20 kHz mark the edges of the corrected bandwidth — beyond that span the correction is deliberately not being applied.

Taken together, these observations point to a single, defensible conclusion: PureSignal 3 is the more faithful correction. It measures at least as well as its predecessor by every test that resembles a real signal, and better in the far-out region on those tests. Any apparent disadvantage exists on the two-tone test, close-in, and only because a periodic stimulus can be flattered by an incidental features. We should be candid about the limits of this explanation: the direction and the mechanism are well established by the measurements above and experience, but the precise size of a difference on any given amplifier also depends on the particular amplifier's gain, phase and memory characteristics and the exact test conditions.

13.6 Interpreting Your Own Measurements

For the operator standing at an analyzer, a few practical guidelines follow from all of the above. Trust the noise and speech behavior of your signal over the two-tone numbers; they are the faithful measure of what you actually transmit. If you do use a two-tone test, avoid tone spacings that are simple sub-multiples of your sample rate, which can stress the calibration artificially. In viewing gain and amplitude plots, don't be alarmed by curves with branches and some scattered samples — these are the results of the amplifier's real memory effects, faithfully shown rather than hidden. And treat a difference of a few decibels in two-tone IMD between one correction and another with appropriate skepticism: it is within the range the test itself can produce, and it is not, on its own, a reliable indication that one correction is genuinely cleaner than the other for the real signal derived from speech.

The most honest single measurement of correction quality remains the one that most resembles the signal you actually send. Drive the transmitter with noise, look at the spectral regrowth on either side of your signal, and listen. A correction that gives a clean, narrow noise spectrum and natural-sounding speech is doing its job.

Revision History

Rev	Date	Changes
0.10	2016-08-05	Initial version of this document; released for comments.
0.11	2016-08-24	Added TXASetSipSpecmode(…) in WDSP and in this document. Added comments on SetPSControl().
0.12	2016-11-28	Added new functions calls for PureSignal 2.0.
1.14	2017-03-17	Added GetWDSPVersion(). Added 1 to the revision number of this document such that it now matches the WDSP revision. Documented calls for EER.
1.15	2017-05-06	Added AM_LSB and AM_USB transmit modes. Added CFC and Phase Rotator audio tools.
1.16	2017-05-28	Added CFC Metering. Moved phase rotator in processing chain.
1.17	2017-08-25	Added Variable Ratio Resampler and Rate Matching in WDSP and document.
1.18	2019-01-21	Added 'dexp', combined VOX and Downward Expander.
1.19	2022-01-27	Added function to get CFC data to display compression vs frequency. For the panadapter/waterfall, added an 'rms' detector type. For the variable ratio resampler, added 9 functions to allow dynamically setting/getting some internal parameters for experimentation and tuning purposes. Added additional arguments in functions getRMatchDiags(...) and create_rmatchV(...).
1.20	2022-02-15	Internal enhancements supporting portability to non-Windows operating systems. Enhanced panadapter/waterfall frequency accuracy for high-zoom cases. Porting Considerations: (1) There are two new files that must be included in the build - calculus.c and calculus.h, (2) for the display, setup for calls to SetAnalyzer(...) is somewhat different for optimum results and parameters fscLin and fscHin should be passed as doubles -- see this Guide.
1.21	2023-04-29	Added a volume limiter to the FM Demodulator to balance signal and noise volume during squelch tail or weak signal reception with squelch OFF. Added Voice Squelch functionality as an alternative especially for SSB reception.
1.22	2023-05-20	Efficiency Improvements - less system calls at the expense of slightly higher average thread and memory allocations.
1.23	2023-06-17	Efficiency Improvements.
1.24	2023-11-08	Added time adjustment for TXA carrier up-slew. Added bandwidth adjustment for FM transmit and receive. Added a function that can be used to clear display buffers. In the phase rotator, added a function to reverse microphone phase (180 degrees).

Rev	Date	Changes
1.25	2024-07-05	Added "Trained" mode for Spectral Noise Reduction along with NSTAT NPE Method.
1.26	2025-06-30	Added Bandpass Characterization Utility to provide a graphical characterization of bandpass filter magnitude. Added Filter Impulse Response Caching to speed-up re-use of previously used filters. Removed requirement to use an external file, zetaHat.bin, for Trained NR2 mode. For TX, added pulsed-tone and pulsed-two-tone capability. Added capability to return the largest bin value from a specified band within the display FFT output.
1.27	2025-09-19	Various code changes to make the CLANG LLVM Static Analyzer checker tool (used for Linux implementations) happier. Addition of psychoacoustic post-processing for NR2 Noise Reduction. Minimum Phase filter responses were improperly cached - this has been removed.
1.28	2025-10-21	Minimum Phase filter caching resolved and added back. Added impulse response generation for Gaussian filters for future testing and potential inclusion in the receiver.
1.29	2026-01-16	To the existing CW APF, added new filter options: DoublePole, Matched, and Gaussian.
2.0.0	2026-07-01	** NEW: PureSignal 3.0 - Major Release: new data structures, very robust algorithms, simpler operation. ** Free-curves - direct control and algorithm improvements of EQ and CFC. ** Phase Rotator - Auto-calibration and Asymmetry pre/post display data. ** Continued CW APF Improvements. ** New RX input decimator - improved output bandwidth, better alias rejection, up to 6144K input rate. ** Broadcast FM Stereo Demodulator. ** Three very informative appendixes in this document. ** (Note: FFTW Wisdom will be re-computed for this version of WDSP.)
2.1.0	2026-09-04	** NEW: Deep Neural Network Noise Reduction. PureSignal 3.0 - algorithm tuning & reduced CPU cycles for calibration. Efficiency improvements in filter generation. Other minor code changes. Added wdsp.h auto-generated header containing function prototypes for exported calls. Added Appendix D in this document.
2.1.1	2026-10-15	Added Standard and Premium CW Models in NNR. SetChannelState(...) enhancement. Documentation additions. Bug fix in fmd.c and emph.c. Porting: #include <limits.h> in wbfm.c; #include "wdsp_align.h" in extrapolate.c, nurbs_fit.c, nurbs_spline.c; enums in wdsp.h.